



Towards a coherent algorithmic theory and implementation of pattern-based land use and land cover change modeling

François-Rémi Mazy

► To cite this version:

François-Rémi Mazy. Towards a coherent algorithmic theory and implementation of pattern-based land use and land cover change modeling. Computer Aided Engineering. Université Grenoble Alpes [2020-..], 2022. English. NNT: 2022GRALM043 . tel-04382012

HAL Id: tel-04382012

<https://theses.hal.science/tel-04382012v1>

Submitted on 9 Jan 2024

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

THÈSE

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITÉ GRENOBLE ALPES

École doctorale : MSTII - Mathématiques, Sciences et technologies de l'information, Informatique

Spécialité : Mathématiques Appliquées

Unité de recherche : Institut National de Recherche en Informatique et en Automatique

Vers une théorie et une implémentation algorithmique cohérente des modèles pattern-based de changement d'usage et de couverture des sols

Towards a coherent algorithmic theory and implementation of pattern-based land use and land cover change modeling

Présentée par :

François-Rémi MAZY

Direction de thèse :

Emmanuel PRADOS

CHARGE DE RECHERCHE, Université Grenoble Alpes

Directeur de thèse

Pierre-Yves LONGARETTI

Astrophysicien, CNRS

Co-encadrant de thèse

Rapporteurs :

BRITALDO SILVEIRA SOARES FILHO

Professeur, Universidade Federal de Minas Gerais

JEAN-FRANÇOIS MAS

Professeur, Universidad Nacional Autonoma de Mexico

Thèse soutenue publiquement le **14 décembre 2022**, devant le jury composé de :

BRITALDO SILVEIRA SOARES FILHO

Professeur, Universidade Federal de Minas Gerais

Rapporteur

JEAN-FRANÇOIS MAS

Professeur, Universidad Nacional Autonoma de Mexico

Rapporteur

BERNARD TOURANCHEAU

Professeur des Universités, UNIVERSITE GRENOBLE ALPES

Président

ELISE ARNAUD

Maître de conférences, UNIVERSITE GRENOBLE ALPES

Examinatrice

Invités :

EMMANUEL PRADOS

Chargé de recherche HDR, INRIA CENTRE GRENOBLE-RHONE-ALPES

PIERRE-YVES LONGARETTI

Astrophysicien, CNRS DELEGATION ALPES



François-Rémi Mazy

TOWARDS A COHERENT ALGORITHMIC
THEORY AND IMPLEMENTATION
OF PATTERN-BASED LAND USE AND
LAND COVER CHANGE MODELING



January 13, 2023

*À mes grands parents,
Hélène et Jacques Zawadzki,
dont je prends aujourd'hui le nom.*

REMERCIEMENTS

Il me semblait naturel que l'écriture des remerciements constituait la toute dernière des tâches à accomplir avant l'achèvement du manuscrit. Pourtant, je crois aujourd'hui qu'il aurait été plus juste pour toutes ces personnes qui m'ont soutenu de les remercier à chaque attention, à chaque instant partagé. Car au-delà de la reconnaissance que je me dois de leur rendre, trois années constituent un temps bien assez long pour qu'il ne soit pas impossible que j'oublie quelqu'un-e dont le chemin a croisé le mien une certaine période puis s'en est relativement éloigné. Que cell-eux-ci veuillent bien m'en excuser.

Pierre-Yves, tu es bien sûr le premier cité de mes remerciements. Tu as su m'accompagner tout au long de cette thèse avec une bienveillance amicale, une disponibilité sans faille et un enthousiasme constant. Cette thèse n'aurait jamais pu être ce qu'elle est sans ta rigueur scientifique et ta contribution considérable. Je souhaite ici t'exprimer toute ma reconnaissance et la fierté que je ressens d'avoir pu ainsi travailler conjointement avec toi.

J'éprouve une certaine mélancolie en repensant au premier échange de mail que j'ai pu avoir avec toi Emmanuel. Instantanément, j'ai perçu avoir trouvé un rivage accueillant pour l'ingénieur en quête de sens que j'étais. Tu fais partie des figures inspirantes que j'ai croisé et qui forment aujourd'hui la personne que je suis. Merci à toi.

J'ai eu la chance d'appartenir à la fabuleuse équipe STEEP qui tente, il me semble, d'inventer à son échelle une autre manière de faire de la science. Certes, des circonstances inédites — mais non surprenantes — ont quelques peu mis à l'épreuve nos liens. Toutefois, je ne pense pas qu'elles expliquent entièrement ma faible présence quotidienne au sein de l'équipe. La vie de doctorant est tout a fait particulière et exigeante et je crois qu'il m'a fallu trouver en dehors de celle-ci les points d'appuis nécessaires pour tenir cette épreuve d'endurance. Excusez-moi et soyez remercié-es pour tous les instants partagés au laboratoire, en séminaire et ailleurs. Je chéris aujourd'hui ces souvenirs avec vous toutes et tous : Marie-Anne, Jean-Yves, Denis, Serge, Nils, Guillaume, Roger, Mathieu, Peter, Sophie, Vincent, Christine, Mathilde B., Antonin, Alexandre, Michela, Louis, Quentin, Léon, Mathilde J.d.P., Emmanuel, Grégoire, Olivier, Yazid, Baptiste, Christophe, Steven...

Je n'oserais énumérer tous-tes les ami-es qui mériteraient absolument d'être cités-és ici. Cell-eux-ci m'ont apporté soutien, reconnaissance, équilibre et je

n'ai cessé de m'imprégner d'ell-eux pour être la personne que je suis et que je serai. Merci à vous, co-habitant-es, militant-es et partageur-euses d'instant de vie. J'espère être à la hauteur de l'amitié dont vous me témoignez chaque jour.

Je remercie ma famille qui m'a toujours suivi dans mes choix. Merci Maman pour la personne que tu es et toutes les valeurs que tu m'apprends encore aujourd'hui. Merci Mamie de m'avoir tant donné, tu es un exemple extraordinaire pour moi. Je pense à Papi qui est parti au début de mes études et dont le souvenir m'est des plus heureux et poignant à la fois. Merci à vous Rachel et Philippe de croire en moi et de m'avoir les premiers donné le goût de la nature. Merci à toi Eric d'avoir incarné peut-être sans le vouloir cette figure du scientifique qui a inspiré l'enfant que j'étais. Merci à vous Pierre-Louis et Clotilde dont la distance nous sépare mais dont je sais que rien n'est jamais perdu. Merci à vous Anne, Christian, Charlotte et Manon de m'avoir accueilli parmi vous en toute simplicité.

Enfin, merci Sophie de vivre ta vie en tandem avec la mienne.

Il semble *a priori* que je clos avec cette thèse une période de vie consacrée notamment à la recherche. Je suis fier de l'avoir réalisée avec vous toutes et c'est empli de votre confiance que je butinerai sur d'autres curiosités.



RÉSUMÉ

Les interactions entre les changements d'usage et de couverture des sols (Land Use and Cover Change, LUCC) et les changements globaux que sont le changement climatique et la perte globale de biodiversité sont reconnues et analysées depuis plusieurs décennies. Ainsi, de nombreux domaines de recherche (environnement, géographie, écologie du paysage, etc.) ou encore le secteur de l'aide à la décision de l'aménagement territorial emploient des modèles LUCC. Leur usage permet de mieux comprendre les moteurs de ces changements et de réaliser des projections en générant des cartes d'usage et de couverture des sols simulées (ces dernières ayant un rôle important dans l'appropriation de ces modèles par les décideurs).

Parmi les différentes approches possibles, cette thèse porte sur les modèles LUCC pattern-based statistiques et spatialement explicite. Ce type de modèle prend en entrée des cartes d'usage des sols et des variables explicatives (comme l'altitude, la distance aux routes, le type de sol, etc.), et retourne en sortie des cartes de probabilité de transition d'un usage du sol à un autre et peut également réaliser une allocation, c'est-à-dire la simulation (et projection future) d'une carte d'usage et de couverture du sol. Les sorties sont déterminées selon les éléments d'entrée (dits de calibration) et d'un scénario qui caractérise une situation (potentiellement inédite) que l'utilisateur souhaite évaluer. Cependant, il a été montré dans la littérature que si on présente un même cas d'étude à différents modèles de ce type, on obtient des résultats différents à des degrés divers. Ces écarts sont significatifs et peuvent questionner sur la fiabilité des modèles et la validité de l'ensemble des résultats obtenus jusqu'alors sur cette base, dans toutes les disciplines où ces outils sont employés.

Dans cette thèse, les modèles LUCC pattern-based sont formalisés selon une architecture modulaire où l'on distingue les étapes de calibration-estimation, d'allocation et de sélection des variables explicatives. Chacun de ces processus est étudié de manière indépendante afin de se concentrer uniquement sur la problématique correspondante. Ainsi, le module de calibration-estimation est mathématiquement formalisé et une méthode basée sur la méthode de Bayes et l'estimation de densité par noyau est proposée (Bayes-eKDE). Un processus d'évaluation basé sur l'analyse d'un problème parfaitement contrôlé est introduit et permet d'évaluer objectivement ce module. Un échantillon représentatif de méthodes implémentées dans les environnements de modélisation existants sont ainsi analysées, comparées et critiquées. Bayes-eKDE se révèle être très performant par rapport aux autres en terme de résultats et de temps de calcul. Le module d'allocation est lui aussi formalisé. La notion de biais d'allocation est

introduite mathématiquement et est à la base d'un processus d'évaluation du module. Deux méthodes d'allocation sans biais (uSAM et uPAM) sont proposées et leur supériorité par rapport aux méthodes implémentées dans une sélection représentative d'environnements de modélisation existants est illustrée dans une série de cas d'étude de complexité croissante. Enfin, une méthode de sélection des variables explicatives qui maximise leurs pertinences tout en limitant leurs redondances est introduite mathématiquement (Cramer-MRMR). Toutes ces méthodes sont implémentées dans le package Python open-source CLUMPY pour *Comprehensive Land Use [and cover] Model in PYthon*. Ce logiciel est un livrable de cette thèse d'ores et déjà utilisable par un utilisateur sur ses propres cas d'étude (avec notamment un GUI).

De nombreuses problématiques actuelles qui relèvent de l'aménagement territorial tendent à davantage s'appuyer sur la modélisation des LUCC (autonomie alimentaire, artificialisation des sols, économie locale, etc). Les avancées dans ce domaine introduites dans cette thèse sont donc essentielles pour garantir la validité des réponses qui y sont apportées.

Pour plus de détails, l'annexe C contient un résumé détaillé de la thèse en français en proposant notamment la traduction des chapitres d'introduction et de conclusion.

SUMMARY

The interactions between land use and cover change (LUCC) and global changes such as climate change and global biodiversity loss have been recognized and analyzed for several decades. Thus, many research fields (environment, geography, landscape ecology, etc.) or the decision support sector of land use planning employ LUCC models. Their use allows users to better understand the drivers of these changes and to make projections by generating simulated land use and cover maps (the latter having an important role in the appropriation of these models by decision makers).

Among the different possible approaches, this thesis focuses on spatially explicit, statistical pattern-based LUCC models. This type of model takes as input land use maps and explanatory variables (such as elevation, distance to roads, soil type, etc.), and returns as output probability maps of transition from one land use to another and can also perform an allocation, *i.e.* the simulation (and future projection) of a land use and land cover map. These outputs are determined from input elements (called calibration elements) and from a scenario that characterizes a (potentially unprecedented) situation that the user wishes to evaluate. However, it has been shown in the literature that if the same case study is analyzed with the help of different models of this type, different results are obtained, to varying degrees. These differences are significant and raise questions about the reliability of the models and the validity of all the results obtained so far in all scientific fields where such tools are used.

In this thesis, LUCC pattern-based modeling approach is formalized in a modular architecture where one distinguishes the calibration-estimation, allocation and explanatory variable selection steps. Each of these processes is studied independently in order to focus only on the corresponding problem. Thus, the calibration-estimation module is mathematically formalized and a method based on Bayes rule and kernel-based density estimation is proposed (Bayes-eKDE). An evaluation process based on a perfectly controlled problem setting is introduced and offers a basis to evaluate objectively this module. Some of the methods implemented in existing numerical modeling environments are analyzed, compared and criticized. The Bayes-eKDE procedure proves to be very efficient compared to the others in terms of results accuracy and computation time. The allocation module is also formalized. The notion of allocation bias is introduced and is the basis of an evaluation process of the module. Two unbiased allocation methods (uSAM and uPAM) are proposed and their superiority over other methods found in a representative subset of modeling software is illustrated in a series of case studies of increasing complexity. Finally, a method

for selecting explanatory variables that maximizes their relevance while minimizing their redundancy is introduced proposed (Cramer-MRMR). Its parameterization is illustrated through a case study. All these methods are implemented in an open-source Python package named CLUMPY for *Comprehensive Land Use [and cover] Model in PYthon*. This software is a deliverable of this thesis and acts as a demonstrator of the methods proposed in this work, while being already usable by an informed user on his own case study. A graphical user interface (GUI) allows to configure a simple case study.

Many current issues in territorial planning tend to rely more and more on LUCC modeling (food autonomy, land artificialisation, local economy, etc.). The advances in this field introduced in this thesis are therefore essential to guarantee the validity of the corresponding answers.

CONTENTS

Remerciements	7
Résumé	9
Summary	11
Contents	13
List of Figures	19
List of Tables	21
Acronyms	22
1 INTRODUCTION	25
1.1 LUCC and global changes interactions	27
1.1.1 LUCC: a global change nexus	28
1.1.2 Impact of global change on land use and cover	31
1.2 LUCC modeling	32
1.2.1 Applications of LUCC modeling	33
1.2.2 Model, simulation and software	34
1.2.3 Model classification	35
1.2.4 Validation and comparison of LUCC models	36
1.3 Pattern-based LUCC modeling frameworks	38
1.3.1 Land use and cover maps	41
1.3.2 Explanatory Variables	43
1.3.3 Spatial considerations	44
1.3.4 Calibration-Estimation	45
1.3.5 Allocation	50
1.3.6 LUCC modeling validation and the question of spatial accuracy	53
1.4 Thesis outline and main features	55
1.4.1 Thesis intended audience	55
1.4.2 Scientific strategy of the thesis	56
1.4.3 Thesis objectives	57
1.4.4 Personal contribution to articles	58
1.4.5 Thesis structure	59

2 AN ACCURATE AND POWERFUL CALIBRATION-ESTIMATION METHOD BASED ON KERNEL DENSITY ESTIMATION	61
2.1 Introduction	62
2.2 Methodological choices and research strategy relevance	64
2.3 Problem Formulation	67
2.3.1 Summary of Pattern-Based LUCC Model Architecture	68
2.3.2 Definitions and Notations	69
2.3.3 Transition Probability Estimation and Bayes' Rule	74
2.3.4 Probability Distribution Properties	77
2.4 Transition Probability Estimation Method	77
2.4.1 Kernel Density Estimation Short Summary	77
2.4.2 Comments	81
2.4.3 Whitening Transformation	83
2.4.4 Boundary Bias Correction	86
2.4.5 Bandwidth Selection	87
2.5 Implementation	89
2.5.1 Efficient Kernel Density Estimation	89
2.5.2 Density Estimator Architecture	93
2.5.3 Bayes Adjustment Process	94
2.6 Evaluation Method	95
2.6.1 Process	98
2.6.2 Comparison Criteria	99
2.7 Illustrative Controlled Problem	100
2.7.1 Problem Definition	100
2.7.2 Transition Probability Estimation	102
2.7.3 Results and Discussion	104
2.8 Conclusion	109
APPENDICES OF CHAPTER 2	113
2.A A more formal definition of probability densities	113
2.B Extending the eKDE calibration procedure to a mix of discrete and continuous explanatory variables	114
2.C Multinomial Sampling Test algorithm	115
2.D Models Parameters	116
2.D.1 Dinamica EGO	116
2.D.2 Idrisi LCM	117
2.D.3 CLUMondo	118
2.D.4 CLUMPY	118

3 ALLOCATION REVISITED: FORMAL FOUNDATIONS AND BIAS IDENTIFICATION	119
3.1 Introduction	120
3.2 Preliminary concepts and notations	122
3.2.1 Definitions and Notations	123
3.2.2 Reminder on relevant elements of probability theory and LUCC probability distributions	125
3.2.3 Ensemble averaging and noise reduction	127
3.2.4 Pixels vs patches: assumptions in brief	128
3.3 Allocation method and bias: definitions and constraints	130
3.3.1 Sampling design, allocation procedure and allocation method	130
3.3.2 Bias: definition	132
3.4 Unbiased Allocation Methods	135
3.4.1 Unbiased Simple Allocation Method (uSAM)	136
3.4.2 Unbiased Patch Allocation Method (uPAM)	136
3.5 A progressive strategy for the identification of allocation biases	139
3.5.1 Error identification strategy	140
3.5.2 Observed Bias	141
3.5.3 Setting test problems of increasing complexity	141
3.6 Conceptual errors and biases in LCM and Dinamica EGO	143
3.6.1 Mono-pixel patch and single transition: pruning bias	144
3.6.2 Mono-pixel patch and multiple transitions: multiple tran- sition bias	148
3.6.3 Multi-pixel patch and single transition: patch merging bias	150
3.6.4 Multi-pixel patch and multiple transitions	153
3.7 Discussion	153
3.8 Conclusion	155
APPENDICES OF CHAPTER 3	159
3.A Patch-related probability distributions	159
3.A.1 Patch probability distribution	160
3.A.2 Identity of pivot-cells and pixels complete distributions	161
3.B Multinomial Sampling Test algorithm	162
3.B.1 Test description	162
3.B.2 Statistical properties of the MuST algorithm	164
3.C A mono-pixel algorithm enforcing quantities of change	166
3.C.1 Sampling without replacement from exactly known pop- ulations	167
3.C.2 Mono-pixel allocation algorithm without replacement and with exact quantities of change enforced	167

3.C.3	A more direct algorithm enforcing quantities of change	169
3.C.4	An alternative algorithm avoiding binning of continuous explanatory variables	172
3.D	Multi-pixel patch allocation without pixel replacement	175
3.D.1	A multi-pixel patch MuST-like algorithm	175
3.D.2	A constrained multi-pixel patch algorithm enforcing quantities of change and avoiding patch merging	177
3.D.3	uPAM algorithm discussion	182
3.E	Dinamica EGO Allocation Method	182
3.E.1	Allocation Method Based on a Pruning Procedure	182
3.E.2	Recoded Pivot Cell Allocation Method	184
3.E.3	Patch parameters	186
3.F	Idrisi LCM Allocation Method	188
3.G	Pruning by probability distribution truncation	189
3.H	Post-allocation probability confidence interval	191
3.I	Patch construction method (patcher)	194
3.I.1	Neighborhood structures	195
3.I.2	Patch elongation	196
3.I.3	Patch design algorithm	198
3.I.4	Patch elongation validation	201
4	A MAXIMUM RELEVANCE / MINIMUM REDUNDANCY SELECTION PROCEDURE OF EXPLANATORY VARIABLES	203
4.1	Introduction	204
4.2	Definitions and notations	205
4.3	Explanatory variables use	207
4.4	A new MRMR selection of explanatory variables method	210
4.4.1	Generalities	210
4.4.2	Test of relevance	213
4.4.3	Test of redundancy	214
4.4.4	MRMR selection of explanatory variables from Cramer and Cramer-like	215
4.4.5	Remarks on the use of Cramer's test in LCM and Dinamica	216
4.5	Explanatory variable selection in practice	217
4.5.1	Illustrative case study short description	218
4.5.2	Results and discussion	219
4.6	Comments on the reliability and relevance...	233
4.7	Conclusion	234
	APPENDICES OF CHAPTER 4	237

4.A	Sufficient condition of relevance of an explanatory variable . . .	237
4.B	Inference statistics in a nutshell	238
4.B.1	Generalities	238
4.B.2	Levels of significance	240
4.C	MRMR selection method: critique of a popular method	241
4.C.1	Short description	241
4.C.2	Critique	242
4.C.3	Conclusion	245
4.D	Categorical approximation of a continuous probability distribution	246
4.D.1	From continuous to categorical distributions	246
4.D.2	Normal limit of samples statistics of a categorical probability distribution	248
4.D.3	Binning for the test of relevance	259
4.D.4	Binning for the test of redundancy	260
4.E	χ^2 and Cramer's tests	260
4.E.1	Goodness-of-fit tests	260
4.E.2	Tests of independence (or association)	262
5	CLUMPY, A COMPREHENSIVE LAND USE MODEL IN PYTHON	265
5.1	Motivation and significance	265
5.2	Software description	267
5.2.1	Software Architecture	268
5.2.2	Software Functionalities	269
5.3	Sample code snippets	270
5.4	Real case study	273
5.5	Intended Use and Expected Future Development	274
5.6	Conclusions	275
6	CONCLUSION AND OUTLOOK	277
6.1	Contribution of the thesis	277
6.2	Perspective for future work	279
6.2.1	LUCC modeling framework	279
6.2.2	CLUMPY	281
6.2.3	Applications	281
A	GISTAM PUBLICATION: AN OVERVIEW OF THE THESIS WORK	283
A.1	Introduction	284
A.2	LUCC model architecture	286
A.2.1	Calibration-estimation module	286

A.2.2	Allocation module	286
A.2.3	Existing LUCC model-building environments	287
A.3	CLUMPY: a new LUCC model-building environment	288
A.3.1	Calibration-estimation	288
A.3.2	Allocation	291
A.4	Evaluation	294
A.5	Results and discussion	297
A.5.1	Case study short description	297
A.5.2	Mean absolute error comparison	298
A.5.3	One-dimensional cut comparison	299
A.5.4	Execution time comparison	300
A.6	Conclusions	302
A.7	Appendices	303
A.7.1	Case Study Transition Probability Function	303
A.7.2	Models Parameters	303
B	SUPPLEMENTARY MATERIAL OF CHAPTER 2	305
B.1	Kernel Density Estimation Implementation	305
C	RÉSUMÉ DÉTAILLÉ EN FRANÇAIS	313
C.1	Introduction	313
C.1.1	Changement d'usage et de couverture des sols et changements globaux	315
C.1.2	Modéliser les changements d'usage des sols	317
C.1.3	Environnement de modélisation LUCC pattern-based	320
C.2	Une méthode précise de calibration-estimation basée sur l'estimation de densité par noyaux	327
C.3	L'allocation revisitée : fondations formelles et identification des biais	327
C.4	Sélection des variables explicatives pertinentes et non redondantes	328
C.5	CLUMPY, un environnement de modélisation de changement d'usage des sols	328
C.6	Conclusion	329
C.6.1	Contributions de la thèse	330
C.6.2	Perspectives de travail futur	331
	BIBLIOGRAPHY	336

LIST OF FIGURES

1.1	Global distribution of LUC cover with a focus on agriculture. Data source: UN Food and Agriculture Organization (FAO). Figure licensed under CC-BY by the authors Hannah Ritchie and Max Roser (2019).	26
1.2	Planetary boundary diagram.	28
1.3	Humanity's direct and indirect effects on the Earth system. . . .	29
1.4	Architecture of the statistical and spatially explicit pattern-based LUCC model CLUMPY. For other software, the estimation box should be omitted.	40
1.5	Detail of two LUC maps at two dates.	41
1.6	Schematic structure of the chapters of the thesis. The chapters order is dictated by their usefulness in the adopted research strategy and by their methodological interdependencies. The arrows indicate the order and dependencies of information when setting up an actual case study.	60
2.1	General architecture of our spatially explicit LUC modeling strategy.	70
2.2	Transition probability estimation architecture.	76
2.3	Illustration of the kernel density estimation method.	79
2.4	Illustration of the uni-variate kernel density estimation.	79
2.5	Illustration of the boundary bias correction in a unidimensional case.	88
2.6	Illustration of the fit step of our eKDE method.	92
2.7	Implementation architecture of the probability density estimation method.	94
2.8	Comparison method of estimation models for complete known artificial cases.	99
2.9	Representation of the probability distribution of the calibration-estimation illustrative case study.	102
2.10	Mean absolute deviation according of four calibration methods.	105
2.11	Probability density estimation along two one-dimensional cuts.	105
2.12	Computation times of different calibration methods.	106

2.13 Outcome of the multinomial sampling test in a simple idealized setting.	116
3.1 General architecture of our spatially explicit LUCC model. . . .	122
3.2 Unbiased Patches Allocation Method (uPAM) workflow.	139
3.3 Pruning allocation bias illustration – maps.	147
3.4 Pruning allocation bias illustration – distributions.	147
3.5 Multi-transition allocation bias illustration – maps.	150
3.6 Multi-transition allocation bias illustration – distributions. . . .	151
3.7 Patch size bias illustrated through resulting LUC maps.	152
3.8 Patch size bias illustrated through distributions.	153
3.9 Outcome of the multinomial sampling test in a simple idealized setting.	163
3.10 Measured elongation of patches produced by Dinamica EGO. .	188
3.11 Patch design procedure workflow.	198
3.12 Allocation example on the basis of the connectedness test. . . .	200
3.13 Measured elongation of patches produced by uPAM.	202
4.1 Test of relevance (direct binning, AGR $\rightarrow$ URB)	220
4.2 Test of relevance (pseudo-binning, AGR $\rightarrow$ URB)	221
4.3 Test of redundancy (pseudo-binning, AGR $\rightarrow$ URB)	222
4.4 Test of relevance (pseudo-binning, AGR $\rightarrow$ FOR)	226
4.5 Test of redundancy (pseudo-binning) AGR $\rightarrow$ FOR)	227
4.6 Second test of redundancy (pseudo-binning, AGR $\rightarrow$ FOR) . .	228
5.1 General architecture of CLUMPY.	268
A.1 General architecture of a spatially explicit statistical LUCC model.	285
A.2 Transition probability one-dimensional cut.	301
B.1 Call tree of all the functions involved in the eKDE implementation.	306
C.1 Distribution mondiale de l'utilisation et de la couverture des sols, avec un accent sur l'agriculture. Source des données : Organisation des Nations unies pour l'alimentation et l'agriculture (FAO). Figure sous licence CC-BY par les auteurs Hannah Ritchie et Max Roser (2019).	314
C.2 Diagramme des limites planétaire.	315
C.3 Les effets directs et indirects de l'humanité sur le système terrestre	316
C.4 Architecture générale d'un modèle LUCC pattern-based statistique et spatialement explicite.	322
C.5 Detail de deux cartes LUC à deux dates différentes	322

LIST OF TABLES

2.1	Summary of notations and definitions.	74
2.2	Dinamica EGO discretization parameters for the synthetic data of section 2.7.	117
2.3	MLP parameters for the synthetic data of the calibration-estimation case study.	118
3.1	Summary of the notations and definitions.	125
3.2	Summary of the notations and definitions used in Appendices 3.A, 3.B, 3.C and 3.D.	160
4.1	Summary of notations and definitions.	205
4.2	Numerical summary of the MRMR explanatory variable selec- tion method for the AGR $\rightarrow$ URB transition.	223
4.3	Numerical summary of the MRMR explanatory variable selec- tion method for the AGR $\rightarrow$ FOR transition.	229
5.1	CLUMPY's Code metadata	267
A.1	Comparison of models for the calibration-estimation and cali- bration/allocation comparison processes.	299
A.2	Computation time of the different modeling environments. . .	301

ACRONYMS

Acronym	Meaning	First occurrence
Bayes-eKDE	Calibration-estimation method based on Bayes rule and KDE	section 1.3.4.4
CLUMPY	Comprehensive Land Use Model in PYthon	section 1.3
Cramer-MRMR	Cramer based Maximum Relevance / Minimum Redundancy explanatory variables selection method	section 4.4.4
DE	Density Estimation	section 2.4
eKDE	efficient Kernel Density Estimation	section 2.5.1
EV	Explanatory Variable	section 1.3.2
GHG	Green House Gas	section 1.1.1
GoF	Goodness of Fit	section 4.4.4
GUI	Graphical User Interface	section 1.4.5
KDE	Kernel Density Estimation	section 1.3.4.4
LCM	Land Change Modeler (model / software)	section 1.3
LU	Land Use	beginning of chapter 1
LUC	Land Use and Cover	beginning of chapter 1
LUCC	Land Use and Cover Change	beginning of chapter 1
MLP	MultiLayer Perceptron	section 1.3.4.4
ToI	Test of Independence	section 4.4.4
uSAM	unbiased Simple Allocation Method	section 3.4.1
uPAM	unbiased Patch Allocation Method	section 3.4.2
WoE	Weights of Evidence	section 1.3.4.4
WT	Whithening Transformation	section 2.4.3.2

CHAPTER 1 | INTRODUCTION

LAND use change is as old as the emergence of civilization. At any time in the history of our species, it is possible to ask what the patterns of land use are, and the answer will be specific to this place and time. Be it for food production, habitat, transport, resource exploitation, land use is a constitutive factor of all human activities. More importantly, change from one type of use to another reflects all sorts of activity shifts, from the most mundane to the most momentous ones.

The trade-off of land covers underpins the ecosystem services that provide our livelihoods in food, energy, and fresh water (Newbold et al., 2015). Moreover, there are more than just these material considerations to take into account. The human species has benefited from intangible contributions from land cover in general, and by extension landscapes, that have shaped societies, cultures, and the general well-being of individuals (Hernández-Morcillo et al., 2013).

However, the other side of the coin is now known and documented: the cultural and biophysical contributions of ecosystems are now significantly declining globally and one of the drivers of this decline is land use and cover change (LUCC) (IPCC, 2019; Parmesan et al., 2022). Thus, the ability of the human species to transform its environment to benefit from material ecosystem services, which we continue to exploit, has now passed a threshold where their self-regulation and intangible contributions are threatened (IPBES, 2018).

Let us define the central topic of this thesis more precisely. A change in land use and land cover is the alteration of the environment of a place. It can be the direct or indirect result of human activity at a given location. Two major historical and world-wide trends are deforestation for agriculture, and the loss of natural or agricultural areas to urban development. Land cover is commonly defined as what is on the surface of the earth at a given location, be it vegetation (natural or planted), water, bare rock or human-made buildings. Land use refers to the operations carried out by human beings in a place characterized by its land cover — including changing this land cover if feasible and useful — with

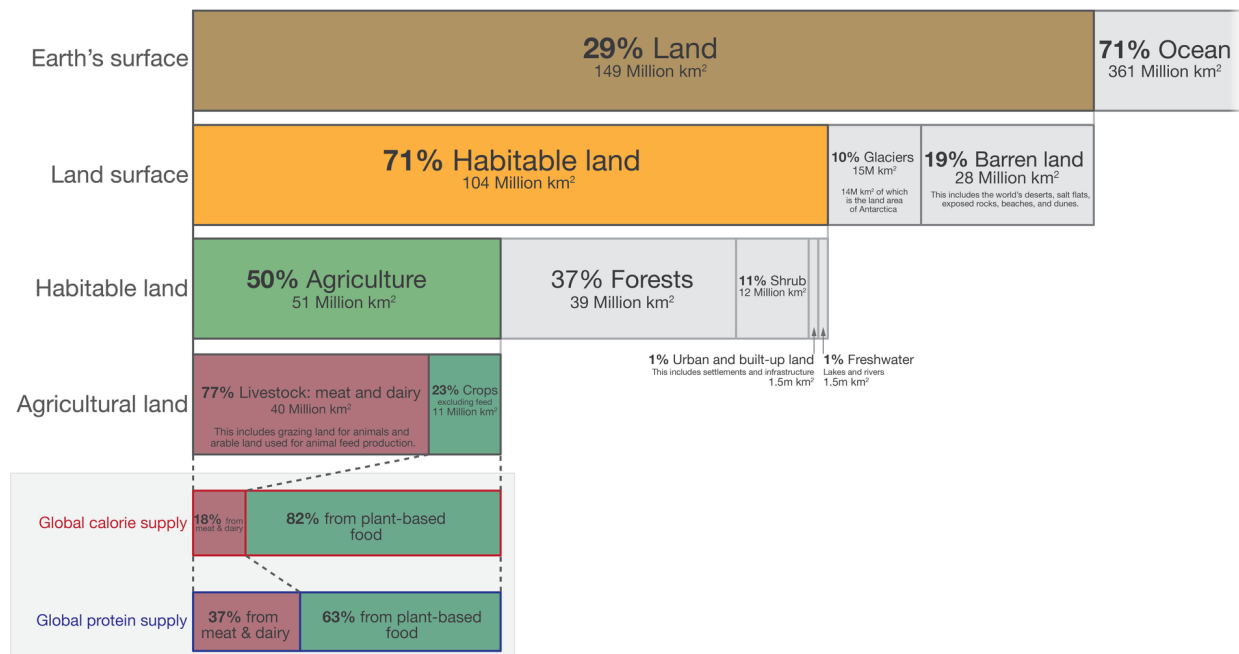


FIGURE 1.1: Global distribution of LUC cover with a focus on agriculture. Data source: UN Food and Agriculture Organization (FAO). Figure licensed under CC-BY by the authors Hannah Ritchie and Max Roser (2019).

the intention of deriving some kind of benefit from it. I mainly use the broad term land use and cover (LUC) in this thesis.

In the history of mankind, the world has been mainly forests, grasslands and mountains. This state of affairs changed drastically during the 20th century when more than 50% of all land started to be affected and modified by man (Ellis et al., 2010). Today, excluding glaciers and barren lands (deserts, salt flats, bare rocks, beaches and dunes), the surface dedicated to agriculture amounts to 50% of the land (Fig. 1.1). This leaves 37% for forest, 11% for bush, 1% for freshwater and 1% for urbanized land. This last quantity may seem surprisingly small and is generally overestimated by non-experts. This one represents 1.5 million km². Then, if we go down another notch in the detail within the agricultural land, we notice a great imbalance: more than three quarters of the surface is dedicated to meat and dairy production (including the production of food dedicated to animals) and the remaining quarter directly to human-consumed food products. We could refine again: within the agricultural surface dedicated to human food, what is the share of field crops, fruit trees, or market gardening?

This introduction is intended to set the context for this thesis by addressing the notion of LUCC at the level of the general framework used in modeling methods and their applications. Thus, I first recall the interaction between global change and LUCC (section 1.1). Then, I introduce LUCC modeling through applications and fundamentals of these models (section 1.2). I intro-

duce next more specifically the statistical and spatially explicit pattern-based LUCC models that are the focus of this thesis (section 1.3). Finally, the thesis objectives are stated; I detail my personal contribution in the articles reproduced here and I describe the structure of this thesis (section 1.4).

As a conclusion of this section, I recall below the definition of major concepts which are handled in this thesis.

LAND COVER (LC) is defined as what is on the surface of the earth at a given location, whether it is vegetation (natural or planted), water, bare rock or human-made buildings

LAND USE (LU) refers to the operations carried out by human beings in a place characterized by its land cover with the intention of deriving some kind of benefit from it.

LAND USE AND COVER (LUC) is a broad term which refers to both land use and land cover.

LAND USE AND COVER CHANGE (LUCC) refers to the transition of a given location from a LUC to another LUC.

1.1. LUCC AND GLOBAL CHANGES INTERACTIONS

The impact of mankind on its environment is so important that it is now endangering the conditions of human life as we know it (IPCC, 2022). Rockström et al. (2009) introduced the notion of planetary limits in which they describe nine thresholds that should not be exceeded in order to be able to live sustainably in a safe ecosystem by avoiding rapid and non-reversible modifications of the environment at the planetary scale (Fig. 1.2). This concept of planetary limits has been updated since (Wang-Erlandsson et al., 2022). Among these limits is the change in LUC. Its loading level is determined by the occupancy rate of intact forest, which is expected to be greater than 75% at the global scale (Steffen et al., 2015). This boundary is refined by distinguishing between boreal forests (85% boundary), temperate forests (50% boundary) and tropical forests (85% boundary). Steffen et al. (2015) estimates as of the date of publication that the global intact forest rate is 62%, below the global limit.

Moreover, there is no simple causal link between LUCC and global change (section 1.1.1) and global change also has an impact on LUC (section 1.1.2). Feedback loops can therefore occur.

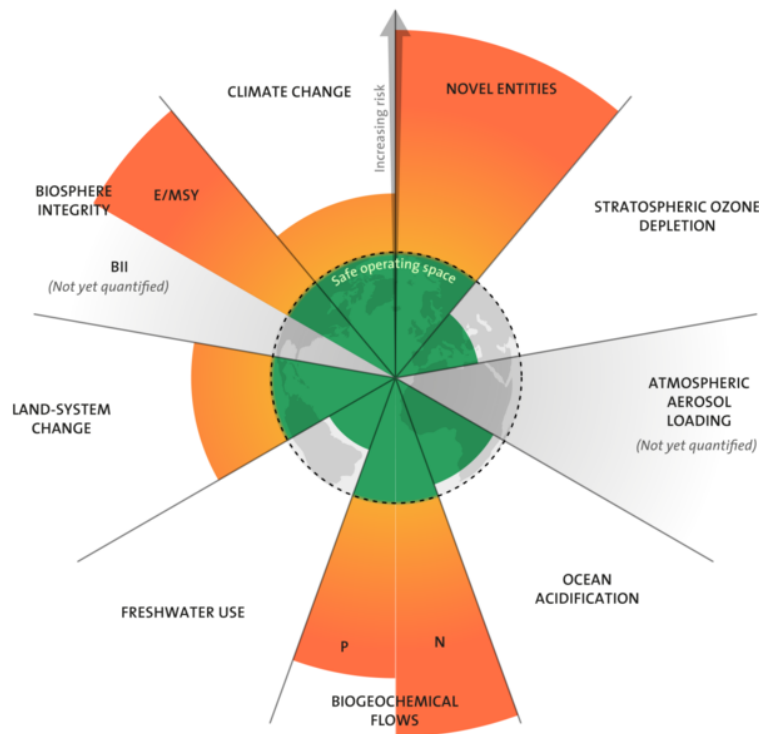


FIGURE 1.2: Planetary boundary diagram. The normalized threshold not to be exceeded is in dotted line. The wedges below this threshold are represented in green while those above it are in orange. Data updated in 2022. Figure source: Stockholm Resilience Centre¹ – CC-BY-4.0.

1.1.1. LUCC: A GLOBAL CHANGE NEXUS

The planet has always undergone slow changes (hundreds to billions of years). In the current understanding, though, the expression “global change” refers to recent phenomena of anthropogenic origin — *i.e.* caused by human activities — that significantly impacts the Earth systems and have been accelerating over the last half-century or so. Two of the major tracers of global change are global warming and biodiversity loss; climate change in particular is both an effect and an amplifying factor of other forms of global change. These two phenomena can rather directly be traced back to human activities, through alterations of biogeochemical cycles (carbon, water and nitrogen cycles), imbalances of biotics, or changes in LUC. Vitousek et al. (1997) proposes a schematic model of major forms of global change and their direct relations (Fig 1.3). Besides being an important form of global change by itself, LUCC has an important and definite impact on climate change and biodiversity loss (Vitousek et al., 1997; IPCC, 2019).

Let us notice that the diagram of Vitousek et al. (1997) (Fig. 1.3) omits any notion of feedback loop. However, it is certain that these global changes have consequences that positively or negatively accentuate their own causes, which

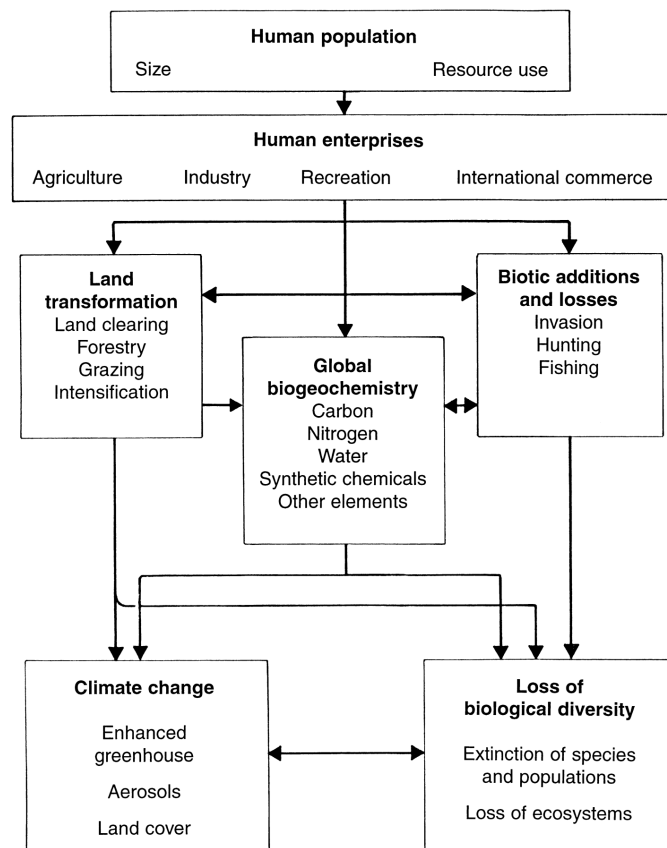


FIGURE 1.3: Humanity's direct and indirect effects on the Earth system. Conceptual model and figure provided by Vitousek et al. (1997).

is precisely a feedback. This is detailed in section 1.1.2 concerning LUCC.

We can quickly distinguish three important types of impact of LUCC among others, discussed in more detail below: action on the local climate, greenhouse gas (GHG) emissions and effects on local biodiversity.

1.1.1.1. LOCAL CLIMATE CHANGE DUE TO LUCC

LUC affects the local climate, contributes to the global climate, and is also involved in extreme weather events (Arneth et al., 2019). Let's take a few examples to illustrate the impacts. In the case of deforestation, the effects are multiple. Some of them go in the direction of global warming: i) emission of CO₂ stored in wood during its combustion; ii) reduction of turbulent heat fluxes due to the reduction of surface roughness; iii) reduction of evapotranspiration² at the origin of clouds, which reduces the filtration of solar radiation. While others, on the contrary, are factors of cooling of the local climate: iv) increase of the albedo and therefore decrease of the absorption of solar radiation; v) reduction of the incoming infrared radiation due to the decrease of the evapotranspiration (Jia et al., 2019). Note that the latitude where the observation is made these different effects should be taken into account when evaluating these phenomena (tropical, temperate or boreal).

It has been shown that in the case of large-scale deforestation such as currently occurring in the Amazon basin, rainfall is significantly negatively affected (Lawrence and Vandekar, 2015; Leite-Filho et al., 2021). Conversely, in the tropics, reforestation increases local rainfall (Perugini et al., 2017).

Local ambient temperatures are also greatly impacted by LUC. It has been shown that a forest is always cooler than any set of lower plant species due to the greater evapotranspiration of trees (Prevedello et al., 2019). Cities and their large expanses of concrete are another primary example; the planting of trees in cities is in fact a widely studied strategy to mitigate so-called urban heat islands (Taha, 1997). Indirectly, cities high temperatures (on average or during a heat wave) also encourages urban populations to use air-conditioning. However, these technologies are themselves high GHG emitters throughout their life cycle (Davis and Gertler, 2015). Note that there is a positive feedback here.

1.1.1.2. GHG AND LAND COVER CHANGE

Throughout their lifetime, plants capture the surrounding CO₂ through the process of photosynthesis. Forests are GHG storage areas and deforestation is therefore a significant source of CO₂ emissions that could have remained

²A biophysical process of transferring water from the surface to the atmosphere by evaporation from the soil and transpiration from plants. It is an essential component of the hydrological cycle.

trapped (even in the case where a part of the cut wood is used as material, a significant part is still burned). Conversely, reforestation increases the overall capture capacity of CO₂. Extreme heat waves can have a negative effect on this normal functioning of forest, by stifling photosynthesis, and leaving the normal respiration of forests (either plants or soil life) dominant (Duffy et al., 2021).

These considerations concern the direct link between CO₂ and LUCC. It is also possible to make a second order link when considering pollution due to human activity on the ground. Due to artificialization of soils, human activities generally emit greenhouse gases, whether through the construction of infrastructures or through the flows and products they generate (road transport, logistic chain, industrial production, commercial surfaces...).

1.1.1.3. BIODIVERSITY LOSS DUE TO LAND COVER CHANGE

The effects of a LUCC on biodiversity are not always to identify although the primary driver is deforestation whose major impact on biodiversity is now well documented. It is sometimes necessary to observe in detail the way in which change takes place. Indeed, LUC can be subtle. For example, the physical closure of plots does not in itself modify LUC but has a direct impact on some species and thus on ecosystems through loss of ecological continuity (Saunders et al., 1991).

1.1.2. IMPACT OF GLOBAL CHANGE ON LAND USE AND COVER

LUC results from a long history of adaptation and development of the human species, in particular with respect the locally dominant climate conditions (Stephens et al., 2019). However, today's global change is disproportionately faster than in the past. Indeed, while previous climate changes correspond to relatively long cycles (thousands to hundreds of thousand years), we are facing today global change over one or two hundred years only. This speed accentuates the effects on plant and animal species, and human societies.

Global change impacts agricultural production regionally (Wheeler and von Braun, 2013) and induces changes in the habitability of ecosystems and changes in LUC (Hulme et al., 1999; Pecl et al., 2017).

Overall, climate change mainly has a negative impact on current land use and cover (Jia et al., 2019).

For example, land considered until now as arid is undergoing desertification (Lickley and Solomon, 2018; Schlaepfer et al., 2017). This leads to forms of agricultural abandonment to the benefit of deserts (Engelbrecht et al., 2015), an accelerating trend worldwide.

Food security is also challenged as cereal yields (wheat, corn, rice...) are expected to decrease by 4-6% for each degree of temperature increase. Also, the increase in extreme events such as droughts, intense precipitation or mid-season frost periods severely affect crop yields (Vogel et al., 2019). These negative effects of climate change, when added together with other factors (geochemistry, biodiversity...), are not yet sufficient to counterbalance the local positive impacts of climate change on agriculture (Lobell et al., 2011; Porter et al., 2014; Beer, 2017), but will eventually be dominant on the longer run. Human activities related to agriculture are therefore subject to adaptations; this leads to LUC changes to meet these food security challenges.

I focus above on impacts directly related to human activities. Other LUCC phenomena can be cited such as the reduction of the glacial surface at the poles (Jeong et al., 2014) or the thawing of the permafrost which constitutes a critical climate tipping point due to the significant quantity of GHGs emitted during its melting (Schuur et al., 2015).

1.2. LUCC MODELING

I have detailed in the previous section the two-way interactions between global change and LUC. This observation is the backdrop of many studies and analyses of LUCC. Indeed, quite a few research fields may be concerned with this type of phenomenon. Moreover, in many cases, it is not only or even mostly a question of analyzing and determining the causes of past LUCC using LUC state maps (i.e., identifying the drivers of change: see, e.g., Meyfroidt et al. 2018), but of interrogating future changes through modeling, on the basis of various scenarios (Geist et al., 2006). In particular, simulations of future LUC maps implementing these scenarios constitute a very common tool to this effect. Quite often LUCC models do not rely on an exact understanding of the drivers (causes) of change, but on the identification of determinants of change, i.e., quantities that correlate to past observed changes. These quantities are often referred to as explanatory variables but one should keep in mind that they do not need to be actual causes of the observed changes.

This section focuses on the ways in which LUCCs are modeled. Thus, I list a number of applications that illustrate the diversity of use of this type of model (section 1.2.1). Then, a definition of the term model and its use is given (section 1.2.2). Also, the classification of the different types of LUCC models is recalled (section 1.2.3). Finally, the last section provides a brief review of the literature on the subject of model comparison and evaluation (section 1.2.4).

1.2.1. APPLICATIONS OF LUCC MODELING

We are not interested in LUCC simply for the phenomenon itself, but more importantly for the reasons that guide these changes and their implications. Thus, LUCC models are to be considered as a tool to answer various research questions that are for the vast majority at the crossroad of environmental science and geography.

In environmental sciences, it is very common to rely on scenarios to guide analyses and modeling (Verburg et al., 2016). A scenario corresponds to a future situation given by the user that he / she wishes to evaluate in light of his research question. Scenario building is a specialized task in its own right. The construction of a scenario is not a trivial task and its design and transposition into a LUCC model will obviously influence the results. In order to avoid these pitfalls, Escobar et al. (2018) proposes a scenario construction guide that tends to standardize their design. The important point for the category of LUCC models of interest in this thesis is that scenarios must not only describe future trends qualitatively but be specific enough to quantify global quantities of change between LUC states at a series of future dates. LUCC simulations will then provide spatially explicit maps of these global future changes

In geography, the use of maps is commonplace. It is a field that has particularly benefited from the democratization of computer tools through the development of Geographic Information System (GIS) software. Let us detail below three areas of application of LUCC models among others.

URBAN GEOGRAPHY

Urban geography studies the development of cities and built-up areas. Urban geographers are interested in how cities are built and how their inhabitants live in them. Urbanization can be considered as one of the driving forces behind many of today's LUCCs (Nuissl and Siedentop, 2021). Its various aspects that can be highlighted by their modeling (Hanzl, 2010; Sun et al., 2007). It is possible to determine the reasons why one parcel is artificialized rather than another and to consider future urbanization scenarios.

LANDSCAPE ECOLOGY

Ecosystems are severely affected by LUCC (see section 1.1.1). Ecology is therefore an important area of application for LUCC models. Indeed, if we are interested in species habitats, their spatial location and distribution on the territory becomes a major issue that can be analyzed through LUC maps. Ecologists can thus make projections according to various scenarios using simulations of LUCC provided by LUCC models. For example,

Newbold (2018) has analyzed the impact of LUCC on vertebrates under various scenarios. In addition, landscapes are now shaped by humans and their study can illustrate the societal or environmental phenomena they face. For example, the historical study of LUCCs in the French Alps highlights the exogenous influence of climate change on the local socio-ecological system (Enora et al., 2021).

FOOD SECURITY

Food security is intimately linked to LUC, which determines agricultural production (Smith et al., 2015; Hurlbert et al., 2019; Bezner Kerr et al., 2022). However, the induced significant impact on the environment that is difficult to reduce without changing diets (Poore and Nemecek, 2018). Only a minor fraction of agricultural production is dedicated to direct human consumption since 77% of it is dedicated to animal feed (Fig. 1.1). Finally, a small but growing proportion of land (some percents in Europe) is dedicated to the biofuel industry, whereas it has been shown that in the United States, LUCC induced by this type of crop results in an overall increase of GHG emissions (Searchinger et al., 2008).

Beyond these observations, LUCC modeling can be used in the search and characterization of alternative agricultural production models, e.g., more local, less meaty and less dependent on pesticides.

There are other applications of LUCC models. Stakeholders decision help is a major one. Faced with global issues, better land planning decisions can be reached on LUCC issues through policies dedicated to this subject (Reid et al., 2006). LUCC models can be very interesting decision support tools because of the easily grasped simulation outputs, in particular through maps of probable LUC future states for a given scenario (Koomen and Borsboom van Beurden, 2011). Interactive tools now offer the possibility to change model parameters and outputs on the fly and to produce quickly illustrative maps, which constitute very useful decision-help tools (Voinov et al., 2016). It is often a question of comparing various contrasting development scenarios to illustrate phenomena and thus help in the decision making process, on local or global scales (Ornetsmüller et al., 2016).

1.2.2. MODEL, SIMULATION AND SOFTWARE

Generally speaking, a model is a simplified representation of a system, designed to help to understand a number of features of the real world with respect to a particular set of questions the investigator has in mind. However, there is

some ambiguity in the term *model*, which, depending on the context, can refer either to the formal method used, the software itself or the specific modeled case studies. Furthermore, there is a source of imprecision between the terms *modeling* and *simulation* which are two distinct concepts. Camacho Olmedo et al. (2018) calls for a better use of the term *model* in the field of LUCC by distinguishing it precisely from simulation. In the present work, the following definitions are adopted. They are in line with common use in the LUCC modeling field, except when more precision is required for our purposes:

A MODELING FRAMEWORK is the type of formal modeling strategy implemented in a given software. It is based on a *theory* which refers to the mathematical background required to formulate the modeling framework³.

A MODELING ENVIRONMENT refers to any of the available software that relies on a specific modeling framework, allowing the user to set up an implementation (or instantiation or case study) of the modeling environment in an actual problem. Such an environment may offer the user a panel of analysis tools than goes substantially beyond a LUCC modeling software, but, in the context of this thesis, the terms modeling environment and LUCC software may be used interchangeably.

A SIMULATION relies on a modeling environment and a scenario applied to a specific case study to make a time-dependent, future LUC projection and thus observe the evolution of the system for the chosen scenario.

The term *model* as commonly used may refer either to the *modeling environment* (for the software), *modeling framework* (for the concepts used and implemented in the software), or to an actual implementation (instantiation) for a particular case study. The previous definitions are adopted to lift this ambiguity when needed.

1.2.3. MODEL CLASSIFICATION

Since the end of the 20th century and the easier accessibility to computer tools, many computer-aided LUCC models have been developed to answer a series of research questions (Sec. 1.2.1). Among these model, some have a global approach through a transition matrix which specify global (study area wide) quantities of change. These LUCC models quantify LUC trade-offs across an entire area, without explicit spatial allocations on maps. In this thesis, I focus on

³Other types of theories are useful or even necessary in LUCC modeling, most prominently from social and environmental sciences (Meyfroidt et al., 2018). However, these are not discussed in this thesis.

the contrary on spatially explicit models, which constitute their own sub-field of research and software innovation (Verburg et al., 2006).

We can observe today a great diversity in these models to the point that it is not easy to constitute a taxonomy, especially since some of them rely on radically different approaches or hybrid methods. The National Research Council (2014) proposed five categories of LUCC models (plus a category of hybrid models) according to the general paradigm that defines them:

1. Pattern-based model (PBM). It is a statistical approach (which may be similar to machine learning in other contexts) that is calibrated to past LUCCs in relation to explanatory variables.
2. Cellular Automata (CA). Here, the model is provided with suitability maps for each LUC, neighborhood parameters and a global transition volume to be reached. The pixels follow a cellular automaton process until they reach the expected transitioned surfaces.
3. Sector-based economic models. LUC is considered to be subject to the economic market of supply and demand. This type of model is particularly considered in inter-regional exchanges.
4. Spatially disaggregated economic approach. Here, we consider LUC as an economic equilibrium that is disturbed by causal phenomena.
5. Agent-based models (ABM). Each economic actor related to or influencing LUC is represented and their interactions with each other and with their environment are modeled. This type of model has grown rapidly in recent years, although they are built specifically for each research question and are not very generalizable (O'Sullivan et al., 2016).

The models of interest in this thesis are hybrid CA-PBM models, i.e., they are statistically calibrated but simulate by allocating the transitions in a manner that falls under the cellular automata category. This hybrid approach turns out to be quite common since it relies both on statistics to analyze changes and on cellular automata to build realistically allocated maps consistent with the statistical calibration (see section 1.3.3 for spatial considerations and section 1.3.5.1 for allocation methods).

1.2.4. VALIDATION AND COMPARISON OF LUCC MODELS

In retrospect, the multiplicity of models has been a growing source of interrogation in the literature, although the number of articles dedicated to this

question is relatively modest. Especially since the different paradigms that are at the origin of the National Research Council (2014) classification are not necessarily exclusive in their possible applications, *i.e.* the same research question can often be answered with two modeling frameworks from different categories. Below, the publications concerning the validation and comparison of models are briefly reviewed. This is not meant to be exhaustive but rather illustrative of the thoughts running through the whole community on this topic.

Starting from the approach of a user who questions the reliability of the results of a LUCC model, Pontius et al. (2008) started to compare 9 models by applying them to 13 distinct case studies. In this way, he was able to propose a first approach to comparing LUCC models. This approach is mainly based on the observation of allocation accuracy at the end of LUCC simulations.

Kelly Letcher et al. (2013) is particularly interested in the reasons why one model is chosen rather than another by proposing a real decision tree to select the most appropriate model for the research question asked (note that its classification is different from that of National Research Council 2014).

Mas et al. (2014) introduced the idea of comparing models on a fully controlled case study that allowed the authors, by its simplicity, to account for the differences between models. However, the origin of the differences was not clearly identified and the models were primarily compared with each other rather than objectively although this was actually possible due to the artificial nature of the study case (I detail this aspect in chapter 3). This article can nevertheless be considered as one of the motivations of this thesis work.

Paegelow et al. (2014) and Camacho Olmedo et al. (2015) have formalized similar methods for comparing LUCC models on real-world case studies. An accuracy score similar to that of Pontius et al. (2008) is used. A form of visual validation is also introduced by observing maps showing allocation successes or failures.

van Vliet et al. (2016) wrote a review of LUCC model calibration and validation methods by analyzing 98 publications covering 114 case studies submitted between 2010 and 2014. This review provides a factual account of the various methods used by the community. The following results are particularly noteworthy: i) the authors observed that the majority of model validations consists in counting local allocation accuracy as introduced by Pontius et al. (2008) and ii) 31% of the papers do not take any validation step, which is considerable.

Paegelow et al. (2018) devote a book chapter to LUCC model validation techniques. These methods can be used to compare models with each other. These authors make a review of the methods presented in the literature. Some models evaluate their results by comparing a validation (observed) LUC map

with an allocated (simulated) map or a probability of change map.

Finally, García-Álvarez et al. (2022) was particularly interested in the sources of uncertainties of 4 LUCC models by comparing the way they treat this subject. These authors conclude, among other things, that no model / software stands out in the way it communicates on uncertainties.

In conclusion of this enumeration, I highlight some elements: i) the comparison of models is not an easy thing to do due to the various natures of the models; ii) this practice is not common in the community, possibly due to the lack of a true validation method; iii) often, the method of comparison / validation is not satisfactory from a statistical point of view (details about spatial accuracy and LUCC models are given in section 1.3.6); iv) the sources of differences between spatially explicit pattern-based models are not clearly identified. These considerations constitute a large part of the motivation of this thesis, as detailed in the description of the adopted scientific strategy (section 1.4.2) and of the scientific objectives (section 1.4.3).

1.3. PATTERN-BASED LUCC MODELING FRAMEWORKS

The subject of this thesis is pattern-based LUCC modeling. This class of models has some particularly interesting characteristics. I list here a few attributes of these models: i) they produce transition probability maps from these with the help of more or less sophisticated statistical tools (see 1.3.4); ii) they produce future LUC maps; such maps can be used for decision help of environmental (e.g., ecological) studies (see section 1.2.1); iii) they are based on explanatory variables — determinants of change — identified by the modeler on the basis of their correlation with past LUC changes and which constitute an important input of LUCC modeling (see section 1.3.2); iv) they do not require specific computer hardware although existing LUCC software do not all run on all operating systems; v) although it is not a specificity, they allow the use of scenarios in producing future projections of LUC changes (see section 1.3.4.3).

I briefly introduce below some of the most common statistical and spatially explicit pattern-based LUCC modeling environments. A particular attention has been devoted to these in this thesis since I compare their results on artificial case studies in chapters 2 and 3.

CLUMONDO

This modeling environment is designed to study the interaction between

exogenous demand (biophysical or economic) and LUCCs and their products (van Vliet and Verburg, 2018). It benefits from a relatively long development and is the last member of the CLUE family (Verburg et al., 1999). CLUMondo's modeling approach is based on a single LUC map at a given time, without analyzing an evolution between two dates. In this way, the choice of CLUMondo can be judicious when only a single LUC map is available. However, it should be noted that CLUMondo does not estimate the probability of LUCC but rather the relevance of LUCs for each pixel. CLUMondo uses the resulting pixel suitability values as change probabilities in its allocation procedure.

CLUMPY

During this thesis, I developed a demonstration software where I was able to implement the different methods we designed with Pierre-Yves Longaretti. At the end of this work, this software named CLUMPY for Comprehensive Land Use [and cover] Model in PYthon reached a sufficient stage of maturity to consider its use by users-modelers. It is a python library and has a graphical interface to set up a case study. Chapter 5 is entirely dedicated to the description of CLUMPY.

DINAMICA EGO

It is a particularly versatile modeling environment that allows the user to design complex models with a user-friendly graphical interface and a graphical model builder (Rodrigues and Soares-Filho, 2018). This software benefits from a long term development expertise (Soares-Filho et al., 2002) and many functions and methods are implemented.

LCM

Lange Change Modeler is part of the former Idrisi and now TerrSet GIS modeling environment and is relatively old in development (Eastman et al., 1995). It was developed as a LUCC projection tool for spatial planning (Eastman and Toledano, 2018b).

Although the models in this category have common characteristics, there are many differences between them, in design, methods used (van Vliet et al., 2016) or results obtained on the same case study (Mas et al., 2014). It is nevertheless possible to outline an architecture that is common to a number of software. This architecture is composed of two main modules: a calibration-estimation module and an allocation module (Fig. 1.4).

The first one allows users to analyze past LUCC patterns to determine pixel transition probabilities. Estimation is often conflated with allocation, and as

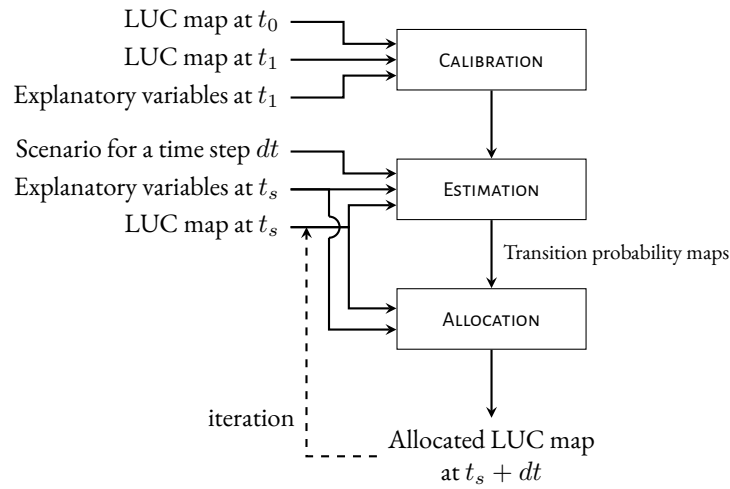


FIGURE 1.4: Architecture of the statistical and spatially explicit pattern-based LUCC model CLUMPY. For other software, the estimation box should be omitted.

such not distinguished from it; the distinction is needed for the new calibration procedure introduced in this thesis (chapter 3). Calibration and estimation use the same method, but on different data.

The second one is a process that models actual LUCCs and thus allows users to allocate actual LUC maps from the transition probabilities returned by the calibration module and from the global quantities of change provided by an external scenario. A simulation is made of consecutive consecutive allocation time steps, spanning the whole future period of interest.

Fig. 1.4 sketches the structure just described. It describes CLUMPY's overall architecture; for other software, estimation is usually not identified as such and would be absent from this sketch.

I detail below the element involved in the main modeling frameworks as well as number of features of statistical and spatially explicit pattern-based LUCC models. Thus, this section constitutes an introduction to the following chapters. LUC maps are first presented in a more formal way (section 1.3.1). Then, explanatory variables are introduced and their selection process is also somewhat discussed (section 1.3.2). I focus next on spatial considerations; this allows me to highlight some fundamental principles of this kind of modeling framework (section 1.3.3). Last but not least, the last two sections introduce the two main modules : the calibration-estimation module (section 1.3.4) and the allocation module (section 1.3.5).

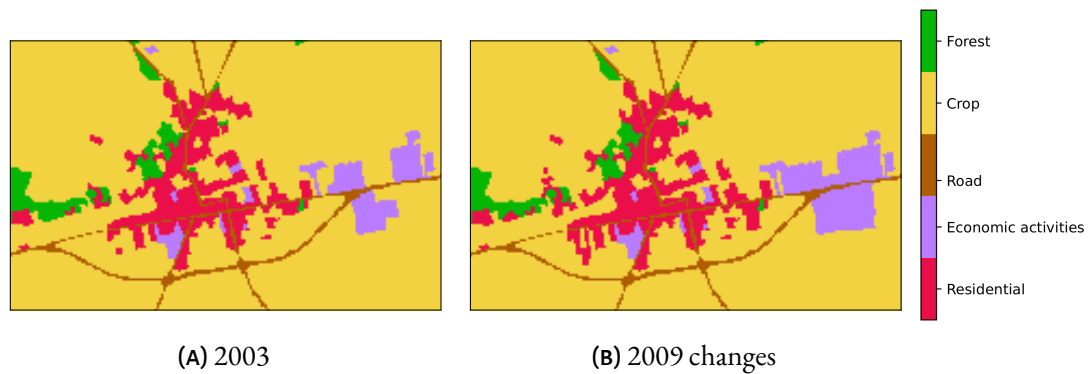


FIGURE 1.5: Detail of two LUC maps at two dates (2003 and 2009). The changes from crop areas to urban and economic activities areas are highlighted in the second map. These maps are used in the illustrative case study in section 4.5.1. It is zoomed on Marcillolles 38260, Rhône-Alpes, France.

1.3.1. LAND USE AND COVER MAPS

Statistical and spatially explicit pattern-based LUCC models require past LUC maps and explanatory variable maps (introduced in section 1.3.2). LUC maps are traditionally in raster (i.e., pixel) format for LUCC modeling purposes. It is possible in principle to design a model based on vector maps (i.e., based on polygons), however, such an approach seems more difficult in terms of mathematical formalization and its advantage is not obvious for LUCC modeling. Pixels have indeed a number of interesting features: i) it is a very classical representation format in computer science; ii) the shape of pixels (square) is not realistic but sufficient for analyses and applications, as long as pixel sizes are small compared to scales of interest (a condition that is not always satisfied).

Generally, in existing modeling environments, the statistical calibration procedure is carried out based on two LUC maps of the same study area at different past dates⁴. As an illustration, a detail of two LUC maps is reproduced in figure 1.5.

1.3.1.1. SPATIAL AND THEMATIC RESOLUTION OF LUC MAPS

It is possible to qualify LUCs with a variable level of detail. This does not have to be as precise as possible, but rather adapted to the question at hand.

Thus, the spatial resolution of the LUC maps will not be the same if we are interested in deforestation at the scale of the entire planet or in a small study area. This choice of a correct scale is necessary to avoid over-sampled or under-

⁴Note that the business as usual transition matrix (see section 1.3.4.3) comes from the differences between these two LUC maps. The time difference has to be taken into account. It is possible to choose a different time step for LUC projections through matrix operations in order to change the transition matrix time step to fit users purposes.

sampled modeling and to be able to complete it from the point of view of the numerical calculation undertaken.

Also, some LUC categories can be more detailed than others, this is called thematic resolution. Indeed, if we are interested in the expansion of urban areas on the outskirts of a metropolis, it may be sufficient to qualify the non-artificialized land by only a few categories: grassland, agricultural, forest, bare rock. On the other hand, if we analyze the agricultural system of a given area, detailing the type of culture on each parcel can be quite relevant. García-Álvarez et al. (2019b) has shown that the result returned by the model is very dependent on this modeling choice. In practice, the thematic resolution is often the result of a compromise between the modeler's objective and the nature, quality and spatial resolution of the available data.

1.3.1.2. CARTOGRAPHY OF LUC MAPS

A map is a representation of reality that may be subject to interpretation or inaccuracy as for any representation. Thus, any LUC map is a source of uncertainty for LUCC models (Prestele et al., 2016; García-Álvarez et al., 2019a). The creation of LUC maps is an upstream task for LUCC modeling. Any uncertainty present at the level of map elaboration will affect LUC modeling results. The production of maps for navigation, transport and economic purposes has progressed significantly with the advent of computers and remote sensing data, and some satellites have instrumentation dedicated to environmental issues while some projects are devoted to producing LUC maps from satellite data.

Most methods of constructing LUC maps are based on remote sensing data from satellite and/or ortho-images taken from plane (Anderson et al., 1976). This can be done by eye, by an artificial intelligence algorithm, or by a hybrid method where a human arbitrates the uncertainties returned by the machine and verifies a representative sample of the whole map. The machine learning methods used are those of the classification type: based on a training dataset that has been classified by a human, each pixel is assigned a LUC state. The color of pixels on satellite / plane images is obviously taken into account (some measurements are not limited to visible light) but other explanatory variables can also help in the decision such as altitude, distance to the road, slope, etc⁵. The related literature is quite extensive (Yang and Newsam, 2010; Helber et al., 2019) and the associated uncertainties are specifically studied (Rwanga and Ndambuki, 2017). I make the choice in this thesis to consider LUC maps as acquired in order to focus on the uncertainties of LUCC models only. The issue of errors

⁵These are also very common explanatory variables in LUCC models.

and uncertainties in LUC maps is a very important topic for LUCC modeling, but this issue is not discussed in this thesis.

Despite the difficulties to be faced in order to produce LUC maps, it is possible today to access such data relatively easily with the rise of open data policies, often implemented by government institutions. For example, the European Copernicus project and its open data set *Corine Land Cover* which provides LUC maps of the entire European continent with a minimum resolution of 100 meters at the following dates: 1990, 2000, 2006, 2012 and 2018 (Girard et al., 2018).

1.3.2. EXPLANATORY VARIABLES

An explanatory variable (EV) is a spatial determinant associated to each pixel – it can therefore be represented in the form of a map – that represents a quantity which is suspected to be correlated to past (and future) observed LUCCs. Some explanatory variables are not defined or available at the pixel level (e.g., socio-economic data), but may still be used for the same purpose.

1.3.2.1. EXPLANATORY VARIABLE TYPES

An EV can be continuous or discrete. A continuous EV is characterized by the fact that the EV can take any real value on its definition range. This continuity is often perceptible in spatial space where close pixels will have the same or related LUC types, a property that may be quantified through spatial autocorrelation functions. Typical examples of spatially continuous EVs are elevation, slope or distance to a particular landscape feature (such as the nearest road). There are also continuous EVs that will not necessarily have spatial continuity, such as land and real estate price. A discrete EV provides categorized information. For example, it is common to use soil type maps, or land tenure maps. Discrete variables are mixed with continuous ones (if any) and it is up to the calibration-estimation method to deal with both kinds of EV.

An EV can also be static or dynamic. A static EV is independent of LUCCs and does not have to be updated (for example, elevation and slope are static EV). Conversely, a dynamic EV is dependent on LUCC and have to be updated at each simulation time step (for example, the distance to the nearest urban center limit is a dynamic EV due to urban sprawl).

1.3.2.2. INDEPENDENCE AND SELECTION OF EXPLANATORY VARIABLES

Because discrete and continuous EVs are deterministic functions of position, the maximum number of statistically independent variables one can find in any

LUCC problem is two, as maps are in two spatial dimensions. In practice, finding two completely independent EVs is all but easy. Still, as a consequence, and although exploring such an issue is a complex endeavor, it may turn out that for the type of EVs and map patterns found in LUCC modeling, a rather limited number of variables may be sufficient to obtain reliable transition probability maps. Whether this conjecture is true or not and to which extent, it is certainly true that the number of explanatory variables eventually selected in actual case study is never large, e.g., less than five or ten.

This also suggests that having an efficient EV selection method that could at the same time identify the most relevant and least redundant (least statistically correlated) variables would definitely help to select a short-list from a wide pre-selection of EVs. In practice, the modeler chooses a set of EVs as being likely to be correlated with the LUCCs he studies. However, the choice of this set of EVs is not necessarily obvious and the list of possibly relevant EVs is open ended, so that one can never be sure that all relevant variables have been identified. On the one hand, an informed modeler may seek to avoid redundancy between the EVs, *i.e.* to ensure that they are relatively statistically independent of each other. On the other hand, even with an expert eye, it is sometimes difficult to know which variables are the most relevant to explain a given LUCC, especially when several of these changes are analyzed at the same time and when they are of various nature (for example, if urban expansion and agricultural abandonment are studied at the same time). Discarding a significant EV can be detrimental to the model. Keeping too many of them also degrades the results because of possible over-interpretation of the data. This pitfall concerning the choice of relevant and non-redundant EVs is the subject of a sub-module of the calibration called selection of explanatory variables. Chapter 4 is entirely dedicated to this subject.

1.3.3. SPATIAL CONSIDERATIONS

Statistical and spatially explicit pattern-based LUCC modeling frameworks aim to model transitions of sets of adjacent pixels (patches) from one LUC state to another. Indeed, these models are based on calibration LUC maps that record transitions observed in the past. Real world LUC changes are represented by a number of pixels on a LUC map. Therefore, one would expect the models to take this into account by analyzing the spatial LUCC patterns of change.

In fact, most models perform calibration on pixels individually without taking into account pixel LUC and LUC change spatial correlations, and analyze them statistically in EV space and not in physical space (or raster space). In light

of the content of the previous paragraph, this may seem paradoxical at first sight. The paradox is resolved with the help of the spatial considerations explained right below. The main idea is that spatial patterns of change are recovered from pixel distributions with respect to EVs through the spatial dependence of EVs and through the procedure of patch formation.

1.3.3.1. PIXELS VERSUS PATCHES

Modeling frameworks of this kind generally focus on pixel statistical properties. But, by design, the pixel probability distributions calibrated in these software carry no information on patch transition. How then can this calibration be at all relevant to actual case studies?

Let us explain this with the example of Dinamica EGO, whose modeling framework is the most explicit about this point, but the explanation is general, with appropriate changes for other software; the logic described here is also followed in the software developed in the course of this thesis. In Dinamica EGO, patches are produced from two basic “bricks”: a seed pixel — or pivot-cell — (*i.e.*, a pixel around which a patch undergoing a change of LUC state is formed), and patch characteristics. To a very high level of approximation, seed pixels are statistically independent of each other⁶, a property which is obviously not true for pixels (there is a high chance that neighboring pixels are in the same LUC state or change state to the same LUC state, for example). Similarly, patch shape characteristics (e.g., patch surface, form parameters, etc) are often assumed to be highly statistically independent of seed pixel ones. And finally, individual seed pixel probability distributions are the same as all individual pixels probability distributions for any given LUC state change⁷. In the end, the fact the seed pixels are independent of each other while spatial correlations are recovered by the patch formation process is ultimately what allows us to focus on individual pixel probability distributions in the calibration process. These somewhat subtle issues are discussed in more detail in Appendix 3.A and in the second half of Appendix 3.D.2.5.

1.3.4. CALIBRATION-ESTIMATION

Calibration-estimation is the first of the modules of a LUCC model that is analyzed in this thesis. In an actual case study, explanatory variables must already be selected before calibration may be performed. Explanatory variable selection is addressed in Chapter 4. This order of analysis will be justified at the

⁶This point is examined in more detail in Appendix 3.A of chapter 3.

⁷This point is also examined in more detail in Appendix 3.A.

end of this introductory material.

As pointed out earlier, one needs to determine the transition probabilities of each pixel. To carry out this process, it is necessary to perform what is called calibration (we also speak of “calibrating the model”). Calibration is a process collecting relevant calibration EV values and their associated pixels. Various preliminary transformations of these EVs are also performed. Estimation is the process providing maps of transition probability. It is possible to distinguish LUCC models by the calibration-estimation method they use.

In the software implementation I have designed, this is divided into two sub-modules (calibration and estimation, see Fig. 1.4) as the estimation process can be called without having to re-calibrate the model. Let me recall here that estimation differs from calibration by the pixels to which it is applied during allocation and their associated EV values, which may not be present in the calibration data; thus estimation requires some kind of an interpolation to be performed from calibration data. I now explain in more detail the structure and functioning of this calibration-estimation module.

1.3.4.1. TWO LUC MAPS AND A SET OF EXPLANATORY VARIABLES

Two LUC maps of the same study area at the same spatial resolution but at two different times are required for calibration. The time difference between the maps is typically a few years⁸. The maps are in raster format and the thematic resolution of LUCs is adapted to the research question of interest (see section 1.3.1).

It is also necessary to provide to the calibration-estimation module a set of EVs selected in a relevant way (see section 1.3.2.2). Depending on the calibration-estimation method chosen (section 1.3.4.4), it may be interesting, or even necessary, to pre-process the EVs in order to have them in an adapted format. Such an approach is implemented in chapter 2.

1.3.4.2. REGIONALIZED CALIBRATION

In some cases, it is interesting to divide the study area into several smaller areas, traditionally called regions in the context of LUCC modeling. Indeed, for the same study case, it may be relevant to calibrate the model in a differentiated way in distinct regions

This approach is common when particular areas are subject to particular regulations. For example, if the territory under study includes a natural area protected by law, this local specificity would not be taken into account if the whole

⁸This is time step is not a constraint for the simulation time step since it will be possible to re-scale adequately the transition matrix (see section 1.3.4.3)

study area were calibrated indiscriminately. However, if two distinct calibrations are carried out, in the protected natural area and outside, the modeling of the dynamics will be more in line with reality.

1.3.4.3. SCENARIO

Scenarios are not needed for calibration but they are required in the estimation process in particular to quantify the probabilities of transition between LUC states.

Scenario modeling is a common feature of LUCC model applications (see section 1.2.1). The modeler constructs one or more contrasting scenarios to try to answer his/her research question.

The first scenario is often an extrapolation one, and may even only be based on changes observed on calibration maps, simply projecting the same patterns of change into the future. It is traditionally called “business as usual” and can be considered as a reference scenario. It is usually easier to construct contrasting scenarios by modifying to various extents the reference one, but this is not a necessity.

Within statistical and spatially explicit pattern-based LUCC modeling frameworks, there are generally four ways in which a “business as usual” scenario can be modified to construct another:

1. Acting on global fractional changes of LUC. This is the most common modification. In this case, we act on the global surface of land that transits from one state of use to another in the study re. This modification is therefore directly involved in what are called LUCC matrices, or transition matrices, which provide the percentages of surface area transited for each distinct LUCC. Note that in a regionalized study case (section 1.3.4.2), this matrix is also regionalized and it is therefore necessary to modify each of its regional declinations to build the scenario.
2. Acting on the regions. In a regionalized case study (section 1.3.4.2), it is possible to modify the regions. Thus, the calibration may be performed on a region and the estimation applied on another given by the scenario. In this way, we can model the enlargement of a protected natural area for example.
3. Acting on EVs. Some scenarios imply the alteration of certain “static” EVs. For example, one can model the impact of global warming by altering an EV indicating the local mean temperature.
4. Acting on the calibration itself. Depending on the calibration-estimation

method, it may be possible to modify the data stored at the end of the calibration and which will be used for the estimation. This approach should only be considered if we want to model a phenomenon that has not been observed in the past with the calibration maps. For example, if one wishes to model a new public policy that aims to encourage the installation of market gardening near residential areas, one could modify the calibrated probabilities and give this transition an adequate probability distribution as a function of the distance to urban centers; such an EV may not even have been part of the initial calibration.

1.3.4.4. CALIBRATION-ESTIMATION METHOD

The main sub-module of the calibration-estimation module performs the calculation of transition probabilities. It is made up of a calibration-estimation method and this method is very often characteristic of the whole model. As a reminder, there are other sub-modules such as EV selection (see section 1.3.2.2) or EV preprocessing. Some software have implemented several calibration-estimation methods and leave it up to the user to choose. This allows to swap sub-modules according to the specific needs of users.

van Vliet et al. (2016) has analyzed the different calibration-estimation methods used by the LUCC model user community. It was found that about one third of the methods used are manual, based on the modeler's expertise, that about half are based on statistical analysis of calibration maps (including hybrid statistical and manual methods), and that a significant number of application articles – 18% – do not specify the method used.

This is where the “statistical” character of the LUCC models I study in this thesis comes in. Indeed, the calibration-estimation method aims at inferring probabilities of LUCC according to changes observed on calibration maps and for a selection of EVs. This approach can be assimilated to machine learning, without excluding very simple statistical methods. Some of the most common calibration-estimation methods are listed below.

CA_MARKOV

It is a hybrid method that combines a cellular automaton and a Markov chain. In this approach, the user's expertise is strongly solicited. Thus, this model is useful when LUC data are lacking or are not adapted to the studied phenomenon (Eastman and Toledano, 2018a).

BAYES EFFICIENT KERNEL DENSITY ESTIMATION (BAYES-EKDE)

Bayes formula allows us to place ourselves in EV space thus to momentarily ignore the spatial characteristics of pixels. Thus, we reduce the problem

to a generic density estimation problem, which has been extensively studied in the machine learning community. This is particularly adapted for continuous variables, which are the most difficult ones to calibrate, and is easily extended to a mix of discrete and continuous variables (a common situation in actual LUCC problems).

In this thesis, we have developed a specific Kernel Density Estimation — or KDE — method. KDE is a non-parametric method that estimates probability densities at any point in EV space. This approach can be considered as a generalization of histograms, except that it does not require discretization. It turns out that avoiding discretization leads to more precise calibrations and estimations for continuous variables.

This method is implemented in our CLUMPY model and is the subject of chapter 2. Generic KDE methods may require a very large number of numerical operations; our method is based on a controlled approximation designed to reduce this number of numerical operations, which may be computationally costly when the number of EV becomes larger than a few. Efficient in the method name does not *a priori* imply that the method is more precise than other methods, but that the modification implemented resulted in a significant improvement of computational efficiency with respect to standard Kernel density estimation (KDE) without noticeable loss of precision.

LOGISTIC REGRESSION BASED ON ONE LUC MAP

The CLUMondo modeling framework relies on a single LUC map and a set of EVs to determine the LUC suitability of each pixel through a logistic regression approach (van Vliet and Verburg, 2018). It is thus a parametric method, and tends to base LUC change on existing LUC states. This results from a trade-off between accuracy and potential lack of data; indeed inferring patterns of change directly from the differences between two maps is by design more precise. Other variables provided by the expert user complete this approach (resistance to conversion, neighborhood effects, trade-off between LUCs) in order to mitigate the LUC change/LUC state assimilation.

MULTILAYER PERCEPTRON (MLP)

Estimating the probability of LUCC for pixels can be related to another machine learning problem. Indeed, it can be compared to a classification problem where the aim is to determine pixels class probability.

To do this, it is possible to use a neural network and the most common one is the MultiLayer Perceptron (Taud and Mas, 2018). This one is notably

implemented in LCM (Eastman and Toledano, 2018b).

Such an approach can return interesting results when appropriately set up, but the user loses however the understanding of how LUCC drivers (i.e., EV) produce the obtained results. This problem is common to many artificial intelligence (AI) methods, and is a major open question in this field.

WEIGHTS OF EVIDENCE

This method is a form of what is called “Naive Bayes classifier” in the machine learning literature. The basic idea is to consider all EVs as statistically independent in order to simplify the calibration of the transition probabilities. In particular, this clearly reduces the noise level of the calibration, but at the cost of an error associated with the unavoidable level of statistical correlations between variables. It must be noted in this respect that even small levels of correlations can produce disproportionately large errors.

In this method, each EV is discretized and the number of observations in each bin gives the evidence weight (which we do not define more precisely here). Then, Bayes formula is used to compute the transition probability of a pixel from the product of the evidence weights.

This method is implemented in Dinamica EGO along with a few others (Soares-Filho et al., 2002), and a very popular one due to its simplicity and ease of interpretation. This approach allows us to easily compute transition probabilities for any pixel, even for EV combinations not found in the calibration data.

1.3.5. ALLOCATION

The calibration-estimation module produces probability maps of LUCC for each pixel. The allocation module allows users to simulate actual LUC maps on the basis of these probability maps. At the pixel level, the term “allocating a pixel” is sometimes used.

Thus, the allocated map contains in some ways less information than the probability maps, since it constitutes a simulation, a drawing, among all the other possible ones that are just as suitable. Nevertheless, some applications require a particular allocation (in landscape ecology for example, see section 1.2.1). Moreover, an allocated map is observed and analyzed visually by the modeler or the recipient of the study (e.g. a policy maker) in an easier way than transition probability maps.

This point implicitly defines a simulation requirement: in addition to being based on LUCC probabilities, allocation maps must be realistic. The realistic character of an allocation refers to landscape structure. A part of this realism requirement bears on the contiguous pixels that have undergone the same LUC state change and that we call a patch. For example, a change in agricultural use of a plot will very often occur on the whole plot, especially in Western Europe where such plots are usually small, whereas the construction of houses is a more limited phenomenon that results in the transition of a smaller number of agglomerated pixels. Evaluating the realism of an allocated map is partially subjective process, except for landscape features that can be quantified through appropriate landscape metrics.

In a more advanced state of the field of LUCC modeling, patch statistical properties might also be quantified through appropriate probability distributions, but there is no consensus and, in fact, no mention, of what the relevant patch parameters might be, beyond the spatial location of their center and their area; occasionally (e.g., in *Dinamica EGO*) deviations from circularity are produced algorithmically and with some randomness. The allocation procedure discussed in this thesis does follow the same requirements — patch position, area, elongation and some randomness.

I will conclude this section with a few unrelated comments. Transition probability maps may constitute a sufficient input to the allocation module for some modeling frameworks. However, other approaches also require the specification of a transition matrix (see section 1.3.4.3) and EV maps. Some of the methods of patch creation also require specific parameters that can be determined from calibration maps or from developer choices⁹.

1.3.5.1. ALLOCATION METHODS

As for the calibration-estimation module, a number of allocation methods has been devised, in order to simulate future LUC maps. These allocation methods must meet two requirements: i) be statistically unbiased, i.e., to produce post-allocation probability distributions that are statistically equivalent to the pre-allocation ones; and ii) be “realistic” (see the discussion of the previous section on this second point). The way these constraints are implemented (in particular the first one) are detailed in the chapter 3.

The first requirement may seem superfluous. However, all LUCC modeling frameworks I studied do violate this point to some extent (see chapter 3). There are quite a few reasons behind this failure, and they vary from one soft-

⁹These inputs to the allocation module, that can be considered secondary, are not shown in Figure 1.4 for the sake of clarity

ware to another. For example, an algorithm may fail to implement an unbiased allocation when several transitions have a non-negligible probability of occurring. And indeed, there is no consensus yet between existing software on how chose between two possible LUC state transitions for a given pixel.

In addition, a patch (set of contiguous pixels) and not a single pixel needs to be allocated. An allocation algorithm must therefore first address the relation between the pixel probability distributions of LUC state transition and the production of a patch of transited pixels. One way to do this is to draw a single “seed” pixel from such a distribution and then produce a patch from contiguous pixels around the seed one and start the process again. Consistency requires to recalculate pixel transition probabilities after each patch is formed. We detail the reason why such an approach is necessary and show it is unbiased in section 3.4 and related appendices.

The choice of a patch construction procedure (“patcher”) is also specific to a given allocation method. Below we briefly describe the possible approaches to patch construction. Only a few seem to have been actually used so far.

CELLULAR AUTOMATA PATCHER

This algorithm constructs pixel by adjacent pixel a patch of given characteristics (area, elongation, etc.) whose probability distribution is either obtained from calibration or given by the user. We propose such a patcher in section 3.I.

SHAPE MEMORY PATCHER

Another approach, which is intended to be very realistic, consists in reproducing the shape of the patches observed during the calibration. A sort of patch shape library could be created. Then, the patcher tries to allocate according to these shapes by applying a similarity transformation (rotation, reflection, scaling) and / or by cutting out a piece if necessary.

Despite the fact that the calibration only provides a limited number of patch shapes, this method allows the user to reproduce the realism constraint as well as possible. However, one can only model previously observed patch shapes in this way. To my knowledge, no LUCC model offers such a patcher at the moment.

TESSELATION PATCHER

It is also possible to perform a tessellation – *i.e.* a partition of the surface of each LUC of the whole map as input to the allocation. Several algorithms (central Voronoi tessellation for example) allow to obtain a tessellation which will be able, according to the needs, to reproduce the behavior of a particular transition with the help of some parameters (in particular

the average surface). Allocating a patch consists in transiting all the pixels of a cell from the tessellation. However an arbitrary tessellation has little chance to fit the constraints of actual patch shapes and sizes, especially when several LUC state transition must be accommodated.

In the real world, a particular form of tessellation is already available: the cadastre. One could thus imagine a patcher that takes precisely the cadastre to allocate entire parcels. Such an approach would be particularly appropriate when modeling crop change on agricultural plots for example. A hybrid method could also slice cadastral parcels into multiple cells to model residential urbanization of an agricultural parcel.

1.3.6. LUCC MODELING VALIDATION AND THE QUESTION OF SPATIAL ACCURACY

In the LUCC modeling community, evaluating models is not a widespread practice (van Vliet et al., 2016), and the most common evaluation method is to test the spatial accuracy of allocation against observed historical changes (see section 1.3.6). A less common approach is to compare probability distributions, either between different allocations by different software (Mas et al., 2014) or between calibrated and allocated probability distributions.

Spatial allocation accuracy is a desirable property of LUCC modeling, however, the problem with validation based on spatial accuracy is that it cannot identify the origin of errors. Probabilistic LUCC modeling is based on the impossibility or extreme difficulty to know the exact causes of LUC changes. Consequently, comparing LUCC models on spatial prediction accuracy effectively results in a confusion between different sources of spatial inaccuracy: data quality, choice of EVs, calibration errors, allocation errors, most notably. In particular, this validation method cannot distinguish between systematic errors due to inefficient or erroneous algorithms, and stochastic sources of errors due to the intrinsic randomness produced by a probabilistic approach. Sources of systematic error result in post-allocation explanatory variable probability distributions that are systematically different from what they are supposed to be, somewhat similarly to using a loaded dice instead of a fair one in a dice throw. In the process, the simulated LUC state changes induce systematic deviations from the expected spatial distribution of these LUC state changes. As a consequence, two different allocation algorithms with different types of errors/biases will produce systematic differences in spatial patterns when their output maps are compared for validation.

In the end, spatial accuracy is ultimately limited by the intrinsic level of randomness of the problem at hand. Furthermore, as allocation consists of a single

experiment, i.e., a single simulation among other possible simulations, spatial analysis is sensitive to statistical noise. Finally, such an approach tends to assimilate a simulation to a prediction, and must therefore be used with care for methods that are intrinsically probabilistic.

The primary question addressed in this thesis is the identification of systematic (formal and algorithmic) errors or lack of precision, and the practical importance of the induced systematic errors in post-allocation spatial accuracy. To achieve this purpose, one needs to separate the systematic effects of formal and algorithmic errors and biases from the statistical effect produced by the intrinsic probabilistic nature of explanatory variables. By eliminating errors and biases in allocation and increasing as much as possible the precision of calibration, systematic sources of discrepancies are eliminated or as much as possible reduced, and only a statistical form of randomness remains, whose magnitude depends on how predictive and how close to deterministic explanatory variables are.

This thesis relies on a very common procedure in many fields (and engineering as well) to assess if a method performs well or not, i.e., testing performances in controlled settings. Setting up problems in which what the answer is known beforehand, and testing how a calibration method performs to recover the information is the only known method to achieve this type of purpose.

Concerning calibration the objective is to test the accuracy of the calibration procedure calibration — and calibration only, i.e., we wish to evaluate the performances of calibration algorithms independently of other concerns. Checking accuracy in controlled settings is not only useful but also mandatory, because, although in actual case studies the underlying probability distributions are not known exactly beforehand, they do exist and the purpose of calibration is precisely to approach them as best as possible. If a method is known to be more precise than another in a number of controlled settings, then one can be reasonably confident that this method will provide a more reliable calibration output than its competitors in actual case studies. Such an evaluation method is proposed in section 2.6.

Concerning allocation, a method is proposed in chapter chapter 3. This method integrates the statistical dimension of the studied module since it requires a certain number of allocated maps, *i.e.* of simulation (Monte Carlo method).

More details on the thesis scientific strategy are given in section 1.4.2.

1.4. THESIS OUTLINE AND MAIN FEATURES

I recalled above the link between global change and LUCC (section 1.1), introduced LUCC modeling (section 1.2), and focused on statistical and spatially explicit pattern-based models (section 1.3). I will now describe the thesis target readership (section 1.4.1), the underlying scientific strategy (section 1.4.2) and the objectives of this thesis (section 1.4.3). Also, I give complementary information concerning the articles submitted for publication which are reproduced in this thesis (section 1.4.4). Finally, the structure of the thesis is presented (section 1.4.5).

1.4.1. THESIS INTENDED AUDIENCE

I am aware that the level of mathematical rigor of this work, made in close collaboration with Pierre-Yves Longaretti, is substantially more demanding than the standard in the field. In the literature, a number aspects of the allocation process have been dealt with on the basis of the developers intuition and choices, with as a result rather large differences in formalism and rigor between different modeling frameworks, whereas these choices are in fact strongly constrained by probability theory, and not as open as assumed in these previous works. As such, this thesis is primarily intended for software developers of LUCC modeling frameworks rather than users who may, however, have an interest in the issues addressed in the work presented here, although they may not (and for good reason) be interested in its technical details.

This level of sophistication was not aimed at idly, but because we truly feel that the main problem we want to address – the important differences of behavior between various software for the same problem and data, and their implications for the reliability of the results – cannot be resolved without such a level of mathematical rigor. However, it is also clear to us that as a consequence our papers – and by extension this thesis – are difficult for or even out of reach of most LUCC software users, and probably also difficult for a number of software developers. Therefore, I have tried, through this introduction and in the three following chapters, to offer some pedagogical background on the applied maths concepts we use and present them in the language of the LUCC community inasmuch as possible.

1.4.2. SCIENTIFIC STRATEGY OF THE THESIS

The main question addressed in this thesis has been already mentioned in section 1.2.4, i.e., that different software produce different results for the same problems and input data, to the point that the results robustness and reproducibility are disturbingly questioned. In section 1.3.6, I discussed the role of formal and algorithmic errors in producing systematic sources of deviations in LUCC post-allocation outcomes. These two issues constitute the point of focus of this thesis with respect to the question just recalled. I also investigate somewhat the role of EV choice. The issue of data quality and of its influence on post-allocation results is a very important one with respect to the main question, but this constitutes a very complex problem that is not discussed in this thesis.

I tend to believe that addressing this problem is of primary importance not only to developers but also to users. After all, every researcher seeks to produce valid understanding and insights for his or her community. The key word here is valid; any software will produce a result once set up for a particular case study, but the spread of results just mentioned raises legitimate concerns on the extent of the reliability of these results.

To make progress on this issue, I broke the problem into several smaller sub-problems, for a specific but widely used type of LUCC software (*i.e.*, statistical and spatially explicit hybrid pattern-based/cellular automata models). A common practice across many research fields is first to examine critically the mathematical apparatus used to set up the modeling framework implemented in the software (which differs widely from the modeling performed by a software user for a specific case study), check whether this implementation itself conforms to its intended purpose (*i.e.*, that it does indeed implement the developer's chosen framework) and then check if the software performs as expected in problems where the other sources of concern (data quality and user potential set-up errors) are eliminated. In principle these conditions should be satisfied by all software, but it will be shown in this thesis, that this is not the case, although some software do perform significantly better than others on this front.

Furthermore, this procedure must be applied to the various pieces making up a LUCC software, because each is rather complex in itself and can be examined largely independently from the others (pieces introduced through the LUCC model architecture in section 1.3). These pieces usually consist in an EV selection module (*i.e.*, quantities that are used to produce projections of future LUC changes), a calibration module (to extract relevant information from the data on these explanatory variables and their relation to past observed patterns of LUCC), and an allocation module (to produce actual LUCC based on this

extracted information), although some software (*e.g.* , the CLUE family) have a simpler structure for various reasons. These three elements are precisely the subject of the following chapters as presented in the section 1.4.5.

1.4.3. THESIS OBJECTIVES

I detail below a number of structured objectives, which are means to the main end described along with its motivation in sections 1.3.6 and 1.4.2 (removing or reducing formal and algorithmic errors in the calibration and allocation process). Obviously, not all of them were clearly defined at the beginning of the thesis and they have not necessarily been solved in the order indicated. However, their enumeration allows to picture quickly the different contributions of this thesis and to find them easily in the text.

1. Formalization of the architecture of spatially explicit statistical pattern-based LUCC models (section 1.3).
2. Formalization of the calibration-estimation module (section 2.3).
 - (a) Definition of the components of the calibration-estimation module (section 2.3).
 - (b) Proposal of a new method for the selection of explanatory variables (sections 4.4).
 - (c) Proposal of an evaluation method for the calibration-estimation module (section 2.6.1).
 - (d) Definition of a measure of precision for the calibration-estimation module (section 2.6.2).
 - (e) Quantification of the precision of existing LUCC models calibration methods (section 2.7).
 - (f) Proposal of a precise and numerically efficient calibration-estimation method (sections 2.4 et 2.5).
3. Formalization of the allocation module (section 3.3).
 - (a) Definition of the allocation module components (section 3.3.1).
 - (b) Definition of a bias for the allocation module (section 3.3.2).
 - (c) Identification of biases in existing LUCC models allocation methods (section 3.6).
 - (d) Proposal of a numerically efficient and nearly (and practically) unbiased allocation method (section 3.4).

- (e) Proposal of an allocation module evaluation method (section 3.5).
- 4. Demonstration software (chapitre 5).
 - (a) Development of an open source library in Python which constitutes an implementation of the formal framework of calibration-estimation and allocation developed in this thesis (section 5.2).
 - (b) Development of a graphical user interface (GUI) for this software.
 - (c) Validation of the implementation of the software on fully controlled case studies (sections 2.7 and 3.5).
 - (d) Use of the software on a real and large case study (section 5.4).

1.4.4. PERSONAL CONTRIBUTION TO ARTICLES

This thesis is largely composed of articles submitted for publication of which my thesis director Pierre-Yves Longaretti is co-author. It is thus appropriate to specify in this section what my personal contribution is in this production through an enumeration of the tasks undertaken during this PhD thesis. This outline is not exhaustive but representative of the distribution of our respective roles in the work carried out.

Generally speaking, it is possible to distinguish the purely mathematical aspect from the rest. Pierre-Yves Longaretti is responsible for the formalism and the mathematical demonstrations used in our articles and *a fortiori* in this thesis.

The formalization of the structure of the LUCC models, (i.e., the outline of their architecture, the definition of the modules or the design of the evaluation methods) is a co-construction between my thesis director and myself. It is the same for the design of the unbiased allocation methods introduced in section 3.4.

The calibration-estimation method based on kernel density estimation was largely introduced by myself (section 2.4). I also introduced the method of approximation of the kernel density estimation (section 2.5.1) which resulted in a substantial increase in numerical efficiency of the method. Also, I have entirely designed and implemented all the case studies, but their analysis and discussion was made jointly with Pierre-Yves Longaretti (sections 4.5, 2.7 and 3.5.3).

Finally, a substantial contribution of this thesis lies in the implementation of our methods within CLUMPY. I am responsible for its design, development and verification (chapter 5). The graphical user interface (GUI) has been co-developed with Roger Pissard, INRIA research engineer and part time member of the INRIA STEEP team where this thesis work was made.

1.4.5. THESIS STRUCTURE

Figure 1.6 schematically represents the structure of this thesis. Three chapters are reproductions of articles submitted for publication¹⁰ in the journal *Environmental Modelling & Software*. Very little has been made in their body text (except for typo correction, correction of some unintentional and always minor and inconsequential errors, and occasionally some clarification of obscure statements). Thus, they can be read independently and consequently, their respective introductory sections have more or less similar contents. Also, no inter-chapter reference is made to any section, but rather citations from our own publications.

The order of the chapters does not follow the natural order of operations a user would follow when setting up a model for a case study (this would be explanatory variables selection, calibration and allocation for validation and/or simulation) but the order dictated by the interdependencies of the methods used; in particular, our calibration method is used both in our study of allocation and in our method of explanatory variables selection.

Chapter 1 (the present one) is dedicated to the introduction of the subject. Chapter 2 (article, in press) deals with the calibration-estimation module and proposes a method based on kernel density estimation. Chapter 3 (submitted article) revisits the allocation module, identifies biases in existing software and proposes an unbiased allocation method. Chapter 4 (submitted article) deals with the selection procedure of explanatory variables by ensuring their relevance and limiting their redundancy. Chapter 5 describes the LUCC demonstration model (CLUMPY) that I developed during this thesis; this software implements the various methods developed in the course of this work, has been tested on a real case study and can already be used for generic LUCC case studies. Finally, chapter 6 concludes this thesis and proposes some perspectives for future formal or software developments of LUCC models.

Finally, a refereed paper given at the 2022 GISTAM conference presents an overview of this thesis work, with an emphasis on calibration issues. This contribution is reproduced in Appendix A and may be used as an alternative and more in-depth reading help and guide to the work presented in this thesis.

¹⁰The first one, reproduced in chapter 2 is already accepted to publication by *Environmental Modelling & Software* at the date of submission of this thesis.

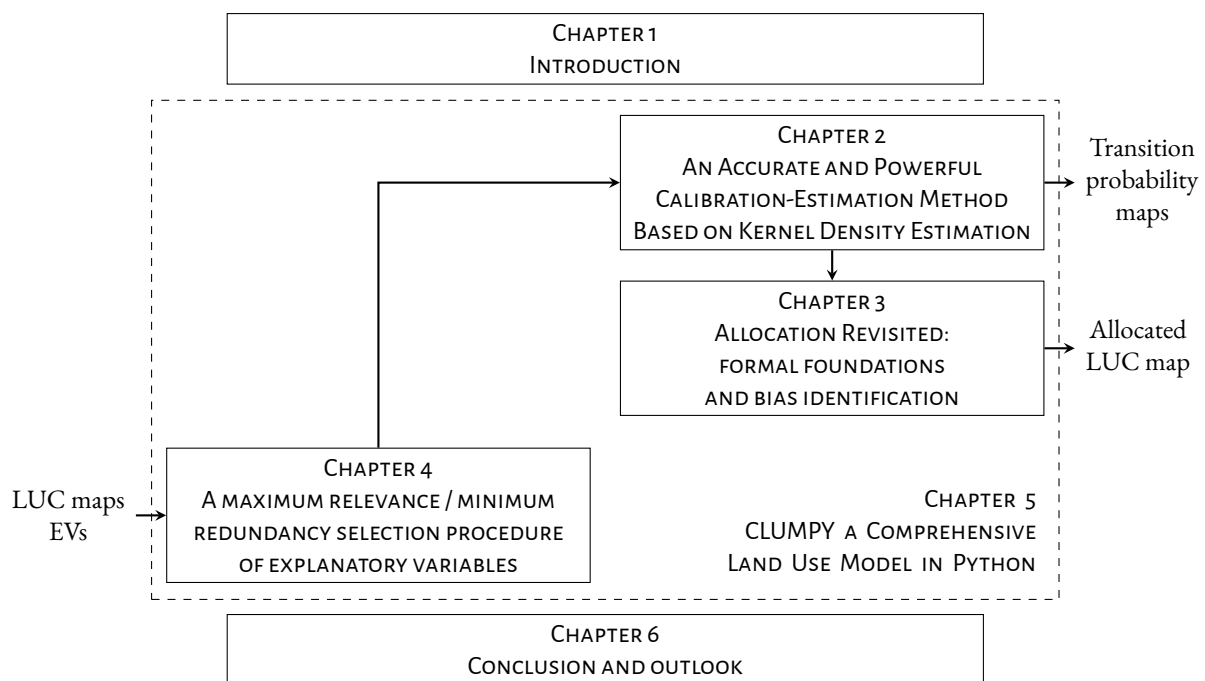


FIGURE 1.6: Schematic structure of the chapters of the thesis. The chapters order is dictated by their usefulness in the adopted research strategy and by their methodological interdependencies. The arrows indicate the order and dependencies of information when setting up an actual case study.

CHAPTER 2

AN ACCURATE AND POWERFUL CALIBRATION-ESTIMATION METHOD BASED ON KERNEL DENSITY ESTIMATION

Mazy, F.-R. and Longaretti, P.-Y. Towards a Generic Theoretical Framework for Pattern-Based LUCC Modeling. An accurate and powerful calibration-estimation method based on Kernel density estimation. *Environmental Software and Modeling*, 2022a

ABSTRACT

Several modeling strategies have been proposed to study Land Use and Land Cover changes (LUCC). However, substantial discrepancies have been noted between different models for the same problem, questioning their overall reliability and reproducibility. To address this challenge, we elaborate a generic, formally correct, theoretical framework for pattern-based LUCC modeling, which is implemented in our own software, CLUMPY (Comprehensive Land Use [and cover] Modeling in PYthon).

The present work focuses on calibration. We devise a kernel density calibration-estimation method (Bayes-eKDE) that is shown on synthetic artificial data to be both accurate and algorithmically efficient. We also introduce a generic evaluation method that allows us to compare the calibration efficiency of existing models. The gain in precision and computational time of our calibration method is precisely quantified in this way.

2.1. INTRODUCTION

Land Use and Land Cover Change (LUCC) models¹ are now widely used in environmental science in a variety of contexts to analyze a large array of issues (Agarwal et al., 2000; Verburg et al., 2006; Bielecka, 2020). In the recent years, a number of studies have undertaken a comparison of model results across a wide range of modeling environment types. These studies have displayed substantial discrepancies in the results, due to differences in input data or modeling strategies (Mas et al., 2014, 2022; Prestele et al., 2016; Alexander et al., 2017; García-Álvarez et al., 2022). These findings question the robustness and soundness of the conclusions drawn from such models (Verburg et al., 2019).

We focus here on pattern-based LUCC models (see, e.g., Camacho Olmedo et al. 2018 for a convenient typology and description of various LUCC modeling framework types). This is a widespread approach, implemented in a number of existing software, based on maps in raster format. Among the most common such software, one can mention the CLUE family (Verburg et al., 1999), the latest member being CLUMondo (van Asselen and Verburg, 2013; van Vliet et al., 2015), Dinamica EGO (Soares-Filho et al., 2002, 2013), or Idrisi LCM² (Eastman et al., 1995). Despite the numerous modeling environments of this type that have been developed over the years, few constituent steps and methodological choices are clearly formalized and exhaustively described in the literature. Furthermore, the calibration and validation methods adopted differ widely from one model to another (van Vliet et al., 2016; Noszczyk, 2019) making it difficult to perform systematic formal and algorithmic comparisons between models. Moreover, some methods and results of other fields could be more extensively used, such as the one described in this work, which is borrowed from the machine learning toolbox.

For these pattern-based LUCC models, substantial differences of behavior between the various modeling environments mentioned above are observed even for the same set of data, explanatory variables selection and scenario (Mas et al., 2014). Such differences necessarily come for the most part from two major

¹In this paper, *modeling environment* refers to any of the available software that relies on software-specific but application-generic modeling choices, allowing users to set up an implementation (instantiation) of the modeling environment in specific case studies. The term *model* refers either to the *modeling environment* (for the software) or *modeling framework* (for the concepts used) thus defined, or to an actual implementation (instantiation) of the software for a particular problem and context. *Theory* refers more specifically to the mathematical and conceptual apparatus that have been used in a specific modeling framework and implemented in its related modeling environment, but other types of theories are useful or even necessary in LUCC modeling, most prominently from social and environmental sciences.

²For the purposes of this study, we make use of Idrisi Selva, a somewhat old version, but for which we do have a licence. LCM is now part of TerrSet, the new software of Clarks Lab.

types of choices: the quantification of the correlation between predictors (explanatory variables) and actual patterns of change (e.g., through logistic regression, weights of evidence, neural networks...) performed in model calibration on the one hand, and the implementation of allocation performed in future projection of LUCC on the other hand. This is true even in controlled settings using artificial synthetic data (as in the analysis of Mas et al. 2014).

The calibration process, responsible for the first type of discrepancies, is the object of the present work. The second type is related to the allocation process and will be addressed in a dedicated paper. The main object of the series of papers initiated here is to analyze errors in modeling choices or in algorithm implementation (or under-optimal choices and implementations) which underlie most of the discrepancies in behavior of the various models. As such, we attempt to provide a firm theoretical basis for the mathematical and algorithmic foundations of pattern-based LUCC modeling. Such an approach should eventually replace the intuitive and often imprecise rules that have been used in the elaboration of existing modeling environments. Mazy and Longaretti (2022c) present an overview of the whole analysis, with an emphasis on the calibration process, and can be used as an introduction to our work.

This article is organized as follows. We start with a methodological discussion explaining in more detail the rationale, relevance and strategy of our approach. Indeed, the work done in the present paper is part of a more global line of research whose objectives and methods are most likely difficult to grasp for most LUCC software users and possible for a number of LUCC software developers as well. We therefore clarify these points, as they are not transparent in the technical work presented in the rest of the paper, and may lead to a misunderstanding of this work relevance (section 2.2). The following section is devoted to a description of the generic architecture of pattern-based LUCC modeling environments (section 2.3). This section also formulates the problem of transition probability estimation and introduces our notations. We then present our method of transition probability estimation (section 2.4). The implementation of our method called Bayes-eKDE is detailed next (section 2.5). Section 2.6 describes the strategy we have adopted to evaluate and compare the performances of our method along with various existing modeling software. Section 2.7 presents our controlled setting based in synthetic data and assumed transition probability distributions; actual comparisons with other modeling environment calibration methods are discussed there as well. The last section summarizes the main results and concludes the paper (section 2.8).

2.2. METHODOLOGICAL CHOICES AND RESEARCH STRATEGY RELEVANCE

Although the intended audience of this paper is LUCC software designers and developers, and more generally anyone interested in the formal foundations of quantitative LUCC modeling, the primary target of this section is LUCC software users. These potential readers are not expected to be interested in the technical details of our work, but may still be interested in understanding its nature and purposes. This may also be true of some software developers. Some of the points made here will by necessity be familiar to some readers.

The starting point of our questioning is the following. If a pattern-based LUCC software fails to some extent to produce expected results (e.g., observed spatial patterns of LUC changes), is this because of a defect in the software conception, of lack of quality of the data used, of incorrect user choices in the problem set-up (such as an incorrect choice of explanatory variables), of intrinsic characteristics of the problem, or a combination of these?

To address this question, it is important to keep in mind that the common validation methods of LUCC software based on various measures of spatial accuracy in the prediction of LUC state changes must be used with caution. Even for perfect data, perfectly known explanatory variables and a perfect software from a calibration and allocation point of view (which is never the case in practice), spatial accuracy can only be expected if the transition probabilities are very inhomogeneous and the fractional amounts of LUCC state change important, i.e., if the probabilistic nature of the problem is not too far from being deterministic. For an extreme but unrealistic context where probabilities of LUC state change are completely spatially homogeneous, actual state changes are completely random. In reality, an actual case study will in effect occupy some middle ground between total randomness and exact predictability. Improving the level of predictability of LUC changes is in fact a major goal of land-use theoretical analyses (such analyses are rooted, e.g., in social and environmental sciences).

Because of this, in the end, some “reasonable” spatial accuracy will be obtained if: i/ the chosen LUCC modeling framework implemented in a given software is mathematically correct; ii/ the problem is well enough understood and the underlying causes of change are known to a sufficient level, so that explanatory variables can be chosen wisely to capture effectively these underlying causes³; iii/ the quality of the data is not a serious issue. A practical consequence

³Explanatory variables need not be truly explanatory in nature, but only efficiently correlated to actual patterns

of this is that one must be able to judge a modeling framework correctness independently of the spatial accuracy of LUC patterns change “prediction”, of explanatory variable relevance and of data quality. This is precisely one of the main objectives of our line of research.

To make progress on this issue, we break the problem up into several parts, in order to analyze if existing software are correctly designed. To this effect, a common practice in other research fields is first to examine critically the mathematical apparatus that was used to set up the modeling framework implemented in the software, to check whether this algorithmic implementation itself conforms to its intended purpose, and finally to check if the software performs as expected in problems where the other sources of concern (here, data quality and user potential set-up errors) are eliminated. Furthermore, this procedure must be applied to the various pieces making up a LUCC software, because each is rather complex in itself and can be examined largely independently from the others. These pieces usually consist in an explanatory variable selection module, a calibration module (to extract relevant information from the data on these explanatory variables and their relation to past observed patterns of LUC changes), and an allocation module (to produce actual LUC changes based on this extracted information), although some software (e.g., the CLUE family) have a simpler structure for various reasons.

The present paper deals with calibration issues only. The way we address this presents at least two potentially paradoxical features from a LUCC software user point of view: we focus only on pixel LUC state changes and we focus on explanatory variable space and not on physical space. The first is paradoxical because LUC changes almost exclusively occur in bunches of contiguous pixels (patches), and not pixel by pixel, independently of each other; the second is paradoxical precisely because some level of spatial location accuracy is an important concern for a LUCC software user. Let us address these two points in turn.

In relation to the first point, note that other software’s calibration procedures also focus on various forms of pixel statistical properties. But, by design, these pixel probability distributions carry no information on patch transition as they carry no information on pixel correlations. How then can this calibration be at all relevant to actual case studies? The answer is related to assumptions that are rarely if ever mentioned by software developers.

Let us discuss this on the example of Dinamica EGO, whose modeling framework is possibly the most explicit and sophisticated and about this point among

of change. However and quite clearly, the closer to actual causal processes they can be (intrinsically or through proxies), the better the resulting correlation.

existing pattern-based LUCC modeling environments. Patches are produced from two basic “bricks” in Dinamica EGO: a seed pixel (i.e., a pixel around which a patch undergoing a change of LUC state is formed), and patch characteristics. Dinamica EGO implicitly assumes that, to a very high level of approximation, seed pixels are statistically independent of each other⁴, a property which is obviously not true for all pixels (there is a high chance that neighboring pixels are in the same LUC state or change state to the same LUC state, for example). Similarly, patch shape characteristics are assumed to be highly statistically independent of their seed pixel location⁵. And finally, individual seed pixel probability distributions are assumed to be the same (or nearly the same) as all individual pixels probability distributions⁶. These assumptions are also made in our own approach to pattern-based LUCC modeling. We will discuss elsewhere why they are reasonably well-satisfied in LUCC case studies⁷. For the time being, we point out that the conjunction of these assumptions justifies, for Dinamica EGO and for our own software (CLUMPY), that their respective calibration procedures focus on characterizing the probability distributions of pixels, while ignoring pixel-to-pixel correlations, i.e., ignoring the fact that transitions occur in patches (patch formation is addressed in the allocation module). For other software, the assumptions are different, and in fact incorrect to varying extents (this point is shown in our article on allocation, Mazy and Longaretti 2022b), but these software nevertheless focus on calibrating some types of pixel probability distribution on such a basis.

As for the second paradox, note that the probability distributions to be characterized, which depend on the modeling environment, are always probability distributions in explanatory space. This follows because explanatory variables are chosen to identify which pixels are more likely to undergo a transition. In explanatory space, and because pixels correlations are ignored in the probability distributions of interest as discussed in the previous paragraph⁸, the fact that pixels are contiguous or not in physical space is immaterial. Contiguity in patch transition is recovered when forming actual patches, while the spatial location of the patches is controlled by the pixel probability distribution that have been

⁴This point has never been proved, but can be checked on a case-by-case basis.

⁵Same comment.

⁶Same comment. Note also that, as pointed out above, the seed pixel correlations are totally different from the correlation properties of the total pixel population. However this information is not encoded in individual pixel probability distributions, which is precisely what allows the seed pixels and total pixel individual probability distributions to be identical to a high degree of precision.

⁷See in particular Appendix 3.A.

⁸Other types of probability distributions do carry information on correlations between contiguous pixels probability of change, but the three necessary properties stated above for our calibration framework to be relevant provide a simpler way to tackle patch transitions from a mathematical and algorithmic point of view than characterizing correlations in a statistical way.

calibrated, through the spatial dependence of the chosen explanatory variables. Observed patterns of change are then recovered from these features, inasmuch as explanatory variables are accurate predictors of these patterns, as pointed out earlier.

We now return to the point made above, i.e., that the relevance and accuracy of calibration procedures (the focus of the present paper) must be evaluated independently of the spatial accuracy of the prediction of patterns of change, of explanatory variable relevance and of data quality. This is achieved through a procedure whose relevance may also not be immediately apparent to a LUCC software user, i.e., by setting up problems in explanatory variable space in which we know beforehand what the answer should be, and testing how our method (and others) perform to recover this information, which is the only known method to achieve this purpose. In practice, we completely specify the relevant pixel probability distributions, allocate changes on the basis of this information in a (demonstrably) statistically correct way, and estimate how accurately these probability distributions are recovered by a number of calibration procedures (including ours) through specific criteria that we have elaborated to this effect (the procedure is detailed in section 2.6.1). The probability distributions chosen for this exercise are not designed to be related to clear and understandable features in physical space, but to test calibration procedures in a rather demanding way. More complete or realistic tests can always be elaborated, but this evaluation of calibration accuracy should be sufficient for its intended purpose, in our experience.

2.3. PROBLEM FORMULATION

We define here in a formal and general way the problem of modeling spatially explicit LUCC. Thus, we introduce a generic architecture of such a model (section 2.3.1) and notations (section 2.3.2). We also frame the specific problem of transition probability estimation (section 2.3.3) that is addressed in this paper as well as the properties of the different quantities involved (section 2.3.4). This probability distribution estimation is non-parametric. This is mandatory for elaborating a calibration procedure able to model LUCC behavior without enforcing at the onset the shape of probability distributions of interest in the problem.

2.3.1. SUMMARY OF PATTERN-BASED LUCC MODEL ARCHITECTURE

Generally speaking, pattern-based LUCC modeling environments display similar architectures in order to produce the two major types of maps required in LUCC analyses and projections: transition probability maps and allocation maps during simulations of future LUC states. This architecture is organized around a number of modules. In the present work, we adopt a structure in three modules: calibration, estimation and allocation (Fig. 2.1), whose purpose is described below. In existing modeling environments, calibration and estimation constitute a single module, although dynamic explanatory variable maps are sometimes recomputed at each time step (e.g., in *Dinamica EGO*), implying the existence of a (re)estimation procedure. Calibration and estimation are formally distinguished here because of the specific characteristics of our procedure and a precise definition of these two processes will be given in section 2.3.2. If some explanatory variable maps evolve and need to be updated, this is also performed at the estimation of transition probability stage.

All modeling environments usually rely on land use and cover (LUC) raster maps of the study area at some initial dates⁹ t_0 and t_1 , usually at least a few years apart, where the area of interest is fully spatially characterized in terms of LUC states. The spatial resolution of these maps is quite variable (a few meters to kilometers). The LUC typology of these initial maps is defined by the user from available data, and adapted to the user's specific needs on a case-by-case basis. This typology collects all LUC states observed at t_0 and t_1 as well as the ones of interest a simulation of future changes (which may or may not be all present at these initial dates).

Several maps of characteristic quantities, called explanatory variables, are also used. Explanatory variables are identified by the modeler prior to the elaboration of a LUCC model. These quantities are selected on the basis of their correlations with observed changes of LUC states between t_0 and t_1 . Some of these explanatory variable maps may be readily available, e.g., the elevation above sea level or slope, or they can be relative to specific characteristics of the LUC maps, e.g., the distance to urban areas, in which case they may evolve along with changes of LUC states. The issue of explanatory variable selection is not addressed in this work; they are assumed to be an input of the method. A new selection method based on the joint analysis of variables relevance and redundancy is proposed in a companion paper (Longaretti and Mazy, 2022).

These three elements (two initial LUC maps and a set of explanatory variable

⁹Some modeling environments, such as CLUMondo, rely on a single map, but must introduce further assumptions to characterize LUC changes from this single map. This strategy is much less precise but unavoidable when data are scarce.

maps) constitute the inputs of the model calibration-estimation process, which aims at producing the probability distributions of explanatory variables, conditional to specific initial (at t_0) and final (at t_1) LUC states. These distributions are not specified in physical space, but in explanatory variable space. Spatial probability distribution maps follow because every pixel in a map is characterized by specific values of the various explanatory variables.

The allocation process explicitly assigns new LUC states at various future dates for all pixels in the study area. This allocation process is initialized from a LUC map at time t_s , usually equal to t_1 at the start of a simulation¹⁰. The explanatory variable maps are also required at time t_s as well as an overall transition matrix that defines the total areas that are expected to change LUC state during the simulation time steps dt , for all possible state changes; such overall changes are usually specified through scenarios. This produces an allocated map at time $t_s + dt$, which serves as a new LUC map when the allocation process is iterated in a simulation. Time steps dt are usually of a few years.

We distinguish two types of modules: the first one for calibration and estimation and the second one for allocation. The first two are grouped together because they are conceptually related, and constitute the object of the present work. The allocation process itself will be examined in a forthcoming paper (Mazy and Longaretti, 2022b).

In some case studies, it is useful to divide the study area into a complete set of non-overlapping spatial regions. In general, the LUC architecture described above is then applied independently in each region. However, such an approach can lead to some spatial discontinuities of transition probabilities at the interfaces between regions. Correcting this potential problem is possible but this question is not addressed in this work.

2.3.2. DEFINITIONS AND NOTATIONS

Our objective requires some mathematical rigor. To this effect, we need to introduce a number of definitions and notations for the quantities that generically appear in pattern-based LUCC modeling. Some of these have already been used in other modeling environments, in particular Dinamica EGO, but we aim here at a higher level of precision and rigor, and consequently, our set of notations is substantially more extensive than usual. These notations are summarized in Table 2.1.

In practice, our first task is to evaluate the probability of change to all pos-

¹⁰Unless an additional data map at t_2 has been used for validation, in which case the simulation often starts at t_2 .

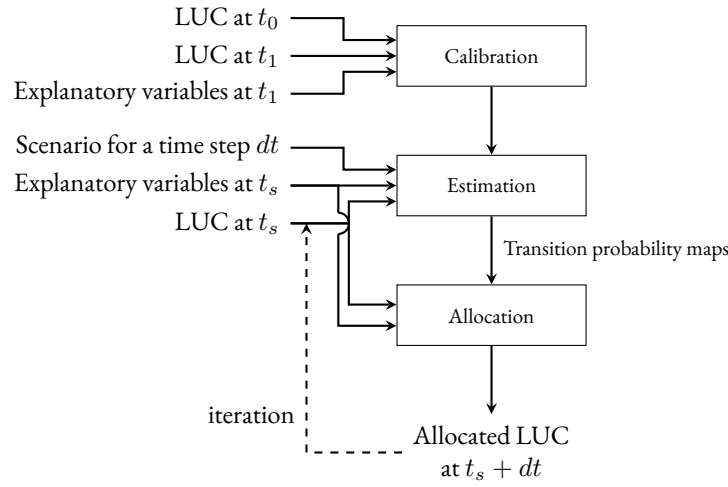


FIGURE 2.1: General architecture of our spatially explicit LUC modeling strategy.

sible LUC states, for each pixel¹¹. Without loss of generality, we can single out a specific initial state for this objective and focus on pixels characterized by this initial state. The process of transition probability characterization must then be repeated for all possible initial states.

LAND USE AND COVER (LUC) STATES

Each LUC state is represented by a positive natural number (the null value is used for unknown state or pixels outside the study area, a common convention in GIS environments). The notation u is used for a pixel state at t_0 (resp. t_s) and v for this pixel state at t_1 .

RASTER QUANTITIES

As stated in section 2.3.1, we use two maps at some initial dates t_0 and t_1 for calibration purposes. Also, because the maps at t_0 (initial calibration map) and t_s (map at some simulation step s) are different, the pixels in both maps for a given initial state u have no reason to be identical. Because of this we introduce specific notations for these relevant sets of pixels associated to the same initial state u at different times.

More precisely, let n_{I_v} be the number of pixels undergoing a change from state u to state v during the period $t_0 \rightarrow t_1$, and let I_v be the set of indices of these pixels¹² $\{1, \dots, n_{I_v}\}$. Each pixel is then identified by its index i .

We use a superscript i to associate any quantity of interest to a particular

¹¹Although actual LUCC occur in patches of contiguous pixels, characterizing single pixel transition probabilities is a required first step. The precise connection between pixel and patch transitions will be examined in another paper in this series.

¹²This implies the existence of an ordering for this subset of all pixels. This ordering can be chosen arbitrarily. For simplicity and by slight abuse of language, pixels are identified to their index in what follows.

pixel i . For example, the spatial coordinates of pixel $i \in I_v$ are denoted $\mathbf{x}^i = (x_0^i, x_1^i)$.

The second set of pixels associated with the same initial state u refers to $t = t_s$. Let n_J be the number of pixels in state u and J be the set of all pixel indices $\{1, \dots, n_J\}$. Each pixel is identified by its index j which can be used as a superscript.

EXPLANATORY VARIABLES

Explanatory (or ancillary, predictor or independent¹³) variables are quantities defined on pixels, and considered relevant for the statistical prediction of LUC changes, due to their correlation with past observed changes. Also, some explanatory variable are dynamic, and their maps need to be updated at each time step in the course of a simulation. For example the shortest distance of pixels to the limits of urban areas might be used as an explanatory variable in a simulation of urban sprawl and such a distance will evolve in the course of such a simulation.

Let d be the number of explanatory variables. For future use, we define the explanatory variable space either as a point space ($\mathcal{D}$) or as a vector space of dimension d ($\mathbb{R}^d$); by construction, $\mathcal{D} \subset \mathbb{R}^d$. The explanatory variables associated to individual pixels define either specific points or vectors in this space.

More precisely, an explanatory variable can have an infinite, finite or semi-finite range of values. When the range is limited in this way, the explanatory variables are viewed as points, or as subsets of the vector space, but do not constitute a sub-vector space. For example, the distance to an urban area is semi-finite because all possible values are ≥ 0 . Let $\mathcal{D}_k \subset \mathbb{R}$ be the k th explanatory variable's domain. In the example just mentioned, it would be equal to $[0, +\infty[$. By construction, $\mathcal{D} = \mathcal{D}_1 \times \dots \times \mathcal{D}_d$.

CALIBRATION AND ESTIMATION

The notions of calibration and estimation have been introduced in section 2.3.1 but not defined. We therefore start by providing a generic definition of these two terms, and will illustrate their specific meaning for the present procedure at various points in this work. *Calibration* is a process collecting relevant calibration explanatory variable values and their associated pixels; the meaning of *relevant* in this sentence will be made clear section 2.3.3. Various preliminary transformations of these explanatory

¹³The term “independent variable” is sometimes used, in opposition to dependent variables (e.g., probability densities) which are functions of independent variables. This does not imply that explanatory variables are statistically independent.

variables that are necessary for estimation are also performed in the calibration module (in particular, the whitening transformation that will be described in section 2.4.3). Note that because some explanatory variables are dynamic (i.e., evolve during the course of a LUCC simulation), the relevant explanatory variable values at the calibration times t_0 and t_1 are the ones stored for the estimation process.

The final objective of the calibration-estimation procedure is to produce maps of relevant probability distributions, in particular maps of various conditional probability distributions of explanatory variables that will be introduced in section 2.3.3. *Estimation* is the process providing these maps from the calibration data. Because some explanatory variables are dynamic, as just pointed out, these maps must be updated at each simulation time step. Such maps can also in principle be produced at the calibration step, but are actually not needed then. This is why in Fig. 2.1 estimation is performed at each simulation time step.

Two different notations are used for explanatory variables: they are designated by $\mathbf{y}$ when the corresponding pixels belong to the calibration set $\mathbf{Y}$ (pixels in LUC state belonging to J) or $\mathbf{Y}_v$ (pixels in LUC belonging to I_v), and by $\mathbf{z}$ when the corresponding pixels belong to the estimation set $\mathbf{Z}$.

More precisely, $\mathbf{y}$ (resp. $\mathbf{z}$) is a column vector collecting the d values y_k (resp. z_k) ($k \in \{1, \dots, d\}$). When used in probability distributions, $\mathbf{y}$ (resp. $\mathbf{z}$) stands instead for the d -tuple collecting all y_k s (resp. z_k s) instead of the column vector, and constitutes in this view a point in explanatory space. To alleviate notations, we do not use different symbols when referring to vectors or d -tuples or points; the difference of meaning should be obvious from the context.

In practice, the set of calibration pixels associated to the calibration explanatory variable values is either the set¹⁴ I_v or the set J . For any such pixel the associated vector of explanatory variables is denoted $\mathbf{y}^i$ while y_k^i refers to the k th explanatory variable value for this pixel. A map of the k th explanatory variable in the calibration phase is obtained for the whole collection of y_k^i values from the related pixel spatial locations $\mathbf{x}^i$. Finally, we define $\mathbf{Y}$ (resp. $\mathbf{Y}_v$) as the n_J -tuple (resp. n_{I_v} -tuple) of d -tuples $\mathbf{y}$. Such a collection accounts for multiple appearances of a given $\mathbf{y}$ value (i.e., a given d -tuple $\mathbf{y}$ may appear a number of times in $\mathbf{Y}$).

¹⁴This implies that some quantities are calibrated (i.e., selected and transformed) and estimated at t_s , while some other are calibrated at t_0 and t_1 and estimated at t_s . The rationale for this will become apparent in section 2.3.3.

In a similar way, the set of pixels associated to the estimation explanatory variable values is always the set J . For a given pixel $j \in J$, z^j , z_k^j and the n_J -tuple $\mathbf{Z}$ of z values taken by pixels in J are defined in the same way as in the previous paragraph. Note that by construction, $\mathbf{Z} = \mathbf{Y}$: for the relevant explanatory variables, the estimation and calibration explanatory variable values are identical.

It is customary in the LUCC modeling literature to bin continuous variables so that continuous and discrete variables can be placed on the same footing. However, although calibration is apparently simpler with binned variables, binning poses its own problems, and we avoid this altogether in the present work. As will be shown in the section 2.7, this also leads to a more powerful and accurate calibration strategy (Bayes rule assisted by efficient Kernel Density Estimation or Bayes-eKDE in short).

In this work we mostly ignore the possible existence of discrete explanatory variables. This follows because discrete variables do not pose any particular calibration problems (to estimate all distributions with respect to such categorical data, the standard method of counting pixels in each bin and normalizing by the total number of pixels is sufficient and well-defined). Appendix 2.B shows how a mix of discrete and continuous variables is dealt with in the framework of our Bayes-eKDE method; in effect, the problem is reduced to a finite number of purely continuous calibrations. For this reason, the remainder of the paper focuses on continuous calibration while ignoring discrete variables, to alleviate notations. Our software (CLUMPY) is set up to deal with both discrete and continuous variables along the lines described in this Appendix.

element	set	definition
u, v	V	LUC state at t_0 and t_1
i	I_v (size n_{I_v})	index of pixels of initial state u at t_0 and final state v at t_1
j	J (size n_J)	index of pixels of state u at t_s
y_k, y_k^i	$\mathcal{D}_k$	k th calibration explanatory variable value taken by pixels in I_v
$\mathbf{y}, \mathbf{y}^i$	$\mathcal{D}$	d -tuple (or column vector or point) of calibration explanatory variables for pixels in I_v
$\mathbf{Y}$	not defined	n_J -tuples of values of $\mathbf{y}$ taken by pixels in J
$\mathbf{Y}_v$	not defined	n_{I_v} -tuples of values of $\mathbf{y}$ taken by pixels in I_v
z_k, z_k^i, z_k^j	$\mathcal{D}_k$	k th estimation explanatory variable value for pixels in J
$\mathbf{z}, \mathbf{z}^i, \mathbf{z}^j$	$\mathcal{D}$	d -tuple (or column vector or point) of estimation explanatory variables for pixels in J
$\mathbf{Z} (= \mathbf{Y})$	not defined	n_J -tuples of values of $\mathbf{z}$ taken by pixels in J

TABLE 2.1: Summary of notations and definitions. All quantities pertain to the same given (and arbitrarily chosen) initial state u .

2.3.3. TRANSITION PROBABILITY ESTIMATION AND BAYES' RULE

The transition probability from u to v during a time step $dt = t_{s+1} - t_s$ for a pixel characterized by the d -tuple of explanatory variables $\mathbf{z}$ is noted $P(v|u, \mathbf{z})$. This notation indicates that this probability is conditional to the knowledge of u and $\mathbf{z}$. Bayes' rule allows us to evaluate this quantity from three other probabilities or probability densities¹⁵:

$$P(v|u, \mathbf{z}) = P^*(v|u) \frac{\rho(\mathbf{z}|u, v)}{\rho(\mathbf{z}|u)}, \quad (2.1)$$

where $P^*(v|u)$ is the targeted global change probability (the fraction of pixels undergoing such a change). It can be extracted at the start of the simulation from the calibration maps at t_0 and t_1 , or, more commonly, it can be specified by a scenario of LUC evolution that one wishes to implement in a LUCC simulation. Whatever method is used, it is assumed in the present analysis that $P^*(v|u)$ is given, and the $*$ superscript is used to highlight the fact that this quantity represents an input by the user, not a quantity to be evaluated in the calibration-estimation process.

¹⁵Strictly speaking, $\rho(\mathbf{z}|u, v)$ and $\rho(\mathbf{z}|u)$ are defined at some specific time t , while $P(v|u, \mathbf{z})$ and $P^*(v|u)$ are defined for this time and a time interval dt . Consequently, the probability densities ρ should also conditionally depend on t and the probabilities P on t and dt in this expression. Furthermore, the calibration explanatory variable values of $\rho(\mathbf{z}|u)$ depend on both t_0 and t_1 . These extra dependencies are dropped to alleviate notations.

Because we make use of continuous variables, Bayes' rule involves probability densities, and not only probabilities: $\rho(\mathbf{z}|u)$ is the probability density of $\mathbf{z}$ for pixels of initial state u , and $\rho(\mathbf{z}|u, v)$ is the probability density of $\mathbf{z}$ for pixels undergoing a state change from u to v . Probabilities themselves are non zero only on a given interval of $\mathbf{z}$. For example, $P(\mathbf{z}_1 < \mathbf{z} < \mathbf{z}_2|u) = \int_{\mathbf{z}_1}^{\mathbf{z}_2} \rho(\mathbf{z}|u) d\mathbf{z}$, where the convention $\mathbf{z}_1 < \mathbf{z}_2$ has been used when $z_{k,1} < z_{k,2}$ for all explanatory variables k . When this “interval” in $\mathbf{z}$ is small, the probabilities usually become proportional to the corresponding small volume in $\mathbf{z}$ space. We conventionally use the letter ρ for probability densities and the uppercase letter P for probabilities. A more formal definition of probability densities is provided in Appendix 2.A.

The use of Bayes' rule is motivated by the fact that the two probability densities $\rho(\mathbf{z}|u)$ and $\rho(\mathbf{z}|u, v)$ on the right-hand side of Eq. (2.1) are much easier to estimate than the probability on the left-hand side, a feature already noted by the designers of the Dinamica EGO modeling environment. It is therefore not surprising that we make use here of the same starting point, Eq. (2.1), although the calibration method outlined below differs substantially from the one used in this software.

A major aim of the LUC calibration-estimation process is — or should be — to estimate these two probability densities. Both $\rho(\mathbf{z}|u)$ and $\rho(\mathbf{z}|u, v)$ are estimated for all pixels with LUC state equal to u at t_s (pixels in J)¹⁶. J is also the set of calibration pixels for $\rho(\mathbf{z}|u)$, while I_v is the set of calibration pixels of $\rho(\mathbf{z}|u, v)$. This defines the relevant sets of pixels involved in the definition of the calibration and estimation processes (section 2.3.2). Consequently, in principle, $\rho(\mathbf{z}|u, v) = \rho(\mathbf{z}|u, v, \mathbf{Y}_v)$ is also conditionally dependent on the calibration pixels of the set $\mathbf{Y}_v$. This dependence is dropped to alleviate our notations.

This difference between the two probability density has a simple but unavoidable motivation. The probability density $\rho(\mathbf{z}|u)$ is defined at each t_s , independently of previous LUC state changes. As some explanatory variables are dynamic, this probability density changes at each time step t_s , as it characterizes a pixel. Both features make the use of J the only possible choice of pixel set for both calibration and estimation explanatory variable values. The probability density $\rho(\mathbf{z}|u, v)$ on the other hand is again defined at t_s but some assumption has to be made in order to specify it for allocation [which is the purpose of $P(v|u, \mathbf{z})$]. The assumption is that, *a priori*, the structure of the underlying causes producing a change is the same between t_s and t_{s+1} (allocation step) and t_0 and t_1 (calibration data). In other words, this is an *a priori* probability den-

¹⁶This is further explained and commented upon in section 2.4.2 for $\rho(\mathbf{z}|u, v)$.

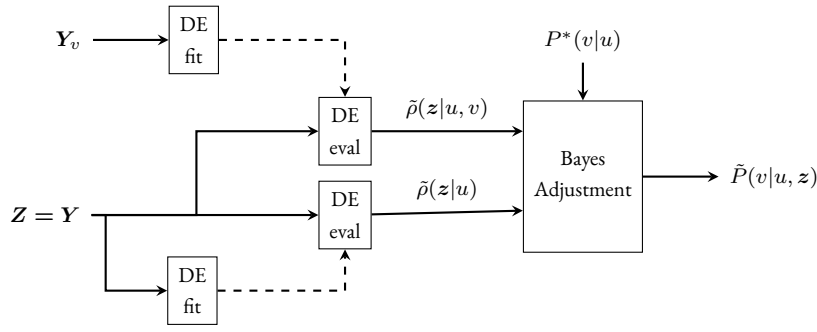


FIGURE 2.2: Transition probability estimation architecture. The density estimation processes fit and eval are detailed in Fig. 2.7. The Bayes adjustment algorithm is detailed in section 2.5.3.

sity for allocation, in the Bayesian sense. One may also impose some *a priori* changes to this *a priori* assumption, if, e.g., they are required by the scenario that is implemented in the simulations of interest. But this does not affect the Bayesian *a priori* meaning of this probability density. This being said, even if the underlying causal structure is assumed to be time-independent, the probability density itself must be updated for each pixel, because some explanatory variables are dynamic (recall that P and ρ are attached to pixels).

The estimation of such probability densities has been extensively studied in the machine learning literature, in particular through the use of density estimators (hereafter DE). The resulting transition probability estimation procedure is outlined in Fig. 2.2. The n_J -tuple $\mathbf{Y} = \mathbf{Z}$ is used as input for the density evaluation of $\rho(\mathbf{z}|u)$, denoted $\tilde{\rho}(\mathbf{z}|u)$, while the n_{I_v} -tuples $\mathbf{Y}_v$ along with $\mathbf{Z}$ are used as input in the density evaluation of $\rho(\mathbf{z}|u, v)$, denoted $\tilde{\rho}(\mathbf{z}|u, v)$. This involves two more detailed sub-processes (fit and eval). The fit procedure tells the estimator what the observed data are, while the eval procedure estimates the probability density from these data and from an appropriate density estimator technique. Both procedures are described in section 2.5.2, drawing on the material of sections 2.4 and 2.5. Finally, Bayes' rule is used to compute the estimate $\tilde{P}(v|u, \mathbf{z})$ of $P(v|u, \mathbf{z})$, from the previous two probability density estimates. This last step requires the use of a correction algorithm (Bayes' adjustment) to cope with the approximations involved in these estimators (this is detailed in section 2.5.3).

2.3.4. PROBABILITY DISTRIBUTION PROPERTIES

The probability density estimates should verify the following closure relations:

$$\int_{\mathbf{z} \in \mathbb{R}^d} \tilde{\rho}(\mathbf{z}|u) d\mathbf{z} = 1, \quad (2.2)$$

$$\int_{\mathbf{z} \in \mathbb{R}^d} \tilde{\rho}(\mathbf{z}|u, v) d\mathbf{z} = 1, \quad (2.3)$$

as the exact (unknown) probability densities ρ do satisfy these very same relations.

Furthermore, although allocation algorithms generally use a method of allocation in patches and not pixel by pixel, the transition probabilities are defined so that the proportion of pixels changing state must be equal on average to the expected level defined by $P^*(v|u)$, i.e.,

$$\int_{\mathbf{z} \in \mathbb{R}^d} \tilde{P}(v|u, \mathbf{z}) \times \tilde{\rho}(\mathbf{z}|u) d\mathbf{z} = \int_{\mathbf{z} \in \mathbb{R}^d} \tilde{\rho}(v, \mathbf{z}|u) d\mathbf{z} = P^*(v|u), \quad (2.4)$$

These relations are exact for the exact probability distributions. It is therefore useful to enforce them as well on the estimations of these distributions. This is achieved through the Bayes adjustment procedure of section 2.5.3.

2.4. TRANSITION PROBABILITY ESTIMATION METHOD

We now flesh out the process generically described in section 2.3.3. We start with the probability density estimation procedure (noted DE in figure 2.2). Density estimation is widely addressed problem in the machine learning literature, and the most common approach, kernel density estimation, or KDE, is described below and adopted here. The presentation is self-contained, and requires no prior knowledge of machine learning techniques.

2.4.1. KERNEL DENSITY ESTIMATION SHORT SUMMARY

Kernel density estimation is a very common non parametric methods used in machine learning to estimate a density distribution. A kernel is a function of finite support or finite support integral. This function is usually monotonically decreasing away from its center, and symmetric with respect to this center; this center itself is a free parameter of the kernel function. The extension of the kernel function is characterized by a parameter called “bandwidth” in the

machine learning literature and we stick to this designation in the remainder of this work. The kernel function integral is normalized to unity. For probabilistic problems, kernel density estimation makes use of observed statistical realizations of the underlying probability distributions. It has been widely studied for both univariate and multivariate distributions (Wand, 1992).

Kernel density estimation proceeds through a kind of moving mask average, applied in the space of the explanatory variables of the problem; this provides an estimate of the actual probability distribution at each point (i.e., individual explanatory variable values) in this space from the location of the calibration points themselves, from an estimate of the local density of surrounding calibration points. In practice, this is achieved by attaching a kernel function of well-defined shape and extent (bandwidth) to each calibration point in explanatory variable space (i.e., using all observation points as Kernel centers, see Fig. 2.3) and summing these kernel functions at all estimation points to obtain an estimate of the real density function (Fig. 2.4). Depending on whether $\rho(\mathbf{z}|u, v)$ or $\rho(\mathbf{z}|u)$ is estimated, the calibration points belong respectively to $\mathbf{Y}_v$ or $\mathbf{Z}$.

As advertised above, when the Kernel is symmetric with respect to its center (a condition satisfied by most kernels) this estimation is easily shown to be equivalent to a weighted-average of calibration points in explanatory variable space, the weight being obtained by the value of the kernel attached to the calibration point, the kernel function itself being attached to the point where the estimate is performed. This equivalent view gives some intuitive feel of what a kernel density estimation actually does. Note that this estimation is performed everywhere in explanatory variable space. This multivariate interpolation based on an estimate of the local density of calibration points is the key feature allowing us to avoid the usual assumption of independence of explanatory variables, without loss of precision.

These two ways of interpreting the kernel density estimation method are equivalent and have immediate consequences. First, as for all estimation methods, the more calibration data one has, the more precise the estimation will be. Second, the bandwidth is a key parameter which directly controls the quality of the estimation process. Indeed, choosing too narrow a bandwidth is a source of over-interpretation (over-fitting) and the estimate will be over-dominated by the particular observations that are available, resulting in noisy estimations. On the other hand, choosing too large a bandwidth leads to over-smoothing and to a loss of information with respect to the data. Determining an adequate bandwidth is therefore a crucial and widely discussed issue in the literature. This is addressed in section 2.4.5.

Multivariate estimations have been discussed by Wand (1992) in a very gen-

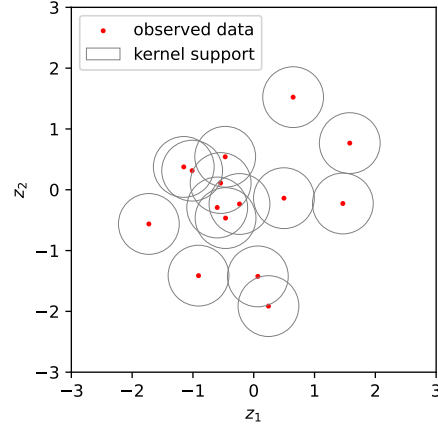


FIGURE 2.3: Illustration of the kernel density estimation (kde) method in a simplified 2-dimensional case. Red points represent observed data and gray circles highlight the kernel support, *i.e.* the radius of influence of the kernel around each observation.

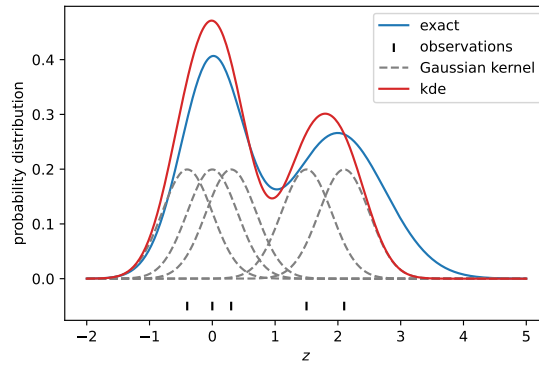


FIGURE 2.4: Illustration of the uni-variate kernel density estimation given 6 observations. The underlying exact probability density is equal to $1/2 \times [\mathcal{N}(0, 0.5) + \mathcal{N}(2, 0.75)]$ where $\mathcal{N}(z, \sigma)$ is a unidimensional gaussian distribution of mean z and dispersion σ .

eral setting. We use here a simplified form of these authors' approach, where the kernel is in fact a product of uniform kernels (Scott, 2015). More precisely, we propose to estimate $\rho(\mathbf{z}|u)$ through:

$$\tilde{\rho}(\mathbf{z}|u) = \frac{1}{n_J h^d} \sum_{j \in J} K \left(\left\| \frac{\mathbf{z} - \mathbf{y}^j}{h} \right\|_p \right), \quad (2.5)$$

where $\|(\mathbf{z} - \mathbf{y}^j)/h\|_p$ is the p -order distance in explanatory variable space, K the kernel function whose integral is equal to one and h is the bandwidth, i.e., the kernel characteristic extension parameter. A commonly chosen distance is the usual second order (Euclidean) one, defined by:

$$\|(\mathbf{z} - \mathbf{y}^j)/h\|_2 = \left[\sum_{k=1}^d (z_k - y_k^j)^2 / h^2 \right]^{1/2}, \quad (2.6)$$

Similarly:

$$\tilde{\rho}(\mathbf{z}|u, v) = \frac{1}{n_{I_v} h^d} \sum_{i \in I_v} K \left(\left\| \frac{\mathbf{z} - \mathbf{y}^i}{h} \right\|_p \right). \quad (2.7)$$

The Gaussian kernel is the most common one in the literature. It is defined as :

$$K_{\text{gaussian}}(r) = \frac{1}{(2\pi)^{d/2}} \exp \left(-\frac{r^2}{2} \right), \quad (2.8)$$

where the second-order distance has been used in Eq. (2.5). However, as will be seen in section 2.4.4, it may be necessary to use a simpler kernel with computationally less intensive partial integrals. We therefore propose to use alternatively a box kernel defined as:

$$K_{\text{box}}(r) = \frac{1}{2^d} \begin{cases} 1 & \text{if } |r| \leq 1 \\ 0 & \text{else} \end{cases} \quad (2.9)$$

for an infinite-order distance in Eq. (2.5), i.e. $\|(\mathbf{z} - \mathbf{y}^i)/h\|_\infty = \max_{k=1}^d (z_k - y_k^i)/h$.

2.4.2. COMMENTS

Eqs. (2.5) and (2.7) are the most important relations in our procedure. As already pointed out in sections 2.3.2 and 2.3.3, the estimation explanatory values $\mathbf{z}$ are associated to pixels in J , i.e., to all pixels in state u at t_s . For $\rho(\mathbf{z}|u)$, calibration and estimation explanatory values are identical, while for $\rho(\mathbf{z}|u, v)$, the calibration values come from the calibration data (explanatory variable values associated to pixels in I_v).

More precisely, the meaning of these two equations is the following. Both are defined at any point $\mathbf{z}$ in explanatory variable space. Eq. (2.5) estimate is based on the local density of calibration points in state u at t_s , i.e., points corresponding to pixels in ensemble, the location of which is specified by $\mathbf{y}^i$ ($i \in J$): the higher the density of calibration points in the vicinity of $\mathbf{z}$, the larger the probability density, the vicinity being defined by the kernel bandwidth. This gives directly the estimate of the probability density $\tilde{\rho}(\mathbf{z}|u)$ at t_s . In practice this estimate is needed at all points $\mathbf{z}^i$ ($i \in J$), i.e., the estimation points are the same as the calibration points, a feature already pointed out in our definitions (section 2.3.2).

Eq. (2.7) is more subtle. Its first, immediate meaning is that it gives the estimate of the probability $\tilde{\rho}(\mathbf{z}|u, v)d\mathbf{z}$ that a given $\mathbf{z}$ within $d\mathbf{z}$ at t_0 leads to the $u \rightarrow v$ transition in the time span $t_1 - t_0$. This estimates results from the density in explanatory space $\mathcal{D}$ of calibration pixels in state u who have effectively transited from u to v between t_0 and t_1 (pixels in I_v). But this estimate applies also to all other points in state u at t_0 , on the basis of this same local density of calibration points in $\mathcal{D}$; as such, because these other points did not actually change state, the estimate has a counterfactual flavor: the other points could have undergone a transition, with this probability for $\mathbf{z}$ within $d\mathbf{z}$ to have been selected.

The second meaning follows from the *a priori* assumption stated in section 2.3.3: this probability density also characterizes the probability that a given $\mathbf{z}$ within $d\mathbf{z}$ at t_s leads to the $u \rightarrow v$ transition in the time span¹⁷ $t_{s+1} - t_s = t_1 - t_0$. Consequently, it can in particular be applied to all $\mathbf{z}$ values associated to pixels in J at t_s . Note finally that in this second meaning, the calibration pixels originally in I_v at t_0 may no longer have the same explanatory variable values at t_s or even may no longer be in state u , which may make Eq. (2.7) paradoxical at first sight. However, this is of no consequence, as the *a priori* assumption does not bear on pixels themselves, but on the relation their $\mathbf{z}$ value bears to the explanatory variable values of the calibration set, in explanatory space $\mathcal{D}$.

¹⁷Other time spans can be envisioned, but this requires to rescale the probability density. Such a rescaling is dealt with, e.g., in Dinamica EGO, and will be addressed in our forthcoming paper on allocation.

(and not in physical space), i.e., to the set $\{\mathbf{y}^i \in \mathcal{D} | i \in I_v\}$: this set exists in explanatory variable space $\mathcal{D}$ independently of time and pixels.

In pattern-based LUCC modeling, time steps are usually small enough so that the fraction of pixels undergoing a specific $u \rightarrow v$ transition is small (e.g., $\sim 10^{-3}$). Consequently, the available data for calibration is rather sparse, which implies that these data tend to be undersampled and the resulting probability estimates to be noisy as soon as more than a couple of explanatory variables are selected (a very common situation). In particular, in relation to undersampling, some combinations of explanatory variable values may not have been observed for a specific transition even though very similar combinations may have. These unobserved combinations have no reason to be *a priori* assigned a null transition probability.

A common way to circumvent undersampling issues is to assume that explanatory variables are independent (as done, e.g., in Dinamica EGO), so that the needed probability distributions can be computed from the product of individual probability distributions: $\rho(\mathbf{z}|u) = \prod_k \rho(z_k|u)$ and $\rho(\mathbf{z}|u, v) = \prod_k \rho(z_k|u, v)$. The individual (marginal) probability distributions are unavoidably much less noisy than the multidimensional ones, so that this does indeed reduce the overall noise level in the estimates. However, this independence assumption has undesirable consequences on the accuracy of the estimates, even when the level of correlation between explanatory variables is low. Namely, this includes in the probability distribution combinations of individual z_k values which would otherwise actually have zero probability of transition, and this reduces the probability distribution of combinations which can actually occur (in order to satisfy the cumulative probability distribution closure relation). These deviations may be substantial even if the level of correlation between explanatory variables is low; e.g., even a 20% correlation as measured by Pearson's correlation matrix (a rather common level) may lead to substantial (factor several) differences in the probability distribution magnitude for some practically important combinations. This counter-intuitive feature can be checked on simple examples. This is the major reason why we have avoided the assumption of statistical independence of explanatory variables.

Kernel density estimations altogether avoid these issues. Undersampled explanatory variable combinations probabilities are corrected for by the interpolation provided by the estimator, and the noise of multidimensional distributions can be reduced in an optimal or near-optimal way by an appropriate choice of the Kernel bandwidth (see section 2.4.5). In particular, our KDE method provides an interpolation in explanatory variable space that provides a natural level of transition probability for unobserved but legitimate combinations, as

explained in relation to our discussion of Eq. (2.7) at the beginning of this section. These features are exemplified in section 2.7.

2.4.3. WHITENING TRANSFORMATION

We have chosen a uniform bandwidth in all dimensions. However, the different explanatory variables, considered individually, would in general require different bandwidths, as the distribution of observed point will depend on the explanatory variable under consideration. In order for the choice of a unique bandwidth to be meaningful, one first needs to normalize the data, so that, in each direction in explanatory space, the data mean is zero and the covariance is the unity matrix. Such a transformation of the data is called a whitening transformation (WT).

This procedure is a generalization of the standardizing of a random variable $\mathbf{x} = (x_1, x_2, \dots, x_d)^T$ (the superscript T refers to the transposed vector), which is generically carried out by defining $\mathbf{X} = \mathbf{V}^{-1/2}\mathbf{x}$ where $\mathbf{V}$ is the diagonal matrix of variances σ_i^2 of x_i : this results in a unit variance for X_i but does not remove the correlations between X_i and X_j . The whitening transform performs both objectives. More precisely, defining the covariance matrix $\mathbf{C}$ of $\mathbf{x}$, the whitening transformation $\mathbf{W}$ of $\mathbf{x}$ transforms $\mathbf{x}$ into $\mathbf{X} = \mathbf{W}\mathbf{x}$, with covariance matrix $\mathbf{W}\mathbf{C}\mathbf{W}^T = \mathbf{I}$ where $\mathbf{I}$ is the identity matrix. From $\mathbf{W}\mathbf{C}\mathbf{W}^T = \mathbf{I}$, one immediately obtains $\mathbf{W}(\mathbf{C}\mathbf{W}^T\mathbf{W}) = \mathbf{W}$, hence

$$\mathbf{W}^T\mathbf{W} = \mathbf{C}^{-1}, \quad (2.10)$$

which is the most important relation defining the whitening transformation. Unfortunately, there is an infinite number of matrices satisfying this constraint, and the problem needs to be further specified from relevant considerations. This is the object of the present section.

To this effect, we rely on the diagonalization of the covariance matrix, which is discussed first. In practice, we need to normalize (in the meaning just defined) data corresponding to both $\rho(\mathbf{z}|u)$ and $\rho(\mathbf{z}|u, v)$. The corresponding number of pixels are n_I and n_{I_v} , respectively. In the next subsection, both cases are treated simultaneously by introducing $n = n_I$ or n_{I_v} .

2.4.3.1. COVARIANCE MATRIX ESTIMATE AND ITS DIAGONALIZATION

Let us first recall the definition of the $d \times d$ covariance matrix $\mathbf{C}$:

$$C_{k,k'} \equiv \text{cov}(z_k, z_{k'}) \equiv E\left((z_k - \mu_k)(z_{k'} - \mu_{k'})\right), \quad (2.11)$$

where $E(\cdot)$ stands for the expectation value (ensemble mean) over all pixels in initial state u (i.e., all pixels in I), and $\mu_l = E(z_l)$ is the sample mean. In vector notation, $\mathbf{C} = E([\mathbf{z} - \boldsymbol{\mu}][\mathbf{z} - \boldsymbol{\mu}]^T)$ where the superscript T refers to the transposition operation. In practice, one chooses to work with centered data, i.e., data with zero mean. To this effect, we define translated vectors

$$\bar{\mathbf{z}} = \mathbf{z} - \boldsymbol{\mu}, \quad (2.12)$$

so that $\mathbf{C} = E(\bar{\mathbf{z}}\bar{\mathbf{z}}^T)$.

In general neither the sample mean nor the expectation values are exactly known, and the covariance matrix is evaluated from its estimate $\hat{\mathbf{C}}$ on the available data¹⁸:

$$\hat{C}_{k,k'} = \frac{1}{n-1} \sum_{i=1}^n (z_k^i - \hat{\mu}_k^i)(z_{k'}^i - \hat{\mu}_{k'}^i), \quad (2.13)$$

where $\hat{\mu}_l = \sum_i z_l^i/n$ is the sample estimate of μ_l . The replacement of the expected mean by a sample estimate creates a bias in the evaluation of the correlation coefficients. Indeed, defining $S_{kk'}^2 = \sum_{i=1}^n (z_k^i - \hat{\mu}_k^i)(z_{k'}^i - \hat{\mu}_{k'}^i)/n$ and $\sigma_{kk'}^2 = E([z_k - \mu_k][z_{k'} - \mu_{k'}])$, one has $E(S_{kk'}^2) = (n-1)\sigma_{kk'}^2/n$. This justifies the normalization adopted in the estimate of Eq. (2.13). In vector notation, $\hat{\mathbf{C}} = n/(n-1) \times \sum_{i=1}^n [\mathbf{z}^i - \hat{\boldsymbol{\mu}}^i][\mathbf{z}^i - \hat{\boldsymbol{\mu}}^i]^T$. We also introduce estimates of the translated vectors¹⁹, $\hat{\mathbf{z}} = \mathbf{z} - \hat{\boldsymbol{\mu}}$ so that

$$\hat{\mathbf{C}} = \frac{1}{n-1} \sum_{i=1}^n \hat{\mathbf{z}}^i \hat{\mathbf{z}}^{T,i} \quad (2.14)$$

As this matrix is real and symmetrical, it is diagonalizable with the help of a unitary matrix $\mathbf{V}$, i.e.,

$$\hat{\mathbf{C}} = \mathbf{V} \mathbf{L} \mathbf{V}^T, \quad (2.15)$$

where $\mathbf{L}$ is the $d \times d$ diagonal matrix of the eigenvalues in decreasing order and $\mathbf{V}$ is an eigenvector matrix of size $d \times d$. Thus by definition, $\mathbf{L}$ is a diagonal matrix whose diagonal elements are the variances of the corresponding variables $z_k - \hat{\mu}_k$.

Let us finally introduce the inverse of the square root of the estimate of the

¹⁸Introducing $\hat{\mathbf{Z}}$, the matrix of size $n_I \times d$ (number of pixels times number of explanatory variables) which contains the data to be transformed, one can write the estimate of the covariance matrix in more compact form: $\hat{\mathbf{C}} = \hat{\mathbf{Z}}\hat{\mathbf{Z}}^T/(n_I - 1)$.

¹⁹This estimate has vanishing expectation value as well as vanishing mean over the sample.

covariance matrix²⁰ $\hat{\mathbf{C}}$:

$$\hat{\mathbf{C}}^{-1/2} = \mathbf{V}\mathbf{L}^{-1/2}\mathbf{V}^T, \quad (2.16)$$

which is directly related to the whitening transformation defined below.

2.4.3.2. WHITENING TRANSFORMATION EXPRESSION

There is an infinite number of possible choices for the whitening matrix $\mathbf{W}$ (Kessy et al., 2018). A possible choice is the following²¹:

$$\mathbf{W} = \hat{\mathbf{C}}^{-1/2}, \quad (2.17)$$

with associated normalized data vector

$$\hat{\mathbf{z}}^* = \mathbf{W}\hat{\mathbf{z}}. \quad (2.18)$$

From Eq. (2.16), $\mathbf{W} = \mathbf{W}^T$ so that $\mathbf{W}^T\mathbf{W} = \hat{\mathbf{C}}^{-1}$ as required from Eq. (2.10). Furthermore, from Eq. (2.18), $\hat{\mathbf{z}}^*$ is centered on the sample because $\hat{\mathbf{z}}$ is. Finally, the estimate $\hat{\Sigma}$ of covariance matrix of $\hat{\mathbf{z}}^*$ is equal to the identity matrix²²:

$$\hat{\Sigma} = \frac{1}{n-1} \sum_{i=1}^n \hat{\mathbf{z}}^{*,i} \hat{\mathbf{z}}^{*,T,i} = \mathbf{W}\hat{\mathbf{C}}\mathbf{W}^T = \mathbf{I}, \quad (2.19)$$

from Eqs. (2.15), (2.16), (2.17) and (2.18). Thus, the chosen matrix $\mathbf{W}$ achieves our normalization purpose: the data are now centered, have normalized dispersion, and vanishing cross-correlations.

Note that both the original and transformed probability distributions must satisfy the closure relations Eqs. (2.2) and (2.3). For example, for $\rho(\mathbf{z}|u)$, one has

$$\int_{\mathbf{z} \in \mathbb{R}^d} \tilde{\rho}(\mathbf{z}|u) d\mathbf{z} = \int_{\mathbf{z}^* \in \mathbb{R}^d} \tilde{\rho}(\mathbf{z}^*|u) d\mathbf{z}^* = 1. \quad (2.20)$$

As the identity of the probabilities holds over any range in random variable

²⁰One can show that this matrix is unique.

²¹This is the first of the five natural choices discussed by Kessy et al. (2018), known as the ZCA-Mahalanobis whitening transformation procedure (ZCA stands for “zero-phase components analysis”). See Kessy et al. (2018) for more details.

²²Note that this does not imply that the related random variables $\hat{\mathbf{z}}_k^*$ are statistically independent, so that the associated probability distributions, $\rho(\mathbf{z}|u)$ or $\rho(\mathbf{z}|u, v)$ cannot be assumed to be separable.

space, the change of variable defined by Eq. (2.18) leads to

$$\rho(\mathbf{z}|u) = |\det \mathbf{W}| \rho(\mathbf{z}^*|u), \quad (2.21)$$

$$\rho(\mathbf{z}|u, v) = |\det \mathbf{W}| \rho(\mathbf{z}^*|u, v). \quad (2.22)$$

The whitening transformation is noted WT in Fig. 2.7 and the whitening transformation scale specified in Eqs. (2.21) and (2.22) is noted *WT scale*. In the remainder of this work, the superscript ^{*} is dropped; unless otherwise specified, all variables are transformed variables and the kernels defined earlier apply to these transformed variables.

2.4.4. BOUNDARY BIAS CORRECTION

A kernel density estimation presents a well-known bias close to an explanatory variable boundary (Diggle, 1985). For bounded variables, the kernel density estimation method will underestimate the density in the vicinity of the variable boundary because the method assimilates the absence of elements beyond the boundary as a probability of occurrence (non-occurrence, in this case) and not as an intrinsic property (Fig. 2.5a). For this reason, the kernel should be truncated beyond the boundary. In effect this amounts to defining a new kernel that is the truncated version of the original one, and this new kernel must accordingly be correctly normalized.

In a multidimensional case, boundaries form hyperplanes which define a set of closed half-spaces; this applies as well to transformed data as discussed in section 2.4.3, and the boundary hyperplanes are also transformed into hyperplanes in the transformed space (which however are no longer aligned with the different explanatory variables). The half-spaces intersection is therefore the domain of interest in the transformed explanatory variable space and is noted $\mathcal{D}$. A given kernel at a particular observation point should then be divided by the cumulative distribution function $\int_{\mathcal{D}} K(\cdot)$ in order to account for this kernel redefinition and normalization (Fig. 2.5b). In other words, the kernels truncated by the boundaries of $\mathcal{D}$ must be renormalized by their effective volume in $\mathcal{D}$, as kernels are necessarily normalized to unity.

Thus, the estimated probability $\tilde{\rho}(\mathbf{z}|u, v)$ defined in Eq. (2.7) can be cor-

rected as follows

$$\tilde{\rho}(\mathbf{z}|u, v) = \frac{1}{n_{I_v} h^d} \sum_{i \in I_v} \frac{K\left(\left\|\frac{\mathbf{z} - \mathbf{y}^i}{h}\right\|_p\right)}{\int_{\mathbf{z} \in \mathcal{D}} K\left(\left\|\frac{\mathbf{z} - \mathbf{y}^i}{h}\right\|_p\right)}. \quad (2.23)$$

and a similar correction is applied to compute $\tilde{\rho}(\mathbf{y}|u)$ from Eq. (2.5).

However, $\int_{\mathcal{D}} K(\cdot)$ may be difficult to calculate efficiently in an exact or approximate way, depending on the nature of the kernel K . Thus, in the case of the Gaussian kernel [Eq. (2.8)], the computation of the integral over the domain $\mathcal{D}$ requires an approximate computational method, e.g. a Monte-Carlo computation of integrals (due to the multi-dimensional nature of the problem) which is not very efficient numerically if one has to repeat this method for each observed element near the boundaries.

In order to avoid this problem, we have adopted the box kernel based on the infinite norm [Eq. (2.9)]. With this choice, computing the integral of the kernel on the domain $\mathcal{D}$ amounts to determining the volume of a hypercube limited by several half-spaces. Cho and Kim (2020) proposed an exact and numerically efficient equation for this calculation. We implemented this method in a dedicated python package HYPERCLIP²³ and used it to implement the edge correction discussed in this section in our algorithms.

2.4.5. BANDWIDTH SELECTION

Determining the optimal bandwidth parameter for the estimation of a probability density from a set of observations is a widely studied problem in the literature. Many methods have been proposed. The vast majority of them are based on the minimization of the squared difference between the estimate and the exact probability. Since the exact probability density is unknown, it is necessary to approximate this difference, such as the Plug-In method (Wand and Jones, 1994; Duong and Hazelton, 2003) which injects an approximation obtained by an estimate made with a pilot bandwidth. Also widespread, the cross-validation method (Rudemo, 1982; Sain et al., 1994; Duong and Hazelton, 2005) uses among other things the leave-one-out estimator. Chacón and Duong (2018) and Schindler (2011) summarize these methods.

However, once the squared difference approximation has been performed,

²³<https://github.com/fmazy/hyperclip>

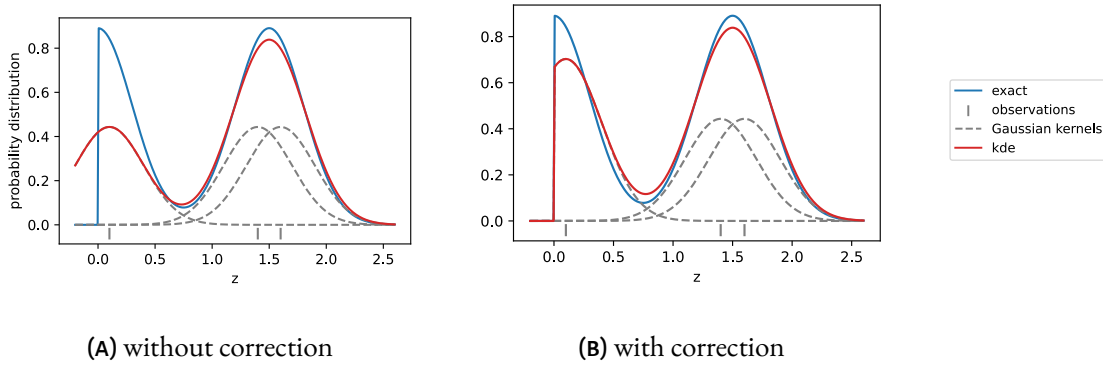


FIGURE 2.5: Very simple illustration of the boundary bias correction in a unidimensional case restricted to the half-space $[0, +\infty)$. In the absence of correction, observations near the border end up in an underestimation of the probability density 2.5a. The boundary bias correction allows us to correct this phenomenon while enforcing the probability closure for the obtained density estimate 2.5b. Although the correction is not perfect, it results in a substantial gain in precision in the estimation of the probability density distributions.

it is necessary to vary the bandwidth and determine a minimum. This step is ambiguous in most real applications. Indeed, in our LUCC problem, as soon as the number of explanatory variables increases, most of the time, no minimum stands out, either because of the presence of several local minima, or because of the monotonicity of the function. Very often, such methods lead to an unsuitable choice of bandwidths, which is often similar to undersmoothing.

In order to circumvent these problems, Terrell (1990) proposed to use a “maximum smoothing principle”. This method is based only on general information about the data such as the standard deviation, but has the advantage of avoiding naive over-interpretation or over-fitting of the data. The bandwidth proposed by Terrel for a multivariate problem once a whitening transformation has been performed is defined as follow :

$$h_{\text{Terrel}} = \frac{(d+8)^{(d+6)/2} \pi^{d/2} \int K^2}{16 n (d+2) \Gamma(d/2+4)}, \quad (2.24)$$

where n is the number of observation data points (pixels), d the number of explanatory variables, Γ is the Gamma function, and $\int K^2 = 2^{-d}$ for the box kernel. In LUCC analyses, avoiding data over-interpretation is often desirable, and the reliability and speed of the calculation makes this a method of choice in this context.

2.5. IMPLEMENTATION

In the previous section, we proposed a theoretical method to estimate the transition probabilities, notably by introducing the kernel density estimation method. However, at first sight, this method is numerically very intensive. Moreover, in an actual case study, we need to take care of inappropriate numerical values in the output of Bayes' formula (e.g., large values in under-sampled regions, in particular in the tails of the distributions). We therefore propose in this section an efficient method for approximating density estimation by kernels and detail a Bayes fitting algorithm designed to deal with the problem just mentioned. To simplify the presentation of the following algorithms, we focus here on the estimation of $\rho(\mathbf{z}|u, v)$. The estimation of $\rho(\mathbf{z}|u)$ is done in a similar way, *mutatis mutandis*.

2.5.1. EFFICIENT KERNEL DENSITY ESTIMATION

A density estimation with the kernel method requires a large number of operations in a naive implementation Scott (2015): calling m the number of calibration data points and n the number of estimation data points, the required number of operations is $\mathcal{O}(mn)$. In the present context, the m training pixels are either the n_J calibration data pixels observed in state u , or the n_{I_v} calibration data pixels having undergone a transition $u \rightarrow v$, while the n estimation pixels are all pixels of interest (i.e., all n_J pixels when considering all possible transitions from a given initial state u).

This estimation can be approximated and accelerated with the use of fast Fourier transforms (O'Brien et al., 2016; Langrené and Warin, 2019) or hashing (Charikar and Siminelakis, 2017; Siminelakis et al., 2019; Backurs et al., 2019). These methods, although attractive, have the disadvantage of generating a plain matrix whose size becomes numerically prohibitive as soon as the number of explanatory variables exceeds 3 which is routinely the case in LUCC modeling (any comparison with these other methods is then impossible because of their incapacity to treat the given problem).

Sub-sampling strategies have been alternatively proposed (Zheng et al., 2013). Various methods have been elaborated to this effect while enforcing a given maximum error level (Phillips, 2013; Phillips and Tai, 2018a,b; Tai, 2022). The simplest such method consists in performing a uniform random sub-sampling, provided that the number of training elements exceeds a certain threshold.

Alternatively, we propose a simple alternative KDE approximation method called efficient KDE (eKDE). The basic principle is the following. We start by

binning the set of observed points at fine grain with respect to the chosen bandwidth. Then, instead of considering a kernel on each of the observed points, we place only a single kernel in the center of each bin, weighted by the number of observed points within this bin. By extension, when we want to estimate the probability density at a given point, we consider instead the center of the bin to which it belongs and identify the bins that are close enough to contribute to the estimation as a simple weighted summation.

This simplification considerably alleviates the numerical burden of the KDE estimation; however, in spite of its conceptual simplicity, the actual algorithmic implementation of this procedure involves a number of delicate points. We describe this implementation in two steps, fit and estimation, designated in this way in Fig. 2.7.

2.5.1.1. EFFICIENT KDE METHOD: FIT

We follow Wells and Ting (2019) who proposed to bin the calibration data points. The idea is to use a sufficiently small binning scale so that the KDE main idea is still relevant. To this effect, let us first define a characteristic scale s such that $\int_{-hs}^{hs} K(r)dr \approx 1$. This is introduced mostly for unbounded kernels. For example, we take $s = 3$ for the Gaussian kernel defined in Eq. (2.8); for bounded kernels, $s = 1$, e.g., for the box kernel defined in Eq. (2.9). Next, we define the bin width w_k for each explanatory variable k with the help of an auxiliary odd integer q_k ; more precisely $w_k = 2hs/q_k$. Note that once explanatory variables have been normalized as described in section 2.4.3, a unique q and w can be chosen.

This binning associates to a calibration explanatory variable y_k (or rather, its centered and normalized version) a range Γ_k of possible integer values, each representing a bin identified by a dummy integer γ_k : $1 \leq \gamma_k \leq \Gamma_k$. This d -tuple identifies a bin in explanatory variable space. The set of calibration bins is denoted Γ_y . Finally, we divide the number of observation data points associated to each bin by the total number of observations which gives us the set of weights of the bins noted $\boldsymbol{\nu}$.

Fig. 2.6 illustrates this fit step with the help of a very simple 2-dimensional example (i.e., we make use of two explanatory variables, z_0 and z_1), using the box kernel. We make use, for simplicity, of only 10 observed points (pixels) and one point for which we wish to estimate the probability density (Fig. 2.6a) — either $\tilde{\rho}(\mathbf{z}|u)$ or $\tilde{\rho}(\mathbf{z}|u, v)$. We choose $q = 3$ for both explanatory variables and perform the binning (Fig. 2.6b) for a chosen q . Finally, we identify the set of nearby non-empty bins Γ_y^* and their weighting $\boldsymbol{\nu}$ (Fig. 2.6c) in the vicinity of the estimation point just defined. We have used q to define the extent of the

vicinity of the current bin, as this defines the maximal extent of our bounded kernel function.

If the domain of interest in explanatory variable space is bounded, the boundary bias correction of section 2.4.4 must be applied. In practice we correct the weights ν in the following way. The bias correction is directly applied to bins: we define bin center by $\mathbf{y}_0$, and attach the kernel function to this center. From the reasoning of section 2.4.4, the kernel must be divided by the cumulative distribution function $\int_{\mathcal{D}} K(\cdot)$ in order to renormalize the kernel function truncated by the boundaries of $\mathcal{D}$, as kernels are necessarily normalized to unity. This kernel renormalization can equivalently be applied to the kernel weight, leading to the following expression for the renormalized weight ν_r :

$$\nu_r = \frac{\nu}{\int_{z \in \mathcal{D}} K\left(\left\|\frac{z - \mathbf{y}_0}{h}\right\|_p\right)}. \quad (2.25)$$

2.5.1.2. EFFICIENT KDE METHOD: ESTIMATION

We can now estimate the probability density for any estimation point. The points we want to estimate are binned in the same way as in the fit step and are noted Γ_z^* . Thus, the probability density is not estimated at the exact estimation point location in explanatory space but at the nearest bin center. This is performed from the set of neighboring bin centers whose kernel's support (bounded by s as explained above) would encompass the bin center where the estimation is made. By construction, this selects q^d bins (since q is an odd integer, there is no ambiguity); this is reduced by selecting the non empty bins in this collection. It is possible to design a numerically efficient algorithm to identify these bins (see our Supplementary Material). For each such bin, the value of the kernel at the estimated point is calculated and multiplied by its weight, defined in the fit step. Finally, we sum these values to obtain the estimated probability density.

This procedure is illustrated on the example presented in the previous section (see Fig. 2.6). We wish to estimate the probability density at a given point represented in orange in Fig. 2.6a. The explanatory variable space is first binned in way explained above. Then we identify the relevant non-empty bins as shown on Fig. 2.6c among all the bins in a $q = 3$ window around the bin of interest in each direction (including this bin). In our example, there are 4 non-empty bins that will be used in the calculation of the probability density estimate. We compute the center-to-center distances of the bins in the non-binarized space to

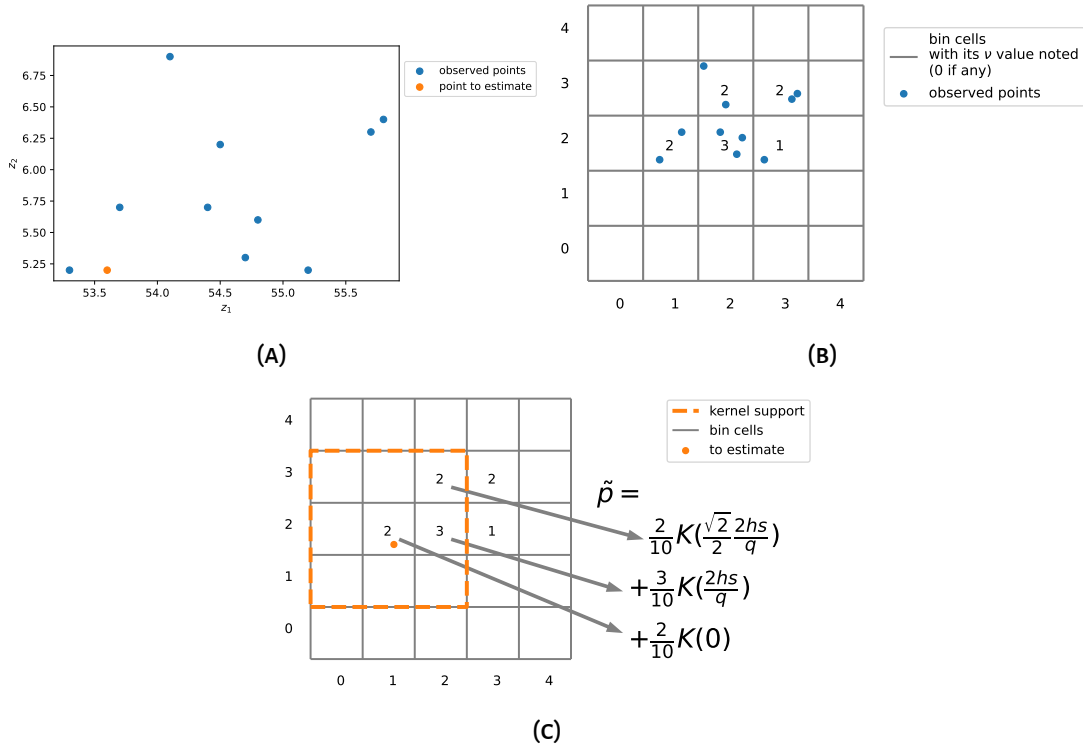


FIGURE 2.6: A very simple example to illustrate the fit step of our eKDE method. We have set 10 calibration points in a 2-dimensional explanatory variable space and one estimation point whose probability density we want to estimate (a). The binning and boundary bias correction are shown in (b). The non-empty bins, associated Γ_z^* and corresponding weights ν are easily identified on the figure: in order to obtain our estimation probability density, we identify the nearby non empty-bins in the dashed-orange delimited square, which contribute to the looked-for estimation (c).

evaluate the corresponding kernels and sum them with their weights to obtain the estimation (Fig. 2.6c).

Such an algorithm seems quite simple at first sight. However, when the number of dimensions increases, it is necessary to find an efficient way to explore the calibration points, which together form a kind of sparse matrix, and this operation is not trivial. A too naive exploration is quickly excluded because of the burden in computation time. We describe an efficient algorithm dedicated to this task in Supplementary Material.

2.5.1.3. PARALLELIZATION

In order to speed up the estimation of transition probabilities, one may parallelize some the operations involved. In our model, we chose to parallelize the estimation step (section 2.5.1.2). Thus, we separate the set of points to be estimated into a certain number of subsets equal to the number of processors that we wish to use. Each processor is assigned to the processing of a single subset and the results are then combined. The case study presented in section 2.7 exemplifies the significant reduction in computation time achieved by increasing the number of processors (Fig. 2.12a).

2.5.2. DENSITY ESTIMATOR ARCHITECTURE

We are now in position to present in a clear and logical fashion the various operations described above and required in our Kernel density estimation method. These are sketched in a sequential manner in Fig. 2.7 and summarized under the “DE fit” and “DE eval” blocks in Fig. 2.2. These blocks are also identified with dotted line boxes in Fig. 2.7. The final objective is to evaluate the probability density of the complete collection of explanatory variables sets of values $\mathbf{Z}$ from the calibration set $\mathbf{Y}$ ($= \mathbf{Y}_v$ or $\mathbf{Z}$ as previously discussed).

The whitening transformation is first obtained (WT transformation, section 2.4.3) and applied to the data. The borders also undergo the same whitening transformation, which defines the transformed domain $\mathcal{D}$ in explanatory variable space; this domain is used in the calculation of the boundary bias correction, sections 2.4.4 and section 2.5.1.1. Once the parameter q is chosen the estimator fit procedure can be performed (KDE fit, section 2.5.1.1). The same whitening transformation is applied to the actual set of interest $\mathbf{Z}$. An estimate of the probability density is obtained from the estimation procedure of section 2.5.1.2, with a bandwidth selected as proposed in section 2.4.5. Finally, the whitening scaling required to relate original and transformed variables probability distributions is applied (section 2.4.3.2).

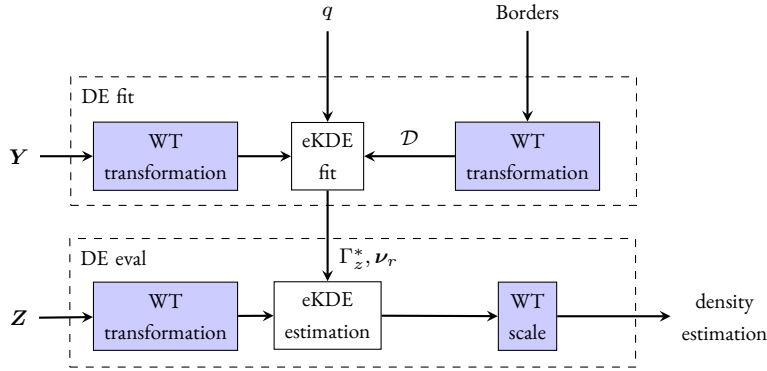


FIGURE 2.7: Implementation architecture of the density estimation method. The dashed-line boxes DE fit and DE eval are the ones referenced in Fig. 2.2. $\mathbf{Z}$ and $\mathbf{Y}$ refer to the estimation and calibration sets of d -tuples of explanatory variable, respectively. WT is the whitening transformation (section 2.4.3) and WT scaling refers to the scaling required to relate original and transformed variables probability distributions (section 2.4.3.2). eKDE is the efficient kernel density estimation method (section 2.5.1) with fit and estimation functions

Our KDE procedure has a single free parameter, q , set by default to $q = 51$ (see Appendix 2.D).

2.5.3. BAYES ADJUSTMENT PROCESS

We now turn to the last process of Fig. 2.2. The exact (and unknown) transition probability obeys Eq. (2.4). As this is a known constraint, one may expect that the estimation of the transition probability will be more accurate if it also required to satisfy this constraint. This requirement translates into the following sum over pixels²⁴:

$$\frac{1}{n_I} \sum_{i=1}^{n_I} \tilde{P}(v|\mathbf{z}^i, u) = P^*(v|u). \quad (2.26)$$

This relation can be justified in the following way. Let us introduce the degeneracy factor g_z which counts all pixels having a same $\mathbf{z}$ value within $d\mathbf{z}$ for a given initial state u . This number is not exactly known, but its expectation value is²⁵: $E(g_z) = \rho(\mathbf{z}|u)d\mathbf{z}/n_I$, by definition (this expectation value is exact in the limit of an infinite number of pixels). This allows us to transform the sum over

²⁴In principle we should make this adjustment directly from the integral of the closure relation Eq. (2.4). Doing this on a pixel sum is nearly equivalent due to the short proof on expectation values that follows, and much simpler and more efficient from an algorithmic point of view.

²⁵This can be defined in an ensemble average meaning, and could be evaluated if one could draw an infinite number of realizations of the observed data, under the same transition probability. This never occurs in reality (there is a unique history), but can be asymptotically performed numerically, and can be used as a basis for this thought experiment.

pixels in an integral over $\mathbf{z}$. Therefore²⁶:

$$E \left(\frac{1}{n_J} \sum_{i=1}^{n_J} P(v|\mathbf{z}^i, u) \right) = \int d\mathbf{z} P(v|\mathbf{z}, u) \rho(\mathbf{z}|u) = P^*(v|u). \quad (2.27)$$

where the last equality follows from the closure relation Eq. (2.4). Enforcing this relation on the estimation of the transition probability itself and not only on its expectation value leads to Eq. (2.26).

This property has very little chance to be exactly verified in actual data because of the approximate nature of the probability density estimation method. Moreover, imposing a too large global transition probability $P^*(v|u)$ can lead to²⁷ $\sum_{v \neq u} \tilde{P}(v|u, \mathbf{z}^i) > 1$ for some pixels i . Thus, in general, the left-hand side of Eq. (2.26) is not equal to the right-hand side. In order to correct for this difference, we propose a simple algorithm which modifies the transition probability estimate by the ratio of the two, in order to guarantee the required equality (algorithm 1). Note that if the requested probabilities $P^*(v|u)$ are particularly large, a saturation phenomenon of the highest transition probabilities occurs (the corresponding pixels will eventually have all changed state). This Bayes adjustment process has been included in Fig. 2.2. The corresponding algorithm is outlined below (Algorithm 1).

2.6. EVALUATION METHOD

Evaluating the results of a LUCC modeling strategy is not a common practice in the literature, which raises questions on the robustness of the obtained results (van Vliet et al., 2016). As an attempt to cope with this issue, it has been proposed to compare different models on a controlled land-use change problem (Mas et al., 2014), from data created by the authors; however, as the transition probability was not specified at the onset, it was only possible to compare the models between them without being able to evaluate their objective quality. This objective quality can only be assessed by evaluating the ability of any modeling strategy to estimate the transition probability $P(v|u, \mathbf{z})$, although this

²⁶In fact, we make use here of successive forms of expectation values: first for the estimate itself, then for the degeneracy factor. A more formal proof will be given in our forthcoming paper on allocation.

²⁷In principle, and for various reasons that will be discussed in our forthcoming allocation paper, spatially explicit pattern-based LUCC models are meaningful only if the relative number of state changes per time step is “small”, in which case the possibility just discussed should not arise. But one may not *a priori* exclude practical applications in which this is not true, at least for some transitions, even though these should be avoided for self-consistency and precision.

Algorithm 1: Bayes Adjustment (BA). It is an outline algorithm without reference to a given programming language.

Input:

for all i , $\tilde{\rho}(\mathbf{z}_i|u)$,
 for all v , and for all i , $\tilde{\rho}(\mathbf{z}_i|u, v)$,
 for all v , $P^*(v|u)$,
 n_c^* (a natural number to stop the algorithm in case of non convergence)

```

1 for all  $i$ ,  $\rho(v|u, \mathbf{z}_i) := \frac{\rho(\mathbf{z}_i|u, v)}{\rho(\mathbf{z}_i|u)} P^*(v|u)$ 
2 for all  $i$ ,  $P(v|u, \mathbf{z}_i) := \frac{P(v|u, \mathbf{z}_i)}{\frac{1}{m} \sum_{i=1}^m P(v|u, \mathbf{z}_i)} P^*(v|u)$ 
3 for all  $i$ ,  $s_i := \sum_{v \neq u} P(v|u, \mathbf{z}_i)$ 
4  $n_c \leftarrow 0$ 
5 while it exists  $i$  such as  $s_i > 1$  and  $n_c < n_c^*$  do
6   for all  $i$  such as  $s_i > 1$  and for all  $v \neq u$ ,  $P(v|u, \mathbf{z}_i) := \frac{P(v|u, \mathbf{z}_i)}{s_i}$ 
7   for all  $i$  and for all  $v \neq u$ ,  $P(v|u, \mathbf{z}_i) := \frac{P(v|u, \mathbf{z}_i)}{\frac{1}{m} \sum_{i=1}^m P(v|u, \mathbf{z}_i)} P^*(v|u)$ 
8   for all  $i$ ,  $s_i := \sum_{v \neq u} P(v|u, \mathbf{z}_i)$ 
9    $n_c := n_c + 1$ 
10 for all  $i$ ,  $P(v = u|u, \mathbf{z}_i) = 1 - \sum_{v \neq u} P(v|u, \mathbf{z}_i)$ 
11 return for all  $i$ ,  $P(v|u, \mathbf{z}_i)$ 

```

quantity is not known in actual case studies. To this effect, we design a completely controlled setting where all relevant transition probabilities are *a priori* known. These are constituted by all quantities on the right-hand side of Bayes rule, Eq. (2.1), i.e., $P^*(v|u)$, $\rho(z|u)$, and $\rho(z|u, v)$, from which the left-hand side of Bayes rule is completely specified as well. This allows us to evaluate the precision of our procedure along with existing ones in the reconstruction of these quantities from pseudo calibration data constructed from given probability distributions.

Actual changes occur in patches of contiguous pixels, but the calibration-estimation process evaluated here focuses on individual pixel probability distributions; patches can then be produced on this basis. Because we work in explanatory variable space, and in line with the explanations given in section 2.2, we ignore here the organization of pixels in physical space. Further arguments can be given to legitimate this procedure. First, the labelling process of all pixels is arbitrary, and the spatial continuity may be disorganized in this labelling process without problem as long as the pixel coordinates are known, and as long as pixels correlations are ignored (this is a feature of the probability distributions involved in Bayes rules). Second, we consider only continuous probability distributions in explanatory variable space (as categorical explanatory variables pose no specific calibration problem). In such a case, if explanatory variables are continuous functions of planar coordinates, the probability distribution represented in physical space will also be continuous. Therefore, one can focus on reconstructing this probability distribution in explanatory variable space, without loss of generality; the associated probability distribution in physical space is recovered from the explanatory variables dependence on spatial location. Finally, ensuring on this basis a correct reconstruction of the pixel probability distributions is sufficient to ensure that patch construction is a well-defined process (see again the explanation given in section 2.2 on this point, which will be more formally proved elsewhere).

Also, allocating final states to pixels individually can be performed by a very simple algorithm. This allows us to focus on the efficiency and precision of the calibration-estimation procedure itself. We will examine the efficiency of patch allocation in our forthcoming paper on allocation.

One last word. Calibration is performed on two given maps, produced at times t_0 and t_1 . The pixel transition probability $P(v|z, u)$ that is produced from these data depends implicitly on the time step $t_1 - t_0$. It is possible to produce a transition probability on a different time-step, with the help of additional assumptions, usually that the distribution of patch sizes produced during the time-step is independent of the time-step but that their total number is pro-

portional to the time-step²⁸.

2.6.1. PROCESS

The outline of the comparison process of the different calibration methods is shown on Fig. 2.8 and is described below. We start with the calibration part of the process. We first produce a LUC map at time t_0 , restricted to initial state u (as our calibration-estimation procedure treats initial states independently of one another). At the start of the process, such maps are “bland”, i.e., they contain only u pixels with no associated explanatory variable values. As we work almost exclusively in explanatory variable space, where the spatial distribution of pixels is unessential, the only important parameter associated to such a map is in fact the number n of pixels in state u .

Next, we exactly specify the function $\rho(\mathbf{z}|u)$. This function must obey the closure condition Eq. (2.2). We then randomly affect to each pixel in state u an explanatory variable d -tuple $\mathbf{y}$ according to this chosen probability density²⁹; the collection of d -tuples $\mathbf{Y}$ will be used as calibration points in explanatory variable space. The distribution $\rho(\mathbf{y}|u, v)$ is also exactly specified and obeys the closure condition Eq. (2.3). Finally, $P^*(v|u)$ is chosen in compliance with the small change hypothesis. We compute the function $P(v|u, \mathbf{y})$ (considered as a function of $\mathbf{y}$) from the inputs $\rho(\mathbf{y}|u)$, $\rho(\mathbf{y}|u, v)$ and $P^*(v|u)$ with the help of Bayes rule Eq. (2.1), and finally allocate to each pixel a final LUC state v according to the probability distribution $P(v|u, \mathbf{y})$ (we use a standard algorithm to this effect, described in appendix 2.C). We thus obtain a LUC map at time t_1 . The LUC maps at time t_0 and t_1 and the explanatory variables $\mathbf{Y}$ complete the elaboration of our calibration data.

We now turn to the comparison part of the comparison process, which essentially consists in the reconstruction of estimates of the inputs $\rho(\mathbf{z}|u)$ and $\rho(\mathbf{z}|u, v)$ from these data, in order to compare it to the actual input we used at the onset of the whole process. The comparison focuses on the distribution $\rho(\mathbf{z}|u, v)$ and its estimate, $\tilde{\rho}(\mathbf{z}|u, v)$, as this is the most important quantity in the whole procedure (it is the quantity most sensitive to possible undersampling, and the most important input in Bayes rule, as it characterizes the dependence of the transition probability on explanatory variables). In practice, we choose a new LUC map at time t_s (which again, for simplicity, contains the

²⁸Both assumptions implicitly assume that time-steps are “small enough” (how small requires to be defined). More sophisticated assumptions can be used if this is not the case. We will not discuss these points here, but will return to this time-step issue in our allocation paper.

²⁹We use only normal distributions or sums of normal distributions, and this random affectation relies on existing dedicated Python algorithms.

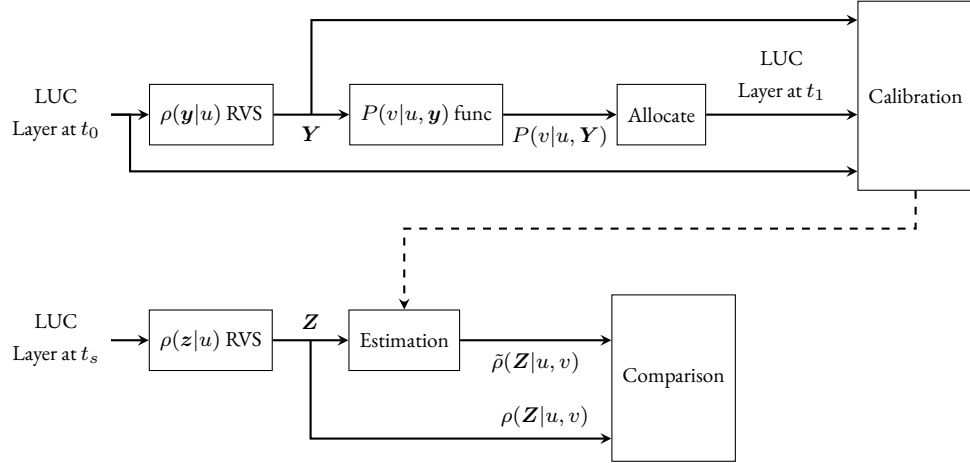


FIGURE 2.8: Comparison method of estimation models for controlled settings with artificial data. The Random Variable Sampling (RVS) process affects to each pixel in state u an explanatory variable d -tuple according the chosen probability distribution $\rho(\mathbf{z}|u)$.

same number n of pixels in state u) and affect to each pixel in state u an explanatory variable d -tuple $\mathbf{z}$ according to $\rho(\mathbf{z}|u)$; the resulting collection $\mathbf{Z}$. Next, from our calibration data and efficient KDE estimation procedure, we produce the estimated probability density $\tilde{\rho}(\mathbf{z}|u, v)$ on the estimation points. As we do know the actual probability density on this estimation set $\rho(\mathbf{z}|u, v)$, we are in position to compare the actual and estimated probability densities. The comparison criteria we use are described in the next subsection.

2.6.2. COMPARISON CRITERIA

The quantification of the estimation error can be performed in various ways. Here, we will only use two of them: a measure of the mean absolute error, and comparison of calibrated probability distributions along one-dimensional cuts. The mean absolute error magnitude ε is given by

$$\varepsilon = \frac{1}{n\rho(\mathbf{z}|u, v)} \sum_{i=1}^n |\rho(\mathbf{z}_i|u, v) - \tilde{\rho}(\mathbf{z}_i|u, v)|, \quad (2.28)$$

where the sum runs on the n pixels in initial state u at t_s , and associated explanatory variable values. This allows us to measure the absolute difference between the exact prediction and the estimate. The absolute error ε is normalized by dividing by the mean value $\overline{\rho(\mathbf{z}|u, v)}$ to ensure that the error obtained for different numbers of explanatory variables in section 2.7 are comparable.

One-dimensional cuts allow us to compare the exact probability and the estimated probability by plotting the estimation on a one-dimensional line in the explanatory variable space.

These two methods of comparison may seem insufficient at first sight. Still, as will become apparent right below when applied to a specific illustration data set in section 2.7, we observe important variations between the models we have tested on the basis of these two criteria. Finally, we have also compared the computation time of these models on the same problem in order to estimate their numerical efficiency. Efficient algorithms are important not only when implementing a LUCC model on specific case studies when the study area is large and well-resolved, but also in order to carry out sensitivity analyses on the results by varying the problem parameters. Inefficient algorithmic implementations make such analyses prohibitive, in spite of their interest in assessing the robustness of the obtained results.

2.7. ILLUSTRATIVE CONTROLLED PROBLEM

Here we focus on a problem where all data are artificially created as explained in section 2.6. We use this problem to compare a number of LUCC modeling environments, namely, Dinamica EGO, CLUMondo, Idrisi LCM and our own allocation-estimation procedure, which is part of our complete LUCC program dubbed CLUMPY (Comprehensive Land Use [and cover] Model in PYthon, see appendix 5). A single case study cannot fully judge the quality of a model. However, we have chosen a simple and general case for which the number of pixels and the number of explanatory variables can be easily parameterized (section 2.7.1). Section 2.7.2 then finds the relevant probability distributions produced by these LUCC modeling environments on this problem. We then conduct an extensive comparison of the behavior of the different methods studied by varying the problem parameters, in particular the initial number of pixels (section 2.7.3).

2.7.1. PROBLEM DEFINITION

The main parameters of our problem are the number of pixels n and the number of explanatory variables d , of unspecified nature; n is varied from 10^5 to 10^7 and d from 1 to 8 in order to compare the various methods that have been tested under more and more demanding conditions. Very large problems ($n \gg 10^8$) have not been explored because some of the existing methods do not converge in 24h or even fail to converge, but our own method still performs as well and fast (within minutes) for such large problems. In principle, it is possible to circumvent such convergence failures by sub-sampling the data, especially for

parametric methods such as the logistic regression of CLUMondo (regressions on millions of data points are clearly unnecessary and even counter-productive). But this requires some extra pre- (and possibly post-) processing steps, and gives less precise results when non-parametric methods such as ours are used. It is therefore preferable to avoid this, if feasible. We did not try to implement such a sub-sampling for any of the calibration methods tested in order to compare them under the same conditions.

Our initial LUC map at time t_0 is uniform and has a single LUC state $u = 1$ as our method treats initial LUC states independently of one another. The probability density $\rho(\mathbf{z}|u)$ is specified by

$$\rho(\mathbf{z}|u = 1) = \mathcal{N}_{\boldsymbol{\omega}, \boldsymbol{\Sigma}}(\mathbf{z}) \quad (2.29)$$

where $\mathcal{N}$ is a multidimensional normal distribution; $\boldsymbol{\omega}$ is the vector of means along each dimension, of size d , and $\boldsymbol{\Sigma}$ is the covariance matrix of the distribution, of size $(d \times d)$, and whose elements are equal to 1.96 on the diagonal and to 0.59 everywhere else. In line with our discussion on the whitening transformation (section 2.4.3), the vector of means $\boldsymbol{\omega} = \mathbf{0}$, so that our explanatory variables are centered but are correlated to each other. We randomly draw the explanatory variables associated to our n pixels according to this probability density.

Next, we define the probability density $\rho(\mathbf{z}|u, v)$:

$$\rho(\mathbf{z}|u, v) = \frac{1}{2} \mathcal{N}_{\boldsymbol{\omega}^-, \boldsymbol{\Sigma}'}(\mathbf{z}) + \frac{1}{2} \mathcal{N}_{\boldsymbol{\omega}^+, \boldsymbol{\Sigma}'}(\mathbf{z}), \quad (2.30)$$

where all the components of the d -dimensional mean vectors $\boldsymbol{\omega}^-$ and $\boldsymbol{\omega}^+$ are equal 1.0 and -1.0 , respectively, and $\boldsymbol{\Sigma}'$ is the covariance matrix whose elements are equal to 0.49 on the diagonal and to 0.245 everywhere else.

Finally, we set $P^*(u|v) = 0.002$. With these definitions, the transition probability $P(v|u, \mathbf{z})$ is fully specified from Bayes rule, Eq (2.1). This allows us in turn to distribute our n calibration pixels between states v and u with the help of a simple multinomial sampling algorithm (see Appendix 2.C). For example, with $d = 2$ (and associated explanatory variables z_0 and z_1) and $n = 800,000$, we have produced in this way a total of 1607 pixels that have transited from u to v , which is consistent with the overall transition probability $P^*(v|u) = 0.002$, considering the statistical noise due to finite numbers³⁰.

Fig. 2.9 represents a 10,000 subset of the calibration pixels in explanatory variable space. The red and cyan color lines are the chosen cuts that will be

³⁰We do not exactly enforce the expected number of transited pixels in the present analysis.

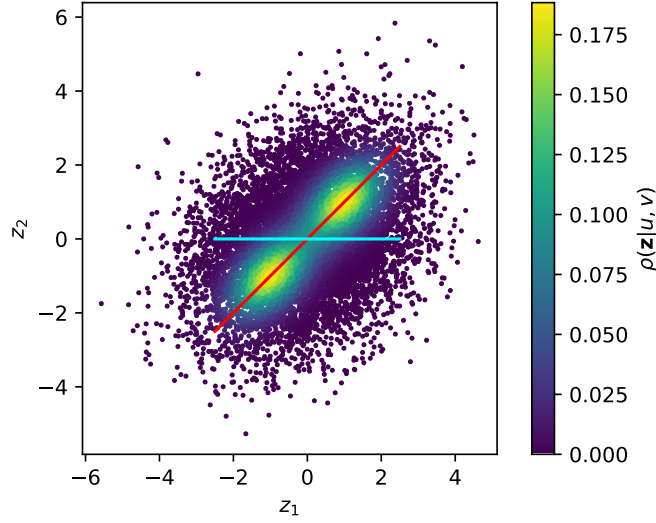


FIGURE 2.9: Representation of the probability distribution of Eq. (2.29) for $d = 2$. A subset of 10,000 pixels has been randomly drawn to produce this graph. The color code corresponds to the probability density Eq. (2.30), considered as a function of $\mathbf{z}$. The red and cyan color lines refer to the one-dimensional cuts that will be used later on to compare the performances of the various LUC modeling environments.

used for the one-dimensional cut comparisons of the original and reconstructed probability density distribution along these cuts for the various modeling environments we have chosen to assess, in order to have a more precise grasp of the accuracy of their calibration procedures and of their performance compared to ours.

In line with the structure displayed in Fig. 2.1, estimation is performed on a second sample of n pixels in initial state u . We again randomly draw their associated d -tuples of explanatory variable values according to Eq. (2.29), calculate the exact probability density $\rho(\mathbf{z}|u, v)$ on this set from Eq. (2.30) and use our calibration data and efficient KDE estimation procedure to produce $\tilde{\rho}(\mathbf{z}|u, v)$ on the same set. The use of a different sample allows us to test the difference between the calibrated and exact transition probability on points in explanatory space that have not been used for calibration, and therefore to test the quality of the interpolation of each method.

We generate data for a set of parameters n and d varying from 10^5 to 10^7 and d from 1 to 8, respectively.

2.7.2. TRANSITION PROBABILITY ESTIMATION

We compare the probability density $\tilde{\rho}(\mathbf{z}|u, v)$ on the second set defined in the previous section, using a panel of calibration methods: Dinamica EGO weights of evidence (Dinamica EGO WE), Idrisi LCM Multilayer Perceptron

(LCM MLP), CluMondo Logistic Regression (CluMondo LR) and our new efficient Kernel Density Estimation (Bayes-eKDE) algorithm. LCM also provides a logistic regression algorithm that we prefer to avoid because of the generic poor performance of this method (the only one proposed by CLUMondo). A poor performance is expected for all methods enforcing a specific *a priori* functional form for the probability distributions of any problem (parametric methods), as this is intrinsically less precise than non-parametric methods. The main advantage of parametric methods is that they can be used in contexts where non-parametric ones would fail due to the scarcity of data.

Dinamica EGO starts by binning explanatory variables; this binning process involves a number of parameters, that are usually left to their default choice by the user. In practice, we improved this default choice by setting the increment parameter to the Terrel bandwidth (section 2.4.5) multiplied by the variance of the explanatory variable (see Appendix 2.D.1). We did find that the choice of this parameter has a substantial influence on the quality of the results, and without this rule of thumb, Dinamica EGO weights of evidence performs substantially less well than reported here.

For LCM MLP, we made two different series of runs. The first one kept the default parameters; these are specified in Appendix 2.D.2. The other one aimed at improving the default behavior. In particular, the default number of neuron³¹ used in the algorithm is 3. However, it is well-known in the machine learning literature that larger numbers do in general perform much better. The alternative set of parameters we have chosen for our second set of runs, noted MLP ref. in what follows, is also specified in Appendix 2.D.2. This second parameterization with 10 neurons in the hidden layer leads to significant improvements in the performances of Idrisi LCM MLP.

CLUMondo parameters are those defined by default and are given in Appendix 2.D.3.

It is sometimes necessary to modify the quantities returned by the different models, as these may not be immediately comparable to the actual probability distributions. More precisely:

- Dinamica EGO does not provide an estimate $\tilde{\rho}(\mathbf{z}|u, v)$. Instead, from its weights of evidence, one can obtain $C\tilde{\rho}(\mathbf{z}|u, v)/\tilde{\rho}(\mathbf{z}|u)$ where C is a constant. From this and from the known $\rho(\mathbf{z}|u)$, one can therefore compute $C\tilde{\rho}(\mathbf{z}|u, v)\rho(\mathbf{z}|u)/\tilde{\rho}(\mathbf{z}|u)$, which is an estimate of $C\rho(\mathbf{z}|u, v)$. This estimate is then normalized to finally obtain the estimate of $\rho(\mathbf{z}|u, v)$.

³¹In a Multi-Layer Perceptron, one can choose to parameterize a number of neurons distributed in one or more hidden layers. By default, we consider a single hidden layer (LCM choice) and the number of neurons indicated in the text corresponds to the number of neurons in this layer.

- CLUMondo's logic is the farthest from our own of all models we have tested. CLUMondo transition probability is made of three contributions, and we have isolated the one produced from explanatory variables, which is the contribution corresponding to our own calibration-estimation logic. This contribution to the transition probability to state v (for whatever state u) is proportional to a quantity that, in our notations, is a parametric logistic regression fit of $\rho(\mathbf{z}|v)$ at $t = t_1$ (i.e., on our second calibration map). Because we focus on a single initial state u , this quantity is thus also a parametric evaluation of $\rho(\mathbf{z}|u, v)$ up to a normalization factor.
- LCM's potential of change is directly proportional to $\tilde{\rho}(\mathbf{z}|u, v)$ so that it only needs to be normalized. The only difficulty in the use of LCM is to enforce a map that differs from the calibration maps at t_0 and t_1 for the estimation procedure. This is done by a manual substitution of the correct map at the right point in LCM's sequence of operations.

Based on this understanding of these modeling environment, the only operation that is finally required to obtain their estimates of $\rho(\mathbf{z}|u, v)$ is a normalization, in order to enforce Eq. (2.3). This is done along the lines of our Bayes adjustment algorithm (section 2.5.3), *i.e.* we rescale the quantities $\tilde{\rho}^i$ we extract from these software for each pixel i in the following way:

$$\tilde{\rho}(\mathbf{z}^i|u, v) = \frac{\tilde{\rho}^i}{\frac{1}{n} \sum_{i=1}^n \tilde{\rho}^i}, \quad (2.31)$$

Once this change of normalization is made, the estimated probability distributions can be compared to the exact probability distribution $\rho(\mathbf{z}|u, v)$.

2.7.3. RESULTS AND DISCUSSION

The mean absolute error is calculated from Eq. (2.28), for various numbers d of explanatory variables (1 to 8), as a function of the number of pixels n . The results are shown on Fig. 2.10 for all four calibration methods. Note that the axes are represented in logarithmic scale.

The lower the value of mean absolute error ε , the more accurate the model estimates of the probability distribution $\rho(\mathbf{z}|u, v)$. Increasing the number of pixels increases the precision of the estimation (at constant d , a larger set of statistical events n reduces the noise level). Conversely, increasing the number of explanatory variables decreases it (for a constant number of pixels n , the studied space is larger if we increase d).

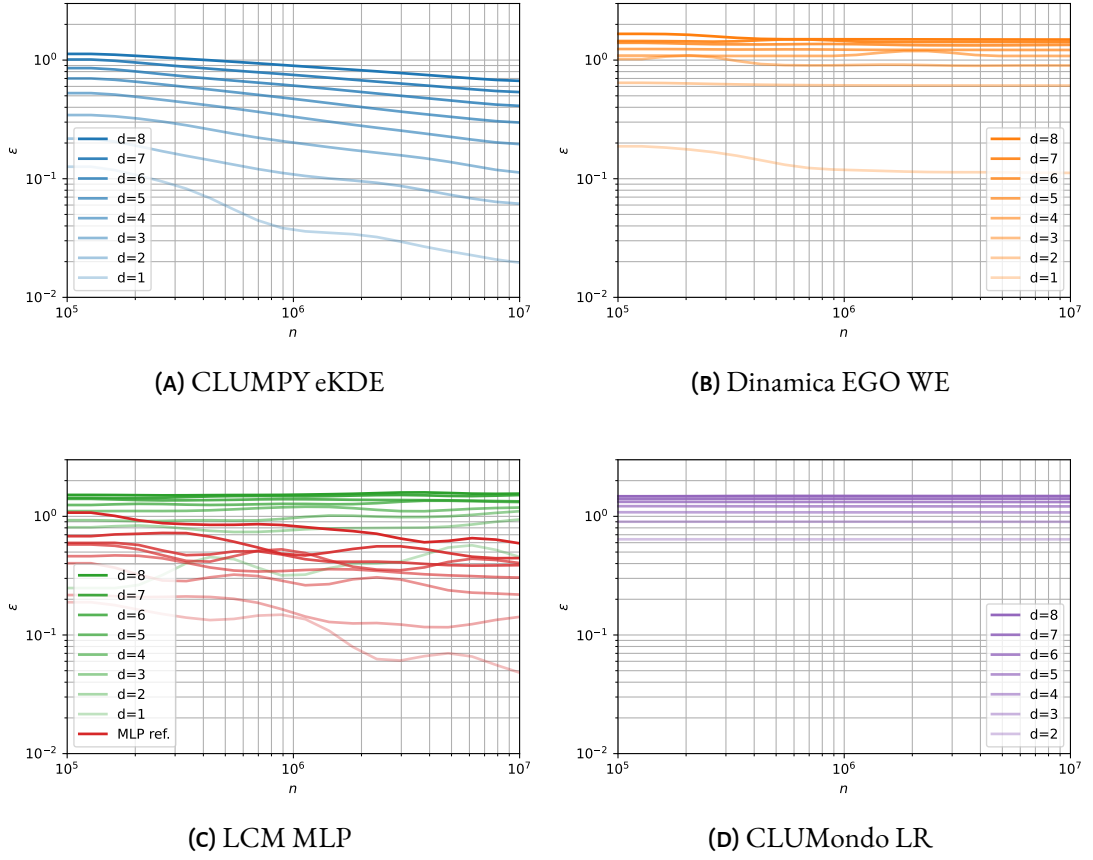


FIGURE 2.10: Mean Absolute Error (ε), as a function of the number of pixels n for various numbers of explanatory variables d , for the four calibration methods tested: (a) CLUMPY efficient Kernel Density Estimation, (b) Dinamica EGO Weights of Evidence, (c) Idrisi LCM Multilayer Perceptron and (d) CLUMondo Logistic Regression. In addition to LCM MLP, we also show in red lines the results of a MLP algorithm with judiciously chosen parameters for reference (MLP ref.). Note that both axis are represented in logarithmic scale, which tends to compress the differences of performance.

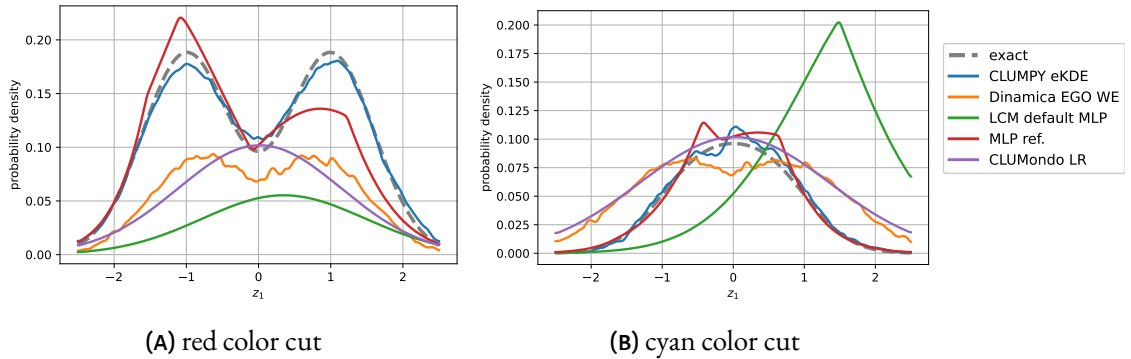
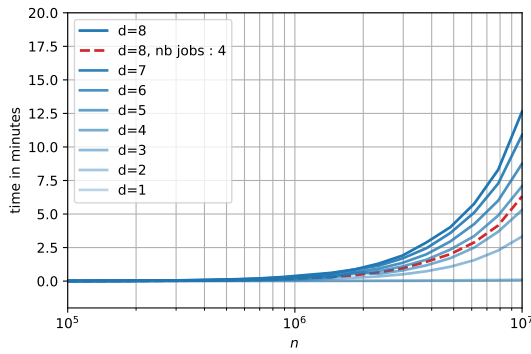
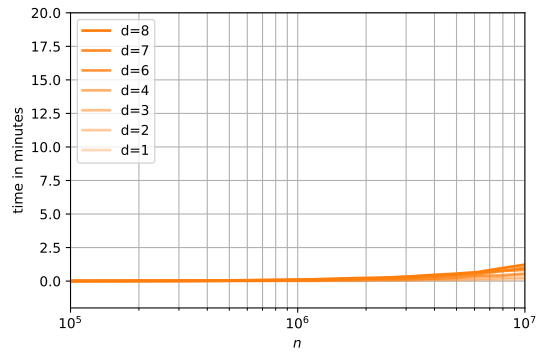


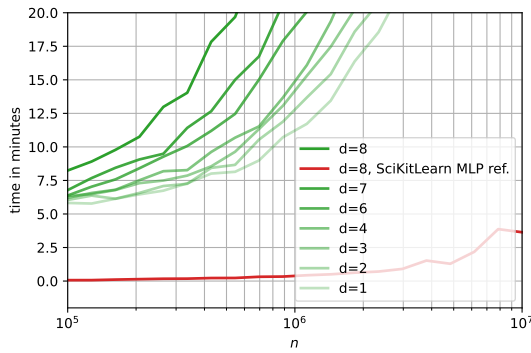
FIGURE 2.11: Probability density estimation along the two one-dimensional lines (cuts) shown in red and cyan color on Fig. 2.9 with parameters $n = 800,000$ and $d = 2$, for four different calibration methods: CLUMPY efficient Kernel Density Estimation, Dinamica EGO Weights of Evidence, Idrisi LCM Multilayer Perceptron, Multilayer Perceptron with judiciously chosen parameters for reference (MLP ref.) and CLUMondo Logistic Regression.



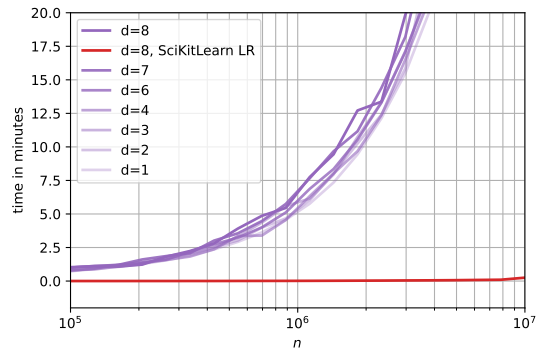
(A) CLUMPY eKDE



(B) Dinamica EGO WE



(C) LCM MLP



(D) CLUMondo LR

FIGURE 2.12: Computation times of different calibration methods as a function of the number of pixels n , for various numbers of explanatory variables d : CLUMPY efficient Kernel Density Estimation, Dinamica EGO Weights of Evidence, Idrisi LCM Multilayer Perceptron (peaks outside the window at 5h20 for $d = 8$) and CLUMondo Logistic Regression (peaks outside the window at 1h for $d = 8$). Also, the computation time of CLUMPY KDE with 4 parallel jobs for $d = 8$ is represented. Note that the abscissa is represented in logarithmic scale.

A more direct comparison of performance is obtained on Fig. 2.11, where the dependence of the transition probability on z along two one-dimensional cuts in z space is shown for $n = 800,000$ and $d = 2$; these cuts are those shown on Fig. 2.9. The exact transition probability is represented by the grey dashed line.

Finally, computation times (minutes) are shown for each calibration method on Fig. 2.12 (note that the abscissa is represented in logarithmic scale).

More specific comments on the various methods are provided in the remainder of this section.

2.7.3.1. CLUMPY EFFICIENT KERNEL DENSITY ESTIMATION

CLUMPY eKDE performs better to significantly better than all other methods. Interestingly, our method keeps improving for increasing n at a given d , while the optimized LCM MLP seems to saturate for a few 10^6 pixels (this however may be an effect of the choice of the number of nodes in this method, or a temporary “plateau” in n).

Also, the computation time is of the order of a few minutes if we stick to a reasonable number of explanatory variables (lower or equal to 6) and this remark is even more valid if we parallelize the computation (Fig. 2.12a).

2.7.3.2. IDRISI LCM MLP

The multilayer perceptron algorithm has interesting characteristics if one does not stick to the default parameters recommended by the developers.

Let us first focus on the results obtained for the default parameters, shown in green on figures 2.10c and 2.11. Increasing the number of dimensions reduces the accuracy as expected. However, increasing the number of pixels does not bring any significant improvement (Fig. 2.10c). Moreover, the estimator is unable to produce an acceptable approximation of the exact probability (Fig. 2.11). Finally, the computation time may become prohibitive: it peaks at 5h20 with 8 explanatory variables and 10 million pixels (out of range in Fig. 2.12c).

In red, we represent the results obtained with the same algorithm (multilayer perceptron) but with better chosen parameters (see Appendix 2.D for a detailed description of the default and non-default parameter choices). We have used the Python open-source package `SCIKIT-LEARN` to produce the MLP ref. result. The improvement shown by the red curve in Fig. 2.10c and 2.11 was obtained by increasing the number of neurons (nodes) from 3 (LCM choice) to 10 (within a single hidden layer in both cases). The related average absolute deviation is even comparable to that returned by CLUMPY (Fig. 2.10a). Moreover, Fig. 2.11 shows that the result is much more precise than all other tested models,

except our own CLUMPY eKDE algorithm, which is somewhat more precise (although increasing further the number of nodes in the hidden layer reduces the gap in performance). Finally, the MLP algorithm implemented by SCIKIT-LEARN leads to reasonable computation times, similar to those of CLUMPY (Fig. 2.12c).

This being said, the MLP algorithm has a significant disadvantage: as for nearly all other artificial intelligence algorithms relying on neuron networks, it is impossible to understand and analyze how the model calibration is actually performed. Also, as will be shown in our dedicated allocation paper, LCM incorrectly uses $\rho(z|u, v)$ for $P(v|u, z)$ for allocation. Still, considering the interest of a well-designed MLP algorithm, it has been implemented in our package as an alternative to our own eKDE algorithm.

2.7.3.3. DINAMICA EGO WEIGHT OF EVIDENCE (WE)

Dinamica EGO WE performs rather poorly, both on mean absolute error and clearly so on one-dimensional cuts. However, we point out that it is the next best alternative after CLUMPY eKDE and our optimized version of the MLP algorithm. A univariate problem ($d = 1$) may be the exception (Fig. 2.10b). Fig. 2.11b highlights Dinamica EGO's choice to consider the explanatory variables as independent: this leads to the existence of two maxima, whereas only one is present in this particular cut.

The speed of Dinamica EGO calibration method is worth pointing out (see Fig. 2.12b). This is due to the hypothesis of statistical independence of the variables, so that the calibration procedure amounts to d times a one-dimensional calibration; conversely, CLUMPY eKDE refrains from making such an assumption, which makes the calibration problem truly d -dimensional and significantly more demanding numerically. Dinamica EGO algorithmic efficiency comes out at the expense of the precision of the estimation.

It is also worth remembering that the parameterization adopted for Dinamica EGO is not the default one, but has been set according to a rule of thumb inspired by the KDE bandwidth (see Appendix 2.D.1). Dinamica EGO does not provide any indication as to how the binning parameters should be chosen (i.e., the parameters used in the method specifying the bin size), while we found that their effect on the quality of the results was substantial.

2.7.3.4. CLUMONDO LOGISTIC REGRESSION

CluMondo LR uses a parametric estimator based on logistic regressions, and is therefore strongly constrained. Quite clearly CLUMondo performs extremely poorly, on both criteria (Fig. 2.10d and Fig. 2.11). CLUMondo returns

a quasi-constant probability distribution on the whole space of the explanatory variables. However, as indicated in section 2.7.2, we multiply the result obtained by the probability density of the explanatory variables $\rho(\mathbf{z}|u)$ in order to produce *in fine* $\rho(\mathbf{z}|u, v)$ and this is why the shape of the curve obtained corresponds more or less to $\rho(\mathbf{z}|u)$.

Dinamica EGO WE, LCM MLP as well as our CLUMPY KDE algorithm are non-parametric models and are more adapted to estimate a large panel of probability density functional forms. As pointed out earlier, the choice of a parametric method may be adapted when data are scarce, and indeed, CLUMondo makes use of a single land use map to identify suitable transition sites. For all practical purposes, CLUMondo uses the resulting pixel suitability scores as transition probabilities. As by definition no transition can be observed with a single map, this requires an extra assumption. Namely, in our language, $\rho(\mathbf{z}|v)$ and $\rho(\mathbf{z}|u, v)$ are proportional, and the transition probability $P(v|u, \mathbf{z})$ is proportional to $\rho(\mathbf{z}|v)$ for all initial states u .

However, it is clear from Bayes rule [Eq. (2.1)] that this cannot be true, as this relation implies that $P(v|u, \mathbf{z}) \propto \rho(\mathbf{z}|u, v)/\rho(\mathbf{z}|u)$. Even for single initial state u for which $\rho(\mathbf{z}|u, v) = \rho(\mathbf{z}|v)$, the probability is proportional to $\rho(\mathbf{z}|v)$ only if $\rho(\mathbf{z}|u)$ is uniform. These are very restrictive conditions. Finally, the parametric method of estimation of $\rho(\mathbf{z}|u, v) = \rho(\mathbf{z}|v)$ (for a unique u) requires that the probability density is maximum at $\mathbf{z} = 0$. This is also violated by our choice, Eq. (2.30). Such a violation may be justified. Consider for example the transition between different species of plants with climate change. In a mountainous region, the change will clearly depend on elevation for any particular species, and has no reason to be maximum at the lowest elevation.

Finally, the computation time is d independent but surprisingly high for a parametric method with a 1h running time for $d = 8$ and $n = 10^7$ (outside the window in Fig. 2.12d). For comparison purposes, we represent in the same figure the time needed for $d = 8$ using the much more efficient python package `SCIKITLEARN`.

2.8. CONCLUSION

This article constitutes the first part of an investigation dedicated to the analysis of the conceptual errors, inaccuracies and inefficiencies of existing pattern-based LUCC software and models. Our overall objective is to propose systematic improvements in both calibration and allocation (the main building blocks in this LUCC approach), based on mathematically rigorous analyses of all as-

pects of the problem, and on efficient algorithmic implementations of the results of these analyses. This in turn will allow us to understand and reduce in a systematic way the difference of results obtained by different modeling environments on any given problem and set of data, a point discussed in the introduction. This series also describes in detail our LUCC model named CLUMPY (Comprehensive Land Use [and cover] Model in PYthon).

We focus here on the calibration of transition probabilities for this type of LUCC models. We introduce a new calibration method based on Bayes rule Eq. (2.1) and on an adaptation of a well-known machine learning method — kernel density estimation (KDE) — referred to as eKDE in the bulk of this paper (the full calibration-estimation method is abbreviated as Bayes-eKDE). We produced a series of artificial case studies from completely specified probability distributions, in order to compare the performance of this method to the calibration methods implemented in existing software, namely Dinamica EGO, CLUMondo and Idrisi LCM MLP (Multi-Layer Perceptron).

Our calibration-estimation algorithm involves a rather complex series of sub-tasks (whitening transformation, section 2.4.3; boundary bias correction, section 2.4.4; bandwidth selection, section 2.4.5). Moreover, its implementation requires other procedures to optimize the execution time (efficient KDE method, section 2.5.1; parallelization, section 2.5.1.3) and to avoid numerical outliers (Bayes adjustment process, section 2.5.3). The architecture of these steps is presented in section 2.5.2. sections 2.4 and 2.5 present a self-contained and exhaustive description of this new method (in the LUCC modeling context) for estimating transition probabilities, summarized in Fig. 2.2.

We also define a new evaluation procedure for calibration methods (see section 2.6) where all probabilities involved are known exactly. This context allows us to implement objective evaluation methods of the accuracy of the transition probabilities estimated with existing calibration procedures, and compare different methods of calibration. For now, only simple comparison criteria are introduced (mean absolute error, one-dimensional cut and computation time) but the proposed evaluation framework is compatible with other comparison methods.

This evaluation procedure is not implemented in physical space, but in explanatory variable space. This is at variance with common validation practices in quantitative geography in general and in pattern-based LUCC modeling in particular. This choice has been motivated in the discussion of section 2.2, but the main points may be worth rephrasing here. Actual spatial locations of allocations in this type of modeling is statistical by design, i.e., many statistically equivalent sites may potentially be chosen for the same LUC state change during a

time interval, but only a few may actually be observed to undergo such a change. The same argument goes for calibration: many other sites for state change could have been chosen instead of the observed ones, with equivalent statistical properties in terms of explanatory variables distribution. The only way to circumvent this feature would be to have large amounts of state change during a reference period (so that the statistical spread is reduced by necessity). There is therefore little reason to expect spatial location to be an appropriate measure of LUCC modeling framework relevance, although this is a desirable property of any LUCC framework instantiation in any case study. On the other hand, producing a robust evaluation of distribution in explanatory space is mandatory, as this is precisely what controls future allocation in a simulation.

This evaluation makes use of a completely controlled test problem (section 2.7). This relies on data created from completely specified transition probabilities. In this setting, we have varied both the total number of pixels (10^4 to 10^6) and explanatory variables (1 to 8) in order to perform our performance comparison under a wide range of conditions. We compare in this way exiting calibration procedures (Idrisi LCM MLP, Dinamica EGO and CLUMondo) to the CLUMPY eKDE method introduced in this work (section 2.7.3). The probability distributions estimated by our CLUMPY eKDE algorithm turn out to be substantially more accurate than the alternative we have tested, whatever the number of explanatory variables and the number of pixels. Also, it appears to be efficient in term of running time, especially if we increase the number of cores dedicated to this calculation; however, we found that an appropriate parameter choice for the MLP calibration method leads to satisfactory results as well. It would be useful to reproduce this evaluation on a larger and more varied number of test problems, but we are confident that our conclusions would stand.

We focused on the accuracy of the estimation and the computation time of the different models. We did not linger over the user experience provided. Without going into details, the configuration of a LUCC case study is often either tedious or constrained in existing software. CLUMondo offers very little flexibility, the default parameters of Idrisi LCM MLP are poorly chosen (section 2.7.3.2) and Dinamica EGO, although very versatile, gives very little indication on the choice of some parameters that are critical to the quality of the results; the versatility comes at the expense of a steeper learning curve. CLUMPY has its own learning curve. It is programmed in a very common language — Python — but programming is an issue in the LUCC modeling community, where most users have little programming experience. We are presently planning to circumvent this problem through the creation of a simple Graphic User

Interface (GUI) for our complete modeling environment CLUMPY. Moreover, there is only one parameter for the transition probability estimation: q , the small-scale binning parameter of our efficient KDE implementation (section 2.5.1.1). From our tests, the recommended default value of this parameter seems to be appropriate in most cases.

This work exemplifies the strategy we adopt in our work on LUCC modeling. On the one hand, we aim at a higher level of mathematical and algorithmic rigor. On the other hand, we devise specifically tailored tests with the aim of achieving a more precise and in-depth comparison of computational performances, conceptual and algorithmic correctness and accuracy than existing methods. In the present work, for example, focusing on comparison in explanatory variable space instead of physical space while using artificial data allows us to benefit from a complete control on the comparisons performed, and to be much more discriminant on this front.

APPENDICES TO CHAPTER 2

2.A. A MORE FORMAL DEFINITION OF PROBABILITY DENSITIES

Continuous variables probabilities are only properly defined on finite intervals of the variables values. Formally, these probabilities are obtained from a so-called probability density that is integrated over these intervals to obtain the required probability. For a univariate continuous random variable z , one can show that there exists a unique function $\rho(z)$ called probability density such that the probability $P(z, \Delta z)$ of obtaining the value z within the range $[z, z + \Delta z]$ is given by

$$P(z, \Delta z) = \int_z^{z+\Delta z} \rho(z) dz. \quad (2.A.1)$$

Equivalently, the probability density is defined by an appropriate partial derivative of P :

$$\rho(z) = \frac{\partial P}{\partial \Delta z}, \quad (2.A.2)$$

where the partial derivative is evaluated at $\Delta z = 0$.

This definition is extended to multivariate random variables $\mathbf{z} = (z_1, z_2, \dots, z_d)$ through the use of a multivariable probability density $\rho(\mathbf{z})$ and the probability is obtained by a multiple integral:

$$P(\mathbf{z}, \Delta \mathbf{z}) = \int_{z_1}^{z_1+\Delta z_1} dz_1 \int_{z_2}^{z_2+\Delta z_2} dz_2 \dots \int_{z_d}^{z_d+\Delta z_d} dz_d \rho(\mathbf{z}). \quad (2.A.3)$$

The derivative expression of the probability density is generalized in the follow-

ing way:

$$\rho(\mathbf{z}) = \lim_{\Delta z_i \rightarrow 0} \frac{P(\mathbf{z}, \Delta \mathbf{z})}{\Delta z_1 \Delta z_2 \dots \Delta z_d}. \quad (2.A.4)$$

2.B. EXTENDING THE EKDE CALIBRATION PROCEDURE TO A MIX OF DISCRETE AND CONTINUOUS EXPLANATORY VARIABLES

We distinguish continuous explanatory variables z_k and discrete ones $\gamma_{k'}$ ($1 \leq k \leq d$ and $1 \leq k' \leq d'$). A range $\Gamma_{k'}$ of possible integer values is associated to each explanatory variable $\gamma_{k'}$. From these quantities, one can form an arbitrary t -uple of discrete variables $\gamma = (\gamma_1, \gamma_2, \dots, \gamma_{d'})$.

The probability distribution of the discrete variables $P(\gamma|u)$ and $P(\gamma|u, v)$ are straightforwardly obtained from pixel counting on calibration maps. The number of pixels with discrete explanatory variables γ and initial state u at t_0 is noted $n_{\gamma, u}$ while $n_{\gamma, u, v}$ is the number of these pixels having undergone a LUC state transition $u \rightarrow v$ between t_0 and t_1 in the calibration maps. Recalling that n_{I_v} is the number of pixels in state u at t_0 and state v at t_1 and defining n_I as the number of pixels in state u at t_0 , one has

$$P(\gamma|u, v) = \frac{n_{\gamma, u, v}}{n_{I_v}}, \quad (2.B.1)$$

$$P(\gamma|u) = \frac{n_{\gamma, u}}{n_I}. \quad (2.B.2)$$

Let us now consider a mixed set of discrete and continuous explanatory variables. One has (Bayes rule)

$$\begin{aligned} P(v|u, \mathbf{z}, \gamma) &= P^*(v|u) \frac{\rho(\mathbf{z}, \gamma|u, v)}{\rho(\mathbf{z}, \gamma|u)} \\ &= P^*(v|u) \times \frac{P(\gamma|u, v)}{P(\gamma|u)} \times \frac{\rho(\mathbf{z}|u, v, \gamma)}{\rho(\mathbf{z}|u, \gamma)}. \end{aligned} \quad (2.B.3)$$

The first factor is provided by the user (or from calibration data if one is only in-

terested in extrapolations of past trends), the second factor is calibrated as specified above. Therefore one just has to calibrate $\rho(\mathbf{z}|u, v, \gamma)$ and $\rho(\mathbf{z}|u, \gamma)$.

For completeness let us indicate how the closure relations are modified when discrete variables are taken into account for the quantities that are calibrated in a mixed continuous/discrete variables context:

$$\int_{\mathbf{z} \in \mathbb{R}^d} \tilde{\rho}(\mathbf{z}|u, \gamma) d\mathbf{z} = 1, \quad (2.B.4)$$

$$\int_{\mathbf{z} \in \mathbb{R}^d} \tilde{\rho}(\mathbf{z}|u, v, \gamma) d\mathbf{z} = 1, \quad (2.B.5)$$

$$\begin{aligned} \sum_{\gamma} \tilde{P}(\gamma|u) \int_{\mathbf{z} \in \mathbb{R}^d} \tilde{P}(v|u, \mathbf{z}, \gamma) \times \tilde{\rho}(\mathbf{z}|u, \gamma) d\mathbf{z} \\ = \int_{\mathbf{z} \in \mathbb{R}^d} \tilde{\rho}(v, \mathbf{z}|u) d\mathbf{z} = P^*(v|u). \end{aligned} \quad (2.B.6)$$

For a given transition $u \rightarrow v$, taking into account discrete variables reduces to applying our eKDE method for each possible γ . Formally, this amounts to substitute $\rho(\mathbf{z}|u, v, \gamma)$ to $\rho(\mathbf{z}|u, v)$, $\rho(\mathbf{z}|u, \gamma)$ to $\rho(\mathbf{z}|u)$ and $P(v|u, \mathbf{z}, \gamma)$ to $P(v|u, \mathbf{z})$ (and similarly for distributions expressed in terms of $\mathbf{y}$ instead of $\mathbf{z}$) in all expressions from section 2.4 onward. The updated closure relations and updated Bayes rule above must be used where relevant instead of the original ones.

At first sight this calibration strategy seems computationally demanding. It is in fact much less demanding than making two eKDE calibrations for $d + d'$ continuous variables, although this is clearly more demanding than making two calibrations for d continuous variables.

2.C. MULTINOMIAL SAMPLING TEST ALGORITHM

The algorithm presented here is well-known (see, e.g., Knuth 1998, section 3.4.1). Let be a pixel and the set of transition probabilities $P(v|u, \mathbf{z})$ for all v . Let us define η_p as the cumulative sum of $P(v|u, \mathbf{z})$:

$$\eta_0 = 0, \quad (2.C.1)$$

$$\forall k, \quad \eta_k = \sum_{v \leq k} P(v|u, \mathbf{z}). \quad (2.C.2)$$

Then, a unique random float is sufficient to test simultaneously all possible transitions for this pixel. For any r , a random float in the half-open interval $[0, 1)$, one can find an index k such that $\eta_{k-1} \leq r < \eta_p$. The state $v = k$ in then

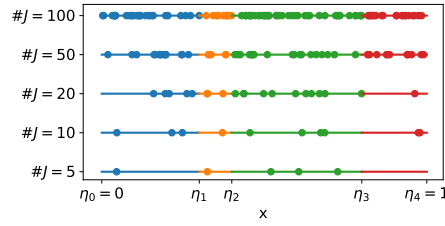


FIGURE 2.13: Outcome of the multinomial sampling test in a simple idealized setting with four possible final output states. These output states are sections of the unit interval. The length of these sections have been chosen arbitrarily. The distribution of random draws along the unit section are represented from bottom to top for an increasing number of draws (5, 10, 20, 50, 100 respectively).

affected to this pixel.

In plain words, we consider output states as sections of the unit interval with length equal to the various transition probabilities; the total length is then equal to one (as the transition $u \rightarrow u$ is included in the process). Intuitively, it is clear that distributing pixels at random on this unit interval will produce relative number of pixels in each possible final state proportional to the length of each interval, on average (Fig. 2.13), and therefore comply with $P(v|u^j, z^j)$, on average. This property will be formally demonstrated in our paper dedicated to allocation (Mazy and Longaretti, 2022b).

This test presents several advantages: all possible final states are tested at the same time, and the order of the sections in the unit interval is irrelevant.

2.D. MODELS PARAMETERS

In section 2.7, we use various LUC change models with the following parameters.

2.D.1. DINAMICA EGO

We used version 6.1.0 of the software. The calibration parameters of Dinamica EGO bear on the binning of the explanatory variables. For each LULC state transition and for each explanatory variable, four parameters are specified: *Increment*, *Minimum Delta*, *Maximum Delta* and *Tolerance angle*. These parameters ensure an independent and adaptive binning with a minimum number of pixels per bin and limited inter-bin statistical variations. However, Dinamica EGO does not provide any guidance on how to determine these parameters. We found a very large spread in the obtained results when these parameters are

modified. The most important parameter is the one named *increment*. As a rule of thumb to choose a more relevant bin size, we have used the size of a box kernel bandwidth defined for our own eKDE algorithm (section 2.4.5) but applied here to the size of the bins; this size was furthermore multiplied by the variance of the marginal distribution of the considered explanatory variable. The rationale of this correction is that the eKDE bandwidth applies to normalized explanatory variables (in particular the variance has been normalized to unity) and must therefore be rescaled to apply to the original data. The other parameters are left at their default value. Table 2.2 summarizes our settings. Our rule of thumb provides satisfactory results in the case study of section 2.7.

parameter	value
Increment	$C_{k,k}h_{\text{Terrel}}$
Minimum Delta	50
Maximum Delta	500000
Tolerance angle ($^{\circ}$)	5.0

TABLE 2.2: Dinamica EGO discretization parameters for the synthetic data of section 2.7.

2.D.2. IDRISI LCM

We have used a somewhat old Idrisi Selva license (version 17.00). Among all the calibration methods provided by LCM, we have only tested the Multilayer Perceptron (MLP) method. In section 2.7 and for comparison purposes, we have also used another MLP (MLP ref.) provided by the SciKitLearn Python package, with different parameters (see Tab. 2.3).

parameter	MLP value	MLP ref. value
Solver	SGD	Adam
Layer 1 nodes – Hidden layer sizes	(3,)	(10,)
Sample size	50%	10%
Use automatic training	yes	
Use dynamic learning rate – Learning rate strategy	dynamic – adaptative	constant
Sigmoid constant a	1.0	
Alpha		10^{-5}
Start learning rate	10^{-2}	10^{-3}
Max iterations	1000	200
Momentum factor	0.5	0.9
RMS – Tolerance	10^{-2}	10^{-4}
Accuracy Rate	100%	

TABLE 2.3: MLP parameters for the synthetic data of section 2.7. The terminology of Idrisi LCM MLP is noted in black and the corresponding terminology of the MLP classifier provided by SCIKITLEARN appears in grey. Some parameters are configurable only with one or the other environment.

2.D.3. CLUMONDO

We have used version 1.4.0 of the software. The sampling parameter is fixed to 30% of all observations. The *number of cells distance between samples* is fixed to 1 (because no spatial correlations are introduced in the case study – see section 2.7.1) with *no data values excluding* and *balanced sample* enabled.

2.D.4. CLUMPY

The eKDE parameter q (section 2.4.1) is fixed to 51. This is the only user-defined parameter in our procedure. This default value should be appropriate for most applications.

CHAPTER 3 | ALLOCATION REVISITED: FORMAL FOUNDATIONS AND BIAS IDENTIFICATION

Mazy, F.-R. and Longaretti, P.-Y. Towards a Generic Theoretical Framework for Pattern-Based LUCC Modeling. Allocation Revisited: Formal foundations and bias identification. *Submitted to Environmental Software and Modeling*, 2022b

ABSTRACT

Pattern-based land use and land cover change (LUCC) modeling is implemented in various software, such as the CLUE family, LCM or Dinamica EGO. These tools are now relatively mature, but their conceptual foundations are little discussed in the literature. In particular, these modeling environments exhibit substantially different behaviors for the same problem and the same calibration data.

These inter-model inconsistencies are revisited *ab initio*, focusing on allocation. The concept of allocation bias is defined and allows us to pinpoint a number of conceptual errors, biases and algorithmic inaccuracies in existing LUCC modeling environments. We describe error- and bias-free allocation methods, implemented in our own software CLUMPY, which is shown to significantly outperform existing software in terms of formal correctness and accuracy.

It is crucial for algorithmic performance evaluation that validation be performed in explanatory variable space, not on maps, contrarily to the most common practice in the field.

3.1. INTRODUCTION

Land Use and Land Cover Change (LUCC) is one of the major drivers of global change worldwide (Lambin and Geist, 2006), and LUCC models are now widespread to study these changes and their impacts (Agarwal et al., 2000; Verburg et al., 2006; Bielecka, 2020). A growing number of studies have been devoted to model comparison (Mas et al., 2014, 2022; Prestele et al., 2016; Alexander et al., 2017; García-Álvarez et al., 2022). Such studies question the robustness and soundness of the conclusions drawn from such models (Verburg et al., 2019).

The present work is part of a systematic analysis of the formal and algorithmic foundations of a specific category of models, namely, pattern-based LUCC models. Our objective is to identify the origin of the discrepancies in results pointed out in the literature just mentioned for a given problem and set of data, in order to reduce the uncertainty in model projections. Among the modeling environments implementing this approach, we focus on CLUE family (Verburg et al., 1999), the latest member being CLUMondo (van Asselen and Verburg, 2013; van Vliet et al., 2015), Dinamica EGO (Soares-Filho et al., 2002, 2013), or Idrisi LCM (Eastman et al., 1995).

Pattern-based LUCC models compute the evolution of a rasterized study area in a discrete time approach through a Markovian assumption. They are organized in a small number of modules, each of which is particularly critical for our purposes. Our own algorithms are organized in three modules: calibration, estimation and allocation. The calibration and estimation modules characterize the probability that a pixel changes state per time step. The allocation module effectively allocates state changes per time step.

The present paper focuses on allocation, while our previous papers dealt with model calibration and estimation (Mazy and Longaretti, 2022a), and selection of explanatory variables (Longaretti and Mazy, 2022). All these papers are self-contained and can be read independently of each other, but important methodological points are given in section 2 of Mazy and Longaretti (2022a) and not repeated here; these points do explain the rationale and relevance of the research strategy adopted in our work.

Our objective is to identify and illustrate errors and biases, either conceptual or algorithmic, in existing modeling environments by contrasting them with an error- and bias-free implementation. Previous analyses (e.g., Mas et al. 2014) dealt with this question by documenting differences in outcome between different modeling environments, on a same set of problems. This is adequate to point out the existence of the issues we address here, but leaves users and

modelers alike without a clear understanding of the origin of the problem, and, more critically, without means to elaborate a correct modeling strategy. Here we identify the origin of the problem by focusing on what pattern-based LUCC models should do at the probabilistic and statistical level, and not at the level of comparison of output maps. This is a more powerful strategy, which furthermore allows us to focus on a single allocation time step (as all time steps are algorithmically equivalent).

Our objective is made possible by going through a step-by-step construction of the required elements of probability theory underlying parameter-free pattern-based LUCC modeling, and devising bias-free algorithmic implementations of these formal results. The present paper introduces the main elements of our allocation method. It is devoted to identifying in a systematic manner the various errors and inefficiencies of existing modeling environments.

The errors and biases we identify and quantify here are not all equally important. A limited subset is most likely sufficient to pinpoint the origin of the difference of behavior and results observed between various modeling environments for quite a large spectrum of case studies, and some are most likely of little practical import in a wide array of actual problems. Still, we try to identify them in a systematic way as it is impossible (at least for us) to elaborate an *a priori* typology of actual contexts in which one or the other of these errors and biases is expected to be truly negligible. By being aware of their existence, model developers and possibly users as well can check *a posteriori* if these sources of error are indeed negligible. In fact, the lack of knowledge of potential sources of conceptual, methodological and algorithmic errors in the existing modeling environment prevents serious and robust error and sensitivity analyses to be performed. This also prevents the reproducibility of obtained results, whereas such reproducibility is a basic requirement of quantitative scientific research.

This problem is partly rooted in the opacity or imprecision in the description of the modeling choices made in existing modeling environments. This is a wide-spread problem: none of the modeling environments we have looked at is described in the literature in a way allowing the reader to reproduce readily the formal choices and algorithmic implementation adopted there.

On the contrary, all our own software (CLUMPY, see Appendix 5) algorithmic choices are described in detail in this work, and, whenever possible, we have clarified or reverse-engineered the choices made in the existing modeling environments we have tested.

This paper is organized as follows. Section 3.2 recalls our notations and the relevant concepts of probability theory we use in our formal analyses, and introduces all required elements to deal with allocation. A generic allocation

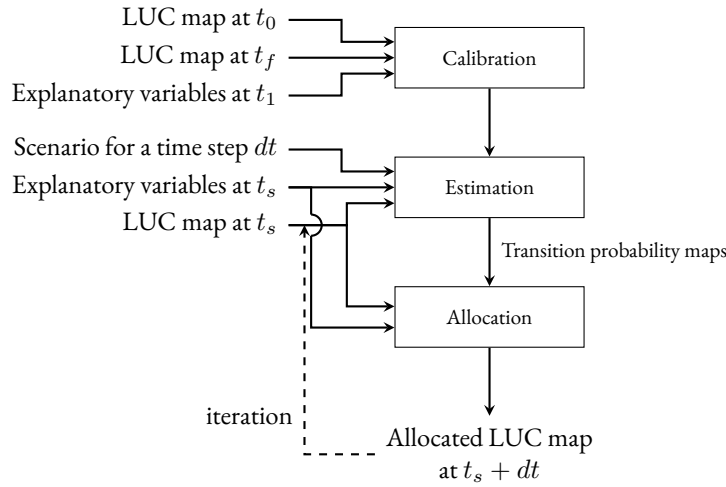


FIGURE 3.1: General architecture of our spatially explicit LUCC model.

framework is then set up (section 3.3). It defines what an allocation method is in principle (or should be), and characterizes the concept of unbiased allocation; this characterization is essential to identify the root of the differences of behaviors between — as well as modeling errors in — existing modeling environments. It also defines what is considered as an allocation bias. Section 3.4 proposes two unbiased allocation method. The first one, designed for mono-pixel patches is presented as a reference method and the second one, devoted to multi-pixel patches, is the current method used in our model. Section 3.5 introduces a generic method to identify major allocation biases through an set of controlled problems of increasing complexity. The penultimate section (section 3.6) examines in a systematic way the errors and biases present in LCM and Dinamica EGO, following the method of the previous section. Our allocation method is included in this evaluation. Our major findings and conclusions are summarized in the last section (section 3.8). More technical matters, in particular formal proofs and detailed descriptions of our algorithm structure, are collected in a number of accompanying Appendices.

3.2. PRELIMINARY CONCEPTS AND NOTATIONS

Our LUCC model is organized in three modules: calibration, estimation and allocation (Fig. 3.1). The first two determine the transition probabilities of the pixels. Calibration is performed using two raster land use and cover (LUC) maps of the same area at two distinct times t_0 and t_1 as well as a set of explanatory variables provided by the user which characterize the changes stud-

ied. Then, the estimation module determines the transition probabilities of a LUC map at time t_s with the same set of explanatory variables. The user also specifies the global transition probabilities (global amounts of change between different LUC states); this is usually done through scenarios. The details of this calibration-estimation procedure is described in a previous article (Mazy and Longaretti, 2022a).

In this paper, we study exclusively the allocation module. This one takes as input the same LUC map at time t_s and it is then a question of generating a new LUC map which is statistically consistent with the transition probability maps obtained previously (Fig.3.1). The details of what this means in terms of probabilities will be detailed in the rest of the article.

Fig. 3.1 is a simplified sketch of this three module structure (calibration, estimation and allocation). In fact, although this is omitted here for simplicity, the allocation module also calls sub-functions of the estimation module. For example, in our method (section 3.4.2), some probability densities are recalculated during allocation.

3.2.1. DEFINITIONS AND NOTATIONS

We introduce a number of definitions and notations for the quantities that generically appear in pattern-based LUCC modeling. Some of these have already been used in other modeling environments, in particular Dinamica EGO. These notations are summarized in Table 3.1.

LAND USE AND COVER STATES

Each LUC state is represented by a positive natural number (the null value is used for unknown state or pixels outside the study area, a common convention in GIS environments). The notation u is used for a pixel state at time t_s (the beginning of the allocation step of interest) and v for this pixel state at the end of the allocation step.

RASTER QUANTITIES

We call J the set of pixels associated with initial state u at $t = t_s$. Let $\#J$ be the number of pixels in state u . Each pixel is identified by its index j which can be used as a superscript. One has $J = \{1, \dots, n_J\}$. Similarly, we use the notation J_v for pixels in initial state u and final state v in the allocation step starting at time t_s .

EXPLANATORY VARIABLES

Explanatory (or ancillary, or predictor) variables – EVs – are quantities defined on pixels, and considered relevant for the statistical prediction of

LUCC models, due to their correlation with past observed changes. Also, some explanatory variable are dynamic, and their maps need to be updated once the allocation time step is completed.

Let d be the number of continuous explanatory variables. We define the explanatory variable space as a point space $\mathcal{D}$. By construction, $\mathcal{D} \subset \mathbb{R}^d$.

Explanatory variables are designated by $\mathbf{z}$ and their set by $\mathbf{Z}$. Specific explanatory variables are identified with a subscript k as z_k . These quantities are indexed by j when they refer to pixel j (z^j, z_k^j).

Discrete variables are important in actual LUCC modeling case studies, but are ignored in the present work. Such variables do not pose any specific calibration problem, and the allocation methods discussed later on can be applied to an arbitrary mix of discrete and continuous variables. Including such variables however makes the various formal expressions more cumbersome. Ignoring discrete variables therefore alleviates notations and formal expressions without any real loss of generality. This is further discussed in section 3.7.

PATCH QUANTITIES

LUC state transitions involve a number of contiguous pixels (patch). Pivot-cells are pixels selected during the allocation method and used as seeds of a pixel patch (see section 3.2.4). We use the c superscript to identify the probabilities and probability densities of these pivot-cells.

Patches are characterized by two parameters: the patch surface σ and the patch elongation e (more precisely defined in section 3.2.4).

ENSEMBLE CARDINALS

As a reminder, the prefix $\#$ is used to designate the cardinal (number of elements) of a finite set. It may also represent the expected number of a set, which may be a real number.

PROBABILITY DISTRIBUTIONS

To alleviate notations, all discrete probability distributions are noted P and all continuous probability density distributions are noted ρ . The various functions are distinguished by their list of variables. When confusion is possible, a superscript is added to P or ρ to resolve the ambiguity.

In this work we ignore the possible existence of discrete explanatory variables. The rationale for this choice is that discrete variables are not very common in the first place, and anyway, they do not pose any particular problem. It is customary in the LUC modeling literature to bin continuous variables so that

continuous and discrete variables can be placed on the same footing. We refrain from doing this, for consistency with our previous work on calibration (Mazy and Longaretti, 2022a).

element	set	definition
u, v	V	LUC state at t_0 and t_1
j	J (size n_J) or J_v (size n_{J_v})	Index of pixels of state u at t_s (resp. initial state u at t_{s-1} and final state v at t_s)
z_k, z_k^i, z_k^j	$\mathcal{D}_k$	k th estimation explanatory variable value for pixels in J
$\mathbf{z}, \mathbf{z}^i, \mathbf{z}^j$	$\mathcal{D}$	d -tuple (or column vector or point) of estimation explanatory variables for pixels in J
$\mathbf{Z}$	not defined	n_J -tuples of values of $\mathbf{z}$ taken by pixels in J
σ, e	$\mathbb{R}$	Patch size and elongation
$\delta\sigma$	$\mathbb{N}$	Pixel size (= 1 by convention)
$\#X$	$\mathbb{N}$	Cardinal (number of elements) of the finite set X . By extension, approximate number of elements in the mean.

TABLE 3.1: Summary of the notations and definitions.

3.2.2. REMINDER ON RELEVANT ELEMENTS OF PROBABILITY THEORY AND LUCC PROBABILITY DISTRIBUTIONS

Two probability distributions play a prominent role in pattern-based LUCC models:

1. $P(v|u, \mathbf{z})$ describes the transition probability of a pixel to state v , knowing the initial state u and the explanatory variables of this pixel.
2. $\rho(\mathbf{z}|u, v)$ describes the probability density¹ of explanatory variables knowing the initial (u) and final (v) states.

These probability distributions are defined on a single but arbitrary pixel. Even though explanatory variables $\mathbf{z}$ are usually deterministic functions of position (and therefore known for each pixel), by drawing pixels at random, one defines a simple but not exclusive way through which the different quantities involved in these probability distributions can be made into random variables.

This being said, it is essential to understand the difference of meaning of these probability distributions between the calibration and simulation stages.

¹Probabilities are dimensionless but probability densities have the dimension of an inverse volume in explanatory space $\mathcal{D}$.

The quantities estimated in the calibration-estimation procedure are $\rho(\mathbf{z}|u, v)$ and $\rho(\mathbf{z}|u)$. These are probability distributions in the frequency sense; for example, $\rho(\mathbf{z}|u)$ counts the number of pixels at $\mathbf{z}$ within $d\mathbf{z}$ with initial state u in explanatory variable space $\mathcal{D}$, divided by the total number of pixels in u . Even though we relied on a kernel density estimation method to evaluate these probabilities, this method can be seen as a weighted pixel counting method, with a weight defined by the kernel function. On the contrary, during allocation, the probability distribution $\rho(\mathbf{z}|u, v)$ is an *a priori* distribution in the Bayesian meaning: one assumes that the distribution of explanatory variables at t_s knowing state u at t_s and for a transition to state v at t_{s+1} is the same as the calibrated distribution. This is not true of $\rho(\mathbf{z}|u)$, which is reestimated at each time step on the same frequency basis.

3.2.2.1. TRANSITION PROBABILITY AND BAYES RULE

The two major probability distributions that have just been introduced above are related through Bayes rule:

$$P(v|u, \mathbf{z}) = P(v|u) \times \frac{\rho(\mathbf{z}|u, v)}{\rho(\mathbf{z}|u)}. \quad (3.1)$$

This formula is particularly useful as it isolates an element with no spatial dependence, $P(v|u)$ – which is usually considered as a scenario parameter during the simulation/allocation phase – from the (usually spatially dependent) probability densities $\rho(\mathbf{z}|u, v)$ and $\rho(\mathbf{z}|u)$ which are determined by the calibration-estimation process. The notation P is reserved for categorical (discrete) random variables, while ρ refers to continuous ones. In general, explanatory variables are continuous, but mixed continuous and discrete ones are possible; in this case, the probability distribution is a probability density as well, because of the continuous ones.

Note that all probabilities involving v are defined per time step, i.e., over the calibration time span during the calibration phase, and the time-step itself during the allocation phase. Usually $\rho(\mathbf{z}|v_t, v_{t+1})$ is defined directly from the calibration data, when the time-step is equal to the calibration time span (modulo possible scenario adjustments, an option which is usually ignored); otherwise, this probability must be accordingly modified, as such probabilities usually depend on the duration of the time step. This question will be further discussed in section 3.6.1.3.

3.2.2.2. CLOSURE RELATIONS

The probabilities and probability densities defined above should verify the following closure relations:

$$\int_{\mathbf{z} \in \mathbb{R}^d} \rho(\mathbf{z}|u) d\mathbf{z} = 1, \quad (3.2)$$

$$\int_{\mathbf{z} \in \mathbb{R}^d} \rho(\mathbf{z}|u, v) d\mathbf{z} = 1. \quad (3.3)$$

Furthermore, although allocation algorithms generally use a method of allocation in patches and not pixel by pixel, the transition probabilities are defined so that the proportion of pixels changing state must be equal on average to the expected level defined by $P^*(v|u)$, i.e.,

$$\int_{\mathbf{z} \in \mathbb{R}^d} P(v, \mathbf{z}|u) d\mathbf{z} = \int_{\mathbf{z} \in \mathbb{R}^d} P(v|u, \mathbf{z}) \times \rho(\mathbf{z}|u) d\mathbf{z} = P^*(v|u), \quad (3.4)$$

These relations are exact for the exact probability distributions. They are enforced as well on the probability distribution estimates produced by our calibration-estimation method.

3.2.3. ENSEMBLE AVERAGING AND NOISE REDUCTION

Among the quantities introduced so far, the number of pixels undergoing a given transition is often treated as a fixed quantity, whereas in principle it is a random variable. In principle, this quantity should be defined as a mathematical average over a large number of different but equivalent histories. Consider for example changes in crop rotation over time in farm land. Many factors come into play (climate, weather, costs, markets, farmer preferences etc). As a consequence, any spatially explicit sequence of state change could have happened differently than observed in the LUCC maps used for calibration, albeit with similar statistical characteristics; this point is at the root of statistical LUCC modeling.

However, we have at our disposal only one such “statistical experience” for calibration, which corresponds to the observed reality. Because of this limitation in available statistics, the estimates of the probabilities obtained in the calibration process are intrinsically noisy, although they are often assumed to represent the underlying probability distributions exactly, and although spatial averaging may reduce this noise to some extent.

Projections of future LUC states in a given area should in principle be performed through a series of simulations for the same time period, this is a standard procedure in all stochastic time evolution problems, in order to obtain sta-

tistically significant averages over different histories. In ergodic contexts, such ensemble averages can be replaced by spatial or time averages (e.g., in the study of statistically steady and spatially homogeneous three dimensional hydrodynamics turbulence). A form of ergodic hypothesis is often implicitly made in LUCC simulations, but its validity has never been tested.

In LUCC simulations, the total number of pixels undergoing a change per times step from state u to state v is subject to statistical variations, as do all stochastic quantities in stochastic evolution problems. The actual expected values of state change $P(v|u)\#J$ is only obtained by averaging over a large enough ensemble of such simulations. To circumvent this feature (often assimilated to a problem), most (possibly all) modeling environments do exactly enforce the expected quantities of state change at each time step in a single simulation; this is done algorithmically and amounts to modifying the transition probabilities in an explicit or implicit way².

However, this procedure introduces biases of its own, a point we shall investigate in the course of this work. Because of this, in order to evaluate the related biases, our bias-free algorithms do not enforce these expected values of state change in an allocation step.

In the remainder of this work, expectation values of any quantity X , $E(X)$, do refer to an average over a large (theoretically infinite) number of independent simulations.

3.2.4. PIXELS VS PATCHES: ASSUMPTIONS IN BRIEF

In fact, pixels do not change state independently, but in patches of contiguous pixels. Dinamica EGO seems to be the only software trying to implement such an approach in a self-consistent way. Patches are characterized by a central pixel, called pivot-cell³, and a number of characteristics, the most important being the patch size and shape.

Let us first introduce our definition of patch merging, as the occurrence of such mergings during allocation is a source of bias, as will be shown later on:

PATCH MERGING

Two patches formed during the same allocation time step are said to be merging if they have common pixels or a partially common boundary, so

²This choice was partially motivated by the fact that, at the present level of computational efficiency of LUCC modeling software, performing series of different simulations for the same time span would be computationally prohibitive, especially that parallel computations are rarely if ever implemented in this field. However, this can be achieved with more efficient allocation algorithms such as the ones described in this work, even with little or no parallelization.

³The developers of Dinamica EGO have introduced this definition.

that the union of the two patches presents no spatial discontinuity⁴.

Finally, we make the following assumptions in the course of the present work:

1. The pivot-cell probability distribution in explanatory variable space is independent of other patches characteristics (size and shape). As any other pixel, a pivot-cell is identified by its index i or j .
2. Multi-pixel patches are characterized by their size (σ) and a shape parameter (e) only, with probability distribution $\rho(\sigma, e|u, v)$. Size and shape are assumed to be independent variables.
3. The various probability distributions of pivot-cells with respect to explanatory variables are identical to the distributions characterizing the entire pixel population: $\rho^c(\mathbf{z}|u, v) = \rho(\mathbf{z}|u, v)$.

Patch sizes can be measured in surface or pixel number. For simplicity, we assume in this work that they are measured in pixel number, so that, conventionally, the size of pixel, $\delta\sigma = 1$. Note that the probability distributions of size and elongation are discrete by nature. However, for simplicity, when these quantities are not obtained from calibration data (the rule so far in LUCC modeling), they are given by a probability distribution provided by the modeler. Such probability distributions are easier to specify with continuous variables. Consequently, probability density distributions — $\rho(\sigma|u, v)$, $\rho(e|u, v)$ — are used to specify the distributions but discrete ones — $P(\sigma|u, v)$, $P(e|u, v)$ — are used everywhere else.

Preventing patch merging is mandatory to obtain a bias-free allocation method (see section 3.6.3). Patch merging is prevented algorithmically in a self-consistent way (section 3.4.2).

These assumptions imply that the overall probability of change of pivot-cells, $P^c(v|u)$, must be related to $P(v|u)$. This relation is specified in section 3.4.2. Furthermore, small enough time steps are required in order for pixels to undergo at most one state change per time-step. This requirement is in general not very constraining.

Note finally that our assumptions imply that Bayes rule for pivot-cells takes the form

$$P^c(v|u, \mathbf{z}) = P^c(v|u) \times \frac{\rho(\mathbf{z}|u, v)}{\rho(\mathbf{z}|u)}. \quad (3.5)$$

These assumptions are discussed in Appendix 3.A.

⁴Patches are also implicitly assumed to be simply connected spatial entities.

3.3. ALLOCATION METHOD AND BIAS: DEFINITIONS AND CONSTRAINTS

Now that the various relevant probability distributions have been defined and their properties and relations clarified, let us return to the discussion of section 3.2.2, which is not sharp enough yet to specify how probabilities should unambiguously be used in an LUCC simulation. For this purpose, it is necessary to introduce a few more definitions.

Occasionally in the remainder of this paper and related appendices, we make reference to some results of sampling theory. The interested reader can find detailed information about this, e.g., in the recently reprinted classical text of Cochran or the most recent and possibly more accessible one by Lohr (Cochran, 1977; Lohr, 2019).

3.3.1. SAMPLING DESIGN, ALLOCATION PROCEDURE AND ALLOCATION METHOD

We introduce several definitions in the following:

SAMPLING DESIGN

A sample is a collection of elements of interest (here, pixels) taken in a set (here, the set J of pixels in state u at t_s). A sampling design is a random process through which these elements are selected to constitute a sample. From a theoretical point of view, a sampling design is defined once the probability of occurrence of all possible samples is specified. From a practical point of view, the problem is to design an algorithm producing samples according to a given law of probability. Quite often, producing an algorithm is a way to define (possibly implicitly) the probability law of all possible samples and its associated characteristics. If instead the desirable properties of the sample probability law are specified beforehand, the difficulty resides in enforcing these properties exactly in the sampling algorithm(s).

ALLOCATION PROCEDURE

An allocation procedure is an algorithmic process through which a final state is attributed to pixels in given sample. An allocation procedure specifies how and in which order pixels in a sample are processed.

ALLOCATION METHOD

In the context of this work, an allocation method is defined by the combination of a sampling design and an allocation procedure.

Our first allocation method applies to pivot-cells only. A simple allocation method dealing with this is presented in 3.4.1. It is well suited to mono-pixel patch allocation. mono-pixel patches are clearly artificial and unrealistic, but this setting nevertheless allows us to identify a number of problems in existing software (sections 3.5 and 3.6). This mono-pixel allocation method has a number of desirable features, in particular it is error- and bias-free (section 3.3.2). Our next allocation method deals with extended patches. The difficulty here lies in enforcing the correct transition probability law while allocating pixels by patches. These methods are presented in section 3.4.

It is implicitly assumed that pixels undergo at most one state change per time step. Generally speaking (and unless some pixel states cannot be changed once reached), this implies in turn that the maps used for calibration must be obtained at close enough dates, in which case all amounts of LUC state change will be small enough. This must also apply to simulation time steps for the small changes quantified in the calibration data to be meaningful in a simulation, especially that a number of explanatory variables are updated along the simulation. Also, this requirement makes the avoidance of patch merging much easier to enforce.

Once some pixels (in state u at time t_s) have undergone a change in a given time-step, they must be taken out of the pool J to ensure that they do not undergo another state change. This is referred to as sampling without replacement in sampling design theory⁵. LUCC models make use of an allocation procedure in an explicit way, but the related sampling design is often either implicit, simplistic, or not clearly described. This is unfortunate, because sampling designs without replacement (the ones of interest here) are well-known to lead to algorithmic difficulties in sampling theory. Because of this, the (explicit or implicit) assumptions made about sampling design often lead to conceptual errors and biases in existing software. These in turn are important to identify, as they figure among the primary reasons of the differences of behavior between different modeling environments for a given problem.

To sum up, an unambiguous use of probability theory in LUCC models relies on unambiguous definitions of sampling designs and allocation procedures (i.e., allocation methods). To proceed further, one first needs to characterize possible biases in the allocation methods, as this serves as an ultimate test of correctness and relevance of a sampling method. This is the object of the next section.

⁵Conversely, a sampling design with replacement puts pixels back in the pool, allowing them to be selected again.

3.3.2. BIAS: DEFINITION

For an allocation method to be accurate, it is necessary that the expected mean number of pixels in each final state from any given initial state, as well as their mean distribution in $\mathbf{z}$, are respected. In other words, for pixels, one requires that⁶

$$E(\#J_v) \equiv \hat{P}(v|u)\#J = P(v|u)\#J, \quad (3.6)$$

$$E(\#J_{\mathbf{z},v}) \equiv \hat{\rho}(\mathbf{z}|u, v)E(\#J_v)\delta\mathbf{z} = \rho(\mathbf{z}|u, v)E(\#J_v)\delta\mathbf{z}, \quad (3.7)$$

where by definition $\hat{P}(v|u)$ (resp. $\hat{\rho}(\mathbf{z}|u, v)$) is the post-allocation value of $P(v|u)$ (resp. $\rho(\mathbf{z}|u, v)$) and $\delta\mathbf{z}$ is a small volume around $\mathbf{z}$. $\#J_{\mathbf{z},v}$ refers to the number of pixels in initial state u and final state v . Note that $\hat{\rho}(\mathbf{z}|u, v)$ is independent of $\delta\mathbf{z}$. These relations therefore requires that the pre- and post-allocation probability distribution of explanatory variables for given initial and final state are identical, while the first requires the same property of the global probability of change.

Similar relations apply to pivot-cells (*mutatis mutandis*)

$$E(\#J_v^c) = P^c(v|u)\#J, \quad (3.8)$$

$$E(\#J_{\mathbf{z},v}^c) = \hat{\rho}^c(\mathbf{z}|u, v)E(\#J_v^c)\delta\mathbf{z} = \rho^c(\mathbf{z}|u, v)E(\#J_v^c)\delta\mathbf{z}. \quad (3.9)$$

$\#J_v^c$ is the number of pivot-cells in final state v , and $\#J_{\mathbf{z},v}^c$ the number of pivot-cells in final state v and with explanatory variables $\mathbf{z}$ within some small volume $\delta\mathbf{z}$.

Similarly, it is desirable that the patch characteristics probability distribution are also satisfied, e.g., for patch surfaces

$$E(\#J_{\sigma,v}) = P(\sigma|u, v)E(\#J_v^c), \quad (3.10)$$

where $\#J_{\sigma,v}$ is the number of patches in final state v and size σ .

It is sufficient to ascertain that these relations are preserved in a single time step, due to the repetitive algorithmic nature of a simulation, once an allocation method is defined.

⁶Note that because the initial state is known, $E(\#J) = \#J$. These relations does not necessarily imply that explanatory variables have been binned, see Longaretti and Mazy (2022) — hence the designation of pseudo-numbers, as these are not required to be integers. Such numbers are used only in the formal proofs. In the tests of section 3.6, these relations are algorithmically tested by using directly our bin-free calibration estimation procedure; see section 3.6 for details.

In Eq. (3.8), $P^c(v|u)$ is related to $P(v|u)$ as follows⁷

$$E(\#J_v^c) = \frac{E(\#J_v)}{E(\sigma)}. \quad (3.11)$$

Consequently:

$$P^c(v|u) = \frac{P(v|u)}{E(\sigma)}, \quad (3.12)$$

so that Eq. (3.8) is automatically satisfied if Eq. (3.6) is satisfied and if the distributions of explanatory variables and patch size are independent, as assumed in this work.

The same argument holds for Eq. (3.9). Indeed, the argument of footnote 7 applies as well to the relation between $\#J_{z,v}^c$ and $\#J_{z,v}$: $E(\#J_{z,v}) = E(\#J_{z,v}^c)E(\sigma)$. Combining this with Eqs. (3.11) and (3.7), and taking into account the assumption $\rho^c(z|u, v) = \rho(z|u, v)$ (discussed and justified to some extent in Appendix 3.A) on top of the independence between explanatory variables and patch sizes leads to Eq. (3.9).

We are now in position to state our definition of an unbiased allocation procedure

UNBIASED ALLOCATION DEFINITION

If the requirements of identity of these pre- and post-allocation probability distributions expressed above in Eqs. (3.6) through (3.10) are satisfied, the allocation method producing the post-allocation distributions is by definition unbiased⁸.

It is useful to elaborate somewhat on the meaning and rationale of these requirements. Let us focus on Eqs. (3.6) and (3.7) for definiteness. Even though these requirements may appear tautological or at least superfluous at first sight, it turns out that none of the existing modeling platforms we have tested do enforce them, making them all biased in one way or another.

These relations distinguish the expectation (or mean) value of the various numbers just mentioned, $E(\#J_x)$, and the actual value in a simulation $\#J_x$,

⁷This relation follows directly from our assumption of independence of pivot-cells and patches statistical properties (at given initial and final LUC state). On the one hand, the average total quantity of change for the state transition considered is $E(\#J_v)$. On the other hand, this same quantity can also be computed as follows. The expectation value (average) patch size is $E(\sigma)$. This applies for every pivot-cell, so that the total quantity of change (averaged over patch sizes) is $\#J_v^c E(\sigma)$. Finally, averaging the number of pivot-cells itself leads to $E(\#J_v^c)E(\sigma)$ for the average total quantity of LUC state change.

⁸In sampling theory, it is pointed out that, especially in contexts where variances can be large (e.g., leading to a factor of error of several in the estimates of quantities of interest), a slightly biased sampling design is acceptable if the variance is substantially reduced. In our experience, variances are not large enough in LUCC modeling to motivate us to look into this question.

which may differ from its mean value. However, these relations *must* be preserved in a statistical sense as $P(v|u)$ and $\rho(z|u, v)$ are the only probability distributions that do define the state change process. More precisely, the requirement here is that the post-allocation probability distributions $P(v|u)$ and $\rho(z|u, v)$ — i.e., measured once pixels have changed state in a time step — are identical to the pre-allocation ones — i.e., as they appear in Bayes rule Eq. (3.1) which defines the transition probability — except for statistical noise. There is nothing mysterious or far-reaching here, this just a self-consistency requirement⁹, i.e., we wish to check that allocation methods do enforce fair sampling of all relevant probability distributions.

However, this basic self-consistency requirement is not satisfied in existing modeling environments, with sometimes rather substantial consequences for the reliability and robustness of LUCC simulations.

Finally, in principle, one should also apply the same requirements to the joint probability distributions used to quantify multi-pixel patch transition (the relevant context in actual case studies), such as, e.g., $\rho^c(z^1, z^2|u^1, v^1, u^2, v^2)$. Such joint probability distributions have never been considered in LUCC modeling, to the best of our knowledge. We first introduced them in our optimal explanatory variables' selection procedure (Longaretti and Mazy, 2022).

In principle, such multi-pixel probability distributions are formally required to define patch transitions. In practice, and in order to avoid some inextricably complex issues, the joint distribution of pivot-cells in a multi-pixel patch context will be implicitly defined algorithmically. This procedure implicitly defines the related post-allocation probability distributions, which we *assume* to be identical to the pre-allocation ones.

Such an assumption could not be made if the pre-allocation multi-pixel distributions were obtained through a calibration procedure, but no existing modeling environment looks into such joint probability distributions. In any case, it is unclear that the present level of quality of calibration maps would allow us to perform such multi-pixel calibration in a precise enough fashion.

For this reason, we leave the related issues aside in the context of the present study, and define patch properties algorithmically; some of these properties (such as patch sizes) may be calibrated on existing data, some (such as patch elongations) may be left to the user preferences if calibration data are insufficient.

⁹The comparison must of course be performed before updating dynamic explanatory variables, to be meaningful. Also, if this self-consistency constraint is not satisfied, the results may again depend on the chosen time step for a given simulation time span, which is unacceptable.

3.3.2.1. IMPORTANT COMMENT

It must be pointed out that comparing performances on, e.g., $\rho(z|u, v)$, i.e., in explanatory variable space $\mathcal{D}$, is a much more powerful way to identify errors and biases than comparing maps in real space. Indeed, on a single allocation time step, and because changes per time step are small, there is little chance that allocations would be performed at the same spatial locations. However, these spatial locations could be equivalent from a statistical point of view. In fact, because statistical LUCC modeling is probabilistic in nature, it is not consistent to expect that changes occur either where they are actually observed or at the same locations across different software. This does not contradict the desire to improve the spatial accuracy of LUCC models pattern projections, with respect to historical observations, but takes into account that this accuracy is limited by our limited knowledge of the (deterministic) causes behind LUC change (a more extended discussion of this point is given in section 2 of Mazy and Longaretti 2022a).

On the other hand, it is mandatory that they occur at statistically equivalent location, otherwise a bias occurs. Such equivalent locations can only be properly identified in explanatory variable space $\mathcal{D}$, not physical space. This is true irrespective of the fact that over the course of a long simulation where a significant fraction of pixels has changed LUC state, some (possibly significant) overlap between projected and actually observed changes must be present (at least if the pool of explanatory variables has been correctly identified).

We are aware that this way of proceeding goes against the most common methods of analysis of LUCC model performances in the literature, but we believe that the power of this approach, that will be apparent in section 3.6, will be sufficiently convincing in itself to convince the reader of the relevance of this departure from current habits.

3.4. UNBIASED ALLOCATION METHODS

Having at hand unbiased allocation methods is necessary to pinpoint the origin of conceptual errors and biases in existing LUCC models and modeling platforms. We describe two such methods here, one for mono-pixel patches (uSAM) and one for multi-pixel patches (uPAM). The mono-pixel patch method is formally shown to be unbiased in Appendix 3.B.2. The multi-pixel patch method is seen to be unbiased on a case by case basis in the error and bias analysis of section 3.6; a formal proof is provided in Appendix 3.D.

The uSAM algorithm enforces quantities of change only statistically per time step, while uPAM enforces them exactly. However, an alternative uSAM-like algorithm enforcing quantities of change exactly is provided in Appendix 3.C, as such a discussion also constitutes a convenient introduction to Appendix 3.D.

3.4.1. UNBIASED SIMPLE ALLOCATION METHOD (USAM)

A mono-pixel bias-free allocation method can be defined following the procedure introduced in section 3.3.1:

SAMPLING DESIGN

Every pixel in J (state u at t_s) is tested for possible selection as a pivot-cell. This is hardly a sampling design, but it turns out that testing all pixels leads to a computationally more efficient allocation method than selecting a smaller subset through pruning¹⁰.

ALLOCATION PROCEDURE

All pixels in the sample (in this case, all pixels) are allocated a final state (which can be identical to the initial one) with the help of the multinomial sampling test algorithm MuST of Appendix 3.B. This algorithm tests all possible final states at once. The resulting mono-pixel patches may be adjacent, as we ignore patch properties in this sampling method and do not try to prevent patch merging for mono-pixel patches.

The fact that this method is bias-free is shown in Appendix 3.B.2.

3.4.2. UNBIASED PATCH ALLOCATION METHOD (UPAM)

A multi-pixel patch allocation algorithm cannot simply consist in adding a patch formation procedure to the mono-pixel patch algorithm described in the previous section. Such a method presents a bias when quantities of change are enforced, and the resulting post-allocation pixel probability distributions will differ from the pre-allocation ones. Also, patch merging must be avoided in order for the post-allocation patch characteristics probability distribution to be also identical to the pre-allocation one.

To avoid these problems, we devised a rather involved sampling design without replacement in order to avoid patch merging. The root of the difficulty is

¹⁰Dinamica EGO introduced the concept and process of pruning, which in effect constitutes its sampling procedure. Pruning is a pre-selection process designed to focus on the pixels most likely to undergo a transition. However, this procedure is problematic and leads to a highly biased allocation as shown in section 3.6.1

due to the fact that probability distributions are defined in $\mathbf{z}$ space, whereas the question of patch merging is a spatial one. This difficulty requires a specific adaptation of standard sampling theory procedures.

Our allocation method involves two different types of probability distribution updating:

- The first type is related to the step by step patch formation just mentioned at a given time t_s . All pixels belonging to the newly formed patch must be removed from the pixel population before the next patch can be formed. In fact, the induced limitations of possibilities for new pivot-cells selection and patch formation while constituting the patch sample is precisely why the overall post-allocation probabilities $\rho^c(\mathbf{z}|u, v)$ and $P^c(v|u)$ can be identical to the pre-allocation ones. In practice, this implies that the pivot-cell probability distributions $\rho^c(\mathbf{z}|u)$ must be updated after every patch construction.
- The second type consists in updating the $\mathbf{z}$ values of dynamic explanatory variables (i.e., variables that change in the course of the simulation, e.g., distance to urban areas due to urban sprawl). This type of updating must be performed *after* the post- and pre-allocation probability distributions are compared to check for possible biases. In particular, $\rho^c(\mathbf{z}|u, v)$ must not be updated after each new patch is formed at time step t_s , as it embodies an *a priori* preference among possible $\mathbf{z}$ values for a given transition $u \rightarrow v$, defined pixel by pixel, and independently of the number of pixels that may or may not have already changed state. This *a priori* distribution holds by definition for the whole time step allocation process. Because of this, in our model architecture, all dynamic updating are performed at the beginning of the next time step (see Fig 3.1).

The structure of our patch allocation method is described below and is represented in a schematic way in Fig. 3.2. It makes use our sampling design and allocation procedure in an iterative way. This iterative strategy differs slightly from our definitions above; it is adopted to produce a formally correct sampling method without replacement:

SAMPLING DESIGN

A subset of potential pivot-cells is created at random by applying the MuST algorithm to the total pixel pool with $P(v|u, \mathbf{z})$ as the probability distribution of final states, and keeping the pixels that actually did undergo a change of LUC state (i.e., pixels with $v \neq u$). We then draw a single such potential pivot-cell at random along with its chosen final state. All other

potential pivot-cells are then dismissed. If state v_0 is selected, MuST is then applied with $P(\sigma|u, v_0)$ to affect a patch size σ to the newly drawn pixel.

ALLOCATION PROCEDURE (“PATCHER”)

Our patching algorithm attempts to create a patch around this pivot-cell with a size and elongation drawn in accordance to their probability distribution (this step is detailed in appendix 3.I). This patch creation can either be a success or a failure. A failure occurs in two cases:

1. the produced patch is adjacent to a previously created patch at the same time step which leads to patch merging;
2. the number of pixels available to create the patch is not large enough to produce the expected patch size (e.g. if the selected falls in too small an enclosed area bounded by different initial states than u). In this case, no pixels are allocated.

Whether it succeeds or fails, this patch creation algorithm returns the subset of pixels that have been processed. If the procedure succeeds, the new patch is kept. In both cases (success or failure), the allocation procedure has been completed and stops. The exact implementation of our patch construction algorithm is presented in Appendix 3.I.

ITERATION AND UPDATING

After completion of the allocation procedure, three updating steps are then necessary before proceeding any further:

1. If the patching procedure was successful, we modify $P(v|u)$ to take into account the allocated area: $P(v|u)$ is updated to $P(v|u) - \sigma / \#J$ where σ is the number of pixels in the newly created patch and $\#J$ is the current number of pixels in u state (i.e., corrected for pixel removals in previous iterations).
2. The subset of pixels returned by the patching algorithm are removed from J (whether or not the procedure was successful) and $\#J$ is accordingly updated.
3. Their removal implies to update $\rho(z|u)$, which is done with our calibration method (Mazy and Longaretti, 2022a).

The sampling design and allocation procedure are then called iteratively until $P(v|u) < 0$ for all $v \neq u$, or the updated set J is empty or the procedure cannot form a new patch. If the total quantities of change are small, the first outcome is by far the most probable.

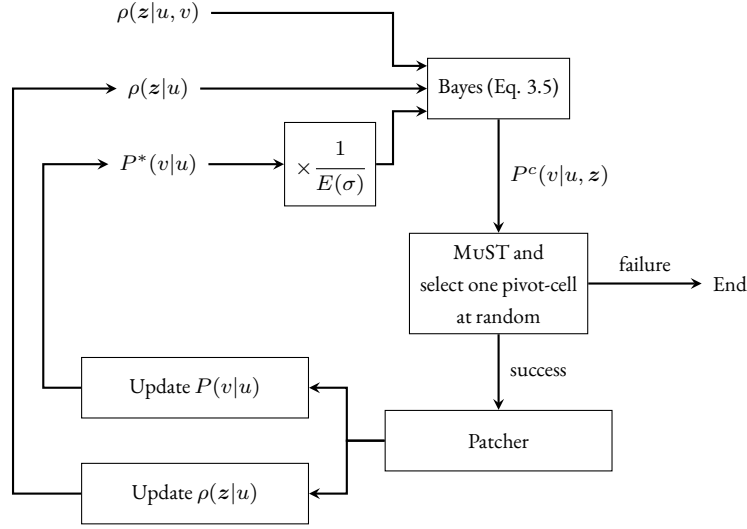


FIGURE 3.2: Unbiased Patches Allocation Method (uPAM) workflow.

It should be noted that if we parameterize this method to generate only one-pixel patches ($\sigma = 1$), the allocation method obtained is equivalent in the mean¹¹ to the one defined for single-pixel patches (section 3.4.1), and exactly equivalent to the algorithm introduced in Appendix 3.D. Also, this allocation procedure covers both the patcher and expander procedure of Dinamica EGO and other software (see Appendix 3.I.3).

The fact that this allocation method is bias-free is proved in Appendix 3.D.

3.5. A PROGRESSIVE STRATEGY FOR THE IDENTIFICATION OF ALLOCATION BIASES

In this section, we describe the method we have devised to identify the origin of allocation discrepancies observed between several existing models (Mas et al., 2014). We first describe how the models are parameterized and how the results are processed (section 3.5.1). Then, our systematic strategy is described (section 3.5.2). Finally, we define a set of case studies specifically designed for this purpose (section 3.5.3).

¹¹The distribution of pivot-cell numbers around the mean will be different but the mean number of pivot-cells will be identical. As we care only about preserving the mean number of pixels in our definition of a bias-free algorithm, this difference is unessential at the level of formal correctness adopted in this work and for the purpose of the present work.

3.5.1. ERROR IDENTIFICATION STRATEGY

First, we specify the transition probability maps $P(v|u, z)$. These maps are exactly known and will therefore be used as reference maps in the analysis of biases in the allocation procedure of various software (including ours). To this effect, the calibration step is ignored in the model by inserting these transition probability maps just prior to the allocation step in the corresponding software¹². This ensures that all modeling environments we have tested do have a common well-defined and controlled starting point for their respective allocation methods, independently of what their calibration methods prescribe from calibration data.

Alternatively, we could instead produce pivot-cells and patches from our assumed probability maps and let the tested software proceed through calibration and allocation. However, this would add noise to the process as well as possible calibration errors and biases. The proposed strategy allows us to focus on allocation issues only.

In section 3.3.2 we defined bias on the basis of expectation values. On the other hand, performing a comparison of pre- and post-allocation probability distributions is unavoidably affected by statistical noise. We can estimate the variance of this noise, but this may not give a sufficiently precise approximation of the required expectation values of the post-allocation probability distributions, Eqs. (3.7) and (3.9). In order to reduce this statistical noise, a Monte Carlo approach is adopted: the same single allocation step is performed a number of times, and the post-allocation probability distributions are obtained by averaging over this series of realizations, thereby substantially reducing the statistical noise and providing more precise estimates of the expectation values we wish to compare across the various allocation methods we have tested. This is presented in more detail in Appendix 3.H, where the setting of these Monte Carlo simulations is specified according to a tolerance level prescribed by the user.

Next, the post-allocation transition probabilities are estimated to compare them to the pre-allocation ones. This amounts to applying the calibration and estimation modules to the pre- and post-allocation maps. We propose to use the calibration-estimation method based on kernel density estimation introduced in Mazy and Longaretti (2022c,a) which was shown to have the best accuracy among all the calibration methods we have tested and compared. Sticking to a single calibration-estimation method to compare different allocation methods

¹²Dinamica EGO and Idrisi LCM provide such a functionality although this requires a non-standard use of Idrisi LCM, whereas CLUMondo does not, so that the allocation procedure of CLUMondo has not been analyzed independently of the calibration one (Mazy and Longaretti, 2022a).

is also essential to ensure that the post-allocation transition probabilities have been computed in the same way, and the accuracy of the method ensures that most observed differences in performance of the allocation methods we have tested is due to the allocation methods themselves.

3.5.2. OBSERVED BIAS

The post-allocation transition probability map is compared to the transition probability map used as input of all the allocation method. In the same way as proposed in Mazy and Longaretti (2022a), we could compute the average of the mean absolute error of each Monte Carlo simulations. This calculation gives an indication of the bias in an integrated way over all pixels. However, in the context of a bias identification process, it is interesting to observe the deviation from the expected probability in a more precise way. We therefore show the pixel distribution probability $\rho(z|u, v)$ itself, averaged over all Monte Carlo simulations. This allow us to point out the identified biases in a more precise manner.

3.5.3. SETTING TEST PROBLEMS OF INCREASING COMPLEXITY

Detecting allocation bias through a real case study is very delicate. It is the same for an artificial case study whose complexity in terms of number of transitions and explanatory variables is close to a real case. Indeed, it can be assumed that several biases of different origin overlap, influence the result and may even interact with each other. Moreover, it would be difficult to identify precisely the origin of a bias when the complexity of the case study prevents us from understanding *a posteriori* the behavior of the allocation algorithm.

Because of this, we focus on a series simple case studies of increasing complexity to perform this analysis. This allows us to detect errors and biases one by one and especially to identify their origin. To this effect, we have chosen to create test problems in graded contexts as follows:

1. Mono-pixel patches and single transition setting.
2. Mono-pixel patches and multiple transitions setting.
3. Multi-pixel patches and single transition setting.
4. Multi-pixel patches and multiple transitions setting.

This progressive method does not ensure that all errors and allocation biases will be captured, but it did help us to identify quite a few, because of its sys-

tematic nature. Also, it illustrates why by comparison our uSAM and uPAM method are error- and bias-free. A series of case studies following the above progression is proposed just below.

3.5.3.1. MONO-PIXEL PATCHES AND SINGLE TRANSITION SETTING

We set up a disk of 300 pixels of radius and spatially uniform initial LUC state u . This represents the initial LUC map of our test problem. The only explanatory variable z is the distance in pixel unit to the center of the disk, *i.e.* the radius. By construction:

$$\rho(z|u) = 2\pi z \# J^{-1} \quad (3.13)$$

We define a single transition $u \rightarrow v$. The probability distribution $\rho(z|u, v)$ is a uni-variate normal law of z , with mean value $z_m = 150$ and standard deviation $\sigma_z = 40$:

$$\rho(z|u, v) = \frac{1}{(2\pi\sigma_z^2)^{1/2}} \exp\left(-\frac{(z - z_m)^2}{2\sigma_z^2}\right). \quad (3.14)$$

We set $P(v|u) = 0.02$. We generate mono-pixel patches with the uSAM allocation method. From the analysis of Appendix 3.H, with a confidence interval of 0.01 for 95% of the distribution, the required number of Mont Carlo simulations is 273.

3.5.3.2. MONO-PIXEL PATCHES AND MULTIPLE TRANSITIONS SETTING

We set up a rectangle of 1000×250 pixels and spatially uniform initial LUC state u as initial LUC map. The only explanatory variable z is equal to the distance in pixel to the left side of the rectangle. By construction, $\rho(z|u) = N_y^{-1}$.

We define two transitions: $u \rightarrow v_1$ and $u \rightarrow v_2$. The probability distributions $\rho(z|u, v_1)$ and $\rho(z|u, v_2)$ are defined as piecewise functions:

$$\rho(z|u, v_1) = \begin{cases} \left(\frac{2}{5}N_y\right)^{-1} & \text{if } \frac{1}{5}N_y \leq z < \frac{3}{5}N_y \\ 0 & \text{else} \end{cases}, \quad (3.15)$$

$$\rho(v_2|u, z) = \begin{cases} \left(\frac{2}{5}N_y\right)^{-1} & \text{if } \frac{2}{5}N_y \leq z < \frac{4}{5}N_y \\ 0 & \text{else} \end{cases}, \quad (3.16)$$

where N_y is equal to the number of pixels in a row, *i.e.* 250. With this setting, some pixels are likely to undergo both transitions with the same probability, some can only transit to one final state, whereas others have no probability to transit. We set $P(v|u) = 0.05$ for both v_1 and v_2 ; this is relatively large but required to observe the multiple transition bias (section 3.6.2). Mono-pixel patches are generated with the uSAM allocation method. From appendix 3.H, with a confidence interval of 0.01 for 95% of the distribution, the required number of Monte Carlo simulations is equal to 420.

3.5.3.3. MULTI-PIXEL PATCHES AND SINGLE TRANSITION SETTING

The synthetic data is exactly the same as the one used in section 3.5.3.1, transition probability included. The only change concerns the patch size expectation. Dinamica EGO's patcher provides only one way to characterize patch sizes which are drawn from a log-normal distribution. *De facto*, we used the same distribution in this case in order to analyze Dinamica EGO's results. We set the log-normal mean to 50 and its variance to 5. Details about the relation between Dinamica EGO's parameters and the log-normal probability law are given in appendix 3.E.3.

Concerning the patch design process, we chose to construct circular patches in order to focus on possible differences between the pre- and post-allocation patch size and $\rho(z|u, v)$ distributions.

3.6. CONCEPTUAL ERRORS AND BIASES IN LCM AND DINAMICA EGO

We identify errors and biases by comparing several allocation methods:

- Dinamica EGO 4.0.10 *Landscape Patcher* block (DE),
- IDRISI Selva Land Change Modeler (LCM),
- Our unbiased Simple Allocation Method (uSAM) developed for this study and described in section 3.4.1 (used for mono-pixel patches).
- Our unbiased Patches Allocation Method (uPAM) developed for this study and described in section 3.4.2 (used for multi-pixel patches).

The uSAM and uPAM methods provide controlled points of comparison, as these have been designed to be both error- and bias-free.

We have reprogrammed the part of Dinamica EGO and the complete LCM allocation methods for comparison with the original software, in order to increase our confidence of our understanding of their workings. This has also allowed us to identify the origin of the sometimes subtle problems we point out. Dinamica EGO and LCM allow us to bypass the calibration step and to directly fill in a transition probability map as the input of the allocation module as discussed in section 3.5.1. Although CLUMondo was evaluated in our analysis of calibration issues (Mazy and Longaretti, 2022a), this model is not discussed in this section for reasons explained in the conclusion of this paper (section 3.8).

3.6.1. MONO-PIXEL PATCH AND SINGLE TRANSITION: PRUNING BIAS

The pruning bias is revealed by the test problem described in section 3.5.3.1. This bias is certainly the most important one. It is observed both in Dinamica EGO and LCM, although they operate in different ways. This bias is not exhibited by design by the uSAM and uPAM algorithms because no pruning is performed. Pruning can be avoided in Dinamica EGO but not in LCM.

3.6.1.1. ANALYSIS

In principle, and without making any specific assumption, the allocation of pivot-cells to final states is a two-step process at any time step t_s in a simulation, as described in section 3.3.1. However, this procedure tests all pixels, and although theoretically correct, this could be inefficient. Indeed, most random draws of pixels are performed uselessly as most pixels will not change state, so that most of the computational time could be spent in checking that no transition occurs. There are at least two ways to deal with this problem: design computationally efficient algorithms to counterbalance this drawback or design a way to reduce the pool of pixels tested for transition. This second option is achieved by relying on the fact that the transition from u to v is more or less

probable depending on z and keeping only the most probable pixels for further allocation testing. The resulting pre-selection is called pruning in Dinamica EGO, and adopted in both Dinamica EGO and LCM.

In this logic, the two-step allocation method described above becomes a three-step one, the first two forming sub-steps of the sampling design:

1. Pruning (pre-selection) of an ensemble of appropriate pixels for transitions from u to other states v .
2. Random selection of a pixel in this pruned ensemble.
3. Allocation of a final state to this pixel.

Both Dinamica EGO and LCM implement pruning by restricting during the pre-selection step to the pixels most likely to undergo a transition. However, they do not agree on what “most likely” means in this statement. They also implement widely different allocation procedures. Dinamica EGO’s pruning approach truncates the probability density function of the focused transition by keeping only the highest probabilities. By construction, the bias is indeed significant for high levels of pruning as will become apparent in the illustrative example discussed next. Truncating at a fixed number of times of the quantity of change (the strategy adopted by Dinamica EGO) does not ensure that the number of pixels pre-selected in this way does encompass most of the probability distribution. A better strategy would have been to truncate at some given fraction of the cumulated distribution (e.g., 90%). This would have given some control on the magnitude of the resulting bias.

LCM is even more radical. It only selects the exact number of pixel with the highest probability of $\rho(z|u, v)$ to achieve the allocation. The post-allocation probability density is strongly biased. The pruning procedures of Dinamica EGO and LCM are described in Appendices 3.E and 3.F and analyzed in more detail in Appendix 3.G.

As previously stated, one can also consider more computationally efficient algorithm to solve the problem of processing large numbers of pixels (e.g., tens of millions). The Multinomial Sampling Test described in appendix 3.B makes use of a unique multinomial test to allocation final states to pixels, and is very efficient. As a result, pruning is no longer needed, and is not used in our uSAM and uPAM methods.

It is also possible in principle to elaborate unbiased pruning algorithms, by uniform pruning instead of truncation of the relevant probability distribution. This option will be discussed elsewhere.

3.6.1.2. ILLUSTRATION

The post-allocation comparison method (sections 3.5.1 and 3.5.2) is applied to Dinamica EGO (DE), LCM¹³ and uSAM. For each models, the post-allocation probabilities have been averaged over 273 allocations, as indicated earlier¹⁴. Fig. 3.3 represents the returned allocated maps and Fig. 3.4 is a graph of the post-allocation distribution $P(v|u, z)$.

The post-allocation results of each model are compared to the expected transition probability distribution. We make here some comments on the outcome of this comparison:

uSAM

The uSAM allocation algorithm output is equal to the expected distribution (Fig. 3.4). A satisfying resulting map is displayed on Fig. 3.3a.

DE Three different pruning factors have been tested for Dinamica EGO : $F = 10$, $F = 20$ and $F \geq 50$. Dinamica EGO orders pixels in decreasing order of $P(v|u, z)$ and selects a number of pixels equal to $F P(v|u) \# J$ in this decreasing ordering. Consequently Dinamica EGO omits a significant part of all pixels (Fig. 3.4). Dinamica EGO's pruning method confines transited pixels to a ring of limited radial extent (Fig. 3.3b). The higher the pruning factor, the lower the bias of the output. In this test, $P(v|u)^{-1} = 50$ was chosen. Thus, $F \geq 50$ corresponds to no-pruning as illustrated on Fig. 3.4. The asymmetry observed for lower F values with respect to the maximum of $\rho(z|u, v)$ is a consequence of the fact that Dinamica EGO prunes by truncation the $P(v|u, z)$ probability distribution instead of $\rho(z|u, v)$ itself as pointed out in appendix 3.G.

LCM

The distribution returned by LCM is highly biased (Figs. 3.4 and 3.3c). The asymmetry on Fig. 3.4 is similar to the one produced by Dinamica EGO. Indeed, as for the other two methods, we have used the probability of transition $P(v|u, z)$ as input of the allocation method. The result would probably have been worse, had we used $\rho(z|u, v)$, as discussed in Appendix 3.G.

The uSAM and uPAM (not illustrated here) algorithms return a better post-allocation probability density function but consider a larger volume of pixels

¹³Even if Dinamica EGO and LCM have been successfully recoded and implemented in our LUCC modeling software CLUMPY, results presented in this paper are real outputs of the software.

¹⁴The Monte-Carlo approach is applied for each models except LCM which is not a random allocation and always returns the same allocated map.

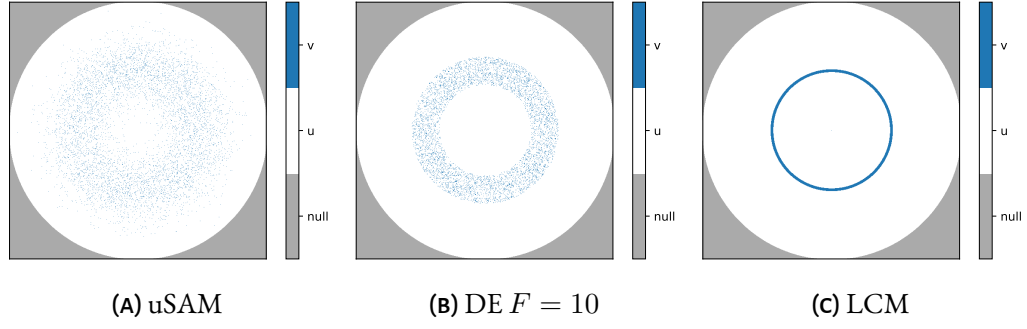


FIGURE 3.3: Pruning bias illustrated through resulting LUC maps of one allocation time step for 3 allocation methods : (a) uSAM; (b) Dinamica EGO (DE) with $F = 10$; (c) Idrisi LCM.

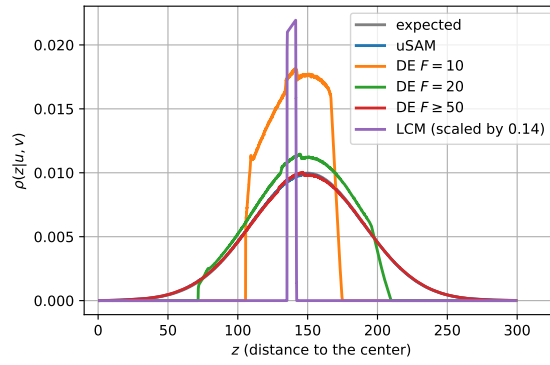


FIGURE 3.4: Pruning bias illustrated through the $\rho(z|u, v)$ post-allocation distribution averaged over 273 Monte Carlo draws compared to the pre-allocation distribution for several allocation methods: uSAM, Dinamica EGO (DE) with pruning parameters $F \in \{10, 20, 50\}$ and Idrisi LCM scaled by 0.14. Note that grey (expected), blue (uSAM) and red (DE $F \geq 50$) lines are nearly exactly superposed.

because no pruning is performed. This is possible due to the high efficiency of modern computers and random number generators, and by optimized programming, even in an interpreted language like Python.

3.6.1.3. MULTI-STEP VS SINGLE STEP INDUCED BIAS

An example showing yet another unexpected consequence of this pruning bias is also provided. It relates to the fact that the post-allocation distribution $P(v|u, \mathbf{z})$ for a given time span depends on the number of steps chosen to perform such a simulation. This is not acceptable, as this number of steps is largely arbitrary: simulation results (in any scientific field for that matter) cannot depend on the (arbitrary) time step of the simulation.

To illustrate this point we performed the same simulation as in the previous section but we have divided the time step by a factor 3 and therefore ran the simulation for 3 time steps. The chosen number of Monte Carlo simulation is still 273. The transition probability map is adjusted in order to have the same

expected distribution after 3 steps than a single step in the previous setting (this mostly consists in adjusting the total quantity of change per time step).

This results of this test are the following. uSAM provides a satisfying post-allocation $P(v|u, z)$ which is exactly the expected one, and the same as previously. On the contrary, a significant additional bias is obtained with Dinamica EGO, for the following reason. By construction, if the same same pruning factor F is kept (as most users do when using Dinamica EGO), the multi-step process selects a lower number of pixels at each sub-time step compared to the original run. This mechanically increases the bias due to pruning. In order to reach the same level of bias, the pruning factor would have to be multiplied by the number of sub-time steps. Indeed, in this example, the bias level obtained for a single step allocation with $F = 10$ (Fig. 3.4) is the same as for a 3 sub-steps allocation with $F = 10 \times 3 = 30$.

This consistency check (independence of the obtained allocation statistical results on the chosen time-step) does lead to the same, extremely biased output for LCM.

3.6.2. MONO-PIXEL PATCH AND MULTIPLE TRANSITIONS: MULTIPLE TRANSITION BIAS

The test problem of 3.5.3.2 made apparent the multi-transition bias. This bias can appear when there is competition on a pixel for several final states that all have a non-zero probability of occurring. If the allocation algorithm was not specifically designed to be bias-free, the pixels that can change state to several final states are *in fine* under-allocated with respect to their transition probability. This bias is ultimately rooted in the way pixels must be removed from the pool in the course of an allocation method without replacement.

3.6.2.1. ANALYSIS

The description of Dinamica EGO's allocation method in appendix 3.E allows us to understand this multi-transition error for this software. If we assume that the unselected pixels are kept in the sample instead of being discarded, the pixels that can undergo two transitions are put back in the pool after the first rejection test twice less often than those that can only undergo one. So their relative probability of being drawn later also decreases while the allocation probability remains unchanged (but not by a factor of 2, much less in fact, since we have to take into account the pruning factor, which makes the transited pixels only a fraction of those removed from the sample). Thus, as the draw progresses, the pixels corresponding to a single final state are drawn relatively more often with

an unchanged allocation probability. In fine, pixels whose possible final state is unique are thus over-represented compared to the pixels can change to more than one final state. By construction, this problem does not arise when only one final state exists.

LCM, on the other hand, uses an overly simplistic allocation method (see Appendix 3.F) that implies such large biases that it inhibits observation of the multiple transition bias.

A method which considers independently all pixels and all final states at once such as uSAM does not present this kind of bias. The uPAM method recalculates between each patch the transition probability by updating the quantity $P(z|u)$ to avoid this bias.

3.6.2.2. ILLUSTRATION

The post-allocation comparison method (sections 3.5.1 and 3.5.2) is applied to Dinamica EGO, LCM and uSAM. For each models, the post-allocation probabilities have been averaged over 420 runs of the same allocation step, as discussed earlier.

The results obtained can be summarized as follows:

USAM

The uSAM algorithm is bias-free for both transitions by design, as checked on Fig. 3.6a. The resulting LUC map is also satisfying (Fig. 3.5a).

DINAMICA EGO

Dinamica EGO's pruning factor is set to $F = P(v|u)^{-1} = 20$ which implies the selection of all pixels with $P(v|u, z) > 0$. In this way, one excludes pruning as a potential source of bias. As described above, the probability to draw a pixel with two possible final states decreases with the ongoing process. Consequently, the high value of $P(v|u)$ allows us to observe less transitions for such pixels. Fig. 3.6b illustrates this bias, whereas it is not visible to the naked eye on the associated map (Fig. 3.5b). This bias is cumulative: it increases with the number of time steps in a simulation. It may therefore affect real case study global simulation to a more significant level than visible here, a point that can only be ascertained through dedicated analysis in more realistic settings.

IDRISI LCM

LCM pruning method described in Appendix 3.F favors pixels with high values of $\rho(z|u, v)$. However, since the transition probabilities which are greater than zero are equal, the process struggle to allocate pixels in a satis-

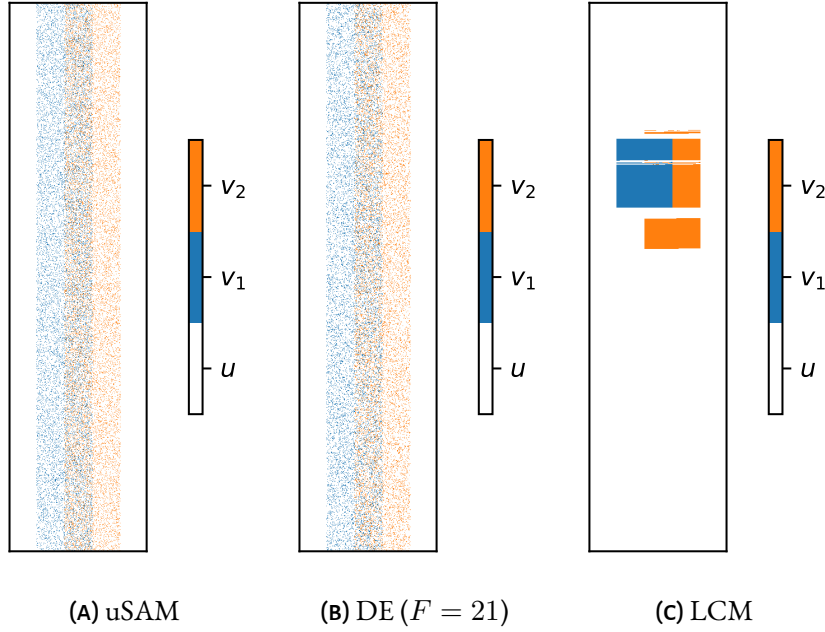


FIGURE 3.5: Multi-transition bias illustrated through resulting LUC maps of one allocation time step for 3 allocation methods : (a) uSAM; (b) Dinamica EGO (DE) with $F = 10$; (c) LCM. As the source of bias is small for Dinamica EGO, it is not apparent on this map, but is visible on the probability distributions shown on the next figure. The source of bias is large for LCM.

fying way. Indeed, as illustrated in Fig. 3.5c, an arbitrary set of pixel is allocated to v_1 and the transition to v_2 is accomplished in a second step. Also, the allocation algorithm breaks the spatial symmetries of the distribution. This follows because probabilities being uniform, the algorithm makes arbitrary selection choices, possibly seeded by round off errors machine precision; this feature is produced by our artificial setting, but is nevertheless a weakness of the allocation algorithm. As a result, Fig.3.6c shows a very biased distribution of $\rho(z|u, v_2)$.

Again, only the uSAM method achieves an unbiased allocation on this multi-transition case study. The uPAM method, not shown here, has the same behavior.

3.6.3. MULTI-PIXEL PATCH AND SINGLE TRANSITION: PATCH MERGING BIAS

The test problem of 3.5.3.3 reveals a bias that is inherent to the generation of multi-pixel patches, by producing spurious patch mergings (patch merging is defined in section 3.2.4).

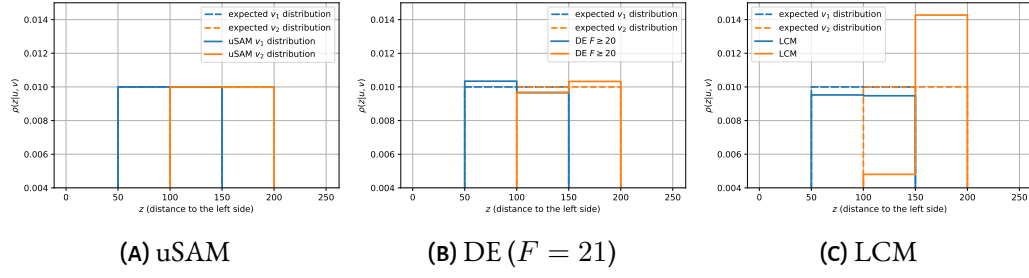


FIGURE 3.6: Multiple transition bias illustrated through the $P(v|u, z)$ post-allocation distribution averaged over 420 Monte Carlo draws compared to the pre-allocation distribution for 3 allocations methods: (a) simple unbiased allocation method; (b) Dinamica EGO (DE) with $F = 21$; (c) LCM.

3.6.3.1. ANALYSIS

During the patch design process, two patches of the same transition can merge and constitute together a single patch whose total size is approximately the sum of the two sizes. More than two patches can also merge. This produces an over-representation of the associated patch sizes compared to the pre-allocation size probability distribution. This constitutes a bias according to the discussion of section 3.3.2.

This bias can be avoided by checking during the patch design process if a merging is occurring in the process of allocating a new patch, as described in Appendix 3.I.

3.6.3.2. ILLUSTRATION

The post-allocation comparison method (sections 3.5.1 and 3.5.2) is applied to Dinamica EGO and uPAM. LCM is excluded from this study because it does not provide any patch construction method. But we have seen in previous subsections that LCM's way of producing contiguous areas of change is intrinsically biased; the problem examined here is of a more complex nature, and is inaccessible to LCM. For each model, the post-allocation probabilities are averaged over 273 repetitions of the same allocation time step.

Post-allocation results are compared to the expected probability distribution, leading to the following results:

uPAM

The uPAM algorithm uses a patch construction process which avoids patch merging (see Appendix 3.I). It returns both the expected $\rho(z|u, v)$ distribution (Fig. 3.8a) and the expected $\rho(\sigma|u, v)$ distribution (Fig. 3.8b)¹⁵.

¹⁵A very slight discrepancy exists for higher values of z , which is not visible on the figure. This is due to the fact that, although the seed pixel has been chosen correctly, the circular patch built around it does not take into account

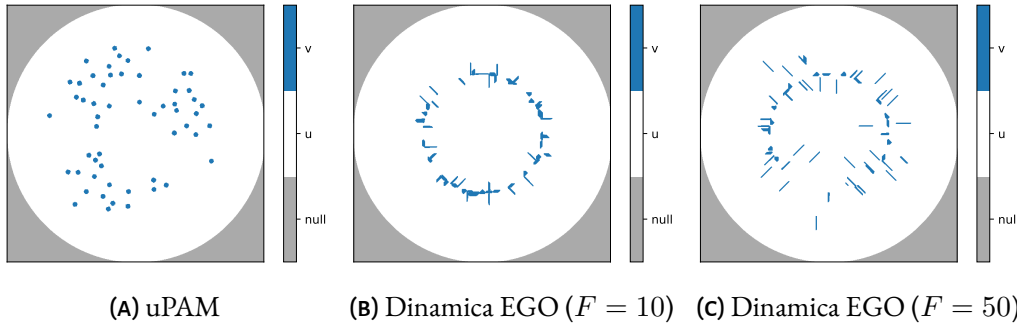


FIGURE 3.7: Patch size bias illustrated through resulting LUC maps for 2 allocation methods : (a) uPAM; (b) Dinamica EGO with $F = 10$; (c) Dinamica EGO with $F = 50$ (no pruning).

DINAMICA EGO

Dinamica EGO's behavior is displayed for two different pruning factors: $F = 10$ which inevitably produces a pruning bias (see section 3.6.1) and $F = 50$ which corresponds to an absence of pruning (all pixels are selected for the allocation process). The patch size distribution presents some small ghost replications of the original distribution at patch sizes that are multiples of the original mean size (Fig. 3.8b). These are produced by patch merging, as described above. More precisely, double ($s \approx 200$), triple ($s \approx 300$) and even quadruple patches are produced for $F = 10$ and are also noticeable on the resulting LUC maps (Fig. 3.7b and 3.7c). A small pruning factor reduces the number of pruned pixels and therefore increases merging occurrences. Moreover, the post-allocation distribution of $\rho(z|u, v)$ is quite different from the one expected one, and this even without pruning for $F = 50$ (Fig. 3.8a). This last point is rather surprising, but we have not been able to pinpoint the origin of this behavior.

Again, the uPAM algorithm is free of bias compared to Dinamica EGO regarding this particular potential source of allocation error. Note also that the patches formed by Dinamica EGO are not circular. The transition probability plays a predominant role in the construction of the Dinamica EGO patch, to the detriment of the elongation distribution which is highly distorted here with respect to the imposed choice (Fig. 3.7c). A related but different observation is also made in the Appendix 3.E.3.2.

the explanatory variable distribution (distance to the center). Selected pixels are slightly more numerous for higher z than for lower ones. This phenomenon is negligible in actual case studies because the transition probability is taken into account by our patch construction method (see Appendix 3.I) and because of the relatively small extension of the patches.

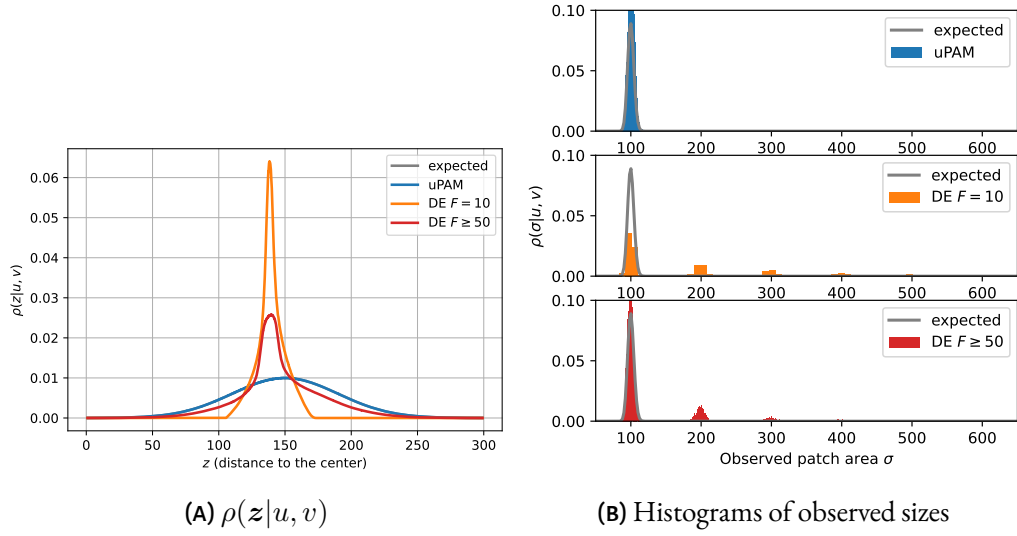


FIGURE 3.8: Patch size bias illustrated through (a) the transition probability $\rho(z|u, v)$, and (b) the size frequency histogram post-allocation distributions averaged over 273 Monte Carlo draws compared to the pre-allocation distributions for 2 allocations methods: uPAM and Dinamica EGO with $F = 10$ and $F = 50$ (no pruning).

3.6.4. MULTI-PIXEL PATCH AND MULTIPLE TRANSITIONS

We do not present any simulation results for this setting. We have checked that the previous biases all occur in such a more realistic context. However, we have not yet been able to identify a possible new source of bias in the test problems we have simulated so far.

3.7. DISCUSSION

Except for the pruning bias, the bias levels associated to the various sources of bias discussed in this section may seem moderate at first glance. However, one must keep in mind that all biases identified here are cumulative, so that their effect will become much more noticeable over the course of a whole simulation than it is for a single time step. Because of this, even apparently small biases may contribute to the difference of results between different modeling environment for the same case study and the same data.

In this work, we have ignored discrete explanatory variables and have also used a single continuous ones. There is no doubt that the sources of bias found are present as well for any mix of discrete and continuous variables. Using more than one variable would definitely be more realistic, but would make the setting more complex, and the display of the bias in explanatory variable space would be

more complex as well. Furthermore, the existence of the biases we have shown is not dependent on the type or number of variables used. We therefore aimed at the simplest configuration: a single continuous explanatory variable.

Formally, and introducing γ to designate a t -uple of discrete explanatory variables, the generalization to an arbitrary mix of discrete and continuous variables is operated in the expressions given in this article by substituting z, γ to z everywhere, and including sums over γ on top of integrals of z wherever appropriate.

A side issue mentioned a few times in the course of this paper and which has a major role in section 3.2.4 is that pattern-based LUCC modeling requires small enough time steps because small fractional total amounts of transitions are needed for the overall procedure to be self-consistent. Small amounts of change are often thought to be difficult to model but in our view, this is largely a validation issue. Validation based on spatial accuracy of allocation is somewhat meaningless for small amounts of change as many statistically equivalent sites are available for producing required LUC changes at each simulation time step; this point is quite clear, e.g., on Fig. 3.4 where qualitative statistical similarities or differences are immediately spotted. In fact, expecting spatial accuracy in such a context amounts to require a high level of predictability, a largely impossible requirement in a single time-step for a modeling approach that is statistical by design. Instead, statistical accuracy should be expected. Although, e.g., Pontius (2002) has proposed a number of methods to cover the position/quantity accuracy spectrum, these are not adapted in light of the preceding remarks for our purposes. Indeed, we have explained elsewhere that although improving the spatial accuracy of projections is an important goal in itself (and one requiring an improvement of the understanding of the causes driving LUC changes), one must find ways to evaluate the correctness of calibration and allocation methods independently of spatial accuracy, data quality or possible errors in model set-up (see section 2 of Mazy and Longaretti 2022a). The criteria of section 3.3.2 constitute a relevant and complete set of validation requirements in this respect. For example, if a validation map is available, comparing the projected and existing probability distributions (once a correct allocation method is available and calibration has been adequately performed) may provide the modeler with a useful tool to track either errors in explanatory variables identification or in scenario formulation, if calibration data are precise enough. The allocation methods presented in section 3.4 are formally and algorithmically correct for both small and large amounts of change.

To summarize this very important point, a major feature of our process of error and bias identification is that *the comparison is performed on the pre- and*

post-allocation probability distributions in explanatory variable space and not physical space, contrarily to the comparison and validation strategies widely used in the field. This follows because, on a single time-step (and to a lesser extent throughout a complete simulation), the amounts of change are small, and physical space comparison will not be able to identify whether the observed changes have the same statistical properties as the expected ones, except for very important sources of bias, such as pruning. This point is also hopefully made clear through the test problems of section 3.6.

3.8. CONCLUSION

The present paper provides another step towards a systematic reinvestigation of pattern-based LUCC modeling from a mathematical and statistical point of view, following our analysis of calibration methods (Mazy and Longaretti, 2022a). This endeavor is motivated by the fact that the discrepancies pointed out in the literature for the projections obtained from various models for the same data and scenarios can be ascribed to differing modeling and algorithmic choices across the available modeling environments. However, a significant number of these choices cannot be freely made by the modeler as they are constrained by probability theory, and the algorithms used must be designed not to violate probabilistic and statistical constraints. Our objective is precisely to pinpoint where such constraints have been ignored or not correctly implemented in existing modeling strategies, in order to provide alternative conceptually correct, quantitatively precise and computationally efficient algorithms for pattern-based LUCC modeling.

Such an undertaking should eventually encompass all the aspects of the problem, namely, calibration, allocation, and formal and algorithmic validation (which, as pointed out earlier unavoidably differs from usual validation methods in LUCC modeling). Furthermore, another overarching element of pattern-based LUCC modeling strategies should also be examined more closely, namely, the Markovian, discrete time formulation of LUCC simulations. The present work focuses on allocation.

Our analysis is designed to bring out the requirements that any modeling framework based on non-parametric probability distributions must satisfy. In particular, allocation methods are characterized by two steps: sampling design and allocation procedure (section 3.3.1). This two step characterization is motivated by concepts in sampling theory, where sampling designs without replacement (the ones of interest for LUCC modeling) are well-known to be difficult

to formalize and implement. This approach constitutes the theoretical backbone of the present paper.

Unbiased allocation methods are defined in section 3.4 for both mono-pixel (unbiased Simple Allocation Method – uSAM) and multi-pixel patches (unbiased Patches Allocation Method – uPAM). In addition, Appendix 3.I presents our patch construction method.

It is customary in LUCC studies to provide only one simulation, which also gives an incorrect impression of prediction to users and land planners. Providing a number of them would instead be both more theoretically correct and would help to dispel this false impression. The algorithms we have elaborated are efficient enough to adopt a Monte-Carlo approach to LUCC simulations instead of the common single simulation one. This would give both better statistical significance to the result and a better grasp of their statistical nature.

The study of existing allocation methods was motivated in particular by the need to identify sources of error and differences of outcome observed *a posteriori* on different software. This is possible only if one has a criterion allowing us to pinpoint errors and biases, and *our bias criterion, based on a necessary consistency requirement (section 3.3.2), is instrumental* in this respect. Furthermore, we propose a rather systematic method of identification of allocation biases. The idea is to compare software outputs in test problems of increasing complexity in order to detect as many biases as possible and at the same time identify their origin. This method imposes *a priori* the relevant transition probabilities in controlled settings, which allows us to focus only on the allocation module.

We have tested our method of allocation bias identification on two non-parametric pattern-based LUCC modeling environments, Dinamica EGO and Idrisi LCM, and compared it to our own uSAM and uPAM algorithms. This process allowed us to identify quite a few sources of errors and biases, and pinpoint their origin in a precise way, and to check that our proposed algorithm are indeed error- and bias-free for any practical purpose. Note that most of the biases found in this way are generic, i.e., not specific to the controlled setting in which they have been identified and studied. Our major results — besides the conception of the uSAM and uPAM algorithms of section 3.4 — are that Dinamica EGO implements an incorrect pruning procedure and a somewhat biased allocation algorithm, a point that is made more obvious when multiple transitions are present. Pruning can be avoided in Dinamica EGO, therefore avoiding its major source of bias; the other sources can be corrected by correcting its allocation algorithm. On the contrary Idrisi LCM is highly biased due to pruning, and its simple allocation procedure is theoretically incorrect; most LCM failures relate directly to its allocation procedure implemented in

this modeling environment. Pruning cannot be avoided in LCM, as it is inherent to its allocation strategy. Another source of problem is that its pruning strategy is not applied to the formally correct probability distribution.

We feel that allocation in both modeling environments could be made error- and bias-free with reasonable additional work, this objective being easier to reach for Dinamica EGO than for LCM. To achieve this purpose, these modeling environments (and others, for that matter) should have access to error- and bias-free allocation methods that are also algorithmically efficient. We point out for now that although no pruning strategy is used in the algorithms we proposed in section 3.4, they are nevertheless computationally efficient and much faster than, e.g., Dinamica EGO, even if our Python reimplementation of the pivot-cell allocation is already more efficient than the original.

CLUMondo (van Vliet et al., 2015) is another popular modeling software and we now discuss it briefly. CLUMondo implements a very different modeling strategy, based on a single LUC map to determine the LUC probability distribution. Moreover, these probabilities are determined by logistic regression (parametric method), which greatly constrains the shape of the distribution, and this is the origin of a major bias in the calibration module (Mazy and Longaretti, 2022a). Allocation follows from the single map probability distribution. Pixels are selected for transition on two criteria: highest probability and a weight depending on the distance to pixels which are already in the final state (no indication about the setting of this parameters are given). In case of multi-transitions, the user can indicate a priority order for each. This very basic approach is highly biased. This being said, we could not include CLUMondo in our benchmark comparison tests because this software does not allow the user to bypass the calibration step, whereas this is required by our protocol. Also, CLUMondo is generally very complicated to configure for testing outside conventional cases which makes it difficult to use in such a benchmark.

Possibly model developers may feel that the issues addressed in this work are less important than the intrinsic imprecision of the calibration data. Even if this is true (which is not obvious by itself), such an assumption also needs to be quantified for LUCC modeling to reach a more robust scientific status. In this respect, note that all the biases brought to light in section 3.6 are cumulative, i.e., they would be much more prominent over the course of complete simulations than over the single time step used in our analysis. Another source of concern comes from the intrinsic variability in results for a specific problem due to the wide spectrum of modeling methods (and not only in pattern-based LUCC models), only briefly mentioned at the beginning of the introduction. We believe — or maybe just hope — that this spread in obtained results could be

significantly reduced if arbitrary choices were eliminated in a systematic way in these various alternative approaches, as was endeavored here for pattern-based LUCC modeling.

The method we have followed may possibly be adapted, *mutatis mutandis*, to such a task. The essence of this method is simple enough when stripped down to its essential core: define in a mathematically precise enough way what a LUCC model of a specific type should do, derive from this definition discriminating self-consistency requirements for any unbiased theoretical or algorithmic implementation of this model type, elaborate one such implementation (without paying much attention to computational efficiency in a first step, or at least not sacrificing correctness for speed), and then elaborate categories of problems of increasing complexity to test this unbiased method and contrast it with existing ones. The difficulty resides in striking a balance between mathematical rigor and practical usefulness, identifying the sometimes subtle formal issues involved in the process, and being able to gather or infer sufficiently detailed information on how existing methods have been elaborated and implemented — this last requirement going contrary to current trends in publishing habits. We did our best to achieve such a balance, but only feedback from LUCC modelers will tell us if we were successful in our attempt. We welcome in advance any such critical and constructive feedback.

APPENDICES TO CHAPTER 3

3.A. PATCH-RELATED PROBABILITY DISTRIBUTIONS

This Appendix introduces more formally patch probability distributions. It also discusses and gives some justification to the assumptions made in section 3.2.4.

This and the following three Appendices introduce a number of quantities that have not been defined earlier. These new quantities are rather numerous, and for convenience and reference, are collected in Table 3.2.

element	set	definition
$\#J_{z,v}, \#J_{z,v}^c$	$\mathbb{N}$	Number of pixels (resp. pivot-cells) in a small volume δz around z in explanatory variable space
$s_j, \delta_j^z, \delta_{i,j}$	$\mathbb{N}$	Bivalued (0 or 1) variables defined for counting purposes
n, n_w, n_b	$\mathbb{N}$	Numbers of balls, white balls, black balls in an urn (Appendix 3.C)
$n^{(p)}, n_w^{(p)}, n_b^{(p)}$	$\mathbb{N}$	Same as above at the p th draw (Appendix 3.C)
N, N_v	$\mathbb{N}$	Number of balls in a generalized urn sampling problem (Appendix 3.D)
$N^{(p)}, N_v^{(p)}$	$\mathbb{N}$	Same as above at the p th draw (Appendix 3.D)
$\Delta, z_\Delta, \delta z_\Delta$	not defined, $\mathbb{R}^d, \mathbb{R}$	Bin in explanatory variable space of (small) volume δz_Δ and with nearly constant explanatory variable value z_Δ
$N_{u,\Delta}; N_u, N_v$	$\mathbb{N}$	Number of pixels in bin Δ , in initial state u ; number of these pixels in final state u, v
$p_{v,\Delta}$	$\mathbb{R}$	Shorthand notation for $P(v u, z_\Delta)$
$E_X(Y)$	$\mathbb{R}$	Expectation value of random variable ¹⁶ $Y(X)$ with respect to the probability distribution of variable X

TABLE 3.2: Summary of the notations and definitions used in Appendices 3.A, 3.B, 3.C and 3.D.

3.A.1. PATCH PROBABILITY DISTRIBUTION

In this work, patch distributions are characterized by their pivot-cell distribution in explanatory space, their size and their elongation (i.e., a minimal way to characterize patch shapes); the last two are defined in real, 2D space. As a matter of fact, patch characteristics do not seem to be a point of much focus in LUCC studies. Our assumptions about patch probability distributions combined to our patch construction method (Appendix 3.I) produces patches with some degree of realism in their randomness.

On the formal level, let us introduce, for each initial and final state pair (u, v) , the patch pivot-cell explanatory variables set z^c , and shape parameters T : $T = (\sigma, e)$, where σ is the patch size and e the patch elongation. The patch probability distribution can therefore be written $\rho^P(z^c, \sigma, e|u, v)$. It is conditional to a given LUC state transition, as indicated by the notation. This distribution generalizes to patches the pivot-cell distribution $\rho^c(z^c|u, v)$ introduced in section 3.2.4. Note that the distribution $\rho(z|u)$ has no patch equivalent, as

patches exist only for a given transition.

Elongation is formally defined and characterized in Appendix 3.I.2. Our definition allows the user to calibrate the elongation distribution, if needed, because elongations are well-defined on any patch, and because our patch construction algorithm does enforce in a statistical way the chosen elongation. The size distribution can also be calibrated from calibration data. However, this calibration approach has apparently not yet been used in the LUCC community for these two patch characteristics, and for the purpose of the present work, specific size and elongation distributions are assumed instead, for simplicity.

The present work also assumes that the three patch parameters $\mathbf{z}^c$, σ and e are statistically independent:

$$\rho^P(\mathbf{z}^c, \sigma, e|u, v) = \rho^c(\mathbf{z}^c|u, v) \times \rho(\sigma|u, v) \times \rho(e|u, v), \quad (3.A.1)$$

where the P superscript is dropped from the distributions of size and elongation, for simplicity. There is no *a priori* reason for this to be the case, but in our experience, this does seem to be at least approximately true. This independence assumption is adopted here for the sake of simplicity. It may be relaxed in the future if needed, and with appropriate calibration data. If this is the case, the allocation algorithms and associated proofs of absence of bias will need to be reconsidered.

3.A.2. IDENTITY OF PIVOT-CELLS AND PIXELS COMPLETE DISTRIBUTIONS

This is the last assumption made in section 3.2.4:

$$\rho^c(\mathbf{z}^c|u, v) = \rho(\mathbf{z}^c|u, v). \quad (3.A.2)$$

This assumption is natural enough, but its conditions of validity are not obvious. There is however a sufficient condition under which it is satisfied. To see this, let us define a characteristic patch scale $L_p = E(\sigma)^{1/2}$, and introduce the smallest of the characteristic explanatory variables scale (in physical space), L_z . If $L_p \ll L_z$, or even $L_p \lesssim L_z$, the distributions $\rho(\mathbf{z}|u, v)$ and $\rho(\mathbf{z}|u)$ will be nearly uniform over a patch. Thus the probability density distribution of all pixels is the same in any patch, and this includes the pivot-cell.

Let us now show this is a slightly more formal way. To this effect, let us introduce some small volume $\delta\mathbf{z}$ around some specific value $\mathbf{z}$ in explanatory variable space. Because continuous explanatory variables are function of spatial location, this small volume will implicitly define some scale L_δ in physi-

cal space¹⁷. We choose $\delta\mathbf{z}$ such that $L_p \ll L_\delta \ll L_z$ (but the argument holds approximately even if these inequalities hold only approximately, i.e., if $L_p \lesssim L_\delta \lesssim L_z$ instead). Let us also define $E(\#J_{\mathbf{z},v})$ as the average number of pixels at $\mathbf{z}$ within $\delta\mathbf{z}$, for give initial and final state u, v . By definition

$$E(\#J_{\mathbf{z},v}) = E(\#J_v) \int_{\delta\mathbf{z}} d\mathbf{z} \rho(\mathbf{z}|u, v) \simeq E(\#J_v) \rho(\mathbf{z}|u, v) \delta\mathbf{z}. \quad (3.A.3)$$

Because all transited pixels belong to a patch, the expected cumulative patch size σ_T at $\mathbf{z}$ within $\delta\mathbf{z}$ is:

$$\sigma_T = E(\#J_{\mathbf{z},v}) = E(\#J_v) \rho(\mathbf{z}|u, v) \delta\mathbf{z}. \quad (3.A.4)$$

On the other hand, by definition of $\rho^P(\mathbf{z}^c, \sigma, e|u, v)$ the same expected cumulative size is given by:

$$\sigma_T = E(\#J_v^c) \int_{\delta\mathbf{z}} d\mathbf{z} d\sigma de \sigma \rho^P(\mathbf{z}^c, \sigma, e|u, v) \simeq E(\#J_v^c) E(\sigma) \rho^c(\mathbf{z}|u, v) \delta\mathbf{z}. \quad (3.A.5)$$

From Eq. (3.11), $E(\#J_v) = E(\#J_v^c) E(\sigma)$ and the comparison of Eqs. (3.A.4) and (3.A.5) leads to $\rho^c(\mathbf{z}|u, v) = \rho(\mathbf{z}|u, v)$, i.e., the required result.

3.B. MULTINOMIAL SAMPLING TEST ALGORITHM

This Appendix deals with monapixel patches. These are rarely if ever encountered in actual maps, but are a useful tool in the present work, either as a first step in multipixel patch design algorithms, or as a tool to identify sources of bias. Because only monapixel patches are dealt with here, the notations ρ^c and ρ , and $\mathbf{z}^c$ and $\mathbf{z}$ can be used interchangeably in the present Appendix.

3.B.1. TEST DESCRIPTION

For any pixel j , u^j and $\mathbf{z}^j$ are known as well as the transition probability $P(v|u^j, \mathbf{z}^j)$ for all v (Eq. 3.1). Let us define η_w as the cumulative sum¹⁸ of

¹⁷It may define several such scales, as $\mathbf{z}$ may be found at several spatial locations, in which case L_δ is the largest of the various scales associated to the various locations.

¹⁸For simplicity, we consider that the set of final states V is ordered and the values of its elements are consecutive. This set is identified to a series of integers $0, 1, \dots$. By convention $w = 0$ is an unaffected state (pixels outside the map, or pixels with unknown state).

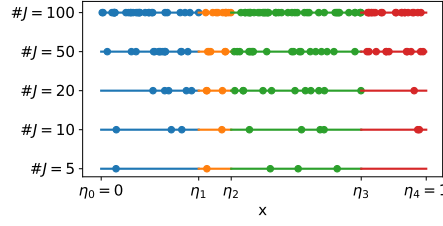


FIGURE 3.9: Outcome of the multinomial sampling test in a simple idealized setting with four possible final output states. These output states are sections of the unit interval. The length of these sections have been chosen arbitrarily. The distribution of random draws along the unit section are represented from bottom to top for an increasing number of draws (5, 10, 20, 50, 100 respectively).

$P(v|u^j, \mathbf{z}^j)$:

$$\forall w \in V, \quad \eta_w = \sum_{v \in V | v \leq w} P(v|u^j, \mathbf{z}^j). \quad (3.B.1)$$

By convention, $\eta_{-1} = 0$ (there is no $w = -1$ state but this starting value is needed in the logic of the test). Then, a unique random float is sufficient to test simultaneously all possible transitions for this pixel. For any real random number ξ in $[0, 1[$, one can find an index w such that $\eta_{w-1} \leq \xi < \eta_w$. The final state $v = w$ is then affected to this pixel. This is a well-known test in probability theory, based on the fact that the random variable Y defined on the cumulative distribution function of a probability distribution $P(x)$ [i.e., $Y = P(x \leq X)$] has a uniform probability. By definition of a random draw, this test enforces Eq. (3.1) for each pixel, and the resulting distribution of N pixels will satisfy Eqs. (3.6) and (3.7) (see next subsection).

In plain words, we consider output states as sections of the unit interval with length equal to the various transitions probability; the total length is then equal to one (as the transition $u^j \rightarrow v = u^j$ is included in the process). Intuitively, it is clear that distributing pixels at random on this unit interval will produce relative number of pixels in each possible final state proportional to the length of each interval, on average (Fig. 3.9), and therefore comply with $P(v|u^j, \mathbf{z}^j)$, on average.

This test presents several advantages: all possible final states are tested at the same time, and the order of the sections in the unit interval is irrelevant. Algorithm 2 right below (Multinomial Sampling Test, hereafter MuST) provides a pseudo-code implementation of this allocation method.

In this algorithm, line 3 attributes a random number to all pixels simultaneously¹⁹. Line 5 is a test of the embedded inequality, and the loop at lines 4-6

¹⁹RAND(0,1) stands for a random number generator in the interval $[0,1]$. It is advisable to use libraries optimized

Algorithm 2: Multinomial Sampling Test MuST

Data:

The set of all pixels indices J ,
 The set of LUC states V ,
 $\{P(v|u^j, z^j)\}_{\forall j \in J, \forall v \in V}$, the transitions probabilities

Result: A new state is drawn for each pixel according to transitions probabilities

```

1  $\forall j \in J, \forall w \in V$ , compute  $\eta_w$  according to (3.B.1)
2  $\forall j \in J, \xi^j \leftarrow \text{RAND}(0, 1)$ 
3 foreach  $v \in V$  sorted in the reverse order do
4    $\lfloor \forall j \in J \mid \xi^j < \eta_v, v^j = v$ 
5 return  $\{v^j\}_{\forall j \in J}$ 

```

stops for the first value v (in decreasing order) that satisfies this inequality.

Note that the same algorithm can be used *mutatis mutandis* for any discrete probability distribution. Note also that this multinomial sampling test does also in effect realize a binomial sampling test for any given final state v : one just has to regroup all segments η_w with $w \neq v$ after the algorithm has run to achieve this purpose.

3.B.2. STATISTICAL PROPERTIES OF THE MUST ALGORITHM

The fact that this algorithm is unbiased is ultimately rooted in the fairness of the random number generator used. Modern random number generators are highly robust in this respect (the number of draws needs to be ridiculously large to achieve a noticeable level of non-randomness in the series of numbers generated in this way). In the proof of absence of bias of the MuST algorithm presented below, this property is represented by the assumption that each particular pixel trial faithfully represents a Bernoulli trial in the mathematical sense.

We can proceed with this proof by introducing another type of samples on the pixel population. They refer to the transition from u to v . Let us define first a random variable S_j with possible values s_j for each pixel j , such that $s_j = 1$ if the pixel has been selected in the sample to change state from u to v , and $s_j = 0$ otherwise; S_j is a Bernoulli variable associated to pixel j . The samples of interest here, $S_{u,v}$, do collect the values of s_j produced by the allocation method: $S_{u,v} = (s_1, s_2, \dots)$.

By construction, the probability distribution of s_j is a Bernoulli probability distribution:

$$p_j = p(s_j) = P(v|u, z^j)^{s_j} \times [1 - P(v|u, z^j)]^{1-s_j}, \quad (3.B.2)$$

to generate such numbers for all pixels at once.

and the probability of realizing a given sample $S_{u,v}$ is given by $\prod_j p_j$ (this fully specifies the sampling design for these new samples, according to our earlier definition). Indeed, if the random number generator is truly random, the result of the trial of pixel j produces the state v with probability p_j , by construction.

This allows us to calculate various statistical quantities on the samples $S_{u,v}$. This makes use in an intermediate step of the number $\#Jg(\mathbf{z}, u)d\mathbf{z}$ of pixels in initial state u and explanatory variables $\mathbf{z}$ within some small (infinitesimal) volume $d\mathbf{z}$. By definition, $g(\mathbf{z}, u) = \rho(\mathbf{z}|u)$ (this quantity is known as the degeneracy factor in physics).

For example, the number of pixels in final state v in a given sample can be obtained from $\#J_v \equiv \sum_j s_j$, and the mean expected number of these pixels follows from

$$E(\#J_v) = E\left(\sum_j S_j\right) = \sum_{s_j, j} s_j p_j = \sum_j P(v|u, z^j) \quad (3.B.3)$$

$$= \#J \int_{\mathbf{z}} P(v|u) \times \frac{\rho(\mathbf{z}|u, v)}{\rho(\mathbf{z}|u)} \times g(\mathbf{z}, u) d\mathbf{z} \quad (3.B.4)$$

$$= P(v|u) \#J \int_{\mathbf{z}} \rho(\mathbf{z}|u, v) d\mathbf{z} = P(v|u) \#J. \quad (3.B.5)$$

The last equality recovers the expected result. This follows essentially from the transformation of a sum over j in an integral over $\mathbf{z}$.

Even though no such compact expression is available for the variance, it is useful to recall the result. Because all S_i are independent random variables, the variance of their sum, $V(\sum S_j) = \sum V(S_j)$, i.e.:

$$V\left(\sum_j S_j\right) = \sum_j P(v|u, z^j) [1 - P(v|u, z^j)] = \sum_j p_j (1 - p_j). \quad (3.B.6)$$

In fact, this formulation allows us to recover much more, as the probability distribution of the $S_{u,v}$ samples is exactly known from Eq. (3.B.2). For example, a slightly different form of the previous calculation gives us the expected probability distribution with respect to z for a given initial and final LUC state of the sample, which, in the mean, should be $\rho(z|u, v)$.

To show this, let us define an indicator variable δ_j^z such that $\delta_j^z = 1$ when $z^j = z$ within some small range δz and $\delta_j^z = 0$ otherwise. Let us define yet another sample, $S_{z,u,v} = (s_1 \delta_1^z, s_2 \delta_2^z, \dots)$. By definition, $S_{z,u,v}$ allows us to collect all pixels that have been drawn with the value z within δz of the set of explanatory variables, and allocated to v from u . One has (with the notation

used in section 3.3.2):

$$\begin{aligned}
E(\#J_{z,v}) &= E\left(\sum_j S_j \delta_j^z\right) = \sum_{s_j,j} s_j p_j \delta_j^z = \sum_{j, z^j=z \text{ within } \delta z} P(v_f|z^j, v_i) = \\
&= P(v|u) \times \frac{\rho(z|u, v)}{\rho(z|u)} \times \#Jg(z, u) \delta z \\
&= P(v|u) \times \rho(z|u, v) \#J \delta z,
\end{aligned} \tag{3.B.7}$$

so that

$$\frac{E(\#J_{z,v})}{E(\#J_v)} = \rho(z|u, v) \delta z. \tag{3.B.8}$$

This last quantity is the fraction of pixels in z within δz for given states u and v . Therefore, the associated probability density $\rho(z|u, v)$ is recovered from relevant expectation values, as advertized.

In other words, the pre-allocation probability distribution $\rho(z|u, v)$ (i.e., before the sample is drawn) is also the post-allocation distribution in the mean, for the transition u to v (i.e., the mean distribution of z values over a large number of samples). Note that consequently our simple allocation method is unbiased, as we have just proven that it satisfies Eqs. (3.6) and (3.7) from Eqs. (3.B.5) and (3.B.7). On the other hand, the assumption that the quantities of change must be small does not seem to have been used anywhere. Remember however that small amounts of change are required for pixels to be expected to undergo a single transition per time step, and this assumption underlies our sampling method.

3.C. A MONO-PIXEL ALGORITHM ENFORCING QUANTITIES OF CHANGE

Because pivot-cells are assumed to be statistically independent from one another, the MuST algorithm does attribute final states independently of how many pixels have (or not) already been selected in the sample for any final state v . As a result, the test does not ensure that the number of pivot-cells will always be equal to the expected mean [Eq. (3.11)] for each final state v , although, as demonstrated in the previous Appendix, this is true on average over a larger number of iterations of the allocation procedure (i.e., iterations of single-step simulations). In fact, as the probability distribution of repeated Bernoulli test is

a binomial distribution (a well-known result of probability theory) the number of selected pivot-cells may turn out to be zero, even if the probability of transition to the intended final state is not vanishing (but the probability of this to occur is usually extremely small).

3.C.1. SAMPLING WITHOUT REPLACEMENT FROM EXACTLY KNOWN POPULATIONS

It is in fact possible to enforce quantities of change exactly, but with a different algorithm. Enforcing such quantities of change is routinely done in LUCC modeling, as usually a single simulation of future projections (allocations) is performed, and as respecting the quantities of change provided through scenarios therefore becomes desirable in such a context. To achieve this purpose, however, one must change the sampling probability at every draw.

To show this explicitly, let us first examine a related but different problem. Namely, let us consider an urn containing initially (before balls are drawn from the urn) $n_w^{(1)}$ white balls and $n_b^{(1)}$ black balls, for a total $n^{(1)} = n_w^{(1)} + n_b^{(1)}$ balls. Balls are drawn without replacement at random from the urn. At the first draw, the probability of drawing a white ball is $p_w^{(1)} = n_w^{(1)}/n^{(1)}$, and similarly $p_b^{(1)} = n_b^{(1)}/n^{(1)}$ for a black ball, with $p_w^{(1)} + p_b^{(1)} = 1$.

The subsequent probabilities of draws are changed, because the number of balls in the urn has changed. Let us assume that, e.g., a black ball has been drawn at this first draw. The urn now contains $n_b^{(2)} = n_b^{(1)} - 1$ black balls, $n_w^{(2)} = n_w^{(1)}$ white balls for a total of $n^{(2)} = n^{(1)} - 1$ balls. The draw probabilities are therefore now changed to $p_w^{(2)} = n_w^{(2)}/n^{(2)} = n_w^{(1)}/(n^{(1)} - 1)$ and $p_b^{(2)} = n_b^{(2)}/n^{(2)} = (n_b^{(1)} - 1)/(n^{(1)} - 1)$. This new probability of drawing a white ball has increased as $n_w^{(1)}/(n^{(1)} - 1) > n_w^{(1)}/n^{(1)}$ and correlatively the new probability of drawing a black ball has decreased as $(n_b^{(1)} - 1)/(n^{(1)} - 1) < n_b^{(1)}/n^{(1)}$. A similar reasoning applies if a white ball has been drawn instead of a black one at the first draw.

Such changes of the probabilities apply at every draw, until the end of the process ($n^{(1)}$ draws performed) and are necessary so that in the end the total number of drawn white and black balls is actually $n_w^{(1)}$ and $n_b^{(1)}$, respectively.

3.C.2. MONO-PIXEL ALLOCATION ALGORITHM WITHOUT REPLACEMENT AND WITH EXACT QUANTITIES OF CHANGE ENFORCED

The problem just discussed provides us with a way to obtain to the desired algorithm for a single LUC state change, and by extension (section 3.C.3.4),

for an arbitrary number of simultaneous state changes. The problem is first discussed in a somewhat informal and intuitive way. More formal approaches are then progressively introduced. This discussion also serves as an introduction to the analysis of our multi-pixel algorithm (Appendix 3.D).

The reasoning is simpler if continuous explanatory variables are first binned. Such a context is relevant in its own right (as binning of explanatory variables is often performed in LUCC modeling software), but also as a stepping stone for the non-binned context of section 3.C.4.

To this effect, we focus on a specific but arbitrary bin Δ of explanatory variables $\mathbf{z}$. For simplicity, the bin is assumed to be small enough so that the value of $\mathbf{z}$ is nearly constant in the bin and approximated as $\mathbf{z}_\Delta$ (say, to the percent level for definiteness; alternatively we can define $\mathbf{z}_\Delta$ such the second equality is true in the following equation). The total number of pixels in initial state u in this bin is thus given by²⁰

$$N_{u,\Delta} = \#J \int_{\Delta} \rho(\mathbf{z}|u) d\mathbf{z} = \#J \rho(\mathbf{z}_\Delta|u) \delta \mathbf{z}_\Delta \equiv \#JP(\mathbf{z}_\Delta|u), \quad (3.C.1)$$

where the integral is performed over all $\mathbf{z}$ in the bin and where $\delta \mathbf{z}_\Delta$ is the small volume of the bin in explanatory variable space. Each pixel in the bin has therefore a probability $p_{v,\Delta} = P(v|u, \mathbf{z}_\Delta)$ of undergoing a transition from state u to v , and a probability $1 - p_{v,\Delta}$ of not undergoing a transition, at the initial draw. Finally, we wish to impose that the total number of transited pixels in the bin is equal to its expectation value, i.e., $p_{v,\Delta} N_{u,\Delta}$.

In order to enforce this total number of transited pixels exactly, it is sufficient to associate to each pixel draw a ball draw similar to the one described above. This can be done by assuming, e.g., that white balls stand for no state transition, and black balls stand for the considered $u \rightarrow v$ transition. Thus, the initial number of balls in the urn is $n^{(1)} = N_{u,\Delta}$, the initial number of black balls is $n_b^{(1)} = p_{v,\Delta} N_{u,\Delta}$ and the initial number of white balls is $n_w^{(1)} = (1 - p_{v,\Delta}) N_{u,\Delta}$ ($\#J$ is assumed to be large enough so that these numbers can be assimilated to the nearest integer with negligible fractional error).

A possible algorithm is then the following:

SAMPLING DESIGN

Draw at random a pixel in the pool of $N_{u,\Delta}$ pixels.

ALLOCATION PROCEDURE

Draw at random a ball from the urn, and associate to the pixel the corre-

²⁰Discrete explanatory variables if any are not included in this expression, to simplify notations. Adding them does not change the argument. This remark holds throughout this Appendix.

sponding outcome of the ball draw (transition or no transition depending on the ball color). Remove the drawn pixel and ball from their respective pools, update the ball numbers and ball draw probabilities as explained in section 3.C.1.

ITERATION

Repeat the process until all pixels (and all balls) have been drawn.

Clearly at the end of the process, the required number of pixels will have transitioned from the discussion of section 3.C.1. The algorithm is then repeated for all possible bins to complete the allocation process. This algorithm is not the only possible one (or even the simplest one) but has relevant features for the algorithm we propose for multi-pixel patch transitions.

3.C.3. A MORE DIRECT ALGORITHM ENFORCING QUANTITIES OF CHANGE

Using an urn and balls is helpful to understand intuitively the necessity of updating pixel numbers and probabilities in sampling without replacement. But this detour is unnecessary, and the algorithm can be formulated directly and equivalently on pixels as there is a one to one correspondence between pixels and balls and their respective probabilities of draw.

3.C.3.1. ALGORITHM STRUCTURE

From this argument, the algorithm can be reformulated as follows.

SAMPLING DESIGN

Draw at random a pixel in the pool of $N_{u,\Delta}$ pixels.

ALLOCATION PROCEDURE

Assign a final state (either u or v) with the MuST test for a single $v \neq u$ final state. After a draw is completed, the pixel is removed from the pixel pool and the probabilities of the MuST algorithm are updated as described in section 3.C.1.

ITERATION

Repeat the process until the required number of transition pixels is obtained.

3.C.3.2. SIMPLIFIED UPDATING

The only requirement is to update the number of pixels in the bin, $N_{u,\Delta}$ and the probability $p_{v,\Delta}$ in the same way as in section 3.C.1. From these two

quantities, the probability of no transition, $1 - p_{v,\Delta}$ and the number of non-transited pixels, $(1 - p_{v,\Delta})N_{u,\Delta}$ is also obtained, and the problem is completely specified at the next draw. This allows us to define an equivalent alternative and more convenient set of updated quantities:

- First, defining $N_{u,\Delta}^{(p)} = n^{(p)}$ the number of pixels in bin Δ and $\#J^{(p)}$ the number of pixels in state u at the p -th iteration, updating $N_{u,\Delta}^{(p)}$ is equivalent to updating $\rho^{(p)}(\mathbf{z}_\Delta|u) = N_{u,\Delta}^{(p)} / (\#J^{(p)} \delta \mathbf{z}_\Delta)$. More formally, this follows because the first equality itself holds independently of the bin size.
- Second, from Bayes rule, Eq. (3.1), updating $p_{v,\Delta}^{(p)}$ is equivalent to updating $P^{(p)}(v|u)$ once $\rho^{(p)}(\mathbf{z}_\Delta|u)$ is itself updated, because $\rho(\mathbf{z}_\Delta|u, v)$ is a probability density defining the suitability of pixels for the $u \rightarrow v$ transition, which, as such, should not be modified in the process (the fact that pixels are removed from the sample does not *a priori* affect the relevance — or lack of — of explanatory variable values for the considered transition).

Conversely one sees that updating these quantities — i.e., $\rho^{(p)}(\mathbf{z}_\Delta|u)$, $\#J^{(p)}$ and $P^{(p)}(v|u)$ — at each draw immediately specifies the new transition probabilities, under the same *a priori* given information. One expects that the updating of the quantities of change is operated through $P^{(p+1)}(v|u) = P^{(p)}(v|u) - \sigma_v^{(p)} / \#J^{(p)}$ where $\sigma_v^{(p)} = 1$ if the removed pixel in bin Δ is in final state v and $\sigma_v^{(p)} = 0$ otherwise. This follows because the total number of pixels undergoing a change at iteration (p) is $\sum_\Delta p_{v,\Delta}^{(p)} N_{u,\Delta}^{(p)} = \#J^{(p)} P^{(p)}(v|u, \mathbf{z}_\Delta) \rho^{(p)}(\mathbf{z}_\Delta|u) \delta \mathbf{z}_\Delta = \#J^{(p)} P^{(p)}(v|u) \int \rho^{(p)}(\mathbf{z}_\Delta|u, v) d\mathbf{z}_\Delta = \#J^{(p)} P^{(p)}(v|u)$ and the quantity of change at iteration (p) is $\sigma_v^{(p)}$.

3.C.3.3. COMMENTS

A yet even simpler version of the algorithm can be produced, if one takes into account that the order in which transitions occur (or not) does not matter. In this case, and because the order in which pixels are drawn is random, one could as well assign the first $p_{v,\Delta} N_{u,\Delta}$ draws to the $u \rightarrow v$ transition and stop the algorithm there, leaving the remaining pixels in their initial state u . From a statistical point of view, this would be strictly equivalent, and this would definitely be much more economical from an algorithmic point of view.

We ignore this route in this Appendix because it is impractical when explanatory variables are not binned (section 3.C.4). Also, the previous algorithm is probably more intuitively obvious.

3.C.3.4. MULTIPLE TRANSITIONS GENERALIZATION

One can generalize the algorithm to an arbitrary number of simultaneous transitions in a simple manner, with the help of the MuST algorithm.

SAMPLING DESIGN

Draw at random a pixel in the pool of $N_{u,\Delta}$ pixels.

ALLOCATION PROCEDURE

Assign a final state with the MuST test for multiple transitions. After a draw is completed, the pixel is removed from the pixel pool and the probabilities of the algorithm are updated as described above, from the updating of all $P(v|u)$ for each v and of $\rho(z_\Delta|u)$, i.e., $\#J^{(p)}$ is first updated $P^{(p)}(v|u)$ is updated to $P^{(p)}(v|u) - 1/\#J^{(p)}$ if the pixel has undergone a state change from u to v and unchanged otherwise [with a similar updating of $P^{(p)}(u|u)$], and $\rho^{(p)}(z_\Delta|u)$ is updated to $\rho^{(p)}(z_\Delta|u) - 1/\#J^{(p)}\delta z_\Delta$.

ITERATION

Repeat the process until the required number of transition pixels is obtained for each transition.

Once a transition has attained its predefined volume of change, the associated transition probability is down to zero and can be removed from the test. The procedure ensures that this occurs before or when all pixels are drawn (this should be transparent from the analogy with sampling without replacement in an urn with the appropriate number of balls of different colors, each color standing for a given transition).

Finally repeat the algorithm for all bins to complete the allocation process.

3.C.3.5. ABSENCE OF BIAS

Let us now show that this last algorithm is unbiased. In the present context, this amounts to verifying that Eqs. (3.6) and (3.7) are satisfied.

From the discussion after Eq. (3.C.1), the number of pixels transiting to state v in bin Δ is $p_{v,\Delta}N_{u,\Delta} = P(v|u, z_\Delta) \times \rho(z_\Delta|u)\#J\delta z_\Delta$. This is true independently of the number of simultaneous transitions considered and follows from the formal correspondence of the algorithm with the urn sampling without replacement and with fixed number of colored balls for each bin; in this expression, $\rho(z_\Delta|u)$ is the initial probability density (the formal correspondence is ensured precisely because the probability and total pixel number are identical to the urn one at each step). Consequently, the total number of pixels transiting

for any $u \rightarrow v$ transition (all bins) is $\#J_v = \sum_{\Delta} P(v|u, z_{\Delta}) \rho(z_{\Delta}|u) \#J \delta z_{\Delta} = \int \rho(v, z|u) dz \#J = P(v|u) \#J$, i.e., the result of Eq. (3.6) without expectation values: the algorithm does indeed enforce the exact quantities of change, as planned.

Finally, concerning Eq. (3.7), and from the number of pixels $P(v|u, z_{\Delta}) \rho(z_{\Delta}|u) \#J \delta z_{\Delta} = \rho(z_{\Delta}, v|u) \#J \delta z_{\Delta}$ transiting to state v in bin Δ just mentioned, one has $\#J_{z,v} = \rho(z_{\Delta}, v|u) \#J \delta z_{\Delta} = \#J_v \rho(z_{\Delta}|u, v) \delta z_{\Delta}$, where the expression of $\#J_v$ just obtained has been used. In other words, the algorithm also exactly enforces the quantity involved in Eq. (3.7), and not only in expectation value.

This completes the proof that the algorithm is bias-free.

3.C.4. AN ALTERNATIVE ALGORITHM AVOIDING BINNING OF CONTINUOUS EXPLANATORY VARIABLES

So far, the argument has glossed over a potential but minor problem. The numbers $N_{u,\Delta}$ of pixels in each bin, as well as the numbers of pixels undergoing each transition in each bin is assumed to be a round number (an integer). There is no guarantee that this is the case because probabilities are real numbers, not integers.

This can be circumvented in various ways. A first approximate way is to assume that the bin sizes are small enough so that z is close to being constant in the bin, but still large enough so that the numbers just mentioned are large enough and can be approximated by an integer with sufficient relative precision. This argument is more and more exact in the limit of an infinite number of pixels, which can be taken at constant bin size, or at constant (integer) numbers per bin in the limit of infinite numbers of pixels and vanishing bin size. These limits are of course never achieved. However, as maps are more and more resolved, very large number of pixels may now be considered (e.g., up to 10^{10}) so that rounding numbers such as $N_{u,\Delta}$ to the nearest integer constitutes a negligible relative error, except in the (themselves negligible) tails of the distributions of interest.

Another way to tackle the problem is to avoid binning altogether, a step also required by the fact that one obtains better estimates of $\rho(z|u)$ (one of the two quantities that need to be updated at each draw) by using our estimator (Mazy and Longaretti, 2022a), which requires unbinned continuous explanatory variables.

The use of binning allowed us to produce an algorithm where the number of pixels changing state was enforced not only globally (for the whole unbinned

pixel population) but also locally, i.e., bin by bin. This second requirement made somehow the question of elaborating an unbiased algorithm simpler, but at the cost of too strong a requirement; indeed, Eq. (3.7) only requires that the distribution is obtained statistically, not exactly for each allocation; Eq. (3.6) is the only one that should be met in actual and not expectation values.

3.C.4.1. ALGORITHM STRUCTURE

The understanding gained in the previous sections of this Appendix suggests a way to elaborate such an algorithm. To this effect, let us modify the algorithm defined in section 3.C.3.4 in the following way:

SAMPLING DESIGN

Draw at random a pixel in the pool J of pixels.

ALLOCATION PROCEDURE

Assign a final state with the MuST test for multiple transitions, and for the z value of the pixel. After a draw is completed, the pixel is removed from the pixel pool and the probabilities of the MuST algorithm are updated as described right below.

ITERATION

Repeat the process until the required number of transition pixels is obtained.

The discussion of section 3.C.3 shows that it is sufficient to update $\#J^{(p)}$, $P^{(p+1)}(v|u) = P^{(p)}(v|u) - 1/\#J^{(p)}$ if the pixel has undergone a state change from u to v and unchanged otherwise and similarly for $P^{(p+1)}(u|u)$, and to update $\rho^{(p)}(z|u)$. This second updating is performed with our eKDE algorithm (Mazy and Longaretti, 2022a). Note that the eKDE method enforces that $\int \rho(z|u)dz = 1$ so that reducing by one the total pixel number ($\#J^{(p+1)} = \#J^{(p)} - 1$) also reduces by one the integrated pixel density $\#J^{(p)} \int \rho^{(p)}(z|u)dz$; this ensures the self-consistency of pixel numbers between the two updatings. Again, these updatings define in turn the updated transition probabilities from Bayes rule, as $\rho(z|u, v)$ is unchanged in the process.

3.C.4.2. ABSENCE OF BIAS

Eqs. (3.6) and (3.7) are satisfied by design of this algorithm. Let us show this in an explicit way.

For Eq. (3.6), the algorithm has been designed to enforce the quantities of change exactly and not only in expectations value. An intuitive proof of this

statement can be obtained by making again use of the correspondence between this algorithm and another form of coupled pixel/colored balls problem. Start with $\#J$ balls labelled by the various states v . The number of balls for any given state is the enforced one, i.e., $P(v|u)\#J$. If the balls were used as in section 3.C.2, the final state of a pixel would be assigned from a random ball draw. Here we use this property in the opposite way: the final state of the pixel is assigned through the algorithm, and a ball of the correct label is assigned at random to the pixel. By design the updating of the probabilities $P^{(p)}(v|u)$ after removal of the ball/pixel is identical to the probabilities obtained from ball counting as in section 3.C.1. This ensures that updating the probabilities and removing balls are equivalent operations, and that the number of balls/pixel of each final state v is $P(v|u)\#J$ once the allocation is completed.

The main feature of this proof is that it allows us to ignore the value z associated to each pixel, as it is irrelevant to the overall updated number of balls/pixels in each state v at each iteration. This is to be contrasted with the proof given in section 3.C.3.5 for the algorithm of section 3.C.3.4, which applied for each individual bin, with a specified z_Δ .

Eq. (3.7) follows because $\rho(z|u, v)$ is enforced exactly at each iteration of the algorithm. Consider a small hypercube volume Δ in explanatory space around a specific value z_Δ . Once all pixels have been assigned a final state v , for any given final state, one can define an index δ^j for each pixel j such that $\delta^j = 1$ if $z^j \in \Delta$ and $\delta^j = 0$ otherwise. The probability $p(\delta^j = 1) \equiv p_\Delta^j = \int_\Delta \rho(z|u, v) dz = \rho(z_\Delta|u, v)\delta z_\Delta$ and the probability $p(\delta^j = 0) = 1 - p_\Delta^j$. This defines again a Bernoulli variable on each pixel, but with another probability distribution p_j :

$$p_j = p(\delta^j) = p_\Delta^j{}^{\delta^j} \times [1 - p_\Delta^j]^{1-\delta^j}, \quad (3.C.2)$$

and leads to the looked for expectation value:

$$E(\#J_{z,v}) = E\left(\sum_{j \in J_v} \delta_\Delta^j\right) = \#J_v \sum_{\delta_\Delta=0}^1 \delta_\Delta p(\delta_\Delta) = \#J_v \rho(z|u, v) \delta z_\Delta. \quad (3.C.3)$$

Note that the logic of the argument differs from the one used in Eq. (3.B.3): the sum has been identified to $\#J_v$ times the expectation value of the same variable. This was possible because the number of transited pixels $u \rightarrow v$ is now enforced to its expectation value. In the process, a detour through an explanatory variable integral has been avoided, thereby avoiding the problem raised by the fact that $P(v|u, z)$ is no longer a constant quantity throughout the allocation

process. This is also to be contrasted with the proof given in section 3.C.3.5.

3.D. MULTI-PIXEL PATCH ALLOCATION WITHOUT PIXEL REPLACEMENT

One of the motivations of Appendix 3.C is to provide an introduction to the discussion of the uPAM algorithm. To this effect, we will first extend the logic of Appendices 3.B and 3.C to the multi-pixel patch context. In particular the logic of Appendix 3.C will be followed, by designing algorithms of increasing scope and complexity until the uPAM algorithm as presented in section 3.4.2 can be shown to be bias-free.

3.D.1. A MULTI-PIXEL PATCH MUST-LIKE ALGORITHM

It is convenient to start with an algorithm that does not enforce the total quantities of change, nor the absence of patch merging, and therefore does not enforce the absence of replacement either. This algorithm is of little practical use, but will allow us to reach our purpose in a progressive way, by pointing out the problems raised. A single final state v is used in this algorithm.

As explanatory variables patch size and patch elongation are statistically independent, applying the MUST algorithm to each of these will provide the looked-for allocation procedure (for simplicity we keep here the same notation for the binned and non-binned quantities):

3.D.1.1. ALGORITHM STRUCTURE

BINNING

The distributions $\rho(\sigma|u, v)$ and $\rho(e|u, v)$ are binned²¹ to $P(\sigma|u, v)$ and $P(e|u, v)$.

SAMPLING DESIGN

The MUST algorithm of Appendix 3.B is applied to all pixels in initial state u with the relevant probability $P^c(v|u, z)$ of Eq. (3.5) for each pixel. This provides a set J_v^c of pivot-cells for state v . For each such pivot-cell j^c , the MUST algorithm is then applied with the relevant $P(\sigma|u, v)$ to affect a

²¹The distribution of patch sizes and elongations, although specified in the continuous limit, are intrinsically discrete as pixels are discrete, but they are more simply specified in the continuous limit, and in any case, one may wish, for various reasons, to bin them in a coarser way than their natural discretization.

patch size σ^{j^c} to the pivot-cell, and again with $P(e|u, v)$ to affect a patch elongation e^{j^c} .

ALLOCATION

The patch is effectively allocated with the help of the patch design algorithm of Appendix 3.I.3 where the patch merging test is omitted. No pixel is removed from the sample in the process.

ITERATION

At each iteration (i.e., for each pivot-cell), the pixels are left in the pool J . The rationale of this choice is to avoid dealing with overlapping patches. Only at the end (once all patch have been assigned) are patches actually constructed and possibly merged on the basis of the selected pixels.

3.D.1.2. PROPERTIES AND LIMITATIONS

At the pivot-cell selection process step, the algorithm is unbiased: the argument of Appendix 3.B.2 applies without change. The same is true with appropriate modifications of the proof for patch sizes and elongation. Let us show this for patch sizes for definiteness (the proof is clearly formally identical for elongation). To this effect, we introduce a new random variable S_j associated to each pivot-cell in the set J_v^c , with two possible values, $s_j = 1$ if the associated patch as size $\sigma = \sigma_0$ and $s_j = 0$ if $\sigma \neq \sigma_0$. The probability distribution of this variable is

$$p_j = p(s_j) = P(\sigma_0|u, v)^{s_j} \times [1 - P(\sigma_0|u, v)]^{1-s_j}, \quad (3.D.1)$$

Consider samples S_σ collecting the values of s_j produced by the allocation method: $S_\sigma = (s_1, s_2, \dots)$. The probability of such a sample is $p(S_\sigma) = \prod_j p_j$. The number of times σ_0 appears in a sample $\#J_\sigma$ has an expectation value over the population of samples of length $\#J_v^c$ given by

$$E(\#J_\sigma) = E\left(\sum_j S_j\right) = \sum_{s_j, j} s_j p_j = \#J_v^c P(\sigma_0|u, v), \quad (3.D.2)$$

which is the required result.

This suggests that the post-allocation distributions of patch sizes and elongations is unbiased. However, this is the case *only if* patch merging is avoided. Indeed, patch merging will necessarily and systematically produce a reduction in the number of pixels that have undergone a transition. It will also alter the patch size distribution, by reducing the number of smaller patches to the benefit of larger ones. Patch merging must therefore be avoided to obtain an unbiased

algorithm.

Patch merging can hardly be avoided in the above algorithm. Indeed, some amount of patch overlap will almost necessarily occur in the process (random pivot-cell draws nearly always produce spatially close pairs of pivot-cells), although the total number of overlaps will be fractionally small for small enough quantities of change.

Also, the patch allocation may fail at step (iii) of Fig. 3.11 if no suitable neighbor is found. This occurs when the location of the pivot-cell is drawn in a very constrained surrounding (few available pixels of the same initial state in the neighboring structures of Appendix 3.I.1 used in the patch design algorithm).

3.D.2. A CONSTRAINED MULTI-PIXEL PATCH ALGORITHM ENFORCING QUANTITIES OF CHANGE AND AVOIDING PATCH MERGING

3.D.2.1. CONTEXT

As in Appendix 3.C, we first wish to produce a bias-free algorithm when explanatory variable space distributions are binned. To this effect, the definitions of Appendix 3.C.2 are adopted and extended. All possible final states v are included in this section.

More precisely, consider a given bin Δ of volume δz_Δ . It is assumed again that $\rho(z|u)$ is nearly constant in Δ . As before, the total number of pixels in the bin is $N_{u,\Delta} = \#J \int_\Delta \rho(z|u) dz \simeq \#J \rho(z_\Delta|u) \delta z_\Delta \equiv \#JP(z_\Delta|u)$, and the number N_v of pixels undergoing a transition from u to v is enforced to $N_v = N_{u,\Delta} P(v|u, z_\Delta) \equiv \#J_{z_\Delta,v}$; for $v = u$ this stands for the number of pixels not undergoing a transition. By construction $N_{u,\Delta} = \sum_v N_v$.

Furthermore, binning is chosen such that the scales L_p and L_z defined in Appendix 3.A.2 obey the ordering assumed there ($L_p \ll L_z$); this assumption is unessential inasmuch as binning is avoided at the end, but helps to simplify the argument. Elongation is left aside: as there is no constraint associated to this, and as it is statistically independent by assumption from the other quantities, including it does not change the argument, but creates a diversion on a side issue. Finally, we assume that patch formation cannot fail in step (iii) and (viii) of Fig. 3.11 due to lack of suitable neighbors. Again, this will be lifted when discussing the uPAM algorithm (section 3.D.3).

The binning just performed allows us to focus on allocation in a single bin at a time. It is convenient in a first step to consider pixel final state allocation by bunch σ while ignoring their spatial location (i.e., ignoring for the time being that they change state by patches of spatially contiguous). This approach will be justified in section 3.D.2.5.

3.D.2.2. GENERALIZED URN AND BALLS ALGORITHM

The problem can first be approached by generalizing the argument of Appendix 3.C.1. Let us consider an urn containing initially (before balls are drawn from the urn) $N_v^{(1)} = N_{u,\Delta} P(v|u, \mathbf{z}_\Delta) \equiv \#J_{\mathbf{z}_\Delta, v}$ balls labeled v (for all possible final states of the problem, i.e., with $v = u$ included) for a total $N^{(1)} = \sum_v N_v^{(1)} = N_{u,\Delta}$ of balls.

Balls are drawn without replacement at random from the urn. At the first draw, the probability of drawing a v ball is $p_v^{(1)} = n_v^{(1)}/n^{(1)} = P(v|u, \mathbf{z}_\Delta)$, with $\sum_v p_v^{(1)} = 1$. At iteration (p) , the number of v balls is $N_v^{(p)}$ for a total number of balls $N^{(p)} = \sum_v N_v^{(p)}$, and with probabilities of draw $p_v^{(p)} = N_v^{(p)}/N^{(p)}$.

Balls are drawn iteratively in a modified way with respect to Appendix 3.C.1. A ball is drawn at random. Let us say that its v label is $v = v_0$. The difference with the previous urn and ball model is that now, we wish to remove at each iteration an arbitrary number $\sigma_{v_0}^{(p)}$ balls from the urn (except for the requirements that $\sigma_{v_0} \ll N_{v_0}^{(1)}$) and with $\sigma_{v_0}^{(p)} = s^{(p)}$ if $v_0 = u$. Usually $s^{(p)} = 1$ but occasionally, $s^{(p)} > 1$ can be chosen to mimick patch construction failure; otherwise $\sigma_{v_0}^{(p)}$ mimicks patch construction. How this is done is not relevant, but, for example, one may look into the urn and pick the remaining $\sigma_{v_0} - 1$ balls at random.

Quite clearly, the updated number of balls are $N^{(p+1)} = N^{(p)} - \sigma_{v_0}$, $N_{v_0}^{(p+1)} = N_{v_0}^{(p)} - \sigma_0$ and $N_v^{(p+1)} = N_v^{(p)}$ for $v \neq v_0$. The new draw probabilities are $p_v^{(p+1)} = N_v^{(p+1)}/N^{(p+1)}$ for all v , with $\sum_v p_v^{(p+1)} = 1$ as expected.

Iterations stop when all $v \neq u$ balls are drawn from the urn (except for possible small mismatches between $\sigma_{v_0}^{(p)}$ and the remaining number of v_0 balls at the penultimate iteration for v_0 ; σ_0 can, for example, be completed by switching the required number of u balls with v_0 balls coming from outside the urn). Obviously, this takes at most $N^{(1)}$ iterations (the number of iterations if balls were drawn one by one from the urn), and at the end, all v balls will have been drawn from the urn (with a possible slight overshoot in v ball numbers as just described).

3.D.2.3. CONSTRAINED MULTI-PIXEL PATCH ALGORITHM STRUCTURE

Following again the lead of Appendix 3.C, we may implement the previous iterative sampling directly on pixels with the same numbers and probability updating, without reference to urns and balls, for each bin Δ . The formal correspondence with the urn model just described will ensure that both the required numbers of pixel state change will be obtained for each final state v , respecting the expected initial probabilities of state change, and that this is achieved pre-

cisely by updating these probabilities and the total number of balls remaining in the urn.

Therefore, the algorithm proceeds as follows:

BINNING

The distribution $\rho(\sigma|u, v)$ is binned to $P(\sigma|u, v)$.

SAMPLING DESIGN

A pixel is drawn at random in bin Δ . The MuST algorithm of Appendix 3.B is applied with $P(v|u, z)$. If state $v_0 \neq u$ is selected, MuST is then applied with $P(\sigma|u, v_0)$ to affect a patch size σ to the newly drawn pixel.

ALLOCATION

If $v_0 \neq u$, a patch of size σ is constructed with the help of the patch design algorithm of Appendix 3.I.3 around the drawn pixel, which becomes the pivot-cell of the newly created patch. We will check at the end that the distribution of pivot-cells is the correct one, although the probability of draws used is the one for the total pixel population of bin Δ , $P(v|u, z_\Delta)$, and not the pivot-cell one $P^c(v|u, z_\Delta)$.

UPDATING

If no state transition occurred, the pixel is removed from the pool. If the patch is effectively allocated, all pixels are removed from Δ . If the patch construction fails (patch merging is detected or too few suitable neighbors), the pivot-cell and all pixels used in the attempted patch construction are removed from Δ . The updating is performed as follows after iteration (p) (the definitions of pixel numbers N , N_v are the same as in section 3.D.2.2):

- If a state transition occurs, $N_{v_0}^{(p+1)} = N_{v_0}^{(p)} - \sigma$. Similarly $N^{(p+1)} = N^{(p)} - \sigma$ starting with $N^{(1)} = N_{u,\Delta}$. For all $v \neq v_0$, $N_v^{(p+1)} = N_v^{(p)}$. Pixels number are initialized to starting with $N_v^{(1)} = N_v = N_{u,\Delta} P(v|u, z_\Delta) = \#J_{z_\Delta, v}$.
- If no state transition occurs, $N_u^{(p+1)} = N_u^{(p)} - 1$ starting with $N_u^{(1)} = N_{u,\Delta} P(u|u, z_\Delta) = \#J_{z_\Delta, u}$ (i.e., $v = u$ for this quantity) and $N^{(p+1)} = N^{(p)} - 1$. For all $v \neq u$, $N_v^{(p+1)} = N_v^{(p)}$.
- If the patch construction fails, the updating removes s pixels from state u where s is now the number of pixels involved in the patch construction until the process fails (it may fail immediately after the draw, in which case only one pixel is removed): $N_u^{(p+1)} = N_u^{(p)} - s$ and $N^{(p+1)} = N^{(p)} - s$. For all $v \neq u$, $N_v^{(p+1)} = N_v^{(p)}$.

- From this updating, one can now update the probability distribution $P^{(p)}(v|u, \mathbf{z}_\Delta) = N_v^{(p)}/N^{(p)}$ for all v ; $\sum_v P^{(p)}(v|u, \mathbf{z}_\Delta) = 1$ as required.
- As $\rho(\mathbf{z}_\Delta|u, v)$ is unchanged (the fact that pixels are removed from the sample does not *a priori* affect the relevance — or lack of — of explanatory variable values for the considered transition), Bayes rule allows us to recover $P^{(p)}(v|u)$ from $P^{(p)}(v|u, \mathbf{z}_\Delta)$, $\rho^{(p)}(\mathbf{z}_\Delta|u) = N^{(p)}/(\#J\delta\mathbf{z}_\Delta)$ and $\rho(\mathbf{z}_\Delta|u, v)$.
- $P(\sigma|u, v)$ is unchanged.

ITERATION

The sampling design and allocation steps are iterated until the required quantity of change is obtained for each transition in bin Δ (i.e., $N_v^{(p)} < 0$ for each v).

The procedure is then repeated for all bins to complete the allocation process. The simplified updating of section 3.C.3.2 applies here as well, for the same reason.

3.D.2.4. ABSENCE OF BIAS

Let us show that this algorithm is bias-free. This is achieved in several steps as one needs to check Eqs. (3.6), (3.7) and (3.10). Note that we have shown in section 3.3.2 that Eqs. (3.8) and (3.9) are satisfied if Eqs. (3.6) and (3.7) are, under the independence assumption of the explanatory variables and patch size distributions adopted in this work.

Eqs. (3.6) and (3.7) are satisfied by design. This is ensured by the formal correspondence with the urn problem of section 3.D.2.2, due to the updating of probabilities and total pixel number $N^{(p)}$ used in the algorithm, which are identical to the ones in the urn sampling problem. In particular, $\#J_{\mathbf{z}_\Delta, v} = N_v$ for bin Δ and for all v is exactly enforced in the algorithm (except for the slight overshoot due to the condition $N_v^{(p)} < 0$ at the last iteration). The total volume of change follows immediately from summing bin volumes of change: $\#J_v = \sum_\Delta \#J_{\mathbf{z}_\Delta, v} = \#J \int \rho(\mathbf{z}|u) P(v|u, \mathbf{z}) d\mathbf{z} = \#J \int \rho(v, \mathbf{z}|u) d\mathbf{z} = \#JP(v|u)$ so that Eq. (3.6) is satisfied exactly and not only in expectation value, as anticipated. Finally, using this expression and Eq. (3.1) (Bayes rule) in the bin quantity of change leads to $\#J_{\mathbf{z}_\Delta, v} = \rho(\mathbf{z}_\Delta|u, v) \#J_v$, i.e., Eq. (3.7) is satisfied exactly in bin Δ and not only in expectation value.

At the end of the algorithm, an unknown number of pivot-cells $\#J_{\mathbf{z}_\Delta, v}^c$ will have been generated in each bin Δ and for each final state v ; the total number

of pivot-cells in final state v , $\#J_v^c = \sum_{\Delta} \#J_{z_{\Delta},v}^c$ is also unknown. To each pivot-cell, a patch size has been affected with the MuST algorithm. The fact that this algorithm is bias-free (see section 3.D.1.2) ensures that $E(\#J_{\sigma,v}) = P(\sigma|u, v)\#J_v^c$ for all pivot-cell samples of given size $\#J_v^c$. Averaging over all possible sample sizes therefore shows that Eq. (3.10) is satisfied because the algorithm avoids patch merging (as per the argument of section 3.D.1.2).

3.D.2.5. SPATIAL ISSUES

The preceding proof leaves in the dark a point that one may find confusing: the spatial structure of patches (contiguous pixels) has been ignored. How can such an essential feature in usual 2D space be so unessential in explanatory variable space? A first element of answer has already been provided in Appendix 3.D.2.5: as all pixels are equivalent in a bin Δ , all collections of such pixels with the right properties (correct total number for each final state) will satisfy the no-bias constraint.

This answer however leaves a point in the dark: not all pixels collection of the right total number are equivalent, because pivot-cells cannot be freely chosen due to the patch merging avoidance condition. Indeed, a collection of $n_{z_{\Delta},v} \simeq E(\#J_{z_{\Delta},v}^c)$ pivot-cells in final state v all grouped in the same spatial area in bin Δ is as probable as any other, in principle (at least at the level of pivot-cell selection), but cannot be realized because patches have a non vanishing spatial extent ($\sim E(\sigma)^{1/2}$): this is prevented by the removal of pixels at the iteration and updating step of the algorithm, which itself is enforced to prevent patch merging.

How is the information of patch merging prevention formally accounted for? The answer lies in the fact that the exclusion of potential pivot-cells is not incorporated in the 1-pixel probability distribution $\rho^c(z|u, v)$, but the joint 2-pixels, 3-pixels, ..., $n_{z_{\Delta},v}$ -pixels probability distributions, that is, $\rho^c(z^1, z^2|u, v)$, $\rho^c(z^1, z^2, z^3|u, v)$, etc.

For example, $\rho^c(z^1, z^2|u, v) = \rho^c(z^1|u, v) \times \rho^c(z^2|z^1, u, v) \neq \rho^c(z^1|u, v) \times \rho^c(z^2|u, v)$. The last inequality follows because $\rho^c(z^2|z^1, u, v) \neq \rho^c(z^2|u, v)$ precisely because $z = z(x)$ (x stands for the coordinates of in physical 2D space) and x^2 cannot be chosen closer than $\sim 2E(\sigma)^{1/2}$ to x^1 , on average. Because the conditional and unconditional probability distributions differ, the 1-pixel distributions do not contain all the information needed to fully describe the problem (in particular, e.g., the distribution of mutual distances between patches, and not only for the same initial and final states), and multiple pixel distributions are required to quantify the missing information.

This limitation is of no direct consequence here, because joint pixel distributions are not yet in the scope of LUCC modeling (except indirectly through

patch distributions and locations). If the information they carry ever becomes a concern, such joint pivot-cell distributions will also need to be calibrated.

Note finally that in the limit of small quantities of change, the mutual patch excluded “volume” in explanatory space is small or even negligible, so that the conditional probability distribution simplifies to $\rho^c(\mathbf{z}^2|\mathbf{z}^1, u, v) \simeq \rho^c(\mathbf{z}^2|u, v)$, and the patch merging problem is only marginally important. However, quantifying this is delicate and problem-dependent, so it is best to err on the side of safety and explicitly prevent patch merging.

3.D.3. UPAM ALGORITHM DISCUSSION

The algorithm uPAM as described in section 3.4.2 results from a combination of the arguments of Appendices 3.C.4 and 3.D.2.3. In particular there are two differences between the algorithm of section 3.D.2.3 and uPAM:

- The drawn pixel is obtained from the MuST algorithm.
- The updating of numbers and probabilities are obtained from the updating of $\#J$, $P(v|u)$ and $\rho(\mathbf{z}|u)$.

The first point follows the logic of Appendix 3.C.4. Because MuST is bias-free, the sampling design of uPAM is bias-free.

The rationale of the second point is explained in Appendix 3.C.3.2 and applies without change in the present context. In particular, the updating of the quantity of change $P^{(p+1)}(v|u) = P^{(p)}(v|u) - \sigma_v^{(p)} / \#J^{(p)}$ applies in the same way, except for the quantity of change $\sigma_v^{(p)}$ which now accounts for the number of pixels removed from the initial pool. The same is true of the way the updating of $\rho(\mathbf{z}|u)$ is obtained from the explanation of Appendix 3.C.4. The absence of bias demonstrated in section 3.C.4.2 applies here as well, once the correspondence with the algorithm of section 3.D.2.2 is substituted to the correspondence with the algorithm of section 3.C.1.

These previous proofs of absence of bias were detailed enough and need not be repeated here.

3.E. DINAMICA EGO ALLOCATION METHOD

3.E.1. ALLOCATION METHOD BASED ON A PRUNING PROCEDURE

Although Dinamica EGO’s code is not open source, the code documentation and discussions through Dinamica EGO’s Google Group allowed us to un-

derstand and reproduce its allocation method²². We build on this understanding to present here a more detailed and transparent flow diagram and algorithm structure of Dinamica EGO's allocation method.

SAMPLING DESIGN

Dinamica EGO ranks pixels by decreasing value of $P(v|u^j, z^j)$ and keeps F times the number of pixels needed for the each transition of interest²³, i.e., the software prunes the number of pixels. F is specified by the user; the default is $F = 10$. If more than one transition is possible, the sum of the probabilities over all v is used instead of a single probability: $\sum_{v \neq u^j \in V} P(v|u^j, z^j)$. More precisely, if one collects all the z values corresponding to the chosen pruning factor into an ensemble Z_F , the corresponding ensemble of pixels, J^F , contains a number of pixels $\#J^F = \sum_{z \in Z_F} \rho(z|u) \#J$.

ALLOCATION PROCEDURE

In Dinamica EGO, allocation follows from a series of simple rejection tests²⁴. Pixels are first chosen at random in J , and the global probability of transition $\sum_{v \in V} P(v|u^j, z^j)$ is tested. If a pixel passes the test, it is kept for the second rejection test, where one of the possible state changes is chosen at random (say, v_0), and the simple rejection test is applied to v_0 with the conditional probability²⁵ $P(v_0|u^j, z^j) / \sum_{v \in V} P(v|u^j, z^j)$. The test is repeated until one of the state changes is accepted. Once a pixel has been selected in this way, it is used as pivot-cell of a new patch (the patch construction procedure is described below). The pixels of the patch are then removed from the pruned ensemble.

Because $J^F \subset J$, in principle, $P(v|u)$ should be changed into $P(v|u) \times (\#J/\#J^F)$ when one uses a pruned ensemble instead of the complete one, in order to reflect the fact that the expected volume of LUC state transitions is unchanged while the number of pixels to test is reduced. In practice, Dinamica

²²Dinamica EGO has two different procedures to create new patches: patcher and expander. The second one is specific to creating new patches in contact with existing areas of the target LUC category. This occurs frequently, and requires indeed a more specific way of handling patch formation. Here we ignore Dinamica EGO's expander functionality, as it does not involve new issues of principle with respect to the patcher one.

²³The Dinamica EGO online documentation (https://www.csr.ufmg.br/DinamicaEGO/dokuwiki/doku.php?id=guidebook_start) is not very clear on these technical points. The most detailed information, still somewhat vague, is obtained through Dinamica EGO's Google Group (https://groups.google.com/forum/#!searchin/DinamicaEGO/allocation%7Csort:date/DinamicaEGO/kL7XE_uCBnI/9E_zWUJWPNMJ). Some more details were provided through mail to one of us (PYL) by one of Dinamica EGO developers (Hermann Rodrigues, *private communication*).

²⁴These tests proceed as binomial tests with only one probability p of acceptance and $1-p$ of rejection, similarly to the multinomial test described above.

²⁵The conditional probability is necessary because the pixel is now known to undergo one of the possible transitions.

EGO works with weights of evidence and enforces the volumes of changes instead of enforcing this correction. The end result is identical with using actual transition probabilities and adjusting the rescaling of $P(v|u)$.

The allocation procedure description leaves a point in the dark: are pixels rejected in the first test taken out of the pruned ensemble, or put back in there? In principle, they should be taken out, following the general logic of sampling without replacement that should be implemented, but it appears that Dinamica EGO's algorithm puts them back in the sample. This question is further discussed in section 3.6.2.1.

3.E.2. RECODED PIVOT CELL ALLOCATION METHOD

The selection and allocation of pivot cells is probably the most critical step in the analysis of potential biases. For this reason, this part of Dinamica EGO's patcher has been recoded for this study, in order to check by cross-comparison between our version and Dinamica EGO's that our understanding of this step is quantitatively correct. This applies to both mono-pixel and multi-pixel patches. We have only used this recoded algorithm for mono-pixel patches, and it is therefore coded only for this context. The resulting pseudo-code is described in algorithm 3.

Dinamica EGO applies its pivot cell allocation method to each initial LUC state separately. Once the pruning process (line 1) is completed, for each possible final state v , the targeted number of pixels $N_{u \rightarrow v}$ that are required to undergo the $u \rightarrow v$ transition is determined (line 3). While all transitions are not performed (line 4), one pixel is randomly selected from the pool of pivot candidates J_v^F (line 5). Then, a rejection test with the sum of all possible transition probabilities is applied (line 6). If the pixel is rejected, it is replaced in the pool (this operation is formally incorrect, a point discussed in section 3.6.2.1). If on the contrary the pixel is accepted, the selection process chooses a final state randomly (line 8) and then it tests its normalized probability against a random value (line 9). If the pixel passes the test, the pixel is used as a seed (pivot cell) for a new patch (line 10). Otherwise, the process chooses a new final state randomly until a transition is accepted (line 7). Once a transition quota is reached, the corresponding transition and its probabilities are removed (line 14).

As a final comment, let us point out that the two stage process of selection of a final state and pivot cells is somewhat inefficient, as one can in fact test all final states in the same test, as in our multinomial simple test described earlier.

Algorithm 3: Reproduced Dinamica pivot cell allocation process**Data:**

The set of all pixels indices J ,
 V , the set of LUC states,
 $\{P^c(v|u^j, z^j)\}_{\forall j \in J, \forall v \in V}$, the transitions probabilities,
 $P^c(v|u)$, the transition scenario matrix,
the patch parameters,
 F , the pruning factor

Result: Allocated LUC map.

```

1  $J^F \leftarrow \text{PRUNING}(J, \{P^c(v|u^j, z^j)\}_{j,u}, F)$ 
2  $\nu \leftarrow V \setminus \{u\} \equiv \{v \in V, v \neq u\}$ 
3  $\forall v \in \nu, N_{u \rightarrow v} \leftarrow P^c(v|u) \# J$ 
4 while  $\exists v \in \nu, N_{u \rightarrow v} > 0$  do
5    $j \leftarrow \text{SAMPLE}(J_v^F)$ 
6   if  $\text{RAND} < \sum_{v \in \nu} P^c(v|u, z^j)$  then
7     while allocation is not made yet do
8        $v \leftarrow \text{SAMPLE}(\nu)$ 
9       if  $\text{RAND} < \frac{P^c(v|u, z^j)}{\sum_{v \in \nu} P^c(v|v_t, z^j)}$  then
10        Allocate pivot cell  $j$ 
11        Decrease  $N_{u \rightarrow v}$  by 1
12        Remove allocated pivot cell  $j$  from  $J_u^F$ 
13        if  $N_{u \rightarrow v} \leq 0$  then
14           $\nu \leftarrow \nu \setminus \{u\}$ 
15 return the allocated LUC map, i.e.  $\{v^j\}_{j \in J}$ 

```

3.E.3. PATCH PARAMETERS

Dinamica EGO characterizes patches through two parameters: the patch size and a shape parameter called “isometry”²⁶.

3.E.3.1. PATCH SIZE

The probability distribution of patch sizes is specified by the software and cannot be changed by the user, except for its parameters. It is a log-normal law:

$$P(\sigma) = \frac{1}{\delta\sigma(2\pi)^{1/2}} \exp\left(-\frac{[\ln(\sigma) - \mu]^2}{2\delta^2}\right), \quad (3.E.1)$$

where μ and δ are the log-normal law parameters and $\ln$ is the natural logarithm (the probability distribution is independent of the pixels initial and final states). However, the parameters specified by the user in Dinamica EGO are the mean $E(\sigma)$ and the variance $V(\sigma)$ of the patch size. The corresponding parameters μ and δ of Eq. (3.E.1) are obtained through the following relations:

$$\mu = \ln\left(\frac{E(\sigma)^2}{(E(\sigma)^2 + V(\sigma))^{1/2}}\right) \quad (3.E.2)$$

$$\delta^2 = \ln\left(1 + \frac{V(\sigma)}{E(\sigma)^2}\right). \quad (3.E.3)$$

3.E.3.2. PATCH ISOMETRY

The shape parameter used by Dinamica EGO is called “isometry” and ranges from 0 (linear patch) to 2 (circular patch).

In order to have a better idea of the impact of this parameter during the patch creation process, we have characterized the shape of the patches produced by Dinamica EGO using our own patch shape parameter, elongation, defined in appendix 3.I.2.

This analysis makes use of very simple synthetic data. We chose a uniform initial LUC map of (1000×1000) pixels with a uniform transition probability distribution toward a unique final state. The global transition probability is set to $P(v|u) = 0.005$, the pruning factor to $F = 200$ (this corresponds to no pruning) and the expected patch size to $\sigma = 50$.

We perform a number of Monte Carlo simulations, defined as follows. We

²⁶This has no relation with the mathematical concept of isometry.

run a large number of times the allocation procedure and average the observed value (here the mean elongation of the patches). The chosen number of Monte Carlo simulations is 100.

This setting is designed to focus on patch shapes. Allocated maps are generated through Dinamica EGO and we measure the elongation, Eq. (3.I.12), of each patch (we exclude patches that have merged in order to avoid the patch merging bias, which will change the post-allocation measured isometry in uncontrolled ways).

The isometry parameter does not have any simple relation to our elongation parameter. However, one would expect some form of monotonic relation between the two if both are to constitute useful ways of representing some kind of mean and progressive deformation of a patch with respect to a purely circular patch. Our elongation parameter being directly based on moments of the patch shape is expected to display this behavior. The question we address here is whether this is also the case for Dinamica EGO's isometry parameter, which has no such direct geometric interpretation.

The result of this analysis is represented on Fig. 3.10 where we plot the measured elongation and its standard deviation obtained in our Monte Carlo simulations as a function of Dinamica EGO's non-circularity parameter (isometry). We observe a bimodal distribution of elongations with a transition around an isometry parameter of 1. The standard deviation of the measured elongation is also relatively high and independent of the non-circularity parameter.

One cannot compare the expected shape parameter with the measured one in Dinamica EGO's logic. Indeed, the isometry parameter does not produce a precisely defined type of deformation. On the other hand, our definition of elongation allows us to check this, and the measured (post-allocation) elongation correlates strictly to the imposed (pre-allocation) one (see appendix 3.I.4). In this respect, if a shape parameter based on patch moments were calibrated on data in LUCC modeling, Dinamica EGO isometry parameter would lead to a bias due to its bimodality. For this reason, and because of the absence of clear correlation between isometry and elongation, while elongation provides a clear shape control to the user, we favor elongation as a shape parameter, although the associated patch construction and characterization is more involved.

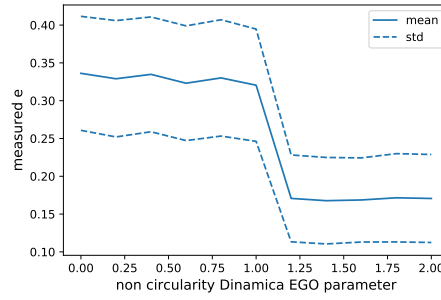


FIGURE 3.10: Measured elongation (average and standard deviation, with rook neighboring structure) of final LUC maps produced by Dinamica EGO as a function of the non-circularity parameter (isometry).

3.F. IDRISI LCM ALLOCATION METHOD

Idrisi LCM allocation method is relatively simple. It is deterministic: no random draws are involved and the allocated pattern will always be the same if the simulation is repeated.

SAMPLING DESIGN

The sampling is reduced to LCM chosen pruning procedure. LCM ranks pixels by decreasing value of $\rho(\mathbf{z}^j | \mathbf{u}^j, v)$ (called potential maps in LCM) and keeps only the number of pixels needed for the specified transition. This is done for all transitions (Eastman et al., 1995). This constitutes an extreme form of pruning, and the sampling design becomes deterministic.

ALLOCATION PROCEDURE

If there is only one transition, there is no random pixel selection nor allocation step in LCM: all the remaining pixels after pruning are allocated. If there is more than one, allocation conflicts can arise. These are solved by a simple rule of thumb²⁷ allocating conflicting pixels to one or the other transition and extending the sample until the targeted number of pixels is obtained for each transition.

The choices performed by LCM ensure that transitions necessarily occur in patches (due to the underlying continuous dependence of $\mathbf{z}$ on spatial coordinates), but at the cost of a potentially strongly biased transition distribution with respect to the distribution quantified in the calibration phase. Indeed, truncating any distribution to the highest value inevitably changes the post-allocation distribution with respect to the pre-allocation one (see section 3.6.1 and Appendix 3.G).

²⁷This rule does not need to be further specified here as this allocation method is not theoretically correct, as will be shown later on. For more details, see Eastman et al. (1995).

3.G. PRUNING BY PROBABILITY DISTRIBUTION TRUNCATION

Pruning is an operation of pre-selection of pixels that are most likely to undergo a given transition. This operation is not strictly necessary but is nevertheless performed by both Dinamica EGO and Idrisi LCM, most likely to reduce the allocation computational time. In our own algorithms, we do not need any pruning as our algorithmic implementation is intrinsically quite efficient, even for large case studies (tens of millions of pixels).

The main tasks of this Appendix are to identify the correct probability distribution that must in principle be used in any kind of pruning strategy, and to show why, on this basis, both Dinamica EGO and Idrisi LCM pruning strategies are necessarily biased.

The problem of identifying the correct probability distribution for pruning has an inherent ambiguity that must first be dispelled. Indeed, two criteria can *a priori* be chosen for pruning, for a given $u \rightarrow v$ transition:

- On the one hand, one may choose the pixels who have the largest transition probability $P(v|u, \mathbf{z})$.
- On the other hand, because explanatory variables are selected on the basis of their correlation with observed transitions, one may wish to choose the pixels whose explanatory variable values maximize $\rho(\mathbf{z}|u, v)$ instead.

It turns out that both pruning criteria are admissible, but not in the same space. Furthermore, choosing one or the other as the distribution to be pruned may lead to differences in truncation boundaries, for the same fraction of total probability pruned.

The proof of these statements follows from a close scrutiny of Eqs. (3.B.3) and (3.B.5). The first equation shows that the pixels with the largest $P(v|u, \mathbf{z})$ do indeed contribute most to the number of transited pixels $N_{u,v}$. The next equation shows that the $\mathbf{z}$ values contributing most to $N_{u,v}$ once expressed as an integral in explanatory variable space $\mathcal{D}$ are those maximizing $\rho(\mathbf{z}|u, v)$ in $\mathbf{z}$ space.

To proceed further, let us assume that pruning is performed by truncating the probability distribution so that a given fraction of the total number of pixels is summed in Eq. (3.B.3). Because $P(v|u, \mathbf{z}) \propto \rho(\mathbf{z}|u, v)/\rho(\mathbf{z}|u)$, if $\rho(\mathbf{z}|u)$ is not uniform, $P(v|u, \mathbf{z})$ and $\rho(\mathbf{z}|u, v)$ will not be maximum for the same $\mathbf{z}$, and the truncation boundaries in $\mathbf{z}$ will not be the same if one prunes by truncation on $P(v|u, \mathbf{z})$ or $\rho(\mathbf{z}|u, v)$. This point is further discussed in section 3.6.1.2.

This argument relies on a proof provided for mono-pixel patches. However, it applies as well to multi-pixel patches as, on the one hand, it applies to multi-pixel pivot cells with straightforward changes (i.e., our MuST algorithm is also an unbiased algorithm of selection of pivot-cells), and, on the other hand, to all pixels in a patch constructed around a pivot-cell as these, on average (over repeated simulations of the same time step allocation process), have the same number of pixels $E(\sigma)$, and nearly the same probability distribution as their pivot cell seed as long as their mean size $\sim E(\sigma)^{1/2}$ is small compared to the spatial characteristic scale of explanatory variables (a condition often if not always satisfied in practice). A more formal version of this argument can be provided but is not needed at the level of rigor of the present discussion.

Therefore, as long as pruning consists in keeping the most probable pixels for any state transition, either the probability distribution $\rho(\mathbf{z}|u, v)$ in $\mathbf{z}$ should be selected for probability ordering in explanatory variable space $\mathcal{D}$ (and keeping the corresponding pixels), or the probability $P(v|u, \mathbf{z})$ if pixels are directly ordered and not the explanatory variables themselves. However, this formal equivalence does not apply to maps. Indeed raster maps can be seen as a particular way to order pixels in a matrix way. Assigning row i_r and column i_c indices to pixels is equivalent to assigning a single index j , and ordering $P(v|u, \mathbf{z})$ with a single index j or two indices i_r, i_c clearly lead to the same pixel ordering and same pixel selection from this ordering. Equivalently, this ordering can be performed on location coordinates instead of row and column indices, as there is a bijection between the two. Finally the ordering and associated selection results in a specific area selection in the raster map, with exactly the same probability values and number of pixels selected. This series of equivalent statements implies that, conversely, if pixels are pruned from a map, the map must represent $P(v|u, \mathbf{z})$, not $\rho(\mathbf{z}|u, v)$.

Alternatively, a pruning of $\rho(\mathbf{z}|u, v)$ can be performed in explanatory variable space, not in physical (map) space with a similar (but not identical) line of reasoning. Here again, these arguments can be presented in a more formal way, but this is not needed at the level of rigor adopted here.

This shows that Dinamica EGO pruning is based on the correct distribution (although if pruning in this software had been performed in explanatory space, the pruning boundaries would differ, as pointed out above), while Idrisi LCM choice is incorrect, as it selects pixels on the basis of a map of $\rho(\mathbf{z}|u, v)$ (transition potential map). Indeed, choosing pixels maximizing $\rho(\mathbf{z}|u, v)$ on a map amounts to summing this quantity over pixels:

$$\sum_i \rho(\mathbf{z}^i|u, v) \propto \int_{\mathbf{z}} \rho(\mathbf{z}^i|u, v) \rho(\mathbf{z}^i|u) d\mathbf{z}, \quad (3.G.1)$$

similarly to Eqs. (3.B.3) and (3.B.5). This has no relation whatsoever with $N_{u \rightarrow v}$.

For definiteness, in the remainder of this discussion we keep the ordering of $P(v|u, z)$ as a choice of pruning strategy, as spatial maps are more natural to use than explanatory variable space $\mathcal{D}$ ordering, especially when the number of explanatory variables exceeds two.

As a limiting case, it may occur that a vast majority of explanatory variable space have zero contribution, that is, $P(v|u, z) = 0$. Removing the corresponding pixels from J does not change anything in the final allocation, but may considerably reduce the required number of random pixel selection. By extension, removing the pixels with small probability distribution $P(v|u, z)$ should have little effect on the result, while still reducing the computational cost of the allocation, by effectively pruning the ensemble J . This being said, one must keep in mind that any method of pruning by truncation of the probability distribution necessarily produces a bias with respect to the expected distribution $P(v|u, z)$: by necessity, the post-allocation distribution will differ from the pre-allocation one as Eq. (3.7) will be violated by virtue of Eqs. (3.B.3) and (3.B.5). Consequently, if this pruning strategy is adopted, the amount of pruning must be decided on the basis of a trade-off between computing efficiency and an acceptable level of bias. Alternatively one may prune more uniformly the probability distribution in order to maintain its shape, but this type of pruning strategy is considerably more involved.

This discussion can be summarized in two points:

- Dinamica LCM does choose the correct probability distribution map for pruning, $P(v|u, z)$ while Idrisi LCM does not, as it chooses $\rho(z|u, v)$ instead²⁸.
- Furthermore, the level of bias of the allocation method of LCM is substantially higher than for Dinamica EGO, because the level of pruning applied by LCM is much more extreme.

3.H. POST-ALLOCATION PROBABILITY CONFIDENCE INTERVAL

We estimate here the statistical noise of a bias-free allocation in order to determine the minimum number of Monte Carlo simulations needed to produce

²⁸For the same reason, comparing directly the maps of Dinamica transition probability with LCM maps of transition potential is meaningless.

a given, predefined level of accuracy in the determination of the relevant post-allocation probability distributions. For definiteness, we focus on a single (pre-allocation) combination $(u, \mathbf{z})$ (within some small range $\delta\mathbf{z}$) and a single final (post-allocation) state v_0 . By construction, this transforms our multinomial probability distribution $P(v|u, \mathbf{z})$ into a binomial one with $v = v_0$ or $v \neq v_0$ (without affecting the transition probability to the chosen state v_0), so that each pixel now undergoes a Bernoulli test during an allocation (this is precisely the case with our mono-pixel allocation method, section 3.4.1, and this remark also applies to the allocation of a final state to a pivot-cell in the multi-pixel patch context of section 3.4.2, as the same algorithm is used for this step).

The pivot-cell transition probability is $p_0 = P^c(v_0|u, \mathbf{z})$ and $1 - p_0$ for no transition. The number of pixels undergoing this Bernoulli test is equal²⁹ to $\#J^*\rho(\mathbf{z}|u)\delta\mathbf{z}$. Thus, if we perform n_{MC} Monte Carlo simulations, this binomial transition probability will have been tested $n_{MC}\#J^*\rho(\mathbf{z}|u)\delta\mathbf{z}$ times in total. In these expressions, J^* is the set of pixels that has been reduced as our iterative allocation method proceeds (section 3.4.2). However, in our iterative allocation method, $\#J^*$ is never very different from $\#J$ because the total number of pixels changing LUC state per time step is always a small fraction of the total $\#J$. For simplicity, we assume that $\#J^* = \#J$ throughout the iterative allocation process. This amounts to approximating our iterative allocation method without replacement with an allocation method with replacement. Although this approximation is not precise enough in some actual case studies, it is sufficient for the present purpose (quantifying a number of Monte Carlo simulations to obtain the post-allocation probability distribution within a predefined confidence interval).

In practice, we perform a series of Bernoulli trials whose characteristics have just been specified. Probability theory then implies that the expected mean for success (i.e., the mean number of transitions from u to v_0) is $n_{MC} \times p_0 \rho(\mathbf{z}|u) \times \#J\delta\mathbf{z}$. Dividing this by the number of trials $n_{MC}\#J^*\rho(\mathbf{z}|u)\delta\mathbf{z}$ gives back the expected probability $p_0 = P^c(v_0|u, \mathbf{z})$.

The actual number of successes will differ somewhat from the expected mean, and dividing by the number of trials will yield an estimate of the expected probability, $\hat{P}^c(v_0|u, \mathbf{z})$. The question boils down to defining the number of Monte Carlo simulations n_{MC} in order for this estimate to lie close enough to the expected probability, within some tolerance level predefined by the user.

This question is best resolved by making use of the known variance V of

²⁹This quantity is not necessarily a natural number, depending on the choice of $\delta\mathbf{z}$. This point is unessential for the argument presented here.

repeated Bernoulli tests, which, in the case at hand, is given by

$$V = \frac{P^c(v_0|u, \mathbf{z})(1 - P^c(v_0|u, \mathbf{z}))}{n_{MC}\rho(\mathbf{z}|u)\#J} \quad (3.H.1)$$

In the limit of large numbers, the Bernoulli test probability distribution is approximated by a Gaussian distribution and a very common way to approximate the needed confidence interval ΔP is based on the central limit theorem³⁰

$$\Delta P = 2p_\alpha V^{1/2}, \quad (3.H.2)$$

where p_α is the $1 - \alpha/2$ quantile of a standard normal distribution and refers to the target error rate α . This p -value quantifies the number of standard deviations required to achieve a given level of confidence α i.e., the cumulative probability distribution within $\pm p_\alpha V^{1/2}$. For the standard choice $\alpha = 0.95$ (95% confidence level), $p_\alpha \simeq 1.96$.

We can now introduce our tolerance ε as the admissible error of the observed transition probability, i.e.:

$$\varepsilon = 2p_\alpha V^{1/2}. \quad (3.H.3)$$

For example, we may choose $\varepsilon = 0.01$ and $\alpha = 0.95$; that is, we require our estimate of $P^c(v_0|u, \mathbf{z})$ to be precise within 1% at an $\alpha = 95\%$ confidence level. This in turn translates into a condition on the number of Monte Carlo simulations that must be performed to achieve the required accuracy:

$$n_{MC} \geq 4 \frac{p_\alpha^2 P^c(v_0|u, \mathbf{z})(1 - P^c(v_0|u, \mathbf{z}))}{\varepsilon^2 \rho(\mathbf{z}|u)\#J}. \quad (3.H.4)$$

For simplicity, this criterion is applied throughout the explanatory variable domain and the maximum value is retained:

$$n_{MC} = \max_{\mathbf{z}} \frac{p_\alpha^2 P^c(v_0|u, \mathbf{z})(1 - P^c(v_0|u, \mathbf{z}))}{\varepsilon^2 \rho(\mathbf{z}|u)\#J}. \quad (3.H.5)$$

The number of Monte-Carlo samples n_{MC} is thus fixed once α and ε are specified. In an actual case study, it is often necessary to exclude pixels whose probability density $\rho(\mathbf{z}|u)$ is too low from the evaluation of Eq. (3.H.5), as these would make n_{MC} ridiculously high from regions in explanatory variable space

³⁰This approximation holds although the transition probability is low, because the typical number of pixels changing state is nevertheless large enough in practice.

$\mathcal{D}$ where $\rho(\mathbf{z}|u)$ is badly estimated anyway. To this effect, we used a simple exclusion method: we normalize the $\rho(\mathbf{z}|u)$ values of each pixel so that their sum is 1, order them in an ascending manner, compute the cumulative sum and exclude pixels whose cumulative sum falls below a certain threshold (5% for example).

3.1. PATCH CONSTRUCTION METHOD (PATCHER)

In the elaboration of our patch design process (or patcher), we chose to build patches pixel by pixel from a given pivot-cell. The main reason for proceeding in this way is its simplicity of algorithmic implementation. This choice is also motivated by a compromise between user control on the patch shape and some level of randomness in the shape construction, in order to give patches a more realistic appearance.

The following properties are expected to hold for any patch construction method:

- The patcher algorithm should not introduce any noticeable bias on the distribution $\rho(\mathbf{z}|u, v)$.
- The predefined patch size (as defined at the previous sampling design step) must be reached.
- A patch elongation parameter is provided, possibly at random if the distribution of elongations has not been calibrated.
- The patch merging bias must be avoided (this bias is discussed in section 3.6.3).
- Patches must be “realistic”. This is a rather subjective criterion. In practice, this means that very regular patch shapes must be avoided, as these are rarely if ever encountered in actual case studies. The construction method we provide does satisfy this criterion in our estimate, but the user has no or little control on this characteristic.

Concerning the last requirement, ideally, one should also define a measure of patch shape randomness and an related probability distribution that should be estimated on calibration data, and design a bias-free algorithm that would enforce this probability distribution during allocation. However, this level of sophistication is not yet required at the present level of precision of LUCC modeling. This is fortunate, as producing an appropriate bias-free algorithm for a shape randomness parameter would be quite challenging.

Very little attention has been devoted so far in the literature to the question of patch construction. The most elaborate existing patcher algorithm (Dinamica EGO's) enforces the distribution of patch sizes, and allows the user to define some kind of an elongation parameter ("isometry"); however this parameter produces an essentially bimodal elongation distribution (see Fig. 3.10). In this respect, our elongation definition and implementation is much more satisfactory, as will become apparent in the remainder of this Appendix.

In principle, both the distribution of patch sizes and elongations may be obtained from calibration data, although quantifying elongations may turn out to be quite computationally intensive. In practice though, our own software (CLUMPY) can either calibrate the distribution of patch sizes, or let the user define the mean and variance of a gaussian probability distribution, a strategy also adopted in Dinamica EGO (with a lognormal distribution instead). Concerning patch elongations, and considering the near absence of attention paid to this parameter in the literature, we do enforce a gaussian distribution, with mean and variance specified by the user. More versatile choices may be implemented in the future, if the need arises. In particular, it is always possible to define more shape parameters if needed.

3.1.1. NEIGHBORHOOD STRUCTURES

If patches are to be constructed from a pivot-cell seed (a strategy adopted here, following Dinamica EGO), it seems natural to take into account the information carried by neighboring pixels in the process. To this effect, it is useful to define first elementary neighborhood structures for every potential seed pixel.

Neighborhood structures are represented by matrices, where any given matrix element corresponds to a neighboring pixel (the pixel of interest sitting at the center of the matrix and represented by $\times$). These matrices have only two possible entries, 0 or 1; 1 means that the corresponding pixel is taken into account in the process, 0 means that it is ignored.

Two neighborhood structures are commonly used. The queen neighborhood structure includes all 8 nearest neighbors around a given pixel:

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & \times & 1 \\ 1 & 1 & 1 \end{pmatrix}, \quad (3.1.1)$$

whereas the rook neighborhood structure only considers 4 of the nearest neigh-

bors

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & \times & 1 \\ 0 & 1 & 0 \end{pmatrix}. \quad (3.1.2)$$

The terms *queen* and *rook* are clearly inspired by the possible directions of motion of the corresponding pieces in a chess game from the central pixel. More extended neighborhood structures may in principle be considered. This option is not explored in the present work.

This notion of neighborhood structure is exploited in the patch creation algorithm of section 3.1.3. For definiteness, we use the rook structure which is the default in our model.

3.1.2. PATCH ELONGATION

A patch elongation can be defined by analogy with an equivalent ellipse. Let us recall that, from elementary geometry, the principal moments of inertia of an ellipse, I_x and I_y are related to the ellipse major axis a (along x) and minor axis b (along y) by $I_x = \pi ab^3/4$ and $I_y = \pi ba^3/4$. The ellipse eccentricity itself is given by $\epsilon = (1 - I_x/I_y)^{1/2} = (1 - b^2/a^2)^{1/2}$. This motivates us to define a patch elongation from its moments of inertia.

More generally, any bidimensional shape can be characterized by its successive moments. Let J_p be a set of pixels constituting a patch. The zeroth order moment is equal to the size of the patch (i.e., the number of pixels in the patch):

$$M_{00} = \sum_{j \in J_p} \delta\sigma_j = \#J_p, \quad (3.1.3)$$

where $\delta\sigma_j = 1$ for pixels in the patch. The first order moments are directly related to the position of the center of the patch ($x_0^c = M_{10}/M_{00}$, $x_1^c = M_{01}/M_{00}$):

$$M_{10} = \sum_{j \in J_p} x_0^j, \quad (3.1.4)$$

$$M_{01} = \sum_{j \in J_p} x_1^j, \quad (3.1.5)$$

where (x_0^j, x_1^j) are the spatial coordinates of pixel j . The normalized second

order moments are given by:

$$\mu_{20} = \frac{1}{M_{00}} \sum_{j \in J_p} (x_0^j - x_0^c)^2, \quad (3.I.6)$$

$$\mu_{02} = \frac{1}{M_{00}} \sum_{j \in J_p} (x_1^j - x_1^c)^2, \quad (3.I.7)$$

$$\mu_{11} = \frac{1}{M_{00}} \sum_{j \in J_p} (x_0^j - x_0^c)(x_1^j - x_1^c). \quad (3.I.8)$$

The normalized inertia tensor³¹ I_p is then defined as:

$$I_p = \begin{pmatrix} \mu_{20} & \mu_{11} \\ \mu_{11} & \mu_{02} \end{pmatrix}. \quad (3.I.9)$$

From this definition, one can introduce the reduced principle moments of inertia of the patch, i.e., the eigenvalues λ_1 and λ_2 of I_p :

$$\lambda_1 = \frac{1}{2} \left(\mu_{20} + \mu_{02} - [(\mu_{20} - \mu_{02})^2 + 4\mu_{11}^2]^{1/2} \right), \quad (3.I.10)$$

$$\lambda_2 = \frac{1}{2} \left(\mu_{20} + \mu_{02} + [(\mu_{20} - \mu_{02})^2 + 4\mu_{11}^2]^{1/2} \right). \quad (3.I.11)$$

Clearly, the ratio of the minor to the major axes of the ellipse having the same reduced principle moments of inertia is given by $(\lambda_1/\lambda_2)^{1/2}$ and its eccentricity by $(1 - \lambda_1/\lambda_2)^{1/2}$.

At this point, we deviate somewhat from the equivalent ellipse analogy and choose to define the patch elongation as

$$e = 1 - \left(\frac{\lambda_1}{\lambda_2} \right)^{1/2}. \quad (3.I.12)$$

This definition is adopted as it turns out that the elongation thus defined has convenient features from a numerical point of view. The patch elongation ranges from 0 (for a circular patch) to 1 (for a linear patch).

³¹If a patch is seen as an actual 2D massive structure with each pixel mass equal to its area, this definition corresponds indeed to the tensor of inertia defined in physics.

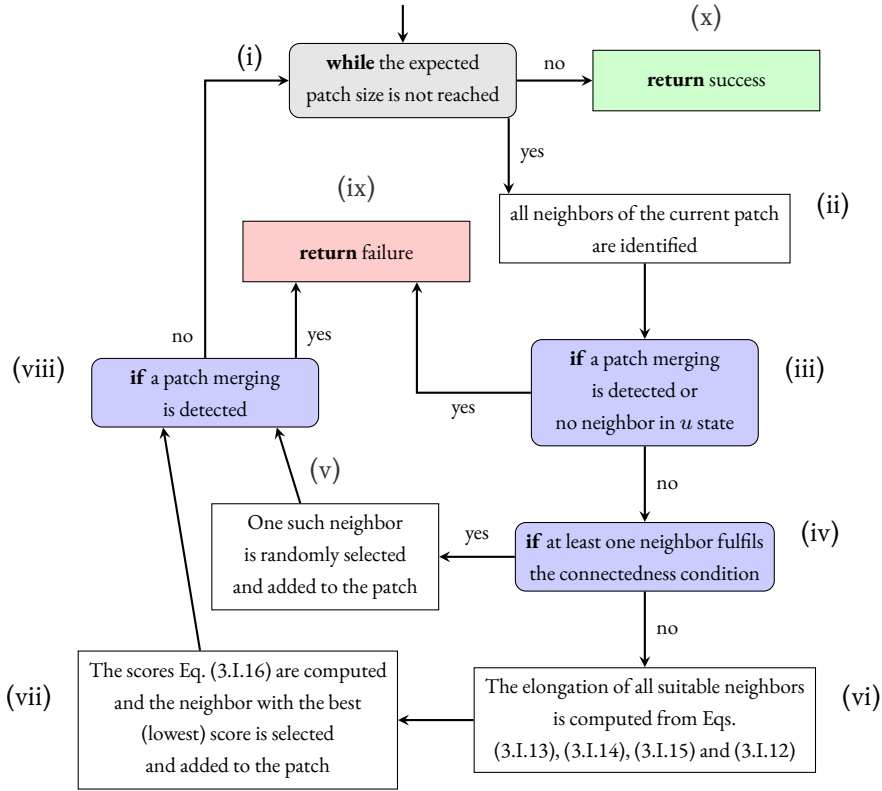


FIGURE 3.11: Patch design procedure workflow. Two parameters are required: the previously randomly selected patch size and elongation.

3.1.3. PATCH DESIGN ALGORITHM

The patch design procedure is represented as a workflow on Fig. 3.11. In the following description, the Roman numerals refer to the numbered steps in this workflow. The process is iterative and begins with the pivot-cell. The objective is to produce some randomness in the patch shape, while preserving as best as possible the patch selected size and elongation. This procedure refers to two subprocesses (patch connectedness condition and neighbor elongation score), described right after the algorithm description.

3.1.3.1. ALGORITHM DESCRIPTION

The loop runs while the expected patch size is not reached (i). As the process goes on, the patch grows pixel by pixel. All neighbors of the current patch at each iteration are identified with the help of the chosen neighborhood matrix defined in section 3.I.1, applied to each pixel of the current patch boundary (ii). Next, all identified neighbors are tested as follows. If at least one neighbor belongs to another patch of the same final LUC state v (iii), a patch merging (defined in section 3.2.4) has occurred; the procedure is then stopped (failure) and no new patch is created (x). The same outcome follows if no neighbor is

in the pivot-cell initial state u (iii and x). Otherwise, there is at least one suitable neighbor, *i.e.* a neighbor with LUC state equal to the pivot cell initial state u (in particular, this excludes pixels who have already undergone another state transition during the same time step). Next, the patch connectedness condition (see below) is checked on all suitable neighbors (iv). If at least one pixel satisfies the connectedness condition, one such pixel (if several) is randomly selected and added to the current patch (v). Otherwise, the patch elongation associated to each pre-selected pixels in step (iv) is computed (vi). Indeed, adding one such neighboring pixel to the patch would change its elongation and one therefore needs to find the most suitable pixel to be added to the patch, if more than one such pixel is available. This is defined on the basis of an elongation score: the pixel with the best (lowest) elongation score is selected and added to the current patch (vii); the computation of the elongation score is described below. At this point, one pixel has been added to the patch, either from the patch connectedness test (v) or from the elongation score one (vii). A final patch merging test is then performed (viii). If a patch merging is detected, the patch is discarded and the procedure stops with a failure outcome (x). Otherwise, the patch growth process starts again until the expected patch size is reached (i). Finally, the patch is allocated and the algorithm returns a successful outcome (xi).

All tested pixels (ending up in either success or failure) are removed from the pool of potential pixels for a $u \rightarrow v$ transition (set J , which therefore is reduced as the iterative allocation method progresses). Whether the patcher procedure ends in success or failure, the allocation method continues as described in section 3.4.2 and in Fig. 3.2.

The patch merging test of steps (iii) and (viii) bears only on merging with other patches produced at the same time step. Merging with pre-existing areas (of the same or a different state) does not lead to a patch rejection. As a consequence, and if the transition probability is correctly estimated in the vicinity of existing areas (as ensured by our calibration-estimation procedure, Mazy and Longaretti 2022a), our patch design procedure can also serve the purpose of the expander patch creation procedure of Dinamica EGO or other software. No specific algorithm is needed for this purpose in our software.

3.1.3.2. CONNECTEDNESS TEST

A “connectedness test” has been implemented in order to prevent the formation of small areas of one or a few non-transited pixels in a patch. To this effect, we consider all pixels that have successfully passed tests (ii) and (iii) in the patch formation procedure just described. The rook neighboring structure is attached to each of these pixels. Next, we determine the number of neighbors

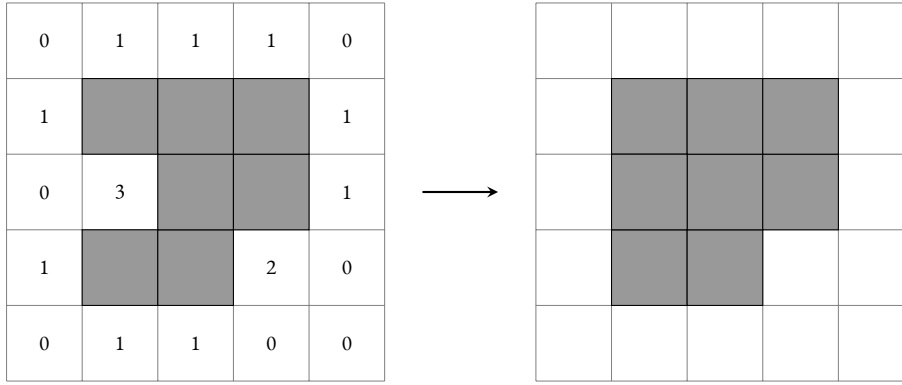


FIGURE 3.12: Allocation example on the basis of the connectedness test. The chosen neighborhood structure is rook and the connectedness condition is set to 3.

n_{rook} in this rook structure, whose LUC state is identical to v (the final state in the transition of interest). We define a threshold number of neighbors n_c . If $n_{rook} \geq n_c$, the pixel under test is selected for allocation in (iv), and one such pixel is randomly selected in (v), if more than one is found.

Fig. 3.12 presents an example of this test with the threshold set to $n_c = 3$ (our default choice). Obviously, this approach is only efficient for small holes and is inefficient in case of large ones. However, such a shape is very unlikely, due to the elongation condition also included in the patch design procedure. As a matter of fact, we have never observed a failure of the test to achieve its intended purpose, but we do not provide a formal proof of this statement.

3.1.3.3. ELONGATION SCORE TEST

In case the connectedness test just described has failed, all pixels selected in steps (ii) and (iii) are considered in steps (vi) and (vii). Each such pixel is tested by provisionally adding it to the current patch. This results in a change of elongation of the patch. The new potential patch elongation is computed under the assumption that the patch centroid does not change. Indeed, the addition of a pixel will only slightly change the center of mass (x_0^c, x_1^c) of the patch (unless it is composed of only a few pixels) and making this assumption greatly simplifies the calculations and leads to a more efficient process. For this purpose, for each neighbor j with coordinates (x_0^j, x_1^j) , one computes new potential second

order moments:

$$\mu_{20}^j = \frac{1}{M_{00} + 1} (M_{00}\mu_{20} + (x_0^j - x_0^c)^2) \quad (3.1.13)$$

$$\mu_{02}^j = \frac{1}{M_{00} + 1} (M_{00}\mu_{02} + (x_1^j - x_1^c)^2) \quad (3.1.14)$$

$$\mu_{11}^j = \frac{1}{M_{00} + 1} (M_{00}\mu_{11} + (x_0^j - x_0^c)(x_1^j - x_1^c)), \quad (3.1.15)$$

where moments M_{20} , M_{02} and M_{11} and centroid coordinates (x_0^c, x_1^c) refer to the current patch. The neighbor elongation e^j is then computed according to the equation (3.1.12).

Next, in order to decide which among all pixels thus characterized is eventually incorporated in the current patch, a test score is computed. The basic rationale of this score is to take two constraints into account: the closer e^j is to the expected elongation patch parameter e , and the higher the transition probability $P(v|u^j, \mathbf{z}^j)$ of this neighbor, the most likely this neighbor is to be selected for inclusion in the current patch.

A possible score satisfying these two conditions is the following:

$$\tau^j = \frac{|e - e^j|}{P(v|u^j, \mathbf{z}^j)} \quad (3.1.16)$$

where v is the final LUCC test in the transition of interest. The neighbor with the lowest τ score is selected and added to the current patch (vii).

3.1.4. PATCH ELONGATION VALIDATION

We are now in position to provide a justification of this rather convoluted choice of patch construction for a given elongation. This is done on an example. Namely, we take again the very simple case study of section 3.5.3.3, except that we now vary the patch elongation parameter from 0 to 1.

It has already been pointed out that the shape parameter of Dinamica EGO had a rather strange and in fact disappointing behavior in this simple problem (Appendix 3.E.3.2). Conversely, our choice of patch construction and elongation definition allows us to obtain a much more controlled result, as shown on Fig. 3.13. The relation displayed on this figure shows the relation between the elongation selected prior to the formation of the patch (expected elongation) and the elongation measured on the actual patch (measured elongation). This

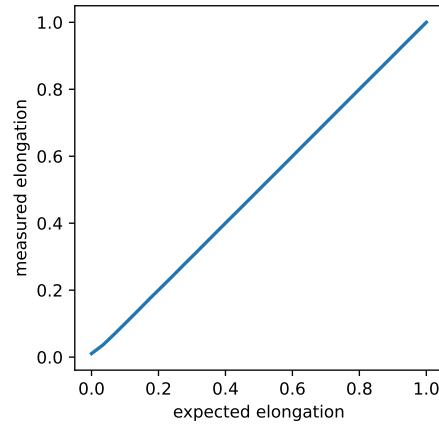


FIGURE 3.13: Measured elongation (averaged) of final LUC maps produced by the uPAM on the case study 3, as a function of the expected mean elongation.

relation is an average over a large number of generated patches with the same expected elongation, in order to reduce statistical sources of noise. A nearly perfect equality between expected and measured elongations is obtained. Furthermore, the normalized standard deviation of all measured elongations is not represented but is very low ($< 5 \times 10^{-3}$). This nearly perfect relation between imposed and measured elongation implies that if some patch shape measure based on second order moments were included in the calibration process, our patch construction parameter would be essentially bias-free.

This justification does not uniquely select the procedure we have defined. We merely checked here that this procedure is satisfactory, not necessary.

CHAPTER 4 | A MAXIMUM RELEVANCE / MINIMUM REDUNDANCY SELECTION PROCEDURE OF EXPLANATORY VARIABLES

N.B.: Appendix 4.C is not included in the submitted version.

Longaretti, P.-Y. and Mazy, F.-R. Towards a Generic Theoretical Framework for Pattern-Based LUCC Modeling. A maximum relevance / minimum redundancy selection procedure of explanatory variables. *Submitted to Environmental Software and Modeling*, 2022

ABSTRACT

Selecting explanatory variables in an optimal way in pattern-based Land Use and Cover Change (LUCC) models is an issue that does not seem to have been dealt with in a careful way in the specialized literature. In this work, we first review the core rationale of the problem and outline a possible strategy to implement an algorithmic approach designed to maximize explanatory variable relevance on the one hand, and minimize redundancy on the other. The few parameters of this semi-automatic method must be chosen carefully and we illustrate this approach in a simple case study. This method of selecting explanatory variables is implemented in our own software, CLUMPY (Comprehensive Land Use [and cover] Model in PYthon).

4.1. INTRODUCTION

In Land Use and Land Cover Change (LUCC) models, explanatory (or independent) variables (EVs) are identified by the user, from published studies and from her/his own expertise. These variables are used as predictors of future LUCC projections, due to their correlations with previously observed patterns of such changes in raster maps of the study area. This correlation is established on the basis of maps of spatially explicit probabilities of change (or dependent variables), that are obtained during the calibration process of the LUCC model under elaboration.

This process is initiated with a broad search of variables that may be relevant to the problem, and may end up with a pre-selection of one or two dozen of explanatory variables. These variables may be of widely different nature: continuous quantities such as distances to geographic features or boundaries of interest, slope, elevation, etc; categorical data such as soil type, etc; socio-economic continuous or categorical data such as real estate prices or employment level, socio-economic categories of population (for projections of urban sprawl), etc. The task facing the modeler at this point is to reduce this multiplicity, as such a high number of variables is in practice not manageable. This may be conducted with two principles in mind:

- Variables may be selected on the basis of decreasing relevance, i.e., on the level of correlation they have with the dependent variable, i.e., the probability of land use and cover (LUC) state transition in a given time interval. A possible tool to evaluate this relevance is a form of Cramer's test. An alternative test suggested in the machine learning literature is the F-test (Ding and Peng, 2005), but there is no lack of potentially relevant statistical tests for such a purpose.
- Variables may be rejected on the basis of redundancy (or, conversely, independence), e.g., by minimizing explanatory variables similarities and ranking the variables in some sort of increasing redundancy order. Redundancy between two distributions may be evaluated with a different form of Cramer's test; the use of the correlation matrix may also be considered, as this is a common enough tool of the trade.

The total number of retained variables for any transition from an initial state u to a final state v may be defined in two different ways: absolute — by fixing the maximum number of variables kept in the final selection — or relative — by

specifying thresholds of relevance score on the one hand, and redundancy score on the other, or of some combined redundancy-relevance score.

These issues constitute the main object of this work, which is organized as follows. Section 4.2 introduces our definitions and notations. Section 4.3 recalls the rationale of the search for explanatory variables; this section formulates in particular a necessary and sufficient criterion for an explanatory variable to be relevant when such variables are not statistically independent. This condition and its simplified form are rarely if ever explicitly stated and this leads to possible confusion in the understanding of the explanatory variable selection strategies in existing software. Section 4.4 constitutes the core of this work and presents our selection criteria and their implementation. Section 4.5 illustrates this procedure and its efficiency with partially synthetic data. Section 4.6 discusses the generic issue of the reliability of explanatory variable search and pre-selection, and section 4.7 concludes the paper by summarizing its main points. Relevant background information, formal proofs and technical details are provided in a number of dedicated appendices.

4.2. DEFINITIONS AND NOTATIONS

element	set	definition
u, v	V	LUC state at t_0 and t_1
i	I	Index of pixels in state u at t_0
j	I_v	Index of pixels in state u at t_0 and state v at t_1
z_k, z_k^i, z_k^j	$\mathcal{D}_k$	k th estimation explanatory variable value for pixels in J_v
d_M, d	$\mathbb{N}$	initial (pre-selected) and final (selected) number of explanatory variables
$\mathbf{z}, \mathbf{z}^i, \mathbf{z}^j$	$\mathcal{D}, \mathcal{D}_M$	d -tuple (resp. d_M) of estimation explanatory variables for pixels in J_v
$\mathbf{Z}$	not defined	$\#J_v$ -tuples of values of $\mathbf{z}$ taken by pixels in J_v
γ_k, Γ_k	$\mathbb{N}$	Index of possible and maximum discretized or discrete value of z_k
γ_z	$\mathbb{N}^d, \mathbb{N}^{d_M}$	d -tuple (resp. d_M -tuple) of indices of discretized or discrete explanatory variables
δ_k	$\mathbb{R}$	Bin size of continuous variable z_k
$n_X, \#X$	$\mathbb{N}$	Number of pixels in set X

TABLE 4.1: Summary of notations and definitions.

Our notations are described below and summarized in Table 4.1.

LAND USE AND COVER STATES

Each LUC state is represented by a positive natural number (the null value is used for unknown state or pixels outside the study area, a common convention in GIS environments). We assume we have at our disposal two calibration maps, at time t_0 and t_1 . The notation u is used for a pixel state at time t_0 (first calibration map) and v for this pixel state at time t_1 (second calibration map). Because explanatory variable selections for any transition is made in the same way, we focus for definiteness of a given pair (u, v) .

RASTER QUANTITIES

We call J the set of pixels associated with initial state u at t_0 . Let $\#J$ be the number of pixels in state u . Each pixel is identified by its index i which can be used as a superscript. One has $J = \{1, \dots, n_J\}$. Similarly, we use the notation J_v for pixels in initial state u at t_0 and final state v at t_1 .

EXPLANATORY VARIABLES

Explanatory (or ancillary, or predictor) variables are quantities defined on pixels, and considered relevant for the statistical prediction of LUCCs, due to their correlation with past observed changes.

Let d_M be the initial (pre-selected) and d be the final (selected) numbers of explanatory variables. We define the explanatory variable space as a point space $\mathcal{D}_M$ (resp. $\mathcal{D}$) of dimension d_M (resp. d). By construction, $\mathcal{D}_M \subset \mathbb{R}_M^{d_M}$, (resp. $\mathcal{D} \subset \mathbb{R}^d$).

Continuous or discrete explanatory variables are designated by z and their set by $\mathbf{Z}$. Specific explanatory variables are identified with a subscript k as z_k . These quantities are indexed by i or j when they refer to pixel i or j (e.g., z^i, z^j, z_k^i, z_k^j).

Selecting explanatory variables is easier if continuous variables are discretized. In this case, one associates to an explanatory variable z_k a range Γ_k of possible integer values, each representing a bin¹ identified by a dummy integer γ_k : $1 \leq \gamma_k \leq \Gamma_k$; the associated bin size is δ_k . The same notations are used for discrete variables, except for the bin size which is not relevant for these variables.

One finally defines $\gamma_z = (\gamma_1, \gamma_2, \dots, \gamma_d)$ or $\gamma_z = (\gamma_1, \gamma_2, \dots, \gamma_{d_M})$, which collects all the indices of the discrete and discretized explanatory variables.

¹The actual range in z_k of these bins does not need to be explicitly specified, and depends on the bin size δ_k , the range of z_k and the total number of bins Γ_k .

ENSEMBLE CARDINALS

All quantities of the form $\#X$ or n_X refer to the number of pixels in a set X .

4.3. EXPLANATORY VARIABLES USE

Changes of LUC states are never randomly distributed on a map. The probability of transition of a pixel from state u to state v is always or nearly always dependent on the position $\mathbf{x}$ of the chosen pixel, and is denoted $P(v|u, \mathbf{x})$, indicating the conditional dependence on the initial state u and the position $\mathbf{x}$.

It is well-known that most if not all of this spatial dependence occurs through explanatory variables $\mathbf{z}$, which are usually deterministic functions of position. This fact is rather easy to spot in actual changes of LUC states, except for categorical and qualitative explanatory variables. For example, urban sprawl occurs in continuity of existing urban areas, so the distance to existing urban areas is an obvious (but not unique) choice for a predictor of the spatial distribution of urban sprawl. Deforestation also often occurs in continuity with previously deforested areas, and/or in relation to access roads, etc.

From this remark, one is motivated to reexpress the transition probability just introduced²:

$$P(v|u, \mathbf{x}) = P(v|u, \mathbf{z}(\mathbf{x}), \mathbf{x}). \quad (4.1)$$

Finally a common assumption is that all spatial dependence occurs through explanatory variables, so that

$$P(v|u, \mathbf{z}(\mathbf{x}), \mathbf{x}) = P(v|u, \mathbf{z}), \quad (4.2)$$

where the explicit dependence of $\mathbf{z}$ on position $\mathbf{x}$ has been dropped. This last property (spatial dependence only through explanatory variables) cannot be proved. It is assumed as it appears to be either true or sufficiently precise in practice, and it does have some intuitive basis (if some direct dependence on position remains, it usually means that some explanatory variable has not yet been identified).

²Categorical and qualitative explanatory variables are included in this expression; for these variables, the dependence on $\mathbf{x}$ is either inexistent, or categorical as well.

Further progress is made through the use of Bayes rule:

$$P(v|u, \mathbf{z}) = P^*(v|u) \frac{\rho(\mathbf{z}|u, v)}{\rho(\mathbf{z}|u)}, \quad (4.3)$$

where $P^*(v|u)$ is the targeted global change probability (the fraction of pixels undergoing such a change), $\rho(\mathbf{z}|u)$ is the probability density of $\mathbf{z}$ for pixels of initial state u , and $\rho(\mathbf{z}|u, v)$ is the probability density of $\mathbf{z}$ for pixels undergoing a state change from u to v . Note that probability densities are required as soon as at least one explanatory variable is continuous. Bayes rule is instrumental in assessing the probability of state change $P(v|u, \mathbf{z})$ as $\rho(\mathbf{z}|u, v)/\rho(\mathbf{z}|u)$ is easier to calibrate from the data.

Consider first the (unrealistic) case where only one explanatory variable z_k is needed for a given transition. The ratio of probability density on the right-hand side of Bayes rule reduces to³ $\rho(z_k|u, v)/\rho(z_k|u)$. Quite clearly, if this ratio is constant, $P(v|z_k, u)$ is independent of z_k and therefore independent of position. More precisely, a necessary and sufficient condition for the transition to be spatially uniform is⁴ $\rho(z_k|u, v) = \rho(z_k|u)$, i.e., the distribution of z_k is independent of the final state v .

The point just made for a single explanatory variable translates readily to a collection of statistically independent explanatory variables. In this case,

$$\frac{\rho(\mathbf{z}|u, v)}{\rho(\mathbf{z}|u)} = \prod_k \frac{\rho(z_k|u, v)}{\rho(z_k|u)}. \quad (4.4)$$

The same condition of dependence of $P(v|\mathbf{z}, u)$ on (or independence from) a given variable z_{k_0} clearly applies.

The problem is somewhat less direct when explanatory variables are not statistically independent, which is the most common situation. Statistical independence is often assumed, but we have shown elsewhere that this may lead to gross errors in the calibration of the transition probabilities (Mazy and Longaretti, 2022a), even for low levels of correlations between the variables. We therefore refrain from making the assumption of statistical independence. In-

³Or $P(z_k|u, v)/P(z_k|u)$ if the variable is discrete. From now on, we ignore discrete variables. The various expressions below are either valid if discrete variables are included, or easily extended to this case. Furthermore, as continuous variables are discretized for the purpose of explanatory variable selection, discrete and continuous variables can be treated on the same footing in most of the present analysis.

⁴The constant ratio just mentioned is equal to 1 because the integral of probability distributions are normalized to unity.

stead, for an arbitrarily chosen explanatory variable k_0 , one has

$$\frac{\rho(\mathbf{z}|u, v)}{\rho(\mathbf{z}|u)} = \frac{\rho(z_{k_0}|\mathbf{z}^*, u, v)}{\rho(z_{k_0}|\mathbf{z}^*, u)} \times \frac{\rho(\mathbf{z}^*|u, v)}{\rho(\mathbf{z}^*|u)}, \quad (4.5)$$

where $\mathbf{z}^*$ is the collection of explanatory variables with $k \neq k_0$. The preceding argument again shows that a necessary and sufficient condition for $P(v|\mathbf{z}, u)$ to be independent of z_{k_0} is $\rho(z_{k_0}|\mathbf{z}^*, u, v) = \rho(z_{k_0}|\mathbf{z}^*, u)$.

Calibrating the probability distributions $\rho(z_{k_0}|\mathbf{z}^*, u, v)$ and $\rho(z_{k_0}|\mathbf{z}^*, u)$ in this equality for all possible k_0 may turn out to be a lengthy and data intensive process, due to their dependence on $\mathbf{z}^*$. This problem is alleviated as one can show that $\rho(z_{k_0}|u, v) \neq \rho(z_{k_0}|u)$ is a sufficient condition for z_{k_0} to be an explanatory variable for the transition $u \rightarrow v$. This statement is explicitly demonstrated in Appendix 4.A.

One cannot prove that it is a necessary condition as well. On the other hand, the condition is both necessary and sufficient for independent explanatory variables. As one looks for the least mutually dependent variables in the process, one may suspect that the condition $\rho(z_{k_0}|u, v) \neq \rho(z_{k_0}|u)$ is also a nearly sufficient condition of dependence of P on z_{k_0} in this context, to some level of approximation that remains to be established. In any case, once a small number of relevant explanatory variables has been selected on this basis, one may always come back to testing the variables relevance on the basis of their conditional probability distributions, and check if the relevance test scores are comparable with their unconditional form.

It would be useful to explore this question in more detail in the future, but we provisionally assume that this condition is sufficiently precise to select relevant explanatory variables. This discussion is summarized as follows:

DEPENDENCE CRITERION

A sufficient and, for all practical purposes, quasi-necessary condition for $P(v|\mathbf{z}, u)$ to be dependent of z_{k_0} is $\rho(z_{k_0}|\mathbf{z}^*, u, v) \neq \rho(z_{k_0}|\mathbf{z}^*, u)$.

As a consequence, the converse $[\rho(z_{k_0}|\mathbf{z}^*, u, v) = \rho(z_{k_0}|\mathbf{z}^*, u)]$ is a necessary and quasi-sufficient condition for P to be independent on z_{k_0} .

4.4. A NEW MINIMUM REDUNDANCY - MAXIMUM RELEVANCE METHOD OF SELECTION OF EXPLANATORY VARIABLES

Selecting explanatory variables relies heavily on inference statistics. Appendix 4.B summarizes the relevant concepts and pieces of information from this field that are needed for our purpose.

The choice of explanatory variables can have a significant impact on the results. Too many explanatory variables can be a source of poor performance, both in terms of accuracy and computation time. It is therefore common in machine learning to reduce if necessary the number of explanatory variables by selecting them according to various criteria or more or less sophisticated algorithms (Chandrashekar and Sahin, 2014).

Among the methods commonly used to this effect, some rely on a prediction score calculated on a subset of explanatory variables. These are called wrapper methods. However, these use the predictor objective function itself as a score metric [here, this would be the $\rho(z_{k_0}|u, v)/\rho(z_{k_0}|u)$ ratio], but there is no natural scale for this in the present context. Thus, we must revert to so-called filter methods which have at least the advantage of being computationally efficient.

Two criteria are taken into consideration in this approach: the relevance and the mutual independence (or conversely, redundancy) of explanatory variables. To this effect, one of the most popular variable selection filter method, called minimal-redundancy/maximal-relevance (mRMR), has been proposed by Ding and Peng (2005) and Peng et al. (2005). This method is also typical of this line of approach. However, on closer inspection, it appears that this method is inappropriate for selecting explanatory variables in the context of LUCC modeling (see Appendix 4.C). Still, as the idea of optimal selection is clearly a relevant framework, an alternative selection method based on the same maximum relevance/minimum redundancy guiding principle is proposed in the next subsection.

4.4.1. GENERALITIES

The selection method presented here relies on the use of the various forms of χ^2 tests, or, equivalently, Cramer's V and a Cramer-like tests. The latter are favored as they are normalized with respect to size effects and allow us to rank explanatory variables easily according to test scores, on top of simply accepting

or rejecting them. χ^2 and Cramer's tests characterize the similarity between two distributions by testing the similarities of data samples drawn from these distributions (or assumed to be drawn from them, the tests being made to check this type of hypothesis). They require categorical data, which implies that continuous variables must first be discretized, transforming their probability distributions into a categorical one (see Appendix 4.D). This transforms $P(v|u, z)$, $\rho(z|u)$ and $\rho(z|u, v)$ into $P(v|u, \gamma_k)$, $P(\gamma_k|u)$ and $P(\gamma_k|u, v)$, respectively.

These tests rely on two assumptions. The first one is statistical independence of the data samples. In practice, this means that if a pixel subsample is drawn at random from the calibration data set, the distribution of the sample and of the parent population are identical, in the mean. This assumption requires some discussion in the LUCC modeling context. On the one hand, it is a certainty that a random subsample of any pixel population (e.g., the pixels with initial state u , final state v , and given range of explanatory variables) will satisfy this assumption, precisely because pixels are randomly drawn from the pixel set. On the other hand, the original pixel population itself does not satisfy the independence assumption: pixels change state in patches, not independently of each other. However, and somewhat paradoxically, this does not in practice affect the statistical independence assumption, for the following reason. Indeed, correlations are spatial. If one takes the same set of transited pixels and reorders them in a random fashion, the spatial correlation will be lost. Still the pixel population will have exactly the same probability distribution, because one looks only at single pixel probability distributions. This indicates that as long as one focuses only on single pixels probabilistic properties and works in explanatory variables space and not in physical space, as one does in the selection of explanatory variables, spatial correlation is unimportant.

This conclusion may seem paradoxical, as the primary purpose of explanatory variables is to explain spatial patterns. The key point here is the assumption that spatial dependence is accounted for only through explanatory variables z . Once this assumption is made, one may as well ignore the spatial dependence of z , which becomes unessential for statistical pixel-by-pixel characterizations. It is only when moving back to spatial characteristics that the deterministic $z = z(x)$ relation comes into play to explain spatial distributions (x being the spatial coordinates). Another important point here is that pixel spatial correlations cannot be accounted for, in the probabilistic sense, at the level of 1-pixel probability distributions, and *all* probability distributions used in LUCC pattern-based modeling are 1-pixel distributions — including the ones involved in Bayes rule, Eq. (4.3). Correlations appear in principle only at the level of n -pixel probability distributions, with n taking all values up to the total

number of pixels in the study area maps. Such an approach is rigorous, but not tractable. In practice, it is much preferable to define pixel correlations through patch statistics. This approach is adopted by Dinamica EGO, see, e.g., Soares-Filho et al. 2002, and in our analysis of allocation biases (Mazy and Longaretti, 2022b). These points will be explored in more detail elsewhere. A more detailed discussion of this point can be found in section 2 of Mazy and Longaretti (2022a).

The tests further assumes that samples mean proportions in each pixel bin follow a normal distribution around the original pixel set means. For example, the original pixel set mean is given by the appropriate probability, e.g., $P(\gamma_z|u, v)$ for the mean proportion in the γ_k bin of pixels having undergone a $u \rightarrow v$ state transition. When drawing an unbiased (random) pixel sub-sample of given size from this distribution, the sub-sample mean proportions will approximate this probability distribution. Drawing many sub-samples of the same size will provide many different measures of these mean proportions. The assumption made here is that the distribution of these sub-sample means (across a large number of samplings of a given size) is normal.

In fact, the distribution of the sample means is more and more normal when increasing the number of data points (the size of the sub-samples), due to the central limit theorem, and the categorical bin probability will be better and better approximated due to the law of large numbers (Papoulis and Pillai 2002, chapter 7). In practice, only $\gtrsim 5$ data points per bin are required for the normal approximation to apply to some reasonable level of approximation. These statements are presented in more quantitative form in Appendix 4.D.

When this minimal sample size per bin is violated, under-sampled bins have an exaggerated influence on the result of the test. For a large enough number of bins, a way to avoid this under-sampling problem is to ignore bins with too few elements. This is possible without significant error if they are located on the tail(s) of the distributions tested. Such bins are not significant, so that ignoring them will not change the result of the test.

Two forms of Cramer's test are of interest here: the Cramer test of independence and the Cramer-like goodness-of-fit test; they are each based on the χ^2 test of the same name. These two different tests are used to reject different null hypotheses, to some significance level. The advantage of Cramer's tests over χ^2 ones is that their scores are normalized between 0 and 1, and are independent of sample sizes. These tests return test scores, to be compared with the critical test score of rejection of the null hypothesis. The larger the score with respect to this critical threshold, the more significant the rejection of the null hypothesis. For Cramer's tests, in practice, 0.1 is a low score, 0.3 a medium one and 0.5 a

large one (cf Hahs-Vaughan and Lomax 2020, table 8.4 p. 312 and related text). The definition of the critical threshold depends on how conservative the user wishes to be; such a threshold replaces the significance level α of the related χ^2 test (see Appendix 4.B.2). This will be further discussed in section 4.4.4, as well as how standardized residuals are constructed in order to assess the origin of the test significance.

The tests involve the comparison of observed bin population numbers (population histograms) resulting from sampling from the categorical distributions mentioned above, or of so-called contingency tables, with hypothesized reference population. The observations are traditionally noted O_i (bins) or O_{ij} (contingency tables) (O stands for “observable”) and their comparison quantities, E_i (bins) or E_{ij} (contingency tables) (E stands for “explanatory”). The tests can be performed in different ways, corresponding to different choices for both O and E and to different null hypotheses.

The exact choice of the test (independence or goodness-of-fit) depends on whether one is interested in explanatory variables’ relevance or redundancy. This is discussed in the next two subsections.

4.4.2. TEST OF RELEVANCE

The less uniform the ratio $P(\gamma_k|v, u)/P(\gamma_k|u)$, the more significant the explanatory variable, from the discussion of section 4.3. One is therefore motivated to choose a test allowing us to compare $P(\gamma_k|v, u)$ and $P(\gamma_k|u)$. Such a comparison is ideally suited for a goodness-of-fit Cramer test. Indeed, the test aims at rejecting the identity between the observed O and explanatory E samples, while an independence test of $P(\gamma_k|v, u)$ and $P(\gamma_k|u)$ would lead to reject the independence hypothesis, i.e., the opposite of the sought for result. Note also that the goodness-of-fit test is both more discriminant and more robust than the F-test suggested by Ding and Peng (2005), as the comparison of the whole distributions is involved instead of a comparison of their means.

It is necessary to bin continuous variables before applying the relevance test. For each variable, the binning method is described in Appendix 4.D.3. This binning makes it possible to evaluate the “observable” (O) and “explanatory” (E) quantities:

$$O_{\gamma_k} = n_{\gamma_k, u, v} \frac{n_I}{n_{I_v}}, \quad (4.6)$$

$$E_{\gamma_k} = n_{\gamma_k, u}, \quad (4.7)$$

where $n_{\gamma_k, u, v}$ and $n_{\gamma_k, u}$ refer to the number of pixels in the bin γ_k in the sets I_v and I , respectively. The n_I/n_{I_v} factor in the first relation ensures that the O and E pseudo sample sizes are identical, as required by this test (these numbers are actually reduced; see Appendix 4.D.3). The relevant forms of the χ^2 test and associated Cramer test are described in Appendix 4.E.1. In particular, the Cramer-like test score V_{GoF} is given by Eqs. (4.E.4) and (4.E.1) with $N = n_I$.

4.4.3. TEST OF REDUNDANCY

One wishes to compare $P(\gamma_k|v, u)$ and $P(\gamma_{k'}|v, u)$ for $k \neq k'$ and this points towards tests of independence (Appendix 4.E.2), which allow us to compare $P(\gamma_k, \gamma_{k'}|v, u)$ with $P(\gamma_k|v, u) \times P(\gamma_{k'}|v, u)$. Indeed, the more correlated the two variables z_k and $z_{k'}$, the more different the bivariate distribution from the product of the univariate ones.

Tests of independence involve a contingency table. The construction and rationale of this table is now discussed. The motivation is to compare maps of $P(\gamma_k|v, u)$ and $P(\gamma_{k'}|v, u)$. Area cross tabulations provide a compact way to achieve such a comparison (Bonham-Carter, 1994). Consider a map A with n_A categories (e.g., discretized elevation) and a map B with n_B categories (e.g., soil type) of the same study area. The area cross table T_{ij} lists the n_A categories in rows and the n_B categories in columns ($1 \leq i \leq n_A, 1 \leq j \leq n_B$). Each element T_{ij} of the table gives the maps number of pixels n_{ij} of the relevant category of maps A and B (given elevation range and given soil type). For the problem at hand, the categories are the bins of explanatory variables k and k' for a given $u \rightarrow v$ transition.

The intuition underlying this summarized description of spatial maps is that if the two quantities shown in maps A and B are highly spatially correlated, the categories themselves will be correlated, with the same pixels belonging to only one given category of each map. The area cross table will therefore have nearly only one non-zero slot per row and per column. Conversely, if the categories in maps A and B are uncorrelated, the joint probability of categories i, j is given by $p(i, j) = p(i)p(j)$, so that, defining N as the number total number of pixels in each map, $N_i = Np(i)$ and $N_j = Np(j)$ are the number of pixels in category i of map A and j of map B, and $N_{ij} = Np(i)p(j) = N_i N_j / N$. More generally, the area cross table is filled with the number of pixels $Np(i, j)$; in the case at hand, $p(i, j) = P(\gamma_k, \gamma_{k'}|v, u)$ and $N = n_{I_v}$ (reduced as indicated in section 4.D.4). Quite clearly, this type of test is more discriminant than, e.g., the use of the explanatory variables correlation matrix (a commonly chosen measure of independence), as it relies on comparisons of whole distributions, and not

simply on a single integrated measure of correlation.

The contingency table O_{ij} of the tests of independence specified in Appendix 4.E.2 is precisely the area cross table T_{ij} just described, for the maps and categories of explanatory variables k and k' whose redundancy (correlation) one wishes to assess, *i.e.* in our specific context:

$$O_{\gamma_k, \gamma_{k'}} = n_{\gamma_k, \gamma_{k'}, u, v}, \quad (4.8)$$

$$E_{\gamma_k, \gamma_{k'}} = \frac{n_{\gamma_k, u, v} n_{\gamma_{k'}, u, v}}{n_{I_v}}, \quad (4.9)$$

The test is fully described in Appendix 4.E.2. In particular, the Cramer test score V_{ToI} is given by Eqs. (4.E.8) and (4.E.5) (where $i = \gamma_k$ and $j = \gamma_{k'}$) with $N = n_{I_v}$ (reduced as indicated in Appendix 4.D.4).

4.4.4. MINIMUM REDUNDANCY - MAXIMUM RELEVANCE SELECTION OF EXPLANATORY VARIABLES FROM CRAMER AND CRAMER-LIKE TESTS

We are now in position devise a selection method inspired from the principles advocated by Ding and Peng (2005) and called Cramer-MRMR. This proceeds as follows:

1. Select a specific transition $u \rightarrow v$.
2. First, rank the potential explanatory variables in decreasing relevance order on the basis of Cramer's goodness-of-fit test (section 4.4.2 and Appendix 4.E.1). Explanatory variables with score tests $V_{GoF} \lesssim V_{GoF}^m \sim 0.1$ should be discarded (the exact choice of V_{GoF}^m is left to the modeler).
3. Next, compute Cramer's test of independence (see section 4.4.3 and Appendix 4.E.2) for each pair of remaining variables starting with the one with the highest relevance V_{GoF} . When the score test $V_{ToI} \gtrsim V_{ToI}^M \sim 0.2$, the variable with the lowest relevance V_{GoF} is discarded (the exact choice of V_{ToI}^M is left to the modeler).
4. Collect all variables selected in this way for all possible end states v from the initial state u , and tag which are relevant for each transition.

This procedure puts more weight on relevance than redundancy, as keeping a redundant variable it is preferable to rejecting a relevant one, but both V_{GoF}^m and V_{ToI}^M are set to conservative limits, the relevance limit being more conservative than the redundancy one. More specifically, the choice of V_{GoF}^m errs on the side

of keeping possibly not strongly relevant variables, while the choice of V_{ToI}^M errs on the side of keeping even moderately correlated variables. These limits can be adjusted if the number of explanatory variables kept in the end is too large for the computational power at hand. The first limit to change is V_{ToI}^M (setting this higher will result in lower redundancy), in line with favoring relevance in the selection over redundancy in the rejection.

Finally, it is always informative to have a better grasp of the test significance. To this effect, one must look, for each transition and each explanatory variable, at the individual probability distribution categories and their weight in the overall test result, which only presents a summary of the test. Standardized residuals provide a very common way to achieve this purpose. Although Sharpe (2015) suggests a number of alternatives or additional indicators, we stick to standardized residuals because of their simplicity and relative ease of interpretation. Standardized residuals are defined in Appendix 4.E for the two χ^2 tests underlying the above procedure.

4.4.5. REMARKS ON THE USE OF CRAMER'S TEST IN LCM AND DINAMICA

Strangely enough, neither Dinamica EGO nor LCM seem to consider at the same time redundancy and relevance as criteria of selection of explanatory variables. Dinamica EGO's guidebook provides some (incomplete) information about what is done, in the section devoted to the analysis of map correlations. Furthermore, a working paper of the Dinamica EGO group (de Almeida et al., 2002) clearly indicates that redundancy is addressed through contingency tables derived from area cross tabulation, as described in section 4.4.3. Various tests are performed to this effect in the software. Dinamica EGO eliminates variables on the basis of their redundancy and does not try to assess their relevance.

Conversely Idrisi LCM (at least the Selva edition that we have tested) seems to evaluate relevance only, from the section of the manual devoted to this issue (p. 233). However, understanding what LCM actually does requires more guesswork than for Dinamica. The manual mentions that high Cramer's test scores indicate relevant variables, and that variables with low V must be discarded. The test performed is most likely a test of independence, as the other one is rarely mentioned in the literature.

One may conjecture that the quantity tested is either $P(z|u, v)$ or $P(z|u)$. The question is, which of the two, and what is it tested against? The Idrisi manual mentions in one sentence that it is compared to the spatial distribution of land cover types in the post-transition calibration data map, and that continuous explanatory variables are binned into 256 categories. The Idrisi tutorial

indicates that tests are made for each possible transition, and starts by comparing a possible explanatory variable with the location of observed changes. We have found no published paper, conference proceedings, technical reports, etc specifying more precisely what LCM actually does.

On the basis of this scant information, the most meaningful possibility of applying a Cramer test of independence is to construct an area cross tabulation between maps of categories of LUCC on the one hand and the 256 bins of z values for $P(z|u)$, i.e., a categorical map of z for state u (the only option to compare the whole set of pixels in initial state u for all possible v). This form of the test is somewhat supported at the qualitative level by the intuition that explanatory variables must somehow be related to observed patterns of change. At the quantitative level, it suffers from two weaknesses. The first is a poor relevance criterion, as what matters is the difference between $P(z|u, v)$ and $P(z|u)$, as shown in section 4.3; the test performed is a degraded form of this comparison. The second is that the test may miss relevant variables, as all patterns of change for all possible transitions from u are tested at once (a similar criticism applies to the procedure proposed by Ding and Peng 2005). Because of this, significant test values will only be reached if a variable is relevant for several transitions. Idrisi's developers are clearly aware of this, as the tutorial indicates that Cramer's test is weakened (for a specific transition) from the fact that all LUC states are compared to the chosen explanatory variable distribution, and that individual classes (i.e. categories) values must be checked. Although this is not specified, the reported individual classes values are most likely some form of the standardized residuals discussed in Appendix 4.E.

4.5. EXPLANATORY VARIABLE SELECTION IN PRACTICE

After this presentation of our method of selection of explanatory variables (EVs), we illustrate it through a case study not meant to be exhaustive but rather intended to be an illustration of our approach to configuring and analyzing our variable selection method. All data involved as well as python script related are freely available on a Github repository⁵.

⁵<https://github.com/fmazy/cramer-mrmmr-illustration>

4.5.1. ILLUSTRATIVE CASE STUDY SHORT DESCRIPTION

We are interested in a relatively small study area located southwest of Grenoble (France) for which we have LUC maps in raster format (resolution of 15 meters for 6.3 million pixels for two different date: 2003 and 2009). This is the same area as the one described and used for illustration purposes in our previous article Mazy and Longaretti (2022c). It is a zone with a relative diversity of landscapes (forest, agricultural plains, valleys and mountains areas, urban areas...). LUC states are restricted here to 7 different ones at the coarsest typology level (water bodies, mineral areas, forests, agricultural areas, urban areas, economic activity areas and other areas). This study area is part of a larger zone that has already been extensively studied in the course of a past project (see Vannier et al. 2016, 2019a,b for more details).

In order for the present case study to be both illustrative and informative, we do not consider all possible LUCC state transitions and all possible explanatory variables, but only a rather small subset of these. Namely, for the present analysis, we restrict ourselves to two transitions: agricultural (AGR) to urban (URB) areas, and agricultural (AGR) to reforested (FOR) areas. These two transitions focus on two *a priori* opposite processes: urban sprawl on the one hand ($AGR \rightarrow URB$), and agricultural abandonment on the other, eventually leading to reforestation on formerly exploited land ($AGR \rightarrow FOR$). There are 3.2 millions agricultural (AGR) LUC pixels in 2003, out of which 17,500 transited to urban areas (URB) and 2,000 to reforestation (FOR) by 2009. For these numbers, the normal limit required by the selected tests is largely satisfied (Appendix 4.D.2).

The explanatory variables considered in this illustrative analysis are: 1- elevation; 2- distance to the nearest urban pixel; 3- distance to the nearest economic activity pixel and 4- distance to the nearest forest pixel. These explanatory variables are typically used in the study of urban sprawl and agricultural abandonment. Explanatory variables are referred to by their number in this list in the following sections. This set of variables is purposely restrictive to include a few relevant and some less relevant variables for each transition, and to illustrate the main features of our explanatory variable selection process, without side-tracking to useless information and details.

No discrete variable is included in our list, but the procedure proceeds in the same way if mixed continuous/discrete variables are pre-selected.

Our method involves very few “free” parameters: the binning parameter ε (Appendix 4.D.2.5), and the test score limits, V_{GoF}^m and V_{ToI}^M (section 4.4.4). These parameters can be left to their default choices, but as explanatory variable selection is not a completely objective process, the user is strongly encouraged

to evaluate the role of these parameters on the results of the selection process. Some guidelines to this effect are provided in the following section. In particular, the “best” choice of these parameters depends on the LUCC state transition considered.

We give a number of decimal places when reporting test scores, but one should keep in mind that changing the binning parameter ε affects the results at the first decimal place (without affecting the ordering); as ε is chosen by the user to obtain “nice” histograms (i.e., not too noisy, see examples below), we have rounded all number to two decimal places, for definiteness.

4.5.2. RESULTS AND DISCUSSION

We describe the results obtained for the two selected transitions, following the selection procedure of section 4.4.4. The three possible approximations introduced in Appendix 4.D.2.5 (mean, median and standard deviation) have been tested. As expected, the results obtained are quantitatively very similar and the outcome of the selection procedure is the same in all three cases. We describe them only for the mean, which was pointed out as the most convenient choice for various reasons in the Appendix.

4.5.2.1. TRANSITION FROM AGRICULTURAL AREAS TO URBAN AREAS (AGR TO URB)

The first parameter to set is ε . This parameter is directly related to the average number of pixels per bins ($\sim 1/\varepsilon^2$, see Appendix 4.D.2.5). If this parameter is too large, there is a risk of under-sampling, which can be checked directly on the level of the noise in the resulting histograms. The acceptable level of noise is somewhat subjective, and the user should adjust this parameter accordingly, by trial and error around the default choice made in the software. In the present case, we have set $\varepsilon = 0.08$. This parameter is set for all univariate and bivariate distributions examined in this exercise, for simplicity.

Also, the advantage of our pseudo-binning method based on our effective Kernel density estimation over direct binning (see Appendix 4.D.1 and the related binning criteria, Appendices 4.D.2.5 and 4.D.2.6) is directly visible by comparing Fig. 4.1 (direct binning) and Fig. 4.2 (pseudo-binning). Pseudo-binning substantially reduces the level of the statistical noise seen in the histograms⁶, which allows us to be much less restrictive on the number of excluded pixels (due to removing undersampled bins; see the quoted Appendices). We therefore apply this pseudo-binning method by default, and only the results

⁶But note that some most low amplitude oscillations remain in the histograms; these are most likely an artifact of the pseudo-binning method itself.

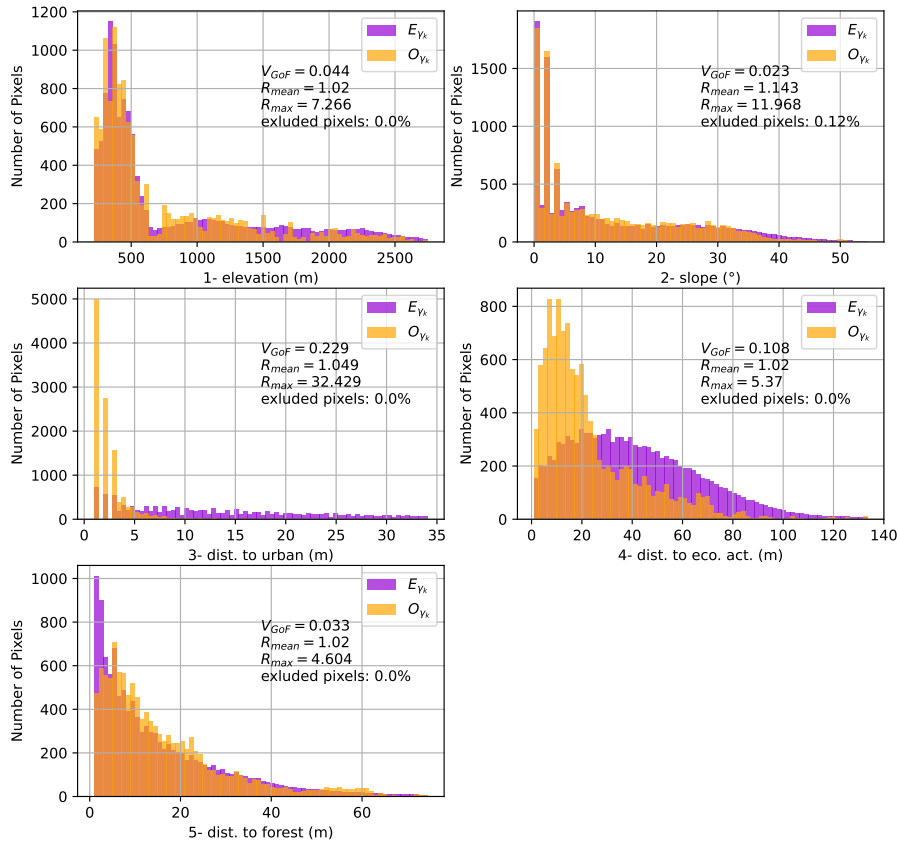


FIGURE 4.1: Test of relevance — direct binning: histograms of $E_{\gamma_k} = n_{\gamma_k, u}$ and $O_{\gamma_k} = n_{\gamma_k, u, v}$ for the transition $u = \text{AGR} \rightarrow v = \text{URB}$, and for all five possible choices of k . V_{GoF} [Eq. (4.E.4)] is given on each graph, as well as the tests of bin optimality (R_{mean} and R_{max}) and the fraction of excluded pixels (highly underpopulated bins).

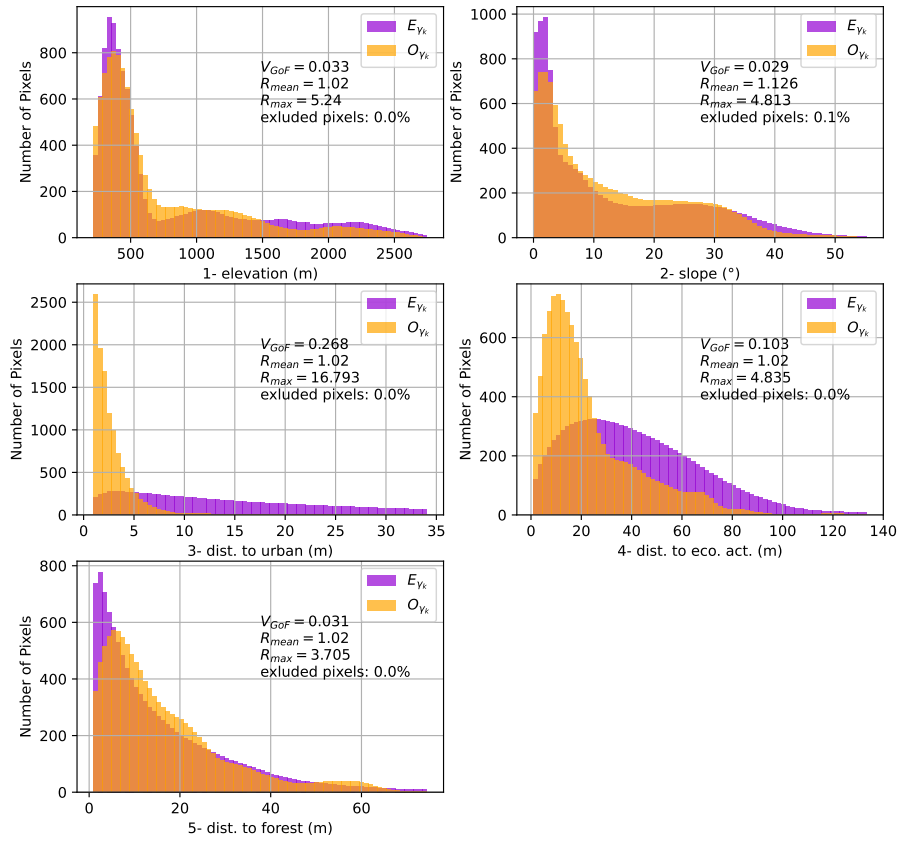


FIGURE 4.2: Test of relevance — pseudo-binning: histograms of $E_{\gamma_k} = \tilde{n}_{\gamma_k, u}$ and $O_{\gamma_k} = \tilde{n}_{\gamma_k, u, v}$ for the transition $u = \text{AGR} \rightarrow v = \text{URB}$, and for all five possible choices of γ_k . V_{GoF} [Eq. (4.E.4)] is given on each graph, as well as the tests of bin optimality (R_{mean} and R_{max}) and the fraction of excluded pixels (highly underpopulated bins).

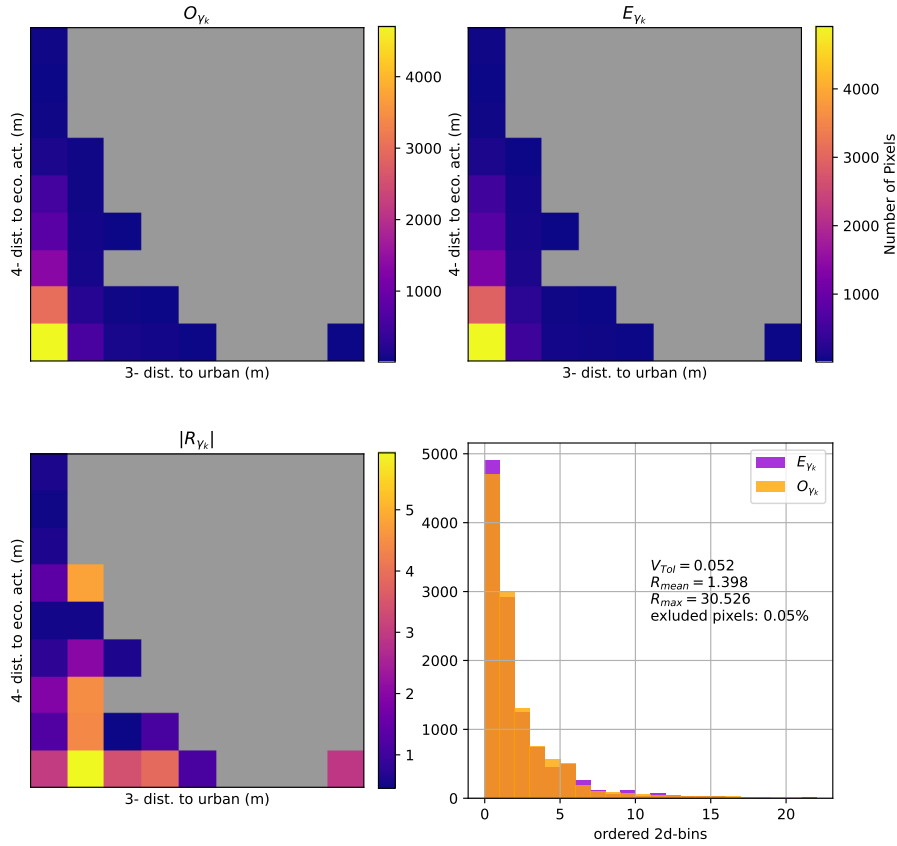


FIGURE 4.3: Test of redundancy — pseudo-binning: representation of $E = \tilde{n}_{\gamma_k, u, v} \tilde{n}_{\gamma_{k'}, u, v} / n_{I_v}$ and $O = \tilde{n}_{\gamma_k, \gamma_{k'}, u, v}$ and residuals R (see Appendix 4.E.2.2) for the transition $u = \text{AGR} \rightarrow v = \text{URB}$ as matrix maps of explanatory variables k and k' ($k = 3$ - distance to existing urban areas, $k' = 4$ - distance to existing economic activity areas). Each cell corresponds to a bidimensional bin and its color to its number of pixels for O , E and to the absolute value of the standardized residual of the bin for R . Underpopulated bins (< 5 pixels) are excluded and appear in gray. The bottom-right graph represents a pseudo-histogram ordered by decreasing values of O . V_{ToI} [Eq. (4.E.8)] is equal to 0.05. The tests of bin optimality (R_{mean} and R_{max}) and the fraction of excluded pixels (highly underpopulated bins) are also given.

TEST OF RELEVANCE, $V_{GoF}^m \simeq 0.1$				
Explanatory variable	V_{GoF}	R_{mean}	R_{max}	excl. pix.
1- elevation	0.03	1.02	5.24	0.0%
2- slope	0.03	1.12	4.81	0.1%
3- dist. to urban	0.27	1.02	16.79	0.0%
4- dist. to eco. act. (?)	0.10	1.02	4.83	0.0%
5- dist. to forest	0.03	1.02	3.70	0.0%

TEST OF REDUNDANCY, $V_{ToI}^M \simeq 0.2$				
Explanatory variable couple	V_{ToI}	R_{mean}	R_{max}	excl. pix.
(3,4)	0.12	1.40	27.01	0.1%

SELECTED EXPLANATORY VARIABLES: 3, 4 (?)

TABLE 4.2: Numerical summary of the maximum relevance / minimum redundancy explanatory variable selection method for the AGR $\rightarrow$ URB transition. The results are very clear cut both in terms of relevance and independence for this transition (AGR to URB). The selected variables are highlighted in bold. The question mark points out a variable (4) that could possibly be excluded as well, due to its marginal relevance score.

obtained in this way are discussed in the remainder of this case study analysis.

With these preliminary adjustments and remarks made, we can now follow the procedure described in section 4.4.4.

TEST OF RELEVANCE

Fig. 4.2 shows the histograms of $E_{\gamma_k} = \tilde{n}_{\gamma_k, u}$ and $O_{\gamma_k} = \tilde{n}_{\gamma_k, u, v}$ (see Appendix 4.D.1) for the transition $u = \text{AGR} \rightarrow v = \text{URB}$, and for all five possible choices of explanatory variable k . The histograms of the distances to existing urban areas and economic activity areas display the most important differences between E_{γ_k} (all pixels in initial state u) and O_{γ_k} (transited pixels), which singles out these two variables as the most relevant. This is confirmed by the Cramer-like test scores V_{GoF} (respectively 0.27 and 0.10), consistently with our significance limit ($V_{GoF} \leq V_{GoF}^m \simeq 0.1$); note that the second of these variables (distances to existing economic activity areas) is only marginally relevant and could also possibly be ignored. The other explanatory variables present significantly lower values (< 0.04). At the end of the relevance test, we keep the explanatory variables 3 (distance to existing urban areas) and 4 (distance to existing economic activity areas). This result is also consistent with expectations for the transition from agricultural to urban areas of interest here.

Some comments on the tests of bin optimality might be useful. The mean

test values of order unity implies that on average, bin sizes are close to being optimal. In fact, if ε is chosen as discussed in Appendix 4.D.2.5 for univariate distributions, this quantity is close to unity. Deviations occur because ε can (and should) be tweaked to give “reasonable” histograms for both univariate and bivariate distributions (i.e., histograms that seem neither over- or sub-sampled). This quantity therefore gives a synthetic global number to characterize such a deviation. As a guideline, if this quantity differs by, say, more than 50% or a factor of 2 from unity, deviations start to be important in the mean, and the user should check that the sampling of histograms remains satisfactory⁷. Deviations from average optimality is characterized by the maximum test, which measures the size of the largest or smallest bin population with respect to the optimal one. A factor of a few indicates a rather modest deviation, overall. A factor of 10 to 15 indicates a deviation that begins to be significant, but this is not necessarily problematic in itself. This is the case, e.g., for the histogram of distance to existing urban areas, but this is simply a consequence of the magnitude of the peak bins in the histogram — thousands, instead of hundreds in the other histograms. Note that this implies that these peak bins are substantially undersampled (with respect to the optimal), but the aspect of the histogram does not imply that this is problematic in terms of histogram resolution.

Note that the histograms shown have the same purpose as the standardized residuals of Eq. (4.E.3), i.e., pointing out which bins contribute most to the relevance (the bins where the O and E histograms differ most). Thus standardized residuals are not shown. Finally, the fraction of excluded pixels is everywhere negligible, and is therefore not an issue.

TEST OF REDUNDANCY

Figure 4.3 shows the values of $E_{\gamma_k, \gamma_{k'}} = \tilde{n}_{\gamma_k, u, v} \tilde{n}_{\gamma_{k'}, u, v} / n_{I_v}$ and $O_{\gamma_k, \gamma_{k'}} = \tilde{n}_{\gamma_k, \gamma_{k'}, u, v}$ and residuals R (see Appendix 4.E.2.2) for the transition $u = \text{AGR} \rightarrow v = \text{URB}$ for the pair of previously selected explanatory variables ($k = 3$ and $k' = 4$). Each cell corresponds to a bidimensional bin and its color measures its number of pixels. A number of underpopulated bins are excluded (< 5 pixels) but they correspond to a very small number of pixels (0.05% of the total). This rejection has therefore no influence on the significance of the results, although it corresponds to a rather large number of bins (these bins contain very little information, so that excluding them does not affect the results). We observe very little difference

⁷Too few or too many bins will lead to gross variations in histograms, especially for O , that should be suspicious-looking to the user, except for pathological explanatory variable distributions.

between O and E (both on the 2D maps and in the pseudo-histogram), implying that these two variables are independent. This is confirmed by the small value of V_{ToI} (0.05), Eq. (4.E.8). Because of this, the map of standardized residuals is not used (see section 4.5.2.4 for generic guidelines on the meaning and use of standardized residuals, when needed).

The previous comments on the optimality tests apply here as well.

In the end, on the basis of these results, the explanatory variables retained for this transition are the distance to the existing urban areas (variable 3) and the distance to the existing economic activity areas (variable 4). It is not surprising that the most relevant explanatory variable for the $AGR \rightarrow URB$ transition is the distance to existing urban areas — as the grow mostly through urban sprawl. Variable 4 seems to be only marginally significant and could be ignored; however its low redundancy score with variable 3 suggests to keep it if the user wishes to err on the safe side. Note that the study area is not mountainous, and this explains why altitude and slope (which play a non negligible role for this transition in other sectors of the Grenoble area) are of no direct interest here.

The numerical results are summarized in Table 4.2.

4.5.2.2. TRANSITION AGRICULTURAL AREA TO FOREST AREA (AGR TO FOR)

The overall amount of change associated to this transition in the calibration maps is substantially smaller than the previous one. Thus, ε must be increased in order to have enough pixels per bin, especially for bivariate distributions. We set it to 0.15 after a few trials. This results from a compromise between the level of residual noise in the univariate distributions histograms, and the number of accepted cells in the bivariate distribution maps. This compromise is less clear-cut than for the previous transition, resulting in a larger fraction of excluded pixels in the bivariate distribution, which however remains always smaller than 10% and does not therefore affect the outcome of the selection process.

The procedure described in section 4.4.4 leads to the following results and comments.

TEST OF RELEVANCE

Fig. 4.4 shows the histograms of $E_{\gamma_k} = \tilde{n}_{\gamma_k,u}$ and $O_{\gamma_k} = \tilde{n}_{\gamma_k,u,v}$ (see Appendix 4.D.1) for the transition $u = AGR \rightarrow v = FOR$, and for all five possible choices of explanatory variable k . Although the results are much less clear cut than in for the previous transition, the relevance score of variable 3 is close to our approximate exclusion threshold, suggesting that it may be disregarded — a point further strengthened by its high level of redundancy with variable 5 (see right below). This would seem logical

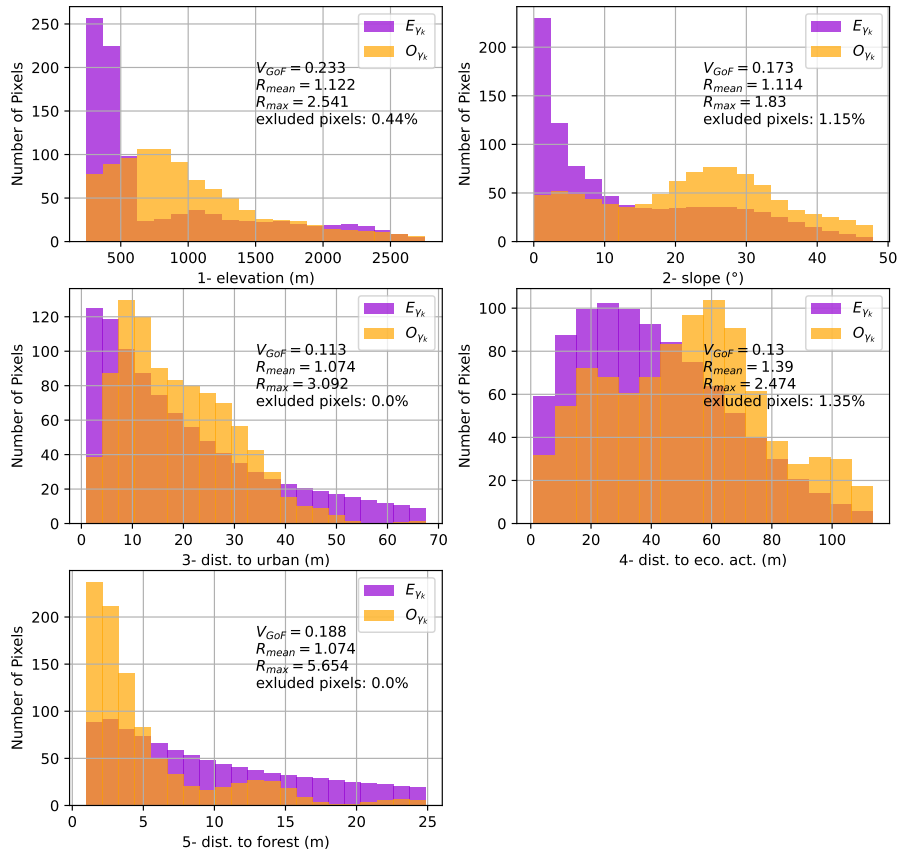


FIGURE 4.4: Test of relevance — pseudo-binning: histograms of $E_{\gamma_k} = \tilde{n}_{\gamma_k,u}$ and $O_{\gamma_k} = \tilde{n}_{\gamma_k,u,v}$ for the transition $u = \text{AGR} \rightarrow v = \text{FOR}$, and for all five possible choices of γ_k . V_{GoF} [Eq. (4.E.4)] is given on each graph, as well as the tests of bin optimality (R_{mean} and R_{max}) and the fraction of excluded pixels (highly underpopulated bins, $n_{\gamma_k} < 5$).

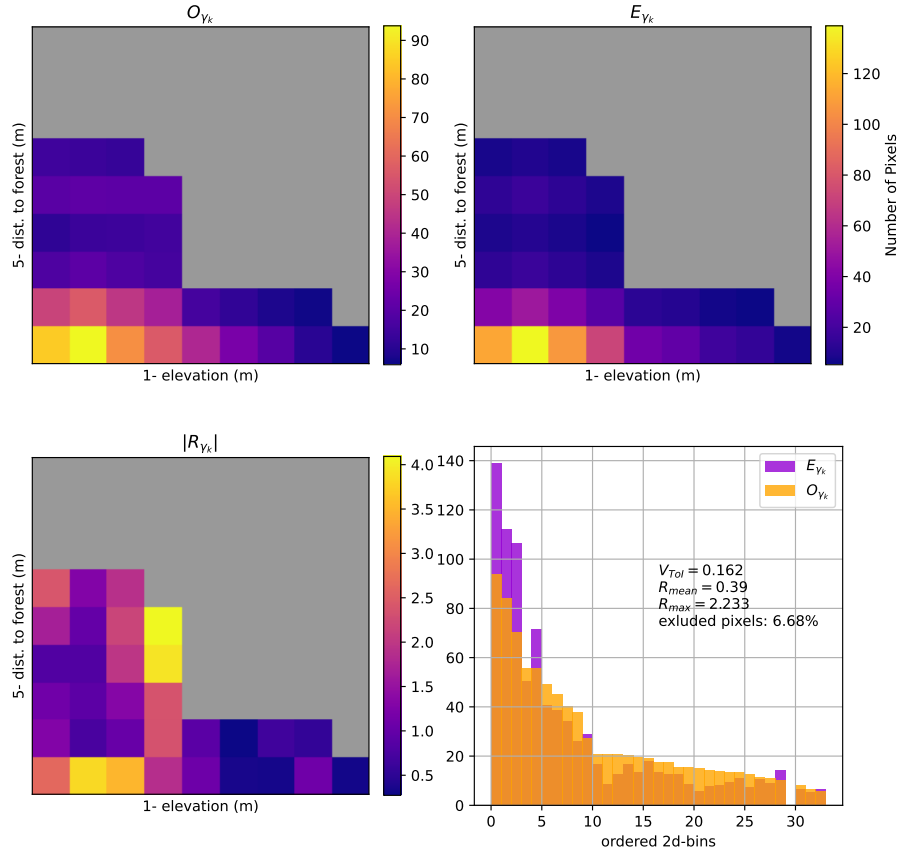


FIGURE 4.5: Test of redundancy — pseudo-binning: representation of E , O and $|R|$ (see Appendix 4.E.2.2) for the transition AGR $\rightarrow$ FOR as matrix maps of explanatory variables γ_k and $\gamma_{k'}$ ($k = 1$ - elevation, $k' = 5$ - distance to forested areas). Each cell corresponds to a bidimensional bin and its color to its number of pixels for O , E and to the absolute value of the standardized residual of the bin for R . Underpopulated bins (< 5 pixels) are excluded and appear in gray. The bottom-right graph represents a pseudo-histogram ordered by decreasing values of O . V_{Tol} [Eq. (4.E.8)] is equal to 0.16. The tests of bin optimality (R_{mean} and R_{max}) and the fraction of excluded pixels (highly underpopulated bins) are also given.

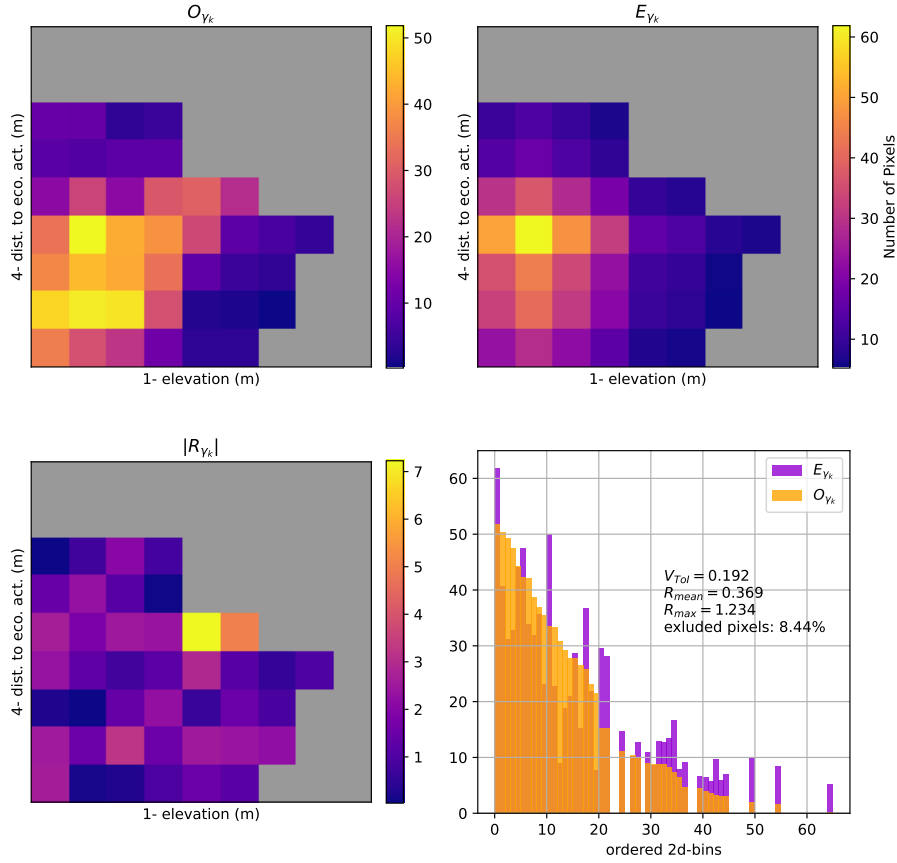


FIGURE 4.6: Test of redundancy — pseudo-binning: representation of E , O and $|R|$ (see Appendix 4.E.2.2) for the transition $AGR \rightarrow FOR$ as matrix maps of explanatory variables γ_k and $\gamma_{k'}$ ($k = 1$ - elevation, $k' = 4$ - distance to economic activity areas). Each cell corresponds to a bidimensional bin and its color to its number of pixels for O , E and to the absolute value of the standardized residual of the bin for R . Underpopulated bins (< 5 pixels) are excluded and appear in gray. The bottom-right graph represents a pseudo-histogram ordered by decreasing values of O . V_{Tot} [Eq. (4.E.8)] is equal to 0.19. The tests of bin optimality (R_{mean} and R_{max}) and the fraction of excluded pixels (highly underpopulated bins) are also given.

TEST OF RELEVANCE, $V_{GoF}^m \simeq 0.1$				
Explanatory variable	V_{GoF}	R_{mean}	R_{max}	excl. pix.
1- elevation	0.23	1.12	2.54	0.44%
2- slope	0.17	1.11	1.83	1.15%
3- dist. to urban	0.11	1.07	3.09	0.0%
4- dist. to eco. act.	0.13	1.39	2.47	1.35%
5- dist. to forest	0.19	1.07	5.65	0.0%

TEST OF REDUNDANCY, $V_{ToI}^M \simeq 0.2$				
Explanatory variable couple	V_{ToI}	R_{mean}	R_{max}	excl. pix.
(1,5)	0.16	0.39	2.23	6.68%
(1,2) (?)	0.19	0.33	1.03	8.23%
(1,4)	0.19	0.37	1.23	8.44%
(1,3)	0.13	0.44	1.59	5.59%
(5,2)	0.15	0.39	1.98	7.36%
(5,4)	0.17	0.51	2.80	5.47%
(5,3)	0.26	0.53	3.50	1.56%
(2,4)	0.16	0.34	1.02	6.03%

SELECTED EXPLANATORY VARIABLES: 1, 5

TABLE 4.3: Numerical summary of the maximum relevance / minimum redundancy explanatory variable selection method for the AGR $\rightarrow$ FOR transition. The question mark indicates that variable 2 is weakly independent of variable 1 and may possibly be included as well, due to its rather high relevance score. Variable 4 is excluded on the basis of its weaker relevance and weak independence from variable 1.

on the basis of the small expected relevance of this variable for the transition under consideration. This is also directly confirmed by the resemblance of the O and E histograms of this variable. The fraction of pixels excluded from the analysis is somewhat larger than for the previous transition (AGR to URB) but still very small ($< 1.4\%$). This is a consequence of the lower number of pixels undergoing in this transition.

The previous comments on the optimality tests apply here as well.

The outcome of this the relevance test is that the explanatory variables 1- elevation, 2- slope, 4- distance to existing economic activity areas and 5- distance to forested areas are provisionally selected, and variable 3 is nevertheless kept to illustrate the outcome of the redundancy score. The presence of variable 4 in this list is somewhat surprising, but this is only the penultimate variable in terms of relevance, and its relevance score is still not so far from the approximate relevance threshold (see the relevance scores gathered in Table 4.3).

TEST OF REDUNDANCY

Fig. 4.5 shows the values of $E_{\gamma_k, \gamma_{k'}}$, $O_{\gamma_k, \gamma_{k'}}$ and $|R_{\gamma_k, \gamma_{k'}}|$ for the pair of most significant variables (1,5), as an illustration of a bivariate distribution analysis for this transition. Distances to existing forested areas (5) and elevation (1) may be expected to be somewhat correlated, in view of the presence of hilly areas in the case study, which indeed concentrate existing woods and forests. This is reflected by the differences between the O and E histograms (recall that identical O and E distributions mean that the two variables are independent). Similarly, elevation (1) and slope (2) are expected to be correlated for this transition. This is reflected by the values of the test scores ($V_{ToI} = 0.16$ for 1 and 5, and 0.19 for 1 and 2, very close to our approximate redundancy threshold). As an other example, Fig. 4.6 focuses on the (1,4) variable couple. The pseudo-histograms are clearly more different than for the (1,5) couple, indicating a higher level of correlation, a point that is also obvious on the differences between the O and E maps and the higher test score $V_{ToI} = 0.19$.

This leads us to exclude variable 4 (distance to economic activities), especially that we pointed out right above that it is the least significant of the provisionally selected variables. There is little doubt on the correctness of this conclusion, but if there were any, it might have been useful to investigate in more detail the origin of the differences between the O and E maps. To this effect, the two cells highlighted in yellow and orange in the standardized residual map would have been the logical starting point, and some GIS treatment would then be needed to pinpoint where the as-

sociated explanatory variable values are located in the calibration maps in physical space (see the discussion of section 4.5.2.4 below for further details).

We also note from Table 4.3 that elevation (1) and slope (2) are rather significantly correlated as well for this transition. This is expected for this transition, considering the nature of forested areas; because of this, it is not necessary to explore in more detail the origin of the bivariate map differences, and the information provided by the standardized residuals is not necessary either. These points suggest that the least significant of the two (slope) may be excluded as well, although this conclusion is less clear cut due to relevance score of this variable, and a prudent user may wish to keep it instead. Finally, although variable 3 (distance to existing urban areas) has high redundancy score (0.26) with variable 5, which confirms that this variable can be safely excluded.

The outcome of this analysis is that the variables selected for this transition are 1- elevation and 5- distance to forest. The numerical results are summarized in table 4.3.

4.5.2.3. PROCEDURE CONCLUSION

In the end, we have collected four of the five pre-selected variables for the two transitions. This large fraction is not a coincidence, the preselection was made in this way to maintain the discussion of this illustrative example within reasonable bounds. The selected variables are however dichotomic: two of the four are relevant for the first transition, while the other two are relevant for the second transition. The end result is consistent with our knowledge of the study area.

For the record, the whole procedure required 1.5 seconds of calculation time.

4.5.2.4. GENERIC COMMENTS ON THE HIERARCHY OF INFORMATION PROVIDED AND ITS PRACTICAL USES

Generally speaking, the procedure provides several levels of information to guide the explanatory variable selection process, in less and less condensed form:

1. Test results V and related test cross-check indicators (R_{mean} , R_{max} , excluded pixel fractions).
2. Histograms (univariate distributions) and pseudo-histograms (bivariate distributions).
3. Bivariate distribution maps

4. Standardized residuals

The simplest and most useful of these are the test results, especially when the results are clear cut, as for the first transition (AGR to URB). These are immediately accessible from the summary tables. The various cross-check indicators are only important when something unusual occurs. As is apparent above, the most common source of deviation from expectation comes from bins that are too populated (i.e., not resolved enough with respect to the optimal bin size). This however, is not necessarily a problem in itself; in particular, in the examples above, the resolution of the histograms is clearly suitable enough to reach correct conclusions.

The histograms and pseudo-histograms provide the most useful disaggregated information, if needed. Note that the information provided goes in different directions for histograms and pseudo-histograms. The more different the *O* and *E* histograms for a univariate distribution, the more relevant the variable and the higher its chances of being selected. Conversely, the more different the *O* and *E* pseudo-histograms for a bivariate distribution, the more likely the rejection of the least relevant variable in the couple. This type of information is especially useful if the user has some direct knowledge of the study area. He or she can then immediately identify where the differences come from, and whether it is reliable or due to, e.g., poor data (parts of the calibration maps may be more reliable than others).

The bidimensional maps are less easy to interpret, but provide powerful information in case of doubt. The dissimilarity of the *O* and *E* maps is more precise and informative than the pseudo-histograms dissimilarity. With some experience with this form of diagnostic, a user can obtain a much firmer decision of inclusion or rejection, in particular by identifying where in physical space the difference comes from (this possibly requires some time-consuming scrutiny of the calibration maps, but the time invested in this task is always useful in terms of understanding the characteristics of the study area).

To this effect, the standardized residuals may provide useful clue to pinpoint more easily where to concentrate efforts, as it helps to identify where the difference between the two *O* and *E* maps is most significant (an information which is not obvious from the maps themselves). This information must be used with care, however. If the most significant difference comes from not highly populated cells in the bivariate maps, the user should first check if this is an effect of noise (undersampling) of these cells. If this is the case, the information should be discarded. Otherwise, the user must identify where this occurs in physical space, which in general should point towards more specific areas than the differences of the maps themselves.

A physical map scrutiny guided in this way should cue the modeler on whether the differences in bivariate maps are artifacts of the data, or a real phenomenon, and guide the acceptance/rejection decision. However, such an involved exercise is only useful when the results from the score tests and histogram comparison are not decisive enough, something we expect to occur only rarely. Such a bivariate map analysis is probably only useful for experienced modelers.

4.6. COMMENTS ON THE RELIABILITY AND RELEVANCE OF THE VARIABLE IDENTIFICATION AND SELECTION PROCESS

An important and generic concern in explanatory variables selection is related to the fact that the list of possibly relevant variables is open ended, so that one can never be sure that all relevant variables have been included in the selection process. If for some reason, relevant explanatory variables are overlooked by the modeler in an actual case study (by ignorance or for practical considerations), the calibration results could be seriously erroneous.

We wish to point out here that, although quite legitimate, this problem may possibly not as severe in the end as one might think at first glance. Because (discrete and continuous) explanatory variables are deterministic functions of position, the maximum number of statistically independent variables one can find in any LUCC problem is two, as maps are bidimensional. In practice, finding two completely independent variables is all but easy: the level of multi-valuedness of these functions almost unavoidably produces a minimum level of variable correlation, this minimum being possibly large; and even for single-valued functions, the possibly complex shape of a study area makes the identification of a couple of independent spatial variables a complex matter.

As a consequence, and for the type of explanatory variables and map patterns commonly found in LUCC modeling (but with a possible caveat here due to our limited experience with actual case studies variability), a small number of variables (say half a dozen or less per LUC state transition) is probably sufficient to obtain reliable transition probability maps. This last statement is difficult to prove, but we believe that this can actually be done or at least shown in a convincing way under generic and mild assumptions about the spatial behavior of explanatory variables. An important point here is that the selected variables need not be actually causal drivers of LUC changes but only to correlate

well with these changes. In particular, even underlying causes may be numerous and unrelated to each other, they may conflate into a smaller number of effective explanatory variables because of the unavoidable correlations between spatial quantities just pointed out. Causes of change that are not spatial in nature (explicitly or implicitly) may be important in LUCC models, but not at the spatial level (e.g., they may drive fractional changes at the level of the whole area), and therefore be best accounted for through scenarios — themselves possibly informed by other types of models — while other factors account for the spatial distribution of such changes.

To the best of our knowledge, these points have never been investigated or even pointed out in LUCC modeling, but they are of direct relevance for the issue of completeness in explanatory variable identification. For this reason, we believe that the initial pre-selection of variables should be as wide and open as possible. In our experience, the type of ranking process proposed here should (or at least may) produce a much more limited list in the end, possibly in a paradoxical manner; e.g., a variable who has a clear causal effect on LUC change may be discarded for one that is a mere proxy of no obvious causal character, but with better correlations to observed LUC changes, if this proxy is more easily spatially quantified. As pointed out in the previous paragraph, such proxies may represent to some extent a number of different underlying causes of change.

An example might be useful here. Consider urban sprawl. The distance to existing urban centers may be a relevant explanatory variable to characterize suburb expansion. This quantity may also effectively encompass to some extent different underlying causes: real estate prices, proximity to nature, etc. The arguments above imply that even if the number of causes of urban sprawl is large, the nature of maps implies that they may be represented by a limited number of variables, if these are chosen with care.

4.7. CONCLUSION

This article is part of a series dedicated to spatially explicit LUCC models. Other papers in this series are dedicated to the calibration and estimation of transition probabilities (Mazy and Longaretti, 2022a) and to an in-depth analysis of allocation errors and biases (Mazy and Longaretti, 2022b), in order to reduce the spread in results obtained from existing modeling environments on a same problem with the same data (e.g., Mas et al. 2014). Here, we focus on the question of semi-automatic selection of explanatory variables. Indeed, the choice of these variables has a direct influence on the quality of the results ob-

tained from pattern-based LUCC modeling. However, there has been very little explicit discussion of this issue in the literature. We tried to bridge this gap by proposing a method of selection of explanatory variables that is apt to identify the most relevant such variables while reducing inter-variable redundancy.

More specifically, we introduce relevance and the redundancy scores based on Cramer's and assimilated statistical tests. Cramer's test is already used in existing software, such as, e.g., Dinamica EGO (Soares-Filho et al., 2002) and Idrisi LCM (Eastman et al., 1995), but no systematic and extensive analysis with the purpose of the present paper in mind has yet been published, to the best of our knowledge. Section 4.4 outlines a procedure to maximize variables' relevance and minimize their redundancy (Cramer-MRMR), with more emphasis on relevance than redundancy (it is preferable to keep redundant relevant variables than to overlook a relevant variable for the sake of reducing redundancy: the former leads to inefficiencies while the latter leads to errors). Special care has been devoted to the verification of the validity of the test assumptions, in particular the assumption of normality of sample mean proportion distributions.

Our proposed tests return a number of more and more sophisticated diagnostics that can be used to guide the explanatory variable selection process. The simplest and most useful are normalized values of test scores that can be compared to two parameters specified by the user, namely, the minimum relevance threshold ($V_{GoF}^m \simeq 0.1$) and the maximum redundancy threshold ($V_{ToI}^M \simeq 0.2$). The typical suggested values are indicative and should not be taken as strict limits, but must be compounded with the user experience and knowledge of the study area. A panel of more sophisticated diagnostics are provided in case of doubt, the more sophisticated requiring the larger expertise. These points have been illustrated through a simple but realistic illustrative LUCC case study (section 4.5).

This method of explanatory variable selection has the advantage of being simple to implement and based on robust tests and analyses. It is implemented in our own LUCC modeling environment, CLUMPY (Comprehensive Land Use [and cover] Modeling in PYthon), which is open source and freely available (see Appendix 5).

APPENDICES TO CHAPTER 4

4.A. SUFFICIENT CONDITION OF RELEVANCE OF AN EXPLANATORY VARIABLE

With the notations of section 4.3, Eqs. (4.3) and (4.5) lead to

$$P(v|\mathbf{z}, u) = P(v|\mathbf{z}^*, u) \Leftrightarrow \rho(z_{k_0}|\mathbf{z}^*, u, v) = \rho(z_{k_0}|\mathbf{z}^*, u). \quad (4.A.1)$$

The first equality expresses that the transition probability is independent of z_{k_0} , the second that the conditional probability distribution of z_{k_0} is independent of v . To alleviate the argument, discrete variables are not included. If present, integrals must be extended by adding the equivalent sums on discrete variables in the expressions below.

From Bayes rule, one has:

$$\begin{aligned} \int d\mathbf{z}^* P(v|\mathbf{z}, u) \rho(\mathbf{z}|u) &= P^*(v|u) \int d\mathbf{z}^* \rho(\mathbf{z}|u, v) \Leftrightarrow \\ P(v|z_{k_0}, u) \rho(z_{k_0}|u) &= P^*(v|u) \rho(z_{k_0}|u, v). \end{aligned} \quad (4.A.2)$$

Let us define the marginal probability distribution $\rho(\mathbf{z}^*|u) = \int dz_{k_0} \rho(\mathbf{z}|u)$. Combining these two sets of equivalences with this definition, one gets:

$$\begin{aligned} \rho(z_{k_0}|\mathbf{z}^*, u, v) &= \rho(z_{k_0}|\mathbf{z}^*, u) \Rightarrow \\ \int d\mathbf{z}^* P(v|\mathbf{z}, u) \rho(\mathbf{z}^*|u) &= \int d\mathbf{z}^* P(v|\mathbf{z}^*, u) \rho(\mathbf{z}^*|u) \Leftrightarrow \\ P(v|z_{k_0}, u) &= P^*(v|u) \frac{\rho(z_{k_0}|u, v)}{\rho(z_{k_0}|u)} = P^*(v|u) \Leftrightarrow \\ \rho(z_{k_0}|u, v) &= \rho(z_{k_0}|u). \end{aligned} \quad (4.A.3)$$

In other words, $\rho(z_{k_0}|u, v) = \rho(z_{k_0}|u)$ is a necessary condition for the equality $\rho(z_{k_0}|\mathbf{z}^*, u, v) = \rho(z_{k_0}|\mathbf{z}^*, u)$ to be true. Conversely (from the negation of the implication), $\rho(z_{k_0}|u, v) \neq \rho(z_{k_0}|u) \Rightarrow \rho(z_{k_0}|\mathbf{z}^*, u, v) \neq \rho(z_{k_0}|\mathbf{z}^*, u)$, i.e., $\rho(z_{k_0}|u, v) \neq \rho(z_{k_0}|u)$ is a sufficient condition for z_{k_0} to be a relevant explanatory variable, from the equivalence of relation (4.A.1).

The reason why only an implication and not an equivalence can be derived is connected to the fact that, if one tries to prove either that $\rho(z_{k_0}|\mathbf{z}^*, u, v) \neq \rho(z_{k_0}|\mathbf{z}^*, u) \Rightarrow \rho(z_{k_0}|u, v) \neq \rho(z_{k_0}|u)$ or $\rho(z_{k_0}|u, v) = \rho(z_{k_0}|u) \Rightarrow \rho(z_{k_0}|\mathbf{z}^*, u, v) = \rho(z_{k_0}|\mathbf{z}^*, u)$, the step from (in)equalities to integrals in the proof does not go through (inequalities of quantities do not necessarily translate into inequalities of integrals); this is explicitly noted by an absence of equivalence in the proof at this step. This also suggests that counter-examples breaking this step may easily be found, taking advantage of the averaging effect of integrals.

4.B. INFERENCE STATISTICS IN A NUTSHELL

In order to propose a method for the selection of explanatory variables in the context of LUCC modeling, we will rely on a number of generic concepts in inference statistics.

4.B.1. GENERALITIES

Inference statistics deals with testing hypotheses such as, e.g., $\rho(z_{k_0}|u, v) = \rho(z_{k_0}|u)$, to a given level of significance (the practical meaning of the significance level will be explained in section 4.B.2). A rather large number of statistical tests have been devised to this effect, and the newcomer may easily feel overwhelmed by their sheer number and variety, and by the rules of decision required to identify the relevant test(s) for a given problem.

A non-specialist introduction to the topic is provided by Hahs-Vaughan and Lomax (2020). This text may seem daunting due to its length, which results from the number of topics covered. However, they are organized in a clear manner, and many chapters and sections are independent of each other. Also, the text devotes quite some room to purely numerical implementation issues with the help of widespread statistics software, and the formal principles underlying the exposition are in fact not so extensive.

Figs. 1.1 and 1.2 of chapter 1 of Hahs-Vaughan and Lomax (2020) (p. 11) allow the reader to grasp in a very quick way the underlying organizational principles of hypothesis testing, subdividing the question into various context-depen-

dent sub-problems, each addressed with its own set of statistical tests and methods. The first level of decision, when one wishes to test an hypothesis such as the one formulated above, is to choose between two different strategies: comparing the mean of the two distributions (Fig. 1.1), or comparing the two distributions themselves (Fig. 1.2). In both instances, it is intuitively clear that estimating similarities or differences will depend on how precisely the distributions themselves are estimated from the data. With a very large number of data points, both the distribution means and the distributions themselves are very precisely determined, so that the variance of the sample mean or the deviations of the observed distributions from the actual ones are very small. Either type of test will therefore be sensitive to small differences between the two means or the two distributions. If the number of data points is small, the variance and deviations will be large, and the tests much less precise. It is therefore intuitively clear that the significance of the test will depend on some measure of the difference between the two quantities to be compared, scaled by some measure of the variance of the data.

Comparing the mean does not require to bin the data, but is less discriminant than comparing the distributions themselves (indeed, different distributions may have the same mean). Confusion is avoided when testing the mean because many if not all these tests require the sample statistics to be normally distributed around the mean: under such a normality assumption, comparing the means and the distributions is about as discriminant. Testing the distributions themselves almost always requires discrete data⁸, transforming a continuous probability distribution into a categorical one.

In spite of a number of contrary statements in the literature, many tests of the mean are sensitive to even small deviations from normality, and a number of alternative, robust estimation methods have been devised in replacement of the standard ones when the normality assumption is problematic (Wilcox, 2020). This issue of violation of normality must be carefully pondered before using a test requiring a normal distribution of data points; Hahs-Vaughan and Lomax (2020) discuss the issue to some extent and provide useful starting points to ponder the role of the normality assumption, before delving into Wilcox (2020) if needed. This point is addressed in detail in the present work for the tests we have selected.

⁸Mazy and Longaretti (2022a) argue that data discretization should be avoided, in order for the calibration process to be more precise. However, selecting relevant explanatory variables is less demanding on this front.

4.B.2. LEVELS OF SIGNIFICANCE

Hypotheses to be tested fall into two categories: the null hypothesis H_0 and the alternative hypothesis H_1 . For example in the present context, the null hypothesis might be $\rho(z_{k_0}|u, v) \neq \rho(z_{k_0}|u)$ (or more strongly, that the two distributions are independent) and the alternative one, $\rho(z_{k_0}|u, v) = \rho(z_{k_0}|u)$ (i.e., that the two distributions are identical, or, more weakly, just correlated). In this example, H_1 is simply the negation of H_0 ($H_1 = \overline{H_0}$), but this is not a generality and more complex sets of hypotheses are quite commonly found.

The objective of a test is always to falsify the null hypothesis to some significance level α ; the significance level is a probability level chosen by the test user, defining the limit of acceptance of the null hypothesis. For example, if $\alpha = 5\%$ (a common albeit arbitrary choice), the test is falsified if the test result is true in less than 5% of cases. In this case, the null hypothesis is rejected.

The ability to quantify probabilities of acceptance or rejection in this way depends on the ability to compute the test score, and on the knowledge of the test score probability distribution. The latter, in turn, depends on assumptions about the sampling distributions being tested; the most important and most common of these is a normality assumption, discussed in section 4.4.1.

Let us illustrate in principle this procedure in the present case. If the probability densities $\rho(z_{k_0}|u, v)$ and $\rho(z_{k_0}|u)$ are discretized into categorical distributions, a χ^2 or Cramer test of independence can be performed (these are presented and discussed in section 4.4). Focusing for the sake of the discussion on the χ^2 test, the test returns a test score value (a χ^2 value). If the various bins of the categorical distribution contain enough data points (see again section 4.4.1), the χ^2 distribution of test scores is satisfied to a sufficient level of precision so that the test score value obeys the χ^2 distribution, which depends on the number of data points as a parameter and on the score value as a variable. For the significance level α and a given number of data points n , the χ^2 probability distribution P defines a critical score value $\chi_{\alpha,n}^2$ (the p -value) such that $P(\chi_{\alpha,n}^2, n) = \alpha$. If the test score result $\chi^2 > \chi_{\alpha,n}^2$, the test falls in the rejection region at the significance level α , and the null hypothesis is rejected in favor of the alternative hypothesis.

More generally, two types of errors can be defined:

- H_0 is true but the test rejects H_0 . This is called type I error, and is quantified by α .
- H_0 is false but the test fails to reject H_0 . This is called type II error, and is quantified by a parameter β similar to α ; $1 - \beta$ is called the test power.

There is a necessary trade-off between type I and type II errors: the smaller α , the larger β . Avoiding errors of type I is usually favored over avoiding errors of type II and the β parameter is usually ignored. The reader is referred to Saporta (2008) chapter 14, and Hahs-Vaughan and Lomax (2020) chapter 6 for more details. In any case, we follow this rule of ignoring type II errors here, so that hypothesis testing is performed by going through the four following steps:

- State the null and alternative hypotheses.
- Select the level of significance (i.e., α).
- Calculate the test score value.
- Make a statistical decision (reject or validate H_0 , at the adopted level of significance).

The choice of the significance level is a highly debated topic. See Saporta (2008), 14.2.7 and Hahs-Vaughan and Lomax (2020), 6.1.1.4 for some discussion of this point. Part of the issue relates to the fact that levels of significance, quantified by p -values, are affected by size effects: the associated score $\chi^2_{\alpha,n}$ depends on the sample size and on the number of degrees of freedom of the problem. Cramer's test is precisely designed to produce a score that is normalized with respect to size effects, and is an example of a generic strategy to this effect (Cohen, 1988).

Finally, it is useful to point out that these tests constitute a very aggregated way of assessing the rejection of the null hypothesis. Once the test is successful in this respect, it is often useful and informative to understand more precisely which categories in the tested probability distribution are responsible for the result, through the examination of so-called standardized residuals (see, e.g., Hahs-Vaughan and Lomax 2020 section 8.2 or Agresti 2002, section 3.3). Standardized residual expressions are provided in this work, for this purpose.

4.C. MINIMUM REDUNDANCY - MAXIMUM RELEVANCE SELECTION METHOD: CRITIQUE OF A POPULAR METHOD

4.C.1. SHORT DESCRIPTION

4.C.1.1. RELEVANCE ASSESSMENT

For continuous data variables, Ding and Peng (2005) propose to assess their relevance through the use of the F-test; in the present context the test must be

applied to the comparison the final LUC state vector and the explanatory variable vector as a basis for a relevance criterion. The F-test is one of the standard statistical tests, and constitutes a common choice for comparing several continuous variables for a classification problem; in the present context, the classification classes cannot be anything but the final LUC states. The F-test requires that the variables tested are normally distributed.

The F-test is a ratio designed to evaluate the significance of the difference of the means of two sampled distributions, taking into account the different variabilities of the various samples and sub-samples of interest.

4.C.1.2. REDUNDANCY ASSESSMENT

Redundancy refers to the more or less extended dependence of the explanatory variables on each other. Indeed, if two explanatory variables are largely dependent, it is advisable to keep only one of the two to avoid redundancy.

Redundancy between variables can be evaluated in different ways. Ding and Peng (2005) propose to use Pearson's correlation coefficient.

4.C.1.3. AGGREGATED INDEX

Ding and Peng (2005) finally make use of an aggregated index which combines their analyses of relevance and redundancy in a single score.

4.C.2. CRITIQUE

4.C.2.1. RELEVANCE ASSESSMENT CRITIQUE

The use of this test for the present purpose poses at least three different and potentially severe problems:

- All possible transitions from a given initial state u to final states v are tested together, so that the test allows for compensation between transitions. For example, a given explanatory variable k_0 may be ill-suited for a given transition. However, if it is very-well suited for other transitions, it may be ranked higher than the best explanatory variable k'_0 for this specific transition (leaving aside for the time being how this best-suited variable may be identified). The ranking process may even discard in the end the best explanatory variable. The point made here is in fact generic in the field of multi-criteria decision analysis, where it is well-known that using averages as indicators tends to lead to discarding the best options. The F ratio, being made up of a ratio of average measures (averaged over the different

transitions), unavoidably encounters this problem. The only way to circumvent this issue is to select relevant explanatory variables on a transition per transition basis, and pool all selected variables in the end. This may select poor or useless variables for some transitions, but will ensure that the modeler will not discard the best explanatory variables for each transition. Avoiding this pitfall is still possible within the framework of the F ratio, by foregoing the summations over groups performed in the denominator and numerator (i.e., keeping only the term corresponding to the transition under consideration). This however will not avoid the other problem that is now described.

- If the differences of the means are significant (i.e., larger than the dispersion, as measured by the F ratio), one may argue that the distributions underlying the mean, i.e., $\rho(z_{k_0}|u, v)$ and $\rho(z_{k_0}|u)$ are sufficiently different for their ratio to be relevant in Bayes rule. However, the converse is not true. Consider, for example, a situation where $\rho(z_{k_0}|u)$ is a normal distribution of zero mean, and $\rho(z_{k_0}|u, v)$ the sum of two gaussians of opposite means. Quite clearly, the ratio of the two will be significantly different from 0 and the corresponding variable k_0 qualifies *a priori* for a *bona fide* explanatory variable. However, by construction, the F ratio (reduced to this single variable, as per the discussion of the previous point) vanishes, and this variable is excluded from the onset from the ranking procedure. This example is somewhat idealized, but shows that there is a non-negligible risk to exclude relevant explanatory variables, if the differences between the means is smaller than the dispersion around the mean. This type of situation is directly connected to the breaking of the normality and homogeneity of variance assumptions that are required for the relevance of the F ratio (Hahs-Vaughan and Lomax 2020, 11.1.4). Indeed, in the example just described, $\rho(z_{k_0}|u, v)$ is clearly not normal. Enforcing normality and requiring that all group samples have the same variance makes it impossible to have relevant explanatory variables without some relevant difference of the means of $\rho(z_{k_0}|u, v)$ and $\rho(z_{k_0}|u)$. It has been argued that non-normality has a small effect on the significance of the F-test (Tiku, 1971), but the present discussion shows that this claim is unwarranted, even for relatively simple and common violations of the normality assumption.
- Ranking the relevance of explanatory variables with the F ratio does not ensure in itself that the variable of highest rank is in fact relevant. To this effect, one must define a null hypothesis (e.g., the variable k_0 distributions

$\rho(z_{k_0}|u, v)$ and $\rho(z_{k_0}|u)$ are identical) and a level of significance of rejection of the null hypothesis to be checked against the test distribution (the F distribution), as for all statistical tests. Without checking if some significance level is achieved, the highest ranking variable may fail to achieve a notable role in Bayes rule. In which case, and even ignoring the problems mentioned before, the problem may arise at different places: the pre-selection of explanatory variables may have been performed in a poor way and this step must be revisited, or some undetected algorithmic error may have crept in, etc.

4.C.2.2. REDUNDANCY ASSESSMENT CRITIQUE

Let us now turn to the use of the correlation matrix for assessing minimum redundancy. The following points limit the usefulness of this strategy:

- Strictly speaking the correlation matrix correctly traces underlying sample correlations when the samples are linearly related to one another. Any nonlinearity will decrease the characterization of this correlation by the correlation matrix. Another (and common to many statistical tools) assumption is that the data points are independent (see Hahs-Vaughan and Lomax 2020, section 10.1 for more details). One can easily construct data that are obviously correlated but for which the standard correlation coefficient is small or even vanishes. This is the case, e.g., when distributing points at random along a circle. Quite clearly, for every point, the cartesian coordinates of the points x and y are function of one another (for each half-circle), and the correlation between x and y is clearly maximum. However, Pearson's correlation matrix gives a vanishing correlation. Such an example is not completely artificial. Consider, for example, the distribution of some quantity that depends only on the distance to some reference point in a map (e.g., a widening circle of mushrooms); a similar conclusion will follow in this case. In such a simple context, the modeler will of course notice the correlation and use the radial distance to the reference point instead of the cartesian coordinates, thereby removing the limitation of the correlation as a measure of redundancy. Still, some similar but less straightforward situation may arise in explanatory space and may not be identified by the modeler, in which case redundancy will be incorrectly assessed by the correlation matrix.
- More generally, Wilcox (2020) chapter 9 points out that the correlation matrix may be problematic not only when points are actually correlated while the correlation coefficient vanishes, as in the previous example, but

also when performing one-sided tests (a topic not discussed in this article) and when evaluating the correlations of all pairs of samples in a multivariate distribution (this is precisely what we aim at doing here). More precisely, Pearson's correlation coefficients are affected by outliers, curvature in the distributions (nonlinearity in a deterministic relation being a particular case), the magnitude of residuals, and restrictions of range. The extent to which these effects do bias (in particular minimize) the actual correlation coefficients between the data is difficult to assess, and must be examined on a case by case basis. Needless to say, this can hardly be implemented in an explanatory variable procedure that is aimed at being generic in most practical applications.

- Correlations are again a single number used to summarize the connection between complete underlying distributions. As for the discussion of comparing means vs comparing the distributions themselves (section 4.B.2), it is clear that comparing the distributions themselves is more precise, although more data intensive.

4.C.2.3. AGGREGATION CRITIQUE

There is little justification for using an aggregated index instead of keeping independently a redundancy and a relevance one. How may the respective weights of these two criteria be chosen? In fact, this question cannot be resolved in a clear-cut, universal manner, and scientific logic here would insist on assessing relevance and redundancy independently, albeit in an iterative way. Also, fixing ahead of the procedure the total number of relevant explanatory variables may be overly restrictive, and this can only be decided on a case-by-case basis, although this is probably an issue of secondary importance.

4.C.3. CONCLUSION

The critique just performed does not imply that the selection procedure devised by Ding and Peng (2005) is useless, only that it is only appropriate in restrictive conditions that are not expected to be commonly satisfied in typical LUCC case studies. This remark is likely to apply to other procedures devised in the machine learning community, because these procedures are often designed with specific applications in mind. This implies in turn that a relevant alternative procedure must be specifically devised for the LUCC modeling context. This is the object of section 4.4.

4.D. CATEGORICAL APPROXIMATION OF A CONTINUOUS PROBABILITY DISTRIBUTION

Several questions are addressed in this Appendix:

- the transformation of a continuous probability distribution [for example, $\rho(z_k|u, v)$ or $\rho(z_k|u)$] into a categorical one through binning [e.g., $P(\gamma_k|u, v)$ or $P(\gamma_k|u)$], and the estimation of the resulting categorical distributions from calibration data; this applies even if discrete variables are included.
- the production of histograms of the obtained categorical distributions from the calibration data for the statistical tests used in the explanatory variables selection process of section 4.4.
- the conditions under which proportions of a random sample of a categorical distribution and related population histograms satisfy the normality assumption required by many if not all statistical tests, in particular, the ones used in this work.

4.D.1. FROM CONTINUOUS TO CATEGORICAL DISTRIBUTIONS

The continuous distributions of interest are $\rho(z_k, z_{k'}|u, v)$, $\rho(z_k|u, v)$ and $\rho(z_k|u)$, i.e., the distributions needed for the selection method of section 4.4 (the joint distribution is needed for the tests of independence described in Appendix 4.E.2).

Only a simple form of binning is required for our purpose. Before proceeding, let us first recall the definitions introduced in section 4.2. A range Γ_k of possible integer values is associated to each explanatory variable z_k . Each integer γ_k in this range represents a bin. For continuous variables, the bin size is δ_k . One of the most important questions to address is the determination of the bin size of such continuous variables, or, equivalently (once the physical range Δ_k of z_k is fixed), the total number of bins $\Gamma_k = \Delta_k/\delta_k$.

Once the bin size is known, two different methods may be considered for producing binned distributions from continuous ones. In the first method, the pixels in a given bin are counted (this applies as well when discrete variables are included). These are denoted $n_{\gamma_k, u, v}$ for $P(z_k|u, v)$, $n_{\gamma_k, u}$ for $P(z_k|u)$ and

$n_{\gamma_k, \gamma_{k'}, u, v}$ for $P(z_k, z_{k'} | u, v)$. The binned distributions follow from

$$P(\gamma_k | u, v) = \frac{n_{\gamma_k, u, v}}{n_{I_v}}, \quad (4.D.1)$$

$$P(\gamma_k, \gamma_{k'} | u, v) = \frac{n_{\gamma_k, \gamma_{k'}, u, v}}{n_{I_v}}, \quad (4.D.2)$$

$$P(\gamma_k | u) = \frac{n_{\gamma_k, u}}{n_I}. \quad (4.D.3)$$

The associated histograms follow directly from $n_{\gamma_k, u, v}$ and $n_{\gamma_k, u}$. Note also that these expressions always apply to discrete variables.

In the χ^2 and Cramer goodness-of-fit tests, one requires that the two populations have identical size. This can be achieved by selecting a random subsample of size n_{I_v} in I , or more simply and accurately for our purpose by rescaling $n_{\gamma_k, u, v}$ by n_I/n_{I_v} as it is actually done in Eq. (4.6). By construction, the probability distribution of this subsample is again $P(\gamma_k | u, v)$ in the mean.

In the second method, the distributions $\rho(z_k | u, v)$ and $\rho(z_k | u)$ are first estimated by the effective Kernel density estimation procedure elaborated in Mazy and Longaretti (2022a). Binning is only relevant for continuous variables, so the following argument applies to these exclusively, except for a particular case addressed right next.

The associated (binned) categorical distributions are then obtained from

$$\tilde{P}(\gamma_k | u, v) = \int_{\gamma_k} dz_k \rho(z_k | u, v), \quad (4.D.4)$$

$$\tilde{P}(\gamma_k, \gamma_{k'} | u, v) = \int_{\gamma_k, \gamma_{k'}} dz_k dz_{k'} \rho(z_k, z_{k'} | u, v), \quad (4.D.5)$$

$$\tilde{P}(\gamma_k | u) = \int_{\gamma_k} dz_k \rho(z_k | u), \quad (4.D.6)$$

where γ_k symbolically refers to the corresponding bin. Note that if one of the two variables in Eq. (4.D.5) is discrete, say k' , the integral on the right-hand side is performed on the continuous variable only:

$$\tilde{P}(\gamma_k, \gamma_{k'} | u, v) = \int_{\gamma_k} dz_k \rho(z_k, \gamma_{k'} | u, v),$$

From now on, we now longer distinguish continuous and discrete variables. Pseudo population histograms are obtained from these alternative estimates of

the categorical distributions by:

$$\tilde{n}_{\gamma_k, u, v} = n_{I_v} \tilde{P}(\gamma_k | u, v), \quad (4.D.7)$$

$$\tilde{n}_{\gamma_k, \gamma_{k'}, u, v} = n_{I_v} \tilde{P}(\gamma_k, \gamma_{k'} | u, v), \quad (4.D.8)$$

$$\tilde{n}_{\gamma_k, u} = n_I \tilde{P}(\gamma_k | u). \quad (4.D.9)$$

By construction, the various $\tilde{n}$ numbers are not integers if continuous variables are involved (hence the “pseudo” qualification). This however does not affect the statistical tests. Note that the total number of pixels in the pseudo histograms associated with $\tilde{P}(\gamma_k | u)$ is n_{I_v} , as required by goodness-of-fit tests⁹.

One may feel that $P(\gamma_k | u, v)$ or $\tilde{P}(\gamma_k | u, v)$ are undefined for parts of the bins. Indeed, there are in general many less pixels undergoing a transition $u \rightarrow v$ in the calibration data than pixels in initial state u ($n_{I_v} \ll n_I$). However, for unobserved transitions (for some γ_k), by construction, $n_{\gamma_k, u, v} = 0$, and the related bins (or pseudo-bins) have zero probability. This point also applies to the joint distribution $P(\gamma_k, \gamma_{k'} | u, v)$. This is accounted for in the calibration procedure of Mazy and Longaretti (2022a).

The normality assumption required by the statistical tests used to select explanatory variables imposes some restrictions on bin histograms. The relevant criteria are established in the remainder of this Appendix. It is not necessary, however, that these criteria be satisfied by all distributions simultaneously, but only by the distribution(s) under consideration for one or the other statistical test.

4.D.2. NORMAL LIMIT OF SAMPLES STATISTICS OF A CATEGORICAL PROBABILITY DISTRIBUTION

4.D.2.1. SAMPLE DISTRIBUTION

The discussion presented here is general, and we use a unique quantity γ to designate either $\gamma_k, \gamma_{k'}$ or $\gamma_k, \gamma_{k'}$, depending on the context.

Consider pixels in a map that are categorically distributed among a number of discrete values of a variable (e.g., LUC states or, more to the point, bins of some continuous explanatory variable). To make the problem general, pixels are numbered in an arbitrary order with an index i ($1 \leq i \leq N$ where N is the total number of pixels), discrete variable values are numbered with an index γ

⁹One may also choose the integer parts of these numbers as an alternative to avoid the question of non-integers bin population numbers. In this case the total number of pixels in pseudo-histograms will be slightly smaller than n_{I_v} , which consistently requires a small change of the binned probabilities for the total probability to be equal to unity.

($1 \leq \gamma \leq \Gamma$ where Γ is the total number of outcomes); π_γ is the proportion of pixels characterized by the discrete variable γ ($\sum_\gamma \pi_\gamma = 1$). One also defines $N_\gamma = \pi_\gamma N$ (N_γ ; by construction, $\sum_\gamma N_\gamma = N$).

When randomly drawing a subsample of n pixels in this population of size N , the proportions p_γ of the subsample will clearly represent more and more closely the exact proportions π_γ as $n \rightarrow N$. The aim of this section is to show that the probability distribution of p_γ is converging to a normal distribution when n increases, and to characterize the domain of validity of the normal approximation. This is required to check if the normality assumption of the χ^2 and Cramer tests used in the course of this work is satisfied.

To this effect, let us focus on a specific, arbitrarily chosen category γ . This simplifies the categorical distribution into simply two categories: a randomly chosen pixel has probability π_γ to fall in category γ and probability $1 - \pi_\gamma$ to fall in the any other category.

The state transition from state u to v in a time step of a LUCC simulation or between two calibration maps should be represented by a random sampling process without replacement: once a pixel has undergone a transition, it cannot be put back into the original set to possibly undergo another transition during the same time step. One can show in a rather straightforward way by direct counting that the number of elements in the γ category under a random sampling without replacement of size $n \leq N$ is an hypergeometric distribution (Cochran 1977, chapter 3). Denoting by n_γ the numbers of elements of the sample falling in the γ category, one has:

$$P(n_\gamma | N, N_\gamma, n) = \frac{\binom{N_\gamma}{n_\gamma} \binom{N-N_\gamma}{n-n_\gamma}}{\binom{N}{n}}, \quad (4.D.10)$$

where the binomial coefficient is given by $\binom{p}{q} = p!/(q!(p-q)!)$ (p and q integers, $q \leq p$).

4.D.2.2. NORMAL APPROXIMATION

The proof that this is approximated by a normal distribution proceeds in two stages: one first proves that in the relevant limit, the hypergeometric distribution is approximated by a binomial distribution; then one invokes the de Moivre-Laplace formula to find the normal limit of the binomial distribution. The second step is standard (see, e.g., Papoulis and Pillai 2002, section 4.5), but the first is less well-known and quite a few more or less complex proofs can be found in the literature. However, this first step is more intuitive than the proof suggests: if $n \ll N$, the sampling without replacement will differ little from a

sampling with replacement, which has a binomial distribution.

We provide here one of the most straightforward proofs of the binomial approximation of the hypergeometric distribution¹⁰. This proof relies on a reorganization of the factorial terms:

$$\begin{aligned}
 \frac{\binom{N_\gamma}{n_\gamma} \binom{N-N_\gamma}{n-n_\gamma}}{\binom{N}{n}} &= \frac{N_\gamma!}{n_\gamma! \cdot (N_\gamma - n_\gamma)!} \cdot \frac{(N - N_\gamma)!}{(n - n_\gamma)! \cdot (N - N_\gamma - (n - n_\gamma))!} \cdot \frac{n! \cdot (N - n)!}{N!} \\
 &= \binom{n}{n_\gamma} \cdot \frac{N_\gamma! / (N_\gamma - n_\gamma)!}{N! / (N - n_\gamma)!} \cdot \frac{(N - N_\gamma)! \cdot (N - n)!}{(N - n_\gamma)! \cdot (N - N_\gamma - (n - n_\gamma))!} \\
 &= \binom{n}{n_\gamma} \cdot \frac{N_\gamma! / (N_\gamma - n_\gamma)!}{N! / (N - n_\gamma)!} \cdot \frac{(N - N_\gamma)! / (N - N_\gamma - (n - n_\gamma))!}{(N - n + (n - n_\gamma))! / (N - n)!} \\
 &= \binom{n}{n_\gamma} \cdot \prod_{k=1}^{n_\gamma} \frac{(N_\gamma - n_\gamma + k)}{(N - n_\gamma + k)} \cdot \prod_{m=1}^{n-n_\gamma} \frac{(N - N_\gamma - (n - n_\gamma) + m)}{(N - n + m)}
 \end{aligned} \tag{4.D.11}$$

Taking the large N limit for fixed $N_\gamma/N = \pi_\gamma$, n and n_γ we get

$$\lim_{N \rightarrow \infty} \frac{(N_\gamma - n_\gamma + k)}{(N - n_\gamma + k)} = \lim_{N \rightarrow \infty} \frac{N_\gamma}{N} = \pi_\gamma, \tag{4.D.12}$$

and

$$\lim_{N \rightarrow \infty} \frac{(N - N_\gamma - (n - n_\gamma) + m)}{(N - n + m)} = \lim_{N \rightarrow \infty} \frac{N - N_\gamma}{N} = 1 - \pi_\gamma. \tag{4.D.13}$$

Inserting these two results in the products of the final expression of the hypergeometric probability distribution right above leads to

$$\lim_{N \rightarrow \infty} P(n_\gamma | N, N_\gamma, n) = \binom{n}{n_\gamma} \pi_\gamma^{n_\gamma} (1 - \pi_\gamma)^{n-n_\gamma}, \tag{4.D.14}$$

which is the looked-for binomial distribution. In practice, the large N limit is very well satisfied as soon as $n \lesssim N/10$. It is still reasonably well satisfied for $n \lesssim N/5$.

The final step relies on the use of the de Moivre-Laplace theorem, which

¹⁰This is reproduced here and checked from <https://math.stackexchange.com/questions/330553/>.

states that

$$\begin{aligned} \lim_{n \rightarrow \infty} \binom{n}{n_\gamma} \pi_\gamma^{n_\gamma} (1 - \pi_\gamma)^{n - n_\gamma} \\ = \frac{1}{[2\pi n \pi_\gamma (1 - \pi_\gamma)]^{1/2}} \exp - (n_\gamma - n\pi_\gamma)^2 / (2n\pi_\gamma(1 - \pi_\gamma)), \end{aligned} \quad (4.D.15)$$

The mean of this distribution is $n\pi_\gamma$ and its standard deviation $\sigma = [n\pi_\gamma(1 - \pi_\gamma)]^{1/2}$. This large n approximation applies as soon as $n\pi_\gamma \gtrsim 5$ and $n(1 - \pi_\gamma) \gtrsim 5$. The first condition is demanding when $\pi_\gamma \rightarrow 0$ and the second when $\pi_\gamma \rightarrow 1$. The approximation will also be well satisfied as soon as the number of bins is large enough, e.g., $\Gamma_k \gtrsim 10$, but even for Γ_k as low as 5, the approximation by a normal distribution is surprisingly accurate.

4.D.2.3. VARIANCE ESTIMATION

A useful result concerning the estimation of the variance of n_γ on the sample can be derived in the binomial distribution approximation. As pointed out earlier this follows by approximating a sampling without replacement by a sampling with replacement. The result derived here can also be found in various texts, such as Cochran (1977), chapters 2 and 3, but the exposition given here is more straightforward.

Let us define the characteristic variable y_i for pixel i such that $y_i = 1$ if i is in the sample, and $y_i = 0$ otherwise. The probability law of y_i is a Bernoulli one:

$$p_i = p(y_i) = \pi_\gamma^{y_i} \times (1 - \pi_\gamma)^{1 - y_i}. \quad (4.D.16)$$

By construction, $\sum_i^n y_i = n_\gamma$ and $E(n_\gamma) = \bar{n}_\gamma = \sum_i E(y_i) = n\pi_\gamma$ (the fact that the random variables y_i are independent in a sample with replacement has been used).

One can also evaluate in a similar way the sample variance of n_γ , $V(n_\gamma) = E([n_\gamma - \bar{n}_\gamma]^2)$. One has (using again the independence of the random variables

$y_i, \sum_i E(y_i - \pi_\gamma) = 0$ and $y_i^2 = y_i$):

$$V(n_\gamma) = E \left(\left[\sum_{i=1}^n (y_i - \pi_\gamma) \right]^2 \right) = E \left(\sum_{i=1}^n (y_i - \pi_\gamma)^2 \right) \quad (4.D.17)$$

$$\begin{aligned} &= \sum_{i=1}^n E((y_i - \pi_\gamma)^2) = \sum_{i=1}^n V(y_i) \\ &= \sum_i E(y_i^2) - 2 \sum_i \pi_\gamma E(y_i) + n\pi_\gamma^2 = n\pi_\gamma - n\pi_\gamma^2 \\ &= n\pi_\gamma(1 - \pi_\gamma), \end{aligned} \quad (4.D.18)$$

which is again the variance of the binomial distribution (as can be checked directly from this distribution from a similar calculation).

When the probability distribution is not known, both the probability and the variance of y must be evaluated from the distribution:

$$\hat{\pi}_\gamma = \frac{1}{n} \sum_{i=1}^n y_i, \quad (4.D.19)$$

$$\hat{V}(y) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\pi}_\gamma)^2. \quad (4.D.20)$$

Note that

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{\pi}_\gamma)^2 = \frac{1}{n} \sum_{i=1}^n y_i^2 - 2 \frac{\hat{\pi}_\gamma}{n} \sum_{i=1}^n y_i + \hat{\pi}_\gamma^2 = \hat{\pi}_\gamma(1 - \hat{\pi}_\gamma). \quad (4.D.21)$$

The quantity $[n\hat{\pi}_\gamma(1 - \hat{\pi}_\gamma)]^{1/2}$ is called the standard error (of n_γ) instead of the standard deviation because it is estimated on the sample.

On the other hand, $y_i - \pi_\gamma = (y_i - \hat{\pi}_\gamma) + (\hat{\pi}_\gamma - \pi_\gamma)$, so that:

$$E(\hat{V}(y)) = \frac{1}{n} V(n_\gamma) - V(\hat{\pi}_\gamma - \pi_\gamma) = \frac{n-1}{n^2} V(n_\gamma) = \frac{n-1}{n} V(y). \quad (4.D.22)$$

These last two results combined with Eq. (4.D.18) show that $n\hat{V}(y_i)/(n-1)$ is a bias-free estimator¹¹ of the variance $V(y) = V(n_\gamma)/n = \pi_\gamma(1 - \pi_\gamma)$ [see also Cochran 1977, Eq (3.8)]. In the limit $n \gg 1$ the factor $n/(n-1)$ can be ignored.

¹¹Note that this does not imply that the standard error itself is a bias-free estimator of the associated standard deviation.

4.D.2.4. DISCUSSION

Non-parametric LUCC modeling can only be implemented if calibration data are precise enough. This implies that calibration maps typically contain millions or tens of millions of pixels. In such a context, the restriction to a given initial state u still commonly involve hundreds of thousands to millions of pixels. Further narrowing to a given transition $u \rightarrow v$ per time step involves thousands or tens of thousands pixels. The conditions $n\pi_\gamma \gtrsim 5$ and $n(1 - \pi_\gamma) \gtrsim 5$ are therefore never a concern in practical problems except in extremely data-poor study areas, for which, anyway, a non-parametric pattern-based LUCC model (the ones of interest here) would be too demanding.

Conversely, at first sight, the condition $n \lesssim N/5$ or $N/10$ might seem less favorable. On the one hand, one may easily sample the categorical distributions discussed in the previous section to satisfy this requirement. On the other hand tests are often performed directly on the complete (unsampled) data set, clearly violating the $n \lesssim N/5$ requirement. Does this imply that the condition of reduction of the hypergeometric distribution to a binomial one fails? The answer is negative, and this can be understood by widening somewhat the scope of what calibration data mean. We focus on the $P(\gamma_k|u, v)$ probability distribution for definiteness in the following argument, but one can easily adapt it to $P(\gamma_k|u)$ or $P(v|u, \gamma_k)$.

Calibration maps can be seen as one out of many historical paths that could have been followed in reality. In this view, the initial state distribution $P(\gamma_k|u, v)$ is just one instantiation of a possible LUCC overall distribution in the study area. Under such a view, and because the number of pixels in state v at t_1 from pixels in state u at t_0 satisfies the $n \lesssim N/5$ requirement, the normal approximation should be valid even if the calibration data is not sampled.

Note that in this argument, the meaning of n and N has been changed: N no longer refers to the pixels in state u at time t_0 and v at time t_1 , but to the (much) larger number n_I of pixels in state u at time t_0 . And, in this counterfactual statement, n is now the actually observed number n_{I_v} of pixels in state v at t_1 . By widening the scope in this way, one necessarily makes the observed probability distribution itself a probabilistic quantity known to some precision (in first approximation, and precisely because of the normal limit just advocated, the square root of the histogram population for each bin).

This argument is largely valid for the distributions $P(\gamma_k|u, v)$ and $P(v|u, \gamma_k)$ because $1 \ll n_{I_v} \ll n_I$. This is less clearly valid for $P(\gamma_k|u)$ for some types of LUC, e.g., for farmed pixels, due to the very large areas devoted to agriculture and cattle raising worldwide. Indeed, in some regions, nearly all available land is devoted to farming. The condition $n_I \gg 1$ will be largely satisfied, but the

condition $n_I \lesssim N_T/5$ (N_T being the total number of pixels in the study area) will usually not be met. This implies that in such a context, the normal approximation will not be very precise, although even for $n_I \sim N_T$, it is not violated by more than a factor of order unity.

The conclusion of this discussion is that the normal approximation of sample means needed to perform χ^2 and Cramer's tests is always valid or reasonably valid, for the type of calibration data that is needed for non-parametric pattern-based LUCC models. In any case, one may always make the argument strictly valid by reverting to the original meaning of n and N in the normality proof above and sampling the distributions accordingly for the tests instead of using the total number of pixels available to make the test.

4.D.2.5. BINNING CRITERIA:

In the course of the preceding discussion, the point concerning the binning width needed to transform continuous probability distributions in categorical ones has been only indirectly addressed, and we now turn to a discussion of this point. We first focus on the standard binning procedure explicited in Eqs. (4.D.1), (4.D.2) and (4.D.3). The pseudo-binning method will be considered in the next subsection.

A rule of thumb to choose a bin width δ_k in the construction of pseudo binned distributions is that the expected fluctuation in the bin proportion does not exceed some limit ε (e.g., $\varepsilon \simeq 10\%$ or some limit of this order), for the bins characterizing the bulk of the distribution¹². From the normal approximation above, this translates, at the n_σ standard deviation level, into

$$\begin{aligned} n_\sigma [n\pi_\gamma(1 - \pi_\gamma)]^{1/2} &\lesssim \varepsilon n\pi_\gamma \quad \text{or} \\ n_\gamma &\gtrsim n_\sigma^2 \frac{1 - \pi_\gamma}{\varepsilon^2}. \end{aligned} \quad (4.D.23)$$

where $n_\gamma = n\pi_\gamma$. This constraint implicitly specifies the minimal bin width for which this minimum n_γ obtains (the wider the bin, the larger n_γ). In practice, and unless otherwise specified, one may choose $n_\sigma = 1$.

In the end, the following rules may be used to define the binning of continuous probability distributions:

¹²Tail bins will always be more noisy, but have little weight in the end and can be removed from the distribution. This point is discussed below.

$$\Gamma_k \gtrsim 10 \quad (4.D.24)$$

$$n_\gamma \gtrsim 5 \quad (4.D.25)$$

$$n_\gamma \gtrsim (1 - \pi_\gamma)/\varepsilon^2 \quad (4.D.26)$$

where n_γ and π_γ symbolically stand for the relevant quantities in Eqs. (4.D.1), (4.D.2) and (4.D.3). The first two conditions are lower bounds for the distribution to be correctly approximated by a normal distribution and the last one is a lower bound for the statistical fluctuations of bin populations to be small enough. This last condition is the one actually constraining bin widths as the first two are expected to be always satisfied in practice (if this is not the case, non parametric LUCC models cannot be used in the first place).

It is useful to recast this last constraint in a different form. First, as this relation is an implicit relation for n_γ , let us rewrite it in explicit form:

$$n_\gamma \gtrsim \frac{n}{1 + n\varepsilon^2} \simeq \frac{1}{\varepsilon^2}. \quad (4.D.27)$$

The last expression applies as soon as $n \gtrsim 1/\varepsilon^2$, a condition which in practice is always satisfied. This has the following meaning. Once $n \gg 1/\varepsilon^2$, it is sufficient to set $n_\gamma \gtrsim 1/\varepsilon^2$ for Eq. (4.D.26) to be satisfied. In the process, all $\pi_\gamma \ll 1$ so that π_γ can be neglected in the right-hand side member of Eq. (4.D.26), and, equivalently, the denominator of the right-hand side member simplifies to $n\varepsilon^2$.

It is useful to assume an equality instead of an inequality in this equation, in order to define a practical threshold:

$$n_\gamma = \frac{n}{1 + n\varepsilon^2} \simeq \frac{1}{\varepsilon^2}. \quad (4.D.28)$$

The rationale for this choice is that keeping the lowest possible number of pixels per bin increases the precision of the discrete representation of the underlying continuous probability distribution.

If one now takes into account Eq. (4.D.25), one is motivated to choose $n_\gamma \geq n_m$ with

$$n_m = \left\lceil \max \left(\frac{n}{1 + n\varepsilon^2}; 5 \right) \right\rceil. \quad (4.D.29)$$

where $\lfloor x \rfloor$ is the integer part of x . One sees that, by construction, the choice $n_\gamma \geq n_m$ satisfies Eqs. (4.D.25) and (4.D.28) [or equivalently Eq. (4.D.26)], as needed.

To proceed further, one must relate the minimum possible number of pixels per bin n_m to the bin size δ_γ , in order to be able to integrate all three conditions above in a single final expression. This relation depends on the continuous distributions $\rho(z)$ or $\rho(z, z')$ involved in Eqs. (4.D.4), (4.D.5) or (4.D.6) (the conditional dependence on u and/or v is ignored to alleviate notations). The two possibilities are examined in turn.

Univariate distributions. Let us focus on $\rho(z)$ first. Defining z_γ as the center of bin γ , one has

$$n_\gamma = n \int_{z_\gamma - \delta_\gamma/2}^{z_\gamma + \delta_\gamma/2} dz \rho(z). \quad (4.D.30)$$

Imposing $n_\gamma = n_m$ produces variable bin widths if $\rho(z)$ is not constant. This choice would definitely produce the most precise type of binning, but, on the other hand, leads to an implicit relation between n_m and δ_γ . On the other hand, we do not need a precise bin width for the qualitative purpose of explanatory variables selection, so that one may replace the exact requirement above by some sort of average in order to define a unique bin width δ . Various approximations are possible here. The most natural choices are the following:

MEAN

ρ is replaced by $\rho_m = 1/\Delta = \int_{z_{min}}^{z_{max}} dz \rho(z) / \int_{z_{min}}^{z_{max}} dz$ where $\Delta = z_{max} - z_{min}$ and stands for the (maximum) range of z values¹³ with $\rho(z) \neq 0$.

MEDIAN

ρ is replaced by $\rho_M/2$, i.e., half the maximum value of ρ .

STANDARD DEVIATION

ρ is replaced by $\rho_\sigma = 1/2\sigma$, the standard deviation of ρ .

Let us designate by $\bar{\rho}$ the actual choice of an average ρ . Eq.(4.D.30) leads to

$$\delta = \frac{n_m}{n\bar{\rho}}. \quad (4.D.31)$$

¹³ $\rho(z)$ may vanish for some limited range(s) between z_{min} and z_{max} but this does not affect the definition.

The associated number of bins is given by $\Gamma = \lfloor \Delta/\delta \rfloor$. If $\bar{\rho} = 1/\Delta$ is chosen, this simply reduces to $\delta = \delta_m = n_m \Delta/n$ and $\Gamma = \Gamma_m = n/n_m$. Otherwise, for the median choice, $\delta = \delta_M = 2n_m/(n\rho_M)$ and $\Gamma = \Gamma_M = n\rho_M \Delta/(2n_m)$, while for the standard deviation choice, $\delta = \delta_\sigma = 2n_m \sigma/n$ and $\Gamma = \Gamma_\sigma = n\Delta/(2n_m \sigma)$.

In the process, some bins may now be found that no longer satisfy the constraint $n_\gamma \geq n_m$. Indeed, imposing a constant $\delta = \Delta/\Gamma$ will lead to a variable number of pixels per bin, instead of the optimum minimum n_m determined above. In practice, we remove all bins with $n_\gamma < n_m$ for consistency in the method. This reduces the actual number of pixels n in the distribution to some new number n_b . The number n_{I_v} must therefore accordingly be rescaled in Eqs. (4.D.1) and (4.D.2). Similarly n_I [Eq. (4.D.3)] must be rescaled to n_b so that the total number of pixels in the O and E distributions of Eq. (4.E.1) become identical.

This bin removal shall affect very little the explanatory variables' selection process. Indeed, most of the distribution will be retained after this removal, and this is sufficient to characterize the distribution shape, which is the only characteristic that matters in the tests we perform. In practice, the fraction of pixels removed in the process is an output of our algorithm. This provides the user with some possibly useful information on the process (if the fraction of pixels removed is too large, something has gone wrong somewhere).

If the standard deviation choice is made, σ can readily be obtained from the calibration pixels. For the median choice, one needs an estimate of the continuous distribution involved. Such an estimate can be obtained from the calibration data, e.g., with the help of the calibration method of Mazy and Longaretti (2022a).

The choice ρ_m has two interesting features. First, the resulting number of bins becomes independent of the distribution detailed characteristics. Second, this clearly leads to the smallest possible average ρ and consequently to the largest possible bin size. The number of pixels removed by the condition $n_\gamma < n_m$ will therefore be minimized. For these reasons, this choice is made by default in our algorithm. However, the bin populations n_γ will be suboptimal, as discusses earlier in relation to Eq. (4.D.28). In order to keep some control on this feature, two ratios measuring the non-optimality of the choice of bin size are provided by our algorithm: $R_{mean} = \sum_\gamma n_\gamma / \Gamma n_m$ and $R_{max} = \max(n_\gamma) / n_m$.

Bivariate distributions. The issue is somewhat more complex for $\rho(z, z')$. Let us first define as before a number of useful auxiliary quantities: the bin center (z_γ, z'_γ) , the bin extent in the two dimensions, δ and δ' , the variable total extents Δ and Δ' in each dimension, the number of bins Γ and Γ' in each dimension,

and finally the total number of bins Γ_T . Furthermore, and taking again advantage of the fact that only a crude estimate of these bin sizes are required, we simplify the probability density by the approximation of independent variables¹⁴, i.e., $\rho(z, z') \simeq \rho(z)\rho(z')$. With these definitions and assumption, Eq. (4.D.30) becomes

$$n_\gamma \simeq n \int_{z_\gamma - \delta/2}^{z_\gamma + \delta/2} dz \rho(z) \int_{z'_\gamma - \delta'/2}^{z'_\gamma + \delta'/2} dz' \rho(z'). \quad (4.D.32)$$

Let us further define Δ, Δ' and $\bar{\rho}, \bar{\rho}'$ for each of these univariate distributions. Eq. (4.D.32) simplifies into

$$\delta \delta' \simeq \frac{n_m}{n \bar{\rho} \bar{\rho}'}. \quad (4.D.33)$$

This motivates us to finally choose

$$\delta \simeq \left(\frac{n_m}{n} \right)^{1/2} \frac{1}{\bar{\rho}}, \quad (4.D.34)$$

$$\delta' \simeq \left(\frac{n_m}{n} \right)^{1/2} \frac{1}{\bar{\rho}'}. \quad (4.D.35)$$

For consistency, one makes the same choice of average ρ (median, mean or variance) for both marginal distributions. As for univariate distributions, our default choice is $\rho = \rho_m$, for the same reasons. This leads to $\delta = (n_m/n)^{1/2} \Delta$, $\delta' = (n_m/n)^{1/2} \Delta'$; $\Gamma = \Gamma' = \lfloor (n/n_m)^{1/2} \rfloor$ and a total number of bins $\Gamma_T = \lfloor n/n_m \rfloor$. Otherwise, for the median choice, $\delta = 2(n_m/n)^{1/2} / \rho_M$, $\delta' = 2(n_m/n)^{1/2} / \rho'_M$; $\Gamma = \lfloor (n/n_m)^{1/2} \rho_M \Delta / 2 \rfloor$, $\Gamma' = \lfloor (n/n_m)^{1/2} \rho'_M \Delta' / 2 \rfloor$ and $\Gamma_T = \lfloor n \rho_M \rho'_M \Delta \Delta' / (4 n_m) \rfloor$. Finally, for the standard deviation choice, $\delta = 2(n_m/n)^{1/2} \sigma$, $\delta' = 2(n_m/n)^{1/2} \sigma'$, leading to $\Gamma = \lfloor (n/n_m)^{1/2} \Delta / (2\sigma) \rfloor$, $\Gamma' = \lfloor (n/n_m)^{1/2} \Delta' / (2\sigma') \rfloor$ and $\Gamma_T = \lfloor n \Delta \Delta' / (4 n_m \sigma \sigma') \rfloor$.

The caveat pointed out for univariate distributions follows: some bins may be underpopulated. We again remove bins (in two dimensions) with $n_\gamma < n_m$. The three outputs mentioned in the univariate binning discussion are also provided in this case by our algorithm (fraction of removed pixels f_r , and the two measures R_{mean} and R_{max} of non-optimality of the resulting bin size), which now read $R_{mean} = \sum_\gamma n_\gamma / \Gamma \Gamma' n_m$ and $R_{max} = \max(n_\gamma) / n_m$.

¹⁴This assumption is *not* made in the test, only to obtain some approximate numbers for the bin size.

4.D.2.6. PSEUDO-BINNING CRITERIA

When applying an upstream density estimation method as in Eqs. (4.D.7), (4.D.8) and (4.D.9), the preceding discussion applies mostly unchanged except for the bin removal criterion.

Indeed, this removal becomes mostly unnecessary in this context. The Kernel density estimation procedure smoothes the number of pixels in each bin and thus reduces the statistical noise of the binning process and the possible non-normality of the number of pixels per bin.

Note that in any case the least populated bins should be ignored because they may strongly bias the tests to be performed. Furthermore, in the evaluation of χ^2 , Eq.(4.E.1), a division by the number of pixels in each bin is performed. To avoid dividing by zero or very small numbers, we exclude the bins with less than 5 pixels, which anyway have a negligible effect on the overall aspect of the distribution, and should therefore not affect the choice of explanatory variables. This again leads to a reduced number of pixels n_b in the effective distribution that comes out of this bin removal process. This rescaling is however much milder than for the standard binning process. In practice, n_{I_v} and n_I in Eqs. (4.D.7), (4.D.8) and (4.D.9) must again be changed to n_b , in order for Eq. (4.E.1) to be meaningful.

These points apply to both univariate and bivariate distributions.

4.D.3. BINNING FOR THE TEST OF RELEVANCE

Our relevance test (section 4.4.2) involves two probability distributions: $P(\gamma_k|u, v)$, where $n_\gamma = n_{\gamma_k, u, v}$ and $n = n_{I_v}$, and $P(\gamma_k|u)$, where $n_\gamma = n_{\gamma_k, u}$ and $n = n_I$ [see Eqs. (4.D.1) and (4.D.3)]. The first probability distributions involves the smaller number of pixels of the two, so that it is chosen to define the bin size. The above discussion leads to

$$n_m = \left\lceil \max \left(\frac{n_{I_v}}{1 + n_{I_v} \varepsilon^2}; 5 \right) \right\rceil, \quad (4.D.36)$$

$$\Gamma_k = \lfloor \max(10, n_{I_v}/n_m) \rfloor. \quad (4.D.37)$$

Once the resulting binning is applied on the set I_v , the set I is binned in the same way. Underpopulated bins ($n_{\gamma_k, u, v} < n_m$ or $n_{\gamma_k, u, v} < 5$, depending on whether direct or pseudo-binning is applied) are removed as explained above. The resulting reduced numbers n_{I_v} and n_I are the ones actually used in Eq. (4.6). The algorithm also returns the fraction f_r of pixels removed and the two non-optimality ratios $R_{mean} = \sum_{\gamma_k} n_{\gamma_k, u, v} / \Gamma_k n_m$ and $R_{max} = \max(n_{\gamma_k, u, v}) / n_m$.

4.D.4. BINNING FOR THE TEST OF REDUNDANCY

Our redundancy test (section 4.4.3) involves two probability distributions: the joint $P(\gamma_k, \gamma_{k'}|u, v)$ one, where $n_\gamma = n_{\gamma_k, \gamma_{k'}, u, v}$ and $n = n_{I_v}$, and the product one, $P(\gamma_k|u, v) \times P(\gamma_{k'}|u, v)$, where $n_\gamma = n_{\gamma_k, u}$ or $n_\gamma = n_{\gamma_{k'}, u}$ and $n = n_I$ (for each distribution in the product) [see Eqs. (4.D.1) and (4.D.2)].

Following the discussion of bivariate binning above, the bivariate distribution is binned as the product of each univariate function involved in the test. This discussion also implied that the number of bins of these univariate probabilities is given by

$$\Gamma_k = \Gamma_{k'} = \lfloor \max(10^{1/2}, (n_{I_v}/n_m)^{1/2}) \rfloor, \quad (4.D.38)$$

where n_m is given by Eq. (4.D.36).

As before, bins of the bivariate distribution satisfying $n_{\gamma_k, \gamma_{k'}, u, v} < n_m$ or $n_{\gamma_k, \gamma_{k'}, u, v} < 5$ (depending on the type of binning used) are removed. The resulting reduced number n_{I_v} is the one actually used in Eq. (4.9). The algorithm finally returns the fraction f_r of pixels removed and the two non-optimality ratios $R_{mean} = \sum_{\gamma_k, \gamma_{k'}} n_{\gamma_k, \gamma_{k'}, u, v} / \Gamma_k \Gamma_{k'} n_m$ and $R_{max} = \max(n_{\gamma_k, \gamma_{k'}, u, v}) / n_m$.

4.E. χ^2 AND CRAMER'S TESTS

This Appendix provides the core definitions, properties and requirements for these two types of test. For more information, see Hahs-Vaughan and Lomax (2020) chapter 8.

4.E.1. GOODNESS-OF-FIT TESTS

4.E.1.1. χ^2 GOODNESS-OF-FIT TEST

The null hypothesis of this test is that the distributions underlying the samples O and E are identical. The alternative hypothesis is that they are not. For the problem of interest here, the distribution of O is $P(\gamma_k|u, v)$ and the distribution of E is $P(\gamma_k|u)$. With the notations of Appendix 4.D.1, one has $O_{\gamma_k} = n_{\gamma_k, u, v}$ and $E_{\gamma_k} = n_{\gamma_k, u}$ for exact binning, or $O_{\gamma_k} = \tilde{n}_{\gamma_k, u, v}$ and $E_{\gamma_k} = \tilde{n}_{\gamma_k, u}$ for pseudo-binning.

With these definitions, the test score χ^2 is obtained from:

$$\chi^2 = \sum_{\gamma_k=1}^{\Gamma_k} \frac{(O_{\gamma_k} - E_{\gamma_k})^2}{E_{\gamma_k}}, \quad (4.E.1)$$

The larger the test score χ^2 , the more different the underlying distributions of O and E . The test score produces a measure of the difference between the bin estimates O_{γ_k} and an assumption E_{γ_k} . If the sample O is drawn from the distribution of E (the null hypothesis), the probability distribution of the test score is the χ^2 probability distribution. The null hypothesis is therefore rejected if the score exceeds the distribution value at some predefined significance level α (the p -value; see section 4.B.2). The validity of the test assumes that the samples O_{γ_k} are normally distributed around their mean value (which should be E_{γ_k} if the two distributions are identical). This assumption is satisfied as argued in Appendix 4.D.2. On top of the assumptions discussed in section 4.4.1, the test requires samples of equal sizes for O and E (i.e., $\sum_{\gamma_k} O_{\gamma_k} = \sum_{\gamma_k} E_{\gamma_k}$).

Finally, as mentioned in section 4.B.2, in environmental sciences, the level of confidence is often chosen to be 5%, but there is substantial discussion on the rationale of this choice. A more informative analysis of the data consists in examining the standard residuals of each category. This adjusted standardized residual is defined by

$$R_{\gamma_k} = \frac{O_{\gamma_k} - E_{\gamma_k}}{[E_{\gamma_k}(1 - \pi_{\gamma})]^{1/2}}, \quad (4.E.2)$$

for the γ_k category (bin). This definition follows from the fact that one compares the difference between the observed and expected population ($O_{\gamma_k} - E_{\gamma_k}$) with the expected population standard deviation ($[n\pi_{\gamma}(1 - \pi_{\gamma})]^{1/2} = [E_{\gamma_k}(1 - \pi_{\gamma})]^{1/2}$) in the normal approximation Eq. (4.D.15). The use of adjusted standardized residuals is advocated in particular by Agresti (2002).

Taking into account that $1 - \pi_{\gamma} \simeq 1$ when the number of bins is large enough (so that the probability of each bin $\pi_{\gamma} \ll 1$), i.e., in practice $\Gamma_k \gtrsim 10$ (see end of section 4.D.2.4), one may use instead the standardized residual

$$R_{\gamma_k} = \frac{O_{\gamma_k} - E_{\gamma_k}}{E_{\gamma_k}^{1/2}}, \quad (4.E.3)$$

which is the quantity directly involved in the definition of χ^2 .

If one chooses again a 5% significance level, but now for the standardized residuals themselves and not the test-score, the normality assumption implies

that categories with $R_{\gamma_k} \gtrsim 2$ are the ones significantly responsible for the refutation of the null hypothesis (Hahs-Vaughan and Lomax 2020, p. 308) due to the approximately normal distribution of O_{γ_k} .

4.E.1.2. SIZE EFFECTS: CRAMER-LIKE GOODNESS-OF-FIT TEST

Strictly speaking Cramer's test derives from the χ^2 test of independence, with the objective to remove the dependence of the test on the sample size and the number of degrees of freedom. However, a similar logic can be applied to produce a goodness-of-fit Cramer's like coefficient V_{GoF} (Hahs-Vaughan and Lomax 2020, 8.2.3.1):

$$V_{GoF} = \left(\frac{\chi^2}{N(\Gamma_k - 1)} \right)^{1/2}, \quad (4.E.4)$$

where N is the sample size (for the present application, $N = n_I$, reduced as indicated in section 4.D.3). The resulting V_{GoF} score, unlike the Cramer score for tests of independence, is not bounded by 1 but is closely related to Cohen's w and rarely exceeds 0.9 (Cohen 1988, p. 218). Instead of comparing the score to a p -value, one may instead note that a score of 0.1 is small, 0.3 is average and 0.5 is large (Cohen, 1988), i.e., 0.5 indicates a very significant rejection of the null hypothesis, as a rule of thumb. A conservative choice would be 0.2 or 0.3; choosing 0.1 implies to keep explanatory variables that differ very little from E , and would possibly lean too much on the safe side.

4.E.2. TESTS OF INDEPENDENCE (OR ASSOCIATION)

4.E.2.1. χ^2 TEST OF INDEPENDENCE

To apply this test, one starts from a two-dimensional, observed contingency table O . For the comparison of $P(\gamma_k|u, v)$ and $P(\gamma_{k'}|u, v)$ of explanatory variables k and k' , as explained in section 4.4.3, the contingency table O is a $\Gamma_k \times \Gamma_{k'}$ matrix where $O_{\gamma_k \gamma_{k'}}$ is the number of pixels undergoing a transition $u \rightarrow v$ and whose values of z_k and $z_{k'}$ belong to the γ_k and $\gamma_{k'}$ bins, respectively. Traditionally, the contingency table elements are indexed by i, j , and we will follow this convention in the remainder of this discussion, to alleviate notations.

One further defines partial (marginal) sums $N_{i.} = \sum_j O_{ij}$ and $N_{.j} = \sum_i O_{ij}$. Finally, the overall total number of data points in the contingency table (i.e., the original sample size) is $N = \sum_i N_{i.} = \sum_j N_{.j}$. In the notations of Appendix 4.D.1, $N_{i.} = n_{\gamma_k, u, v}$, $N_{.j} = n_{\gamma_{k'}, u, v}$ and $O_{ij} = n_{\gamma_k, \gamma_{k'}, u, v}$ for exact binning, and $N_{i.} = \tilde{n}_{\gamma_k, u, v}$, $N_{.j} = \tilde{n}_{\gamma_{k'}, u, v}$ and $O_{ij} = \tilde{n}_{\gamma_k, \gamma_{k'}, u, v}$ for pseudo-binning

($i = \gamma_k$ and $j = \gamma_{k'}$). In both cases the total number of pixels is normalized to $N = n_{I_v}$ (reduced as indicated in section 4.D.4).

If the distributions $P(\gamma_k|u, v)$ and $P(\gamma_{k'}|u, v)$ underlying the contingency table are independent, we have shown (section 4.4.3) that $O_{ij} = E_{ij} = N_{i.}N_{.j}/N$; E_{ij} therefore defines in quantitative way the null hypothesis: what is tested is how closely O_{ij} compares to E_{ij} , i.e., how closely $P(\gamma_k, \gamma_{k'}|u, v)$ compares to $P(\gamma_k|u, v) \times P(\gamma_{k'}|u, v)$. The null hypothesis is that the probability distributions $P(\gamma_k|u, v)$ and $P(\gamma_{k'}|u, v)$ are independent. The alternative hypothesis are that they are correlated.

With these definitions and remarks, the test score reads

$$\chi^2 = \sum_{i=1}^{\Gamma_k} \sum_{j=1}^{\Gamma_{k'}} \frac{(O_{ij} - N_{i.}N_{.j}/N)^2}{N_{i.}N_{.j}/N}. \quad (4.E.5)$$

As before (section 4.E.2.1) this quantity follows a χ^2 distribution in the limit of large numbers. If the two samples trace the same underlying probability distribution, this number is “small”. The more different the distributions, the larger the score. The comment following Eq. (4.E.1) applies here as well.

As for goodness-of-fits tests, in environmental sciences, the level of confidence is often chosen to be 5%, and a more informative analysis of the data consists in examining standard residuals of each category. These are defined again by, with the same meaning as for goodness-of-fit tests:

$$R_{ij} = \frac{O_{ij} - E_{ij}}{[E_{ij}(1 - \pi_{i.})(1 - \pi_{.j})]^{1/2}}, \quad (4.E.6)$$

for the adjusted standard residual, where $\pi_{i.} = N_{i.}/N$ and $\pi_{.j} = N_{.j}/N$. The denominator involves the standard error as it is evaluated on the sample itself: $E_{ij}(1 - \pi_{i.})(1 - \pi_{.j})$ is an unbiased estimator of the standard deviation $N\pi_{\gamma_k}(1 - \pi_{\gamma_k})\pi_{\gamma_{k'}}(1 - \pi_{\gamma_{k'}})$ in the limit $N \rightarrow \infty$ and $N_{i.}, N_{.j} \gg 1$, for uncorrelated $N_{i.}$ and $N_{.j}$ (see Appendix 4.D.2.3).

Eq. (4.E.6) can be approximated by

$$R_{ij} = \frac{O_{ij} - E_{ij}}{E_{ij}^{1/2}}, \quad (4.E.7)$$

with the same justification as in section 4.E.1.1. Choosing again a 5% significance level for the standardized residuals significance, categories with $R_{ij} \gtrsim 2$ are the ones most responsible for the refutation of the null hypothesis.

4.E.2.2. CRAMER'S V TEST OF INDEPENDENCE

The χ^2 test of independence is dependent on sample sizes and on the number of degrees of freedom. Cramer's V_{ToI} test score allows us to remove this dependence, and is defined by

$$V_{ToI} = \left(\frac{\chi^2}{N(\Gamma_{min} - 1)} \right)^{1/2}, \quad (4.E.8)$$

where $N = n_{I_v}$ (reduced as indicated in section 4.D.4) and Γ_{min} is the minimum number of lines and columns in the contingency table O_{ij} . For the problem at hand, $\Gamma_{min} = \min(\Gamma_k, \Gamma_{k'})$. A subscript ToI has been added to distinguish it from the Cramer-like test score for goodness-of-fit.

V_{ToI} is bounded between 0 and 1. The remarks, limits and meanings of the values of V_{GoF} apply here in the same way.

CHAPTER 5 | CLUMPY, A COMPREHENSIVE LAND USE MODEL IN PYTHON

ALL the methods introduced in the chapters 2, 3 and 4 have been implemented within a Python package called CLUMPY for Comprehensive Land Use [and cover] Model in PYthon. It can be set up for a case study in two different ways: the first one allows an advanced user to model a problem by using our methods through a Python script and the second one is a graphical user interface (GUI) which enables the user to configure a simple case study. I used this package to model all case studies presented in this thesis.

This relatively short chapter¹ has the ambition to provide more details on CLUMPY. The software motivations and relevance are first discussed (section 5.1). Then, a description of the main software features is provided (section 5.2). An illustration of a basic use is then presented through a sample code snippet (section 5.3). Finally, the impact CLUMPY could have in the community of users and model designers is explained (section 5.5) before concluding this chapter (section 5.6).

5.1. MOTIVATION AND SIGNIFICANCE

There is a great diversity of LUCC models / software among which CLU-Mondo (van Vliet and Verburg, 2018), Dinamica EGO (Rodrigues and Soares-Filho, 2018) or Idrisi LCM (Eastman and Toledano, 2018b) have been extensively analyzed in the course of this thesis work. I have experienced them as a user through idealized case studies discussed in the previous chapters. These software are affected by shortcomings of varying importance; these are summa-

¹It is likely that this chapter will be used as a basis for the elaboration of an article specifically dedicated to CLUMPY.

rized here in two major points: i) calibration-estimation methods can fail infer LUCC probability distributions (section 2.7); ii) allocation methods have statistical biases, some of them significant (section 3.5.3). Also, their implementation is often inefficient in terms of execution time, especially when the number of pixels is larger than one million (section 2.7).

It is also necessary to address the ease of use of this software from a user point of view, while keeping in mind the common user knowledge and experience. The vast majority of LUCC model users come from the environmental sciences and geography (section 1.2.1). It is fair to assume that they have experience with such software only through a graphical user interface (GUI) and this is precisely what the aforementioned existing software offers. LCM and CLUMondo have a relatively rigid graphical interface where the user fills in the parameters of his/her case study in a fixed framework which corresponds to a standard LUCC case study. I define a standard case study as including a calibration step based on two LUC maps at different dates and a set of EVs, the estimation of a change probability map and the allocation of a simulated map (within a single region). There are several advantages to using such an interface: it is relatively simple to develop, is easy for users to manage, and is able to deal with a large panel of LUCC case studies. On the other hand, it is not possible to implement a more exotic case that goes beyond what designers had in mind². In contrast, Dinamica EGO provides an extremely versatile interface that allows users to build a LUCC case study by graphically manipulating blocks in a dedicated model builder interface. This very powerful approach gives the user a great deal of freedom, and a very large number of spatial and mathematical operations are also implemented. This is the most advanced software I have experienced, although it is more difficult to master for a beginner, who may be intimidated by the large number of parameters available.

Taking into account this double observation (methods and interface), I developed CLUMPY which integrates all the methods we developed in chapters 2, 3 and 4. Thus, it provides a mathematically correct and numerically efficient LUCC modeling environment. Users can model a case study with CLUMPY in two ways: i) directly by calling the methods of the package as commonly done in Python or any other interpreted computer language; ii) by configuring it through the graphical user interface (GUI). The first method allows users to benefit from the versatility of a Python script. This method is therefore to be preferred for an advanced use of the model and allows to reach the same complexity of settings as Dinamica EGO (without the dedicated GUI which is very

²It is however sometimes possible to force the software to act in a unintended way with a few tricks. In particular, I had to do this to set up the controlled settings of sections 2.7 and 3.5.3

long to develop). On the other hand, CLUMPY's GUI allows users to build a standard case study which should be adequate in a large number of contexts, but not exhaustive in terms of functionalities.

5.2. SOFTWARE DESCRIPTION

CLUMPY is written in Python 3 and its metadata are detailed in table 5.1. The dependencies of the package are the following:

- ekde is a personal package³ implementing the efficient approximation of the kernel density estimation method introduced in section 2.5.1 and whose architecture is described in appendix B. It takes advantage of a Cython routine for numerical efficiency.
- Matplotlib (Hunter, 2007), for graph drawing.
- Numpy (Harris et al., 2020) and Scipy (Virtanen et al., 2020), for very efficient matrix routines.
- Pandas (Reback et al., 2022), for some data operations.
- Rasterio (Gillies et al., 2013–), for reading map files in `.tif` format, a very common file extension for GIS.
- Scikit-learn (Pedregosa et al., 2011) and Scikit-image (van der Walt et al., 2014), for machine learning and image analysis methods (useful for LUC maps).

Nr.	Code metadata description	Please fill in this column
C1	Current code version	0.1.0
C2	Permanent link to code/repository used for this code version	https://github.com/fmazy/clumpy
C4	Legal Code License	GNU General Public License v3.0.
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python, Cython
C8	If available Link to developer documentation/manual	https://clumpy.readthedocs.io/en/latest/
C9	Support email for questions	fremi@protonmail.com

TABLE 5.1: CLUMPY's Code metadata

³<https://github.com/fmazy/ekde>

5.2.1. SOFTWARE ARCHITECTURE

The architecture of CLUMPY is made up of a set of classes (object-oriented programming) which interact between them and whose general description is not a trivial thing. We distinguish mainly two types of classes: i) process classes constituted by the statistical methods implemented in the program; ii) structure classes to describe a case study. Process classes can be called independently from structure classes and it is thus possible to make a very advanced use of the package. However, the construction of a LUCC case study can be very complex due to the multiplicity of parameters when dealing with several regions and several transitions from different LUCs simultaneously. This is why structure classes facilitate the use of process classes.

I present below in more detail these two categories in order to introduce the architecture of the package. This presentation is not meant to be exhaustive⁴ but makes it possible to grasp the general architecture of CLUMPY represented in Fig. 5.1. The documentation of the package provides a more detailed description of all its constituent elements.

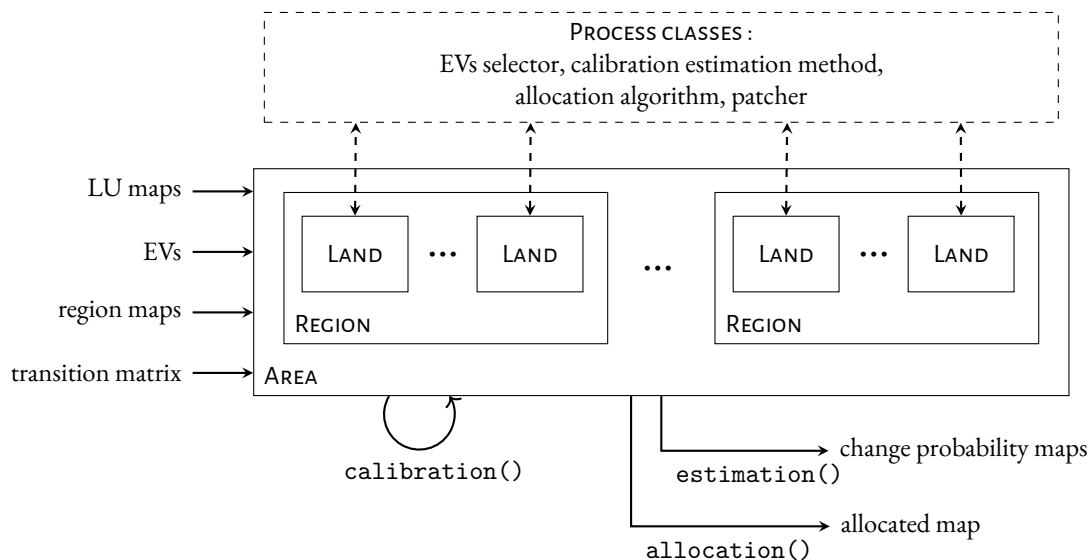


FIGURE 5.1: General architecture of CLUMPY. For the sake of clarity, the specific input arguments to each of the generic functions `calibration()`, `estimation()` and `allocation()` are not detailed. It is possible to distinguish the regions where the calibration takes place from the regions used for estimation and allocation. The double dotted arrows indicate that the Land comes to build objects of the process classes according to its needs and the parameters provided by the user. Each Land builds its own process objects in a compartmentalized way.

⁴I omit for example the layers classes which act as input/output of the LUC maps, EVs and transition probability maps.

5.2.1.1. PROCESS CLASSES

The process classes integrate in a very operational way the statistical methods used by the LUCC models. These classes have a number of parameters and provide functions that require various input arguments. The raw use of these classes requires to be a skilled user but allows a great flexibility of use. This way of doing enables the user to consider independently statistical methods and this is particularly interesting when studying a particular method as I did in the different case studies (section 2.7 and 3.5.3).

5.2.1.2. STRUCTURE CLASSES

The structure classes allow to build a LUCC case study. The general architecture of the use of CLUMPY is linked to these classes (fig. 5.1). Generally, one defines a certain number of regions (section 1.3.4.2) which will be calibrated in an independent way (class *Region*). Within these regions, all pixels of a given initial LUC state u are considered altogether (class *Land*). Thus, there are several objects *Land* in an object *Region*. Finally, the class *Area* contains all the regions. The minimal structure consists of a single *Land*, included in a single *Region*, itself included in an *Area*.

Each *Land* object has a number of parameters which corresponds to the process classes (double dotted arrows in fig. 5.1). This is how the link between the process classes and the structure classes is made. In this way, the user can, for example, choose the calibration-estimation method among several proposed and make a different choice depending on the *Land*, *i.e.* all pixels of a particular LUC within a particular region.

5.2.2. SOFTWARE FUNCTIONALITIES

As indicated in the section 5.2.1.2, the structure classes make it possible to set up a relatively complex case study. The *Area* class allows generic functions to be called. These functions browse the structure and call the process classes according to the *Land* parameters (fig. 5.1). The function `calibrate()` calibrates the case according to two calibration LUC maps, a set of EVs and the calibration region map. Once calibration is done, it is possible to call the `estimate()` and `allocate()` generic functions. These two functions have the same set of input argument: a starting LUC map (which can differ from the second calibration LUC map), the same set of EVs as for calibration (possibly updated), the region map (possibly modified by the current scenario) and the transition matrix (given by the current scenario). The function `estimate()` generates the change probability maps. The function `allocate()` generates a simulated

map.

Moreover, for each process, I have implemented different methods (as a reminder, the choice of the method used is made at the level of the parameters of the Land object). For example, for calibration-estimation, the Bayes-eKDE described in section 2.5 and the standard MLP methods are implemented. The two unbiased allocation methods (uSAM and uPAM) are also implemented (section 3.4) and the patcher introduced in section 3.1 is provided. The only EV selection method implemented is the one introduced in section 4.4.4. The architecture of the package makes it very easy to implement new methods.

The parameters of the user-defined scenario (section 1.3.4.3) are specified at the level of the input arguments of the generic functions `estimate()` and `allocate()`. This can be done in two ways which can be combined. The first is to play on the transition matrix requested for each region. One can thus indicate the percentage of pixels that one wishes to see transiting from one LUC to another. The other way is to modify the region map, which can be different from the one used during the calibration phase.

It is possible to develop a Python script that will take advantage of the generic functions and configuration possibilities. For example, it is very easy to set up an iterative allocation on several successive time steps to perform a complete simulation.

Finally, the configuration of a standard case study is made easy with the accompanying GUI. This one is built completely independently of the structure of CLUMPY. It generates a `json` file which can be interpreted by CLUMPY in a second step. It is also possible to proceed without the interface and to write directly the `json` file in a relatively easy way.

It should be noted that CLUMPY is not in any case a Geographic Information System (GIS) software even if it can display in a very crude way the various LUC maps involved in the process. It is advisable to rely on a dedicated software, such as QGIS or ArcGIS, in order to benefit from a powerful GIS for any pre-processing or post-processing operations.

5.3. SAMPLE CODE SNIPPETS

As an illustration of the use of CLUMPY, I reproduce below an example of a standard case study which I describe briefly.

The structure of the case is described in the file `.json` reproduced on code listing 5.1 below. This code listing is precisely what the CLUMPY's GIU builds and returns. The tabulation of the file makes it possible to appreciate the im-

brications between area, region and land. Here are some comments to facilitate the reading and interpretation of this code.

Two regions are defined here:

- `unprotected_areas` (line 4),
- `protected_areas` (l. 24).

These are represented by the values 1 and 2 on the region map. In the first region, two objects `Land` are defined. They respectively concern land uses with values 4 (line 8) and 5 (line 15). Next, for LUC state 4, the methods used are detailed. For the LUC state 5, only the estimated calibration method is indicated (by indicating the value of one of the parameters of the method) and the other methods are left to their default choice. Finally, the region `protected_areas` has only one studied land and its definition is made in the same way (line 28).

Once this file is built, it is opened and exploited through code listing 5.2. The opening of the file generates the objects of the structure classes (line 2). Then, in a very explicit way, the calibration procedure is called. This one will take into account the indicated methods and the possible final states indicated for each LUC type. Finally, in an equally understandable way, the functions `estimate()` (line 14) and `allocate()` (line 24) are called. CLUMPY's GUI also provides easy ways to call these generic functions.

It should be noted that this example illustrates an extremely simple use of CLUMPY. Many implemented functionalities (not presented here) return intermediate results in order to adjust certain parameters like those involved in the EV selection procedure for example.

CODE LISTING 5.1: A .json file which describes a CLUMPY case.

```

1 {
2   "regions": [
3     {
4       "name": "unprotected_areas",
5       "value": 1,
6       "lands": [
7         {
8           "initial_LUC": 4,
9           "select_EVs_method": "Cramer_MRMR",
10          "calib_est_method": "Bayes_eKDE",
11          "alloc_method": "uPAM",
12          "patcher": "cellular_automata",
13        },
14        {
15          "initial_LUC": 5,
16          "calib_est_method": {
17            "method": "Bayes_eKDE",
18            "params": {"q": 101}
19          }, # all other parameters are set to default
20        },
21      ],
22    },
23    {
24      "name": "protected_areas",
25      "value": 2,
26      "lands": [
27        {
28          "initial_LUC": 4,
29          "calib_est_method": "MLP", # all other parameters are set to
                                     default
30        },
31      ],
32    },
33  ],
34 }

```

CODE LISTING 5.2: A simple CLUMPY Python script which based on a .json file case.

```

1  # open json
2  area = clumpy.open_case("case.json")
3
4  # calibrate
5  area.calibrate(initial_luc_map="luc2015.tif",
6                  final_luc_map="luc2021.tif",
7                  evs=["dem.tif",
8                      "slope.tif",
9                      4, # distance to 2 LUC state
10                     5],
11                  region="region_calib.tif")
12
13 # estimate
14 area.estimate(luc_map="luc2021.tif",
15               evs="distances_updated", # the EVs set is the same
16                                       # but distances are updated
17               region="region_alloc.tif",
18               transition_matrix={
19                   "unprotected_areas": "tm_region_1.csv",
20                   "protected_areas": "tm_region_2.csv"},
21               output="change_probability_maps.tif")
22
23 # allocate
24 area.allocate(luc_map="luc2021.tif",
25               evs='distances_updated', # the EVs set is the same
26                                       # but distances are updated
27               region="region_alloc.tif",
28               transition_matrix={
29                   "unprotected_areas": "tm_region_1.csv",
30                   "protected_areas": "tm_region_2.csv"},
31               output="simulated_luc2027.tif")

```

5.4. REAL CASE STUDY

In addition to being tested on a series of fully controlled case studies (sections 2.7 and 3.6), CLUMPY has been tested on a real case study in the framework of a collaboration with another PhD student, Nicolas Elleaume, in the course of his own PhD thesis⁵. The aim was to model LUCCs in a mountain territory in the context of the adaptation of social-ecological systems to global

⁵*Adaptation of the socio-ecosystems of the Maurienne valley (Savoie) to global climate change through ecosystem services*, Nicolas Elleaume, LECA, CIFOR - CIRAD, Université Grenoble Alpes, supervised by Sandra Lavorel (LECA) and Bruno Locatelli (CIFOR - CIRAD)

changes according to various scenarios (a historical study has already been published: Enora et al. 2021). The study area is the Maurienne valley, in the French Alps (Savoie region). This valley bottom falls from 1840m of elevation at the start to 290m at its end, and stretches over 125km. It harbors a panel of activities (residential, agriculture, pasture, protected natural area, ski resort). Calibration was performed from 2 LUC maps produced for the years 2006 and 2016 with a spatial resolution of 15m, for a total of 9 million pixels. Finally, the study area was divided into several regions (pastoral land association, protected natural area, ski resort). We were particularly interested in agricultural and pastoral abandonment (transition from useful agricultural area to forest or from grassland to forest) and urban expansion. Four contrasting scenarios were defined for each region through transition matrices and designed to reflect various possible futures of the valley (business as usual, sanctuary of natural areas, tourism, agricultural development).

This is a moderately large case study that allowed me to test CLUMPY and to optimize, in particular for the efficient management of large LUC maps. I was able to apply on this case all the functionalities of the model for multiple and simultaneous transitions (selection of EVs, calibration, probability estimations according to a scenario, allocation on several time steps).

5.5. INTENDED USE AND EXPECTED FUTURE DEVELOPMENT

CLUMPY has not been designed to compete with existing LUCC software but foremost to provide a Python package where all our methods are implemented and easily used. Although CLUMPY can be used as is for a panel of case studies (and we do use it in this way), it is primarily intended to be a demonstrator of the methods we have developed, and a point of comparison for other software, and furthermore a point of comparison of quantified accuracy. We hope this will inspire the developers of historical software to reconsider some of their methods and their modeling strategy. In particular, it seems that the methods introduced in this thesis can be implemented relatively easily in a software like Dinamica EGO, thus helping it to get rid of its various sources of error and bias.

LUCC software are not easily analyzed, due to the present lack of explicit enough description of their modeling choices algorithmic implementation. For example, LCM is a proprietary software with a paying license and CLUMondo

and Dinamica EGO, although being both academic software, are not open-source. Thus, with Pierre-Yves Longaretti, we had to carry out a real reverse engineering campaign to analyze certain behaviors of these software. CLUMPY on the contrary is an open-source package, whose methods have been described in detail in dedicated articles (Mazy and Longaretti, 2022a,b; Longaretti and Mazy, 2022) which form the core of this thesis, with technical and operational documentation. I hope that this approach will encourage contributions from other developers and lead to the improvement of CLUMPY. However, these contributions will need to meet the same standard of accuracy and formal correctness, in order to be included in CLUMPY main branch.

CLUMPY also allows users to set up case studies through specific scripts, and their execution is relatively fast from a computational point of view (section 2.7.3). Thus, it is possible and easy to set up large series of experiments where one can vary the various input parameters and thus carry out, for example, sensitivity analyses on the parameters. To carry out this type of study with LCM or CLUMondo is very laborious, even unthinkable. Dinamica EGO performs quite well in this aspect despite some slowness in distance map calculation.

Moreover, the vast majority of LUCC model users have a very operational approach to software use. For them, changing software can be intimidating and requires a learning phase. However, the qualitative and quantitative improvements brought by the statistical methods introduced in this thesis may constitute a sufficient motivation to experiment with this new modeling framework.

Finally, the license of CLUMPY authorizes its use in a very broad panel of contexts, including commercial applications pertaining to a consulting office for example.

5.6. CONCLUSIONS

CLUMPY (Comprehensive Land Use [and cover] Model in Python) provides a versatile framework for pattern-based LUCC modeling. It integrates all the methods introduced in this thesis as well as some methods already used in the community. Its implementation has been tested on fully controlled case studies which have allowed to validate the relevance of the methods but also their good software development. It has also been tested and optimized on large real world case studies. A documentation of the code is also available. Finally, this package brings forward three major new contributions, of interest to the LUCC modeling scientific community. These are briefly detailed below.

CLUMPY provides an interface that makes it easy to build a standard case study without any programming effort. More expert users will be able to benefit from more direct and more powerful programming in Python. It is thus possible to build a more complex case study, to take advantage of all the features of the package. Also, because of its nature, it is easy to integrate CLUMPY in a larger modeling framework, e.g., upstream, from econometric models specifying scenario parameters, or downstream, by coupling LUCC projected outputs to ecological models or other types of analyses. Note that the process classes used in CLUMPY are compatible with the well-known machine learning package SciKit-Learn.

For an experienced user, the use of CLUMPY can be very efficient, both in the construction of a case study through a Python script and in the reasonable execution times. Note that the good results of CLUMPY on this second front results from the effectiveness of the algorithms implemented in Cython for some of them, but also from the use of very powerful and optimized Python packages such as Numpy.

Also, the architecture of the package is an opportunity to put forward in a practical way the formalization of the statistical and spatially explicit pattern-based LUCC modeling discussed in the previous chapters of this thesis. Thus, from a practical point of view, its use allows the user to appropriate the concepts and the architecture of this approach to LUCC modeling.

Finally, I stress again that I do not aim at building an all-purpose software (an illusory purpose anyway). On this front, I only wish to provide a correct and efficient point of comparison for other software developers, with all the necessary mathematical and algorithmic background for other software developers to analyze by themselves if and where their own software and ours needs improvements. Furthermore, as I show that CLUMPY is more correct and precise in what it does than the ones we have analyzed and compared it too, I do expect that some users will be wanting to use it in actual case studies.

CLUMPY is an open-source package, open to new contributions by others. I therefore invite users to give me feedback on this software, and to contribute to the development of new methods. I definitely invite other developers to improve their own software on the basis of the work presented in this thesis and implemented in CLUMPY.

CHAPTER 6 | CONCLUSION AND OUTLOOK

This thesis focused on mathematical modeling of LUCC through a statistical and spatially explicit pattern-based approach. This does not imply in any way that geography and land-use conceptual analyses and theories (and other social sciences as well) are unimportant regarding LUCC phenomena. In fact, they are of primary importance in their own sake. In the more limited context of LUCC software-based modeling, they are essential for developing a clear understanding of what a LUCC software can and cannot do, and provide a major guide to set up correctly a LUCC case study where such software is used. In this respect, a LUCC software can bring useful information if used correctly, but is no substitute to prior (land-use, social sciences, etc) analyses of LUCC drivers and related implications. But with Pierre-Yves Longaretti, we focus on the part we can help improving: applied math and algorithmic implementation. This type of improvement can be quite substantial in terms of results relevance and accuracy, and should not be *a priori* dismissed, even if the scope of LUCC software application is intrinsically limited.

Having reached the end of this thesis manuscript, it may be salutary to step back and draw some conclusions from this work. The respective conclusions of the chapters are not repeated here but summarized by focusing on the contributions of this thesis (section 6.1). Also, new elements of perspective will be introduced to outline possible future developments in LUCC modeling but also in applications (section 6.2).

6.1. CONTRIBUTION OF THE THESIS

The rather detailed list of objectives (section 1.4.3) can be a quantitative enumeration of the contributions of this thesis that I will not repeat here. The first question that led to the elaboration of this thesis topic was based on the obser-

vation of significant differences in results between existing models for the same problem and data. It was therefore a question of identifying the origins of these differences, the existence of which raises serious questions about the validity of the results obtained so far with this type of model. Was it because of the software, the data used, the users' choices, formal mathematics or a combination of these reasons? With Pierre-Yves Longaretti, we therefore undertook a study of these models that was intended to be as complete as possible. To my knowledge, such an undertaking is unprecedented in the community.

We first clarified the architecture of the statistical and spatially explicit pattern-based LUCC models (section 1.3). Thus, we were able to split the problem into different modules that could be studied independently to simplify the study of the overall model. Then we followed the same scheme for each of them: criticize the mathematical apparatus chosen by the model, check its implementation and its correct functioning (sometimes by reverse engineering) and then propose alternative more precise (calibration) and unbiased (allocation) methods that we compare with the existing one. This was done for the calibration (chapter 2), allocation (chapter 3) and EV selection (chapter 4) modules (however without the comparison step for the latter). The idea was to cover as much ground as possible in order to ensure that all the issues relevant to LUCC modeling were addressed. Of course, one has to know where to stop in the refinement of the modeling and this is why we have kept to the first and second order questions but not beyond, which is more than sufficient considering the current type of applications of these models.

Also, it seems to me essential to come back to the way in which one can evaluate these modules. Indeed, to my knowledge, there was no formal method within the LUCC model literature capable of qualitatively and quantitatively evaluating a calibration-estimation or allocation method. In order to check the correct behavior of each of the modules, we have thus developed evaluation protocols. The one dedicated to the evaluation of the calibration-estimation module is based on the use of a fully controlled study case where all probabilities are perfectly known (section 2.6). This approach is quite common in many domains (in machine learning in particular) and is to my knowledge unprecedented for LUCC models. However, it is the only suitable way to assess objectively a probability estimation method. Concerning the evaluation of the allocation module, it is a question of measuring the difference between the probability density put as input of the module and the probability density obtained through a series of simulations. We have in this way defined the concept of allocation bias (section 3.3.2). This approach is also new in the community.

Thus, we characterized and showed the competitiveness of our calibration-

estimation method (Bayes-eKDE, section 2.5) and of our unbiased allocation methods (uSAM and uPAM, section 3.4). These methods have shown results superior or equal to all the others in terms of results obtained and numerical computation time (sections 2.7.3 and 3.7). I have implemented all these methods in a Python package named CLUMPY (chapter 5) which can be considered as a co-product of this thesis. This package covers the basics of LUCC modeling. It may be used by modelers with some experience (as the task streaming is not automated) and mostly by developers to test more extensively the properties of our algorithms and compare them to their own methods (see section 1.4.1 on the recipients of this thesis. In particular, this software will provide users and developers alike of an accurate and unbiased comparison point.

We found that all developers make, to varying and sometimes minor extent and probably unknowingly, unwarranted and in fact incorrect mathematical assumptions in the modeling framework implemented in their software. This is a major source of error and bias. I can finally answer the questions previously raised. The differences in results observed between the different LUCC models are mainly due (in decreasing order of importance) to¹: i) mathematical choices intrinsic to the models (pruning, calibration-estimation method...); ii) user parameters whose choices are not trivial and lack indication (binning, pruning, MLP...); and iii) questionable software implementations (pruning, patcher...).

6.2. PERSPECTIVE FOR FUTURE WORK

Having summarized the main contributions of the thesis, I wish now to outline briefly a few possible directions of future research work. The CLUMPY software, ambassador of the methods introduced in this thesis, is also the subject of concrete perspectives of extension and improvement (section 6.2.2). Finally, I take the liberty here to imagine new applications that could take advantage of some results of this thesis (section 6.2.3).

6.2.1. LUCC MODELING FRAMEWORK

A formalization of pattern-based LUCC models was undertaken in this thesis. The adopted research strategy was motivated by two considerations: to propose a valid and mathematically correct formal theory while being understandable by the community of developers of existing models. However, some

¹Excluding considerations regarding the quality of input data which are taken for granted in this thesis.

fundamental mathematical points have been left out. These are either taken for granted by the community or totally unthought of, and so we have not addressed them in this thesis to focus on elements that are immediately understandable and mobilizable by model developers. Yet these concepts form the basis of LUCC models. In particular, all pattern-based models consider that studying the probability distribution of pixels is equivalent to studying the probability distribution of patch seed pixels (a concept quickly introduced in the section 1.3.3.1). However, this postulate contains an approximation that should be examined at the formal level in order to evaluate its importance and possible consequences of its violation, if any; other related assumptions have been mentioned (e.g., statistical independence between seed pixel and all other patch parameters) that should also be formally analyzed. Moreover, I mentioned in section 1.3.2.2 that there are in fact only two truly independent explanatory variables without developing formal proofs. A mathematical study of the explanatory variables and their “ideal” number could also be the subject of an interesting formal development. Another assumption that has never been formally scrutinized is that future LUCC projections can be simulated in a Markovian approximation. Assessing this last point however would most probably involve sophisticated methods that may, e.g., be borrowed to the fields of statistical physics, physical kinetics, stochastic differential equations, etc, but would probably be way beyond the approaches and mathematical level of everything that has been done so far. The benefit of justifying to some extent the Markovian approximation must be balanced with the fact that such an analysis would be too abstruse for the community.

I also point out some perspectives in the methods used in the different modules. New approaches can be envisaged in the proposed framework such as an alternative to Bayes-eKDE which is more efficient numerically for example. One can also imagine proposing patch construction procedures based on other paradigms than the use of cellular automata such as a shape library (mentioned in section 1.3.5.1). These would not be intended to completely replace the current method but could be particularly adequate to model particular LUCC phenomena. The modeling framework in modules proposed in this thesis allows users and developers to consider the methods as interchangeable and to choose the one that is the most adapted to the situation for each transition of the same study case. This interchangeability must then be allowed by the software to be operational (see section 6.2.2).

It could also be interesting to propose mechanisms to help users to configure model parameters. Indeed, LUCC models can be complex to use and their validity is often strongly dependent on users’ choices. We have outlined such an

approach in the illustration of our method of selecting explanatory variables by describing the choice of the various parameters (section 4.5.1). However, this approach must be taken with caution. Indeed, tools do not by themselves make good science. Understanding the ins and outs of a problem is a must, and one should never use a software as a black box, *i.e.*, without sufficient understanding of what it actually does. This remark invites users to be cautious, but also the designers of modeling frameworks to propose tools designed in this way.

6.2.2. CLUMPY

The CLUMPY software is delivered with this thesis is a first stable version (1.0.0). Although it can be used as is by users, there are numerous perspectives of future development. CLUMPY is an open-source software and is thus open to contributions, especially by adding new methods. Its architecture treats methods as interchangeable bricks and it is easy to elaborate and implement a new one as long as it complies with the requested inputs and outputs.

Also, the GUI proposed in this first deliverable is rudimentary and only allows to design a simple case study (see section 5.2.2). Anyone is welcome to improve it by taking care not to make the user experience too arduous, which is a real challenge considering the complexity of some case studies. It would also be advisable to avoid an entirely automated black box approach by proposing real choices of parameters and by graphically alerting the user when the values of these are unsuitable.

Finally, the documentation of CLUMPY has an important role in its dissemination and accessibility to potential users. The writing of this documentation must bring the keys of comprehension for a good use of the package through tutorial, case of illustration and enumeration of all the functions and classes. The user feedback should allow to improve the documentation according to their experiences. Currently, the documentation is exhaustive concerning functions and objects (API documentation) but does not yet include tutorials.

6.2.3. APPLICATIONS

CLUMPY being a Python package makes it possible for an experienced and knowledgeable user to make a very advanced use of the software. Indeed, Python is a computer language particularly appreciated in scientific research. It has the advantage of being high-level, versatile and relatively easy to use. Moreover, it is possible to call functions written in C or to write files in Cython to take advantage of the speed of execution of C for certain critical operations. Thus, given

CLUMPY's numerical efficiency in performing a large number of estimates and allocations in a reasonable amount of time, it is quite feasible to perform parameter sensitivity analysis with this software to improve one's understanding of LUCC drivers. The fact that CLUMPY is written in Python also allows it to be easily interfaced and included in a larger model that considers it as a function with its own inputs and outputs, or to couple it with other models (e.g. econometric ones) that would specify a scenario from other scientific inputs.

Thus, I hope that all the methods and processes introduced in this thesis as well as CLUMPY will have a positive influence on the community of LUCC model developers but also on the accuracy and validity of the results obtained on future case studies. The applications of LUCC models are expected to take an important place in the coming years, especially in the evaluation of land planning public policies. Indeed, as I explained in the introduction (section 1.1), LUCC are bidirectionally linked with global changes that threaten the conditions of life on Earth as we know them today. Thereby, the possibility to evaluate beforehand land planning public policies is decisive and LUCC models can bring elements of answers to practical questions of land planning and while providing simulation maps that are highly understandable by stakeholders (either political leaders, civil servants or citizen assemblies). Thus, it is for example possible to interrogate with LUCC models new constraints to favor the relocation and food autonomy of a territory, or to plan to stop deforestation and soil artificialization. This thesis has provided the first few steps required to enable statistical and spatially explicit pattern-based LUCC models to be up to the challenge of the questions they claim to help answering.

APPENDIX A | GISTAM PUBLICATION: AN OVERVIEW OF THE THESIS WORK

Mazy, F.-R. and Longaretti, P.-Y. A Formally Correct and Algorithmically Efficient LULC Change Model-building Environment. In *Proceedings of the 8th International Conference on Geographical Information Systems Theory, Applications and Management - GISTAM*, pages 25–36. SciTePress, 2022c

ABSTRACT

The use of spatially explicit land use and land cover (LULC) change models is widespread in environmental sciences and of interest in public decision-help. However, it appears that these models suffer from significant biases and shortcomings, the sources of which can be mathematical, conceptual or algorithmic. We formalize a modeling environment that distinguishes a calibration-estimation module and an allocation module. We propose an accurate calibration-estimation method based on kernel density estimation and detail an unbiased allocation algorithm. Moreover, a method of evaluation of LULC change models is presented and allows us to compare them on various fronts (accuracy, biases, computational efficiency). A case study based on a real land use map but with known (enforced) transition probabilities is used. It appears that the estimation error of the methods we propose is substantially improved over the best existing software. Moreover, these methods require the specification of very few parameters by the user, and are numerically efficient. This article presents an overview of our LULC change modeling framework; its various formal and algorithmic constituents will be detailed in forthcoming papers.

A.1. INTRODUCTION

Land use and land cover (LUCC) change is a major driver of global change alongside the more visible issues of climate change and biodiversity loss. The study of LUCC is of major interest in analyzing and understanding a variety of socio-environmental phenomena but also for decision-help on mitigation and adaptation policies, and the literature on LUCC studies is by now quite substantial. Different LUCC modeling strategies have been developed over the last few decades to address these research issues (static, dynamic, agent-based, local or global scale, *etc...*).

In the present work, we focus on spatially explicit, statistical LUCC model building. Such models generally consists of estimating transition probabilities from one land-use state to another based on (usually) two land-use maps at different dates that reflect changes in the past. These models are designed to simulate new land use and cover (LUC) maps, in scenario-driven projections of future LUC and cover. Such allocated maps can be used for a variety of purposes. For example, they may be coupled to models of ecosystem services to produce evaluations of their future evolution. In fact, producing accurate spatially explicit projections of the effects of public decisions bearing on social-ecological problems is a current issue for such models Verburg et al. (2019).

Statistical and spatially explicit LUCC models constitute a popular approach to LUCC modeling, and various model-building software have already been devised in this framework. Among the most well-known, one may quote Dinamica EGO Soares-Filho et al. (2002), Idrisi LCM Eastman et al. (1995), or the CLUE family of models Verburg et al. (1999), the last member being CLU-Mondo van Vliet et al. (2015). However, these software (and others) exhibit substantial differences in results for the same case studies Mas et al. (2014); Prestele et al. (2016); Alexander et al. (2017), which raises an important concern on the reliability of the LUCC modeling process itself.

No study to date has clearly identified the origin of the differences of behavior and outcome displayed by the existing model-building software on a same problem, let alone proposed a formally correct theoretical framework for this type of LUCC modeling. Moreover, comparing different LUCC models on their spatial outcomes is a poorly mastered operation in the literature van Vliet et al. (2016) and more precise evaluation methods are needed. Nevertheless, as will be illustrated in the course of this work for the three modeling environment just mentioned, these discrepancies can be traced to differences in the formal and algorithmic choices made in their elaboration. In fact, quite a few of these choices were made without paying sufficient attention to the constraints im-

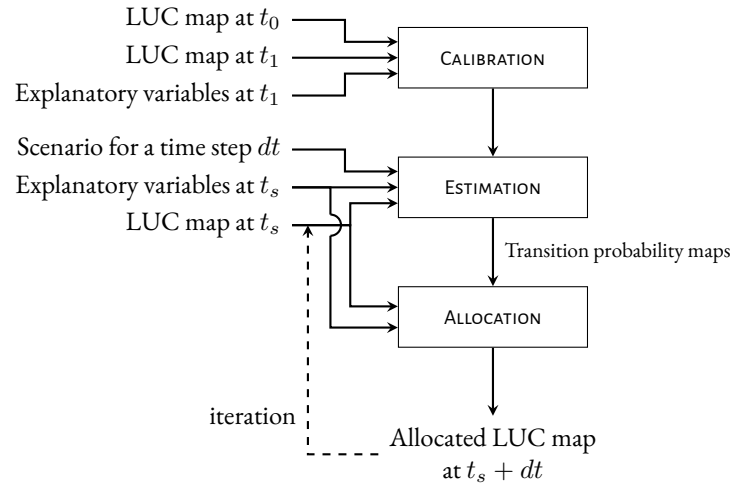


FIGURE A.1: General architecture of a spatially explicit statistical LUCC model.

posed by a correct *ab initio* formal investigation of the probabilistic foundation of the problem, leading to a number of errors and biases.

More precisely, we address these issue by focusing successively on the calibration-estimation and allocation processes in order to propose a fully formalized and conceptually correct foundation for statistical spatially explicit LUCC analyses. We also provide efficient and bias-free algorithmic implementations of these processes, which constitute the core of this approach to LUCC model-building. The present work illustrates some of our main results on these two fronts. A detailed exposition of the underlying formal analyses and algorithmic implementations will be given in forthcoming dedicated papers. The first one will formalize in a rigorous ways statistical LUCCs by groups of contiguous pixels (patches), starting from the more common pixel by pixel probability distributions, and introduce an allocation algorithm to this effect, that will be explicitly shown to be bias-free. The second one will be dedicated to the calibration and estimation of probability transitions. Finally, the last article will provide a systematic method of identification and a systematic analysis of the biases present in the software mentioned above.

This paper is organized as follows. The typical structure of statistical spatially explicit LUCC models is recalled in section A.2. Next, a new and accurate calibration-estimation method and an unbiased allocation methods are briefly overviewed in section A.3. We then propose an evaluation method for LUCC models that allows us to compare their results and performances; the most salient features are described in section A.4. The next section introduces a case study to illustrate the methods of the previous sections, and presents some of our results (section A.5). Finally, the last section summarizes these findings along with other important results on these issues that will be established in our

forthcoming papers on the topic.

We assume that a number of preliminary modeling choices are made prior to the comparison of performances mentioned right above: elaboration of the LUC typology, choice of explanatory variables, and choice of the discretization (pixel) scale. Thus, we exclude from the perimeter of our analysis the obvious influence of these choices on the results García-Álvarez et al. (2019b).

A.2. LUCC MODEL ARCHITECTURE

Spatially explicit statistical LUCC models are generally (but not always) organized around two main modules: calibration-estimation, and allocation (Fig. A.1). This allows us to investigate in turn their underlying logic, formal foundation and algorithmic implementation.

A.2.1. CALIBRATION-ESTIMATION MODULE

We start from two raster (i.e., pixelized) LUCC maps at two different dates t_0 and t_1 as well as maps of the d explanatory variables of interest, characterizing LUCCs (e.g., altitude, distance to the roads, *etc.*). Pixel sizes can vary from a few meters to a few tens of kilometers depending on case studies; the largest maps may involve tens of millions of pixels. The dates t_0 and t_1 differ typically by a few years.

Although changes occur in patches of contiguous pixels, the calibration-estimation process focuses on individual pixel probability distributions; patches are then produced on this basis in the allocation module. Pixels undergo a change from LUC u to LUC v . Each pixel i is characterized by a collection (vector or tuple) $\mathbf{z}_i$ of values of the d explanatory variables. The transition probability distribution must be evaluated for all possible explanatory variables combinations $\mathbf{y}$ present in the calibration maps, including those which were not observed to undergo a transition but may be expected to in the future. To this effect, one needs to evaluate the conditional pixel transition probability $P(v|u, \mathbf{y})$. This involves a form of interpolation in explanatory variable space, called estimation (Fig. A.1).

A.2.2. ALLOCATION MODULE

A new LUC map is simulated from an initial one (possibly different from the calibration maps), from the explanatory variables (when needed) and from the

probability of LUCC maps for each possible LUCC state change (Fig. A.1). Allocations are made in groups of contiguous pixels (patches). The process involves a sampling algorithm and attributes a possible new LUCC state on the basis of the associated transition probability distribution and some patches constraints. The map obtained in this way is a particular instantiation of this probability distribution; many others could have been produced for the same time step, and all such maps are statistically equivalent, except for statistical noise. This point is usually ignored in existing model-building environments, although it is crucial to identify errors and biases, as will be discussed later on. The details of our patch construction method are involved but not critical for software performance comparison (both in terms of accuracy and execution time) and will therefore not be presented here.

In the process the user provides a transition matrix that characterizes the rates of change per time step between each LUC state, and characterized by an overall probability $P(v|u)$ for all possible LUCC initial and final states u, v . This matrix is usually specified through scenarios.

A.2.3. EXISTING LUCC MODEL-BUILDING ENVIRONMENTS

In the present work we focus on three of the most widely used software van Vliet et al. (2016): Dinamica EGO, Idrisi LCM and CLUMondo. Our objective is to contrast their performances (accuracy and computational efficiency) to the modeling strategy and algorithmic implementation we propose in the next section.

Dinamica EGO fits reasonably well in the archetypal structure of Fig. A.1. Transition probabilities are usually estimated from the use of weights of evidence, applied after binning the user-defined explanatory variables Soares-Filho et al. (2002). This binning step requires to specify a number of parameters, which turn out to have a very strong impact on the obtained results. Moreover, this method of estimating transition probabilities is based on the strong assumption of statistical independence of the explanatory variables; it turns out that even modest amounts of cross-correlation between these variables may lead to non negligible errors in the estimated transition probabilities. This software implements an allocation method relying on a pixel pre-selection process (pruning), implemented to reduce computational time; such pruning also produces significant biases in the results.

Idrisi LCM Eastman et al. (1995) also implements a calibration/allocation architecture, while leaving the user with less control on modeling choices than Dinamica EGO. This strategy is adopted on purpose, in order for users with

little expertise to nevertheless be able to implement a LUCC model. The particular LCM version we have used is the one bundled in Idrisi Selva. This software proposes different models for estimating transition probabilities: logistic regression (LCM LR), SimWeight (LCM SW) and Multi-Layer Perceptron (LCM MLP). The LCM allocation module is very simple and deterministic, as the allocation algorithm implemented in this software essentially ignores the statistical nature of the process. This results in simple but strongly biased allocation rules.

Finally, the CLUE family of software Verburg et al. (1999), the latest of which is CLUMondo van Vliet et al. (2015), chooses to estimate the probabilities of change from a single LUC map, from which it defines transition probabilities through a logistic regression on all explanatory variables. Also, CLUMondo does not allow the modeler to access its allocation module independently.

A.3. CLUMPY: A NEW LUCC MODEL-BUILDING ENVIRONMENT

We present here in a succinct manner the methods used in our calibration-estimation and allocation modules. The closest analog in the existing model-building environments is Dinamica EGO. The main differences come from our more sophisticated but more accurate and efficient calibration-estimation method, our demonstrably error- and bias-free allocation method, and significantly more efficient algorithms leading to substantial gains in computation time in large problems.

These modules constitute the core of our own model-building environment, called CLUMPY (Comprehensive Land Use Model in Python).

A.3.1. CALIBRATION-ESTIMATION

A.3.1.1. BAYES RULE

The probability of transition from LUC u to LUC v of a pixel characterized by the d -tuple of explanatory variables $\mathbf{y}$ is noted $P(v|u, \mathbf{y})$. This notation emphasizes the fact that this probability is conditioned by the knowledge of the values of u and $\mathbf{y}$. Bayes rule allows us to express this probability in a more

convenient way for calibration purposes:

$$P(v|u, \mathbf{y}) = P(v|u) \frac{p(\mathbf{y}|u, v)}{p(\mathbf{y}|u)}. \quad (\text{A.1})$$

The various factors on the right-hand side member of this equation are defined as follows. $P(v|u)$ is the global transition probability, specified by a scenario provided by the user. This quantity can also be computed directly from the observed transitions between the calibration maps at times t_0 and t_1 . Next, $p(\mathbf{y}|u)$ is a conditional probability density. We use probability densities because explanatory variables are usually continuous quantities, and we do not bin them in the end, in order to produce a more accurate calibration procedure. By definition $p(\mathbf{y}|u)d\mathbf{y}$ is the probability of observing $\mathbf{y}$ within the small volume $d\mathbf{y}$ of explanatory variable space, for pixels of initial state u . Similarly, $p(\mathbf{y}|u, v)$ is the probability density of $\mathbf{y}$ for pixels making the transition from states u to v . Note that the calibration process, which is based on observed transitions, uses these probabilities in the frequency of occurrence meaning, while the allocation module applies Bayes rule in a Bayesian sense.

The interest of using Bayes rule lies in the fact that $p(\mathbf{y}|u)$ and $p(\mathbf{y}|u, v)$ are much simpler to estimate than $P(v|u, \mathbf{y})$. Indeed, it is then a question of estimating density functions on $\mathbf{y}$ for a set of pixels of initial state u for $p(\mathbf{y}|u)$ or potentially undergoing a LUCC state transition from u to v for $p(\mathbf{y}|u, v)$. This problem is widely addressed in machine learning and is called density estimation.

Consider a set of n calibration pixels, made of all the pixels that actually underwent a LUCC state change from u to v and are directly extracted from two calibration maps at time t_0 and t_1 . Each of these pixels is characterized by its explanatory variables noted $\mathbf{z}$. We wish to estimate the transition probability of all m pixels in our maps; these are by definition evaluation pixels. Each of these pixels is characterized by its explanatory variables noted this time $\mathbf{y}$. The problem is therefore to estimate $P(v|u, \mathbf{y})$.

The idea is to calibrate a density estimator with the calibration pixels for both probability densities and then to estimate these probability densities for all pixels with the obtained density estimator. Finally, as the global transition probability $P(v|u)$ is given, we can apply Bayes rule to obtain $P(v|u, \mathbf{y})$.

A.3.1.2. DENSITY ESTIMATION BY KERNEL DENSITY ESTIMATOR (KDE)

Estimating a probability density is a widely addressed problem in the machine learning literature Scott (2015). A simple and not very precise method is

to make histograms in explanatory variable space. This requires to bin explanatory variables; unfortunately, the choice of the bin size has a strong impact on the results. Some related methods such as averaged shifted histograms have been proposed to circumvent the problem Chamayou (1980).

More sophisticated methods perform density estimates by positioning a kernel function K characterized by a user-specified width parameter h (called bandwidth) on each calibration data point and summing over all these kernels at the desired locations of estimation (in explanatory variable space). This is called kernel density estimation Wand (1992), or KDE in short. This very efficient method is however highly computationally intensive as soon as the number of explanatory variables and elements (here, pixels) increases. Many kernel functions may be studied (e.g., gaussian kernels), and various methods have been proposed to approximate the resulting family of estimators O'Brien et al. (2016); Charikar and Siminelakis (2017). However, these relatively complex methods often require large amounts of memory since they allocate full matrices representing the entire space of explanatory variables. Thus, when confronted with more than a few explanatory variables, these methods can be inapplicable due to lack of memory space on standard machines; the computation time also increases quite fast with the number of dimensions.

Instead, we implemented a hybrid binning/KDE method, that keeps part of the simplicity of simple histograms with only a small degradation of the performance of pure Kernel density estimator methods. The binning is performed on a scale h/q smaller than the bandwidth h Wells and Ting (2019), where $q > 1$ is an odd integer. A kernel density estimator is then applied to the small bins themselves instead of the original individual elements (pixels) in explanatory variable space. One can then show that the larger q , the smaller the approximation error. This hybrid method maintains the spirit of continuous density estimation, while being much more computationally efficient than direct KDE estimators (both in terms of computational time and memory use). This is what we used in our estimations of $P(\mathbf{y}|u)$ and $P(\mathbf{y}|u, v)$.

The choice of the kernel width h for the KDE method is a very important issue due to its influence on the quality of the estimation. Indeed, choosing a too narrow bandwidth results in an over-interpretation of the observational data and in noisy estimates. On the other hand, a too broad bandwidth leads to under-fitting (or over-smoothing) and to a degradation of the estimate of the probability density. The determination of the optimal width of the kernel is a non-trivial problem and is widely discussed in the machine-learning literature Wand and Jones (1994); Rudemo (1982); Sain et al. (1994). However, these methods are computationally very expensive as soon as the number of pixels is

large and the number of dimensions exceeds 3, which is very frequently the case in LUCC studies.

This being said, in the LUCC context, a slight over-smoothing is not a problem and can even be interesting, because transition probabilities are usually undersampled, and therefore noisy. Consequently, we chose to determine the KDE bandwidth h from the principle of maximum smoothing of Terrel Terrell (1990), leading to

$$h_{\text{Terrel}} = \frac{(d+8)^{\frac{d+6}{2}} \pi^{\frac{d}{2}} \int K^2}{16 n (d+2) \Gamma\left(\frac{d+8}{2}\right)}, \quad (\text{A.2})$$

where K is the kernel density estimator. The slight oversmoothing involved turns out to be essentially unnoticeable in our tests, while this prescription considerably reduces the computational burden of KDE methods.

We have checked on various test problems, and will soon show, that even simple kernel functions (such as a square box, a triangle or a gaussian kernel) give much more accurate results than existing calibration methods.

Note finally that, because explanatory variables are of widely different nature and not statistically independent, it is necessary to normalize them in order to work with data of zero mean and covariance matrix equal to the identity in explanatory variable space. This operation is called “whitening transformation” in the machine learning literature. It makes it legitimate to use a unique bandwidth in all dimensions in the transformed explanatory variable space and greatly simplifies the numerical implementation of our calibration-estimation method.

A.3.2. ALLOCATION

In general, the allocation module takes as input a LUC map as well as the transition probability maps obtained in the calibration-estimation module. The allocation method presented here also requires the knowledge of the explanatory variables of the input map.

The simulation of an allocated map produces a specific statistical sampling of the transition probabilities. So far, we have focused on pixel transition properties. However, as already mentioned, we allocate pixel patches. Following the logic initiated in Dinamica EGO, a first pixel is selected according to the transition probability distribution obtained by our calibration-estimation procedure. This pixel is called a “core pixel” or “pivot-cell”. Then, a specific procedure is applied to create a patch around this core pixel. This procedure is character-

ized by different parameters such as the surface of the patch, and its elongation. Patches created in this way reproduce some of the statistical properties of actually observed transition patches, but are defined algorithmically to provide some randomness in their shape.

The allocation modules implemented in the existing software turn out to be all biased to various extents. Such biases have not yet been identified in the literature, first because a biasing criterion has not been formulated, and second because the details of these software allocation procedure is not fully documented. We have circumvented this last problem by a combination of literature analysis, educated guesses, retro-engineering, questions to model developers, and re-implementation (when feasible) of these allocation algorithms to check that our understanding of their structure and content exactly reproduces the outcome of the original software on a number of test problems. We have also formulated a simple but powerful “no-bias” criterion, which, in essence, requires that the post-allocation probability distributions are identical to the pre-allocation ones. This is in fact more of an unavoidable self-consistency requirement, but it turns out that none of the existing LUCC model-building software does satisfy it.

This allowed us to implement a strategy of systematic identification and characterization of the various biases present in existing model-building software. We illustrate this process on a particularly important bias related to pruning, relying on an efficient bias-free algorithm which avoids the need for pruning.

A.3.2.1. PRUNING

LUCC models may involve a very large number of pixels (e.g., tens of millions). Therefore, it can be interesting to pre-select a limited number of pixels in order to speed up the allocation procedure. This pre-selection is called pruning and is implemented by Dinamica EGO and LCM in two different ways, both of which turn out to be significant sources of bias.

Dinamica EGO’s pruning method consists in ranking pixels by decreasing order of transition probability $P(v|u, \mathbf{y})$. Pixels are then pre-selected in this order in this list; the number of pre-selected pixels is equal to the number of pixels necessary to reach the targeted LUCC surface defined by the user selected scenario, multiplied by a pruning parameter F . The default value is $F = 10$ (ten times as many pixels as needed for the various transitions are pre-selected).

LCM ranks the pixels by decreasing order of the probability density (or probability distribution if binned) of the explanatory variables for this transition, $p(\mathbf{y}|u, v)$. Then LCM keeps the exact number of pixels that are necessary to reach the targeted transition surface defined by the selected scenario, and has a specific procedure to resolve conflicts of allocation for the same pixel. Thus,

LCM's pruning method is also its allocation method since all pixels selected in this way are directly allocated without any further consideration. This procedure has the advantage of simplicity and ensures that transitions occur in patches (due to the spatial continuity of the explanatory variables probability densities). However, the maps allocated by LCM often suffer from a severe lack of realism, and they always violate our self-consistency no-bias requirement.

Although establishing which probability ordering should in principle be used for pruning is not an obvious task, we have proved that Dinamica EGO's choice $[p(v|u, \mathbf{y})]$ is the theoretically correct one¹; this applies although the correct allocation probability distribution is by definition $P(v|u, \mathbf{y})$. This being said, both Dinamica EGO and LCM pruning strategies are strongly biased because they modify to various extents the probability distributions which should be enforced exactly. LCM is the more biased of the two.

An unbiased pruning method necessarily consists in a random sampling performed according to $p(\mathbf{y}|u, v)$, and selects the number of kernel pixels needed to reach the targeted number of transited pixels. Thus, the probability distribution of this pixel subsample will be statistically representative of $p(\mathbf{y}|u, v)$ and there will be no unwarranted truncation of this probability density distribution.

The motivation for applying a pruning procedure lies in the numerical acceleration of the allocation method for a very large number of pixels. However, the implementation in Python of the our allocation procedure as presented in section A.3.2.2 proves to be numerically very efficient without pruning, even though the number of pixels in some of our case studies is very large ($> 10^8$); this efficiency relies on the use of dedicated Python functions, which perform nearly as fast as equivalent C codes. Still, we have designed a bias-free pruning algorithm, for possible use in particular problems.

A.3.2.2. UNBIASED ALLOCATION

In addition to pruning, other biases can occur in the allocation procedure. In particular, the creation of patches around a core pixel automatically excludes these patch pixels from the rest of the allocation procedure. However, they could just as well be selected afterwards.

In order to resolve such potential conflicts, we designed the following iterative allocation procedure for a given initial LUC state :

1. We know the transition probability $P(v|u, \mathbf{y})$ of each pixel for all (u, v) .
We can therefore apply a generalized Von Neumann rejection sampling,

¹This statement corrects an error in the original GISTAM publication. See Appendix 3.D.

which allows us to test all possible states v at the same time for any given u . We obtain an unbiased sample of kernel pixels for each of the transitions studied. If no kernel pixel is selected, the allocation procedure is terminated at this point.

2. A single kernel pixel is randomly and uniformly drawn; its associated transition has been determined in the previous step.
3. The procedure of patch creation around this core pixel is applied next. The selected pixels are actually allocated on the simulated map.
4. $P(v|u)$ is updated, taking into account that a certain number of pixels have already been allocated. This probability is therefore reduced for the rest of the allocation procedure in the considered time step.
5. $p(\mathbf{y}|u)$ is updated next because some elements have already been allocated, which has modified the probability density distribution of the explanatory variables.
6. $p(v|u, \mathbf{y})$ is then recalculated from Bayes rule Eq. (A.1); the probabilities involved in steps 4 and 5 are also updated. We then start again at step 1.

We have shown that a carefully designed algorithm of this type is bias-free. This procedure is however demanding since it requires to re-estimate very frequently $p(\mathbf{y}|u)$. We therefore update this probability distribution only when the percentage of allocated pixels is significant enough that the estimated probability is too far from the real distribution (in practice, after a fixed small percentage of state changes has been achieved). This may introduce a slight bias.

A.4. EVALUATION

A review of the literature shows that validating the results obtained by LUCC models is an uncommon practice. This deficiency underlies some of the doubts that may be raised on the robustness of LUCC modeling van Vliet et al. (2016). In any case, before specific results may be validated, the modeling framework itself should be validated. A first attempt along these lines has already been carried out on artificial data Mas et al. (2014) but the lack of exact knowledge of the transition probabilities involved precluded any detailed evaluation of the LUCC models that were tested in this earlier work. Indeed, one of the main problems of statistical LUCC modeling is the estimation of the probability distribution $P(v|u, \mathbf{y})$ (section A.3.1), and, furthermore, it is impossible to know

this probability distribution exactly in a real case study. Validating this estimation of $P(v|u, \mathbf{y})$ therefore remains an essential objective.

We propose here a simultaneous validation method for the calibration-estimation and global calibration-estimation/allocation procedures, that allows us to objectively compare different model-building strategies. This is achieved by quantifying the difference between the transition probabilities obtained from our hybrid KDE estimation in the calibration phase or in the post-allocation one, and exactly specified pre-calibration transition probabilities. We may proceed to this effect from semi-real or completely artificial data. We start from a (real or artificial) LUCC map at $t = t_0$. We adopt an exactly known transition probability distribution, $P^*(v|u, \mathbf{y})$. This exact transition probability may be defined analytically or numerically. Then, a new LUCC map at time $t = t_1$ is produced with our allocation procedure (based on this exact probability distribution and our patch creation algorithm).

This allows us to implement two validation processes:

CALIBRATION-ESTIMATION COMPARISON

We select a calibration-estimation method from an existing LUCC model-building environment, and produce from the two maps just specified at t_0 and t_1 an estimate $\hat{P}(v|u, \mathbf{y})$ of $P^*(v|u, \mathbf{y})$. We repeat this process for all the calibration methods we want to compare, including ours.

CALIBRATION-ESTIMATION/ALLOCATION COMPARISON

We now wish to evaluate the relative efficiencies of these modeling environments over the whole calibration-estimation/allocation process. We thus use as inputs the LUCC map at $t = t_1$ and the probability distribution $\hat{P}(v|u, \mathbf{y})$ determined by the previously described calibration-estimation comparison process, for the modeling environments tested. We produce next a new LUCC map at $t = t_2$ from the associated allocation procedure. We recover a new post-allocation estimate $\tilde{P}(v|u, \mathbf{y})$ of $P^*(v|u, \mathbf{y})$ from the calibration-estimation method of the same modeling environment. We repeat this process for all the modeling environments we want to compare, including ours.

The comparison of the exact (enforced) probability distribution with the estimated ones can be done in various and more or less sophisticated ways. In this article, we limit ourselves to two very simple approaches. The first one consists in producing graphs of the estimated distributions $\hat{P}(v|u, \mathbf{y})$ and the exact one $P^*(v|u, \mathbf{y})$ considered as a function of $\mathbf{y}$, for various one-dimensional cuts in explanatory variable space, e.g., by fixing all explanatory variables but one. This

gives a direct check of the accuracy of the various methods, but only on a limited (although randomly chosen) set of locations in explanatory variable space.

The second validation method is more global and consists in calculating the average of the absolute error throughout explanatory variable space:

$$\begin{aligned}\varepsilon_{\text{calib}} &= \frac{1}{m} \sum_{i=1}^m \left| P^*(v|u, \mathbf{y}^i) - \hat{P}(v|u, \mathbf{y}^i) \right|, \\ \varepsilon_{\text{tot}} &= \frac{1}{m} \sum_{i=1}^m \left| P^*(v|u, \mathbf{y}^i) - \tilde{P}(v|u, \mathbf{y}^i) \right|\end{aligned}\quad (\text{A.3})$$

where m is the number of pixels where the difference is evaluated (generally, all pixels concerned by the $u \rightarrow v$ transition in a map), and where the subscript “calib” or “tot” refers either to the calibration-estimation comparison process or to the global calibration-estimation/allocation one.

By construction Eq. (A.3) tends to underestimate large but localized differences. Using one-dimensional cuts minimizes this possibility to some extent. We could also, e.g., identify the largest absolute difference, and count the number of pixels where this difference is achieved within a given tolerance. We avoid relative differences because they might be very large where the transition probability is low, but this would not necessarily reflect a notable inaccuracy in the estimation itself.

In any case, we can measure the difference between the exact and estimated probability distributions, a validation test that has been never been performed so far, and thus have a first global evaluation of the relative robustness of various calibration-estimation and allocation modules (ours as well as the ones implemented in existing model-building software).

Our own calibration-estimation and allocation procedures are tested simultaneously and not independently in both comparison processes (as the second map needed in these tests is produced by our allocation procedure). One may therefore ask whether they are both validated in this way. Several lines of arguments show that this is the case, relying on *a priori* and *a posteriori* analyses. First, we have formally proved that our allocation procedure is bias-free (the proof will be given elsewhere), i.e., that it enforces the correct transition probability distribution. Also, the KDE density estimation procedures have been shown to converge exactly to the correct density distribution in the machine-learning literature in the limit of a large number of available points (this form of convergence is weaker than for allocation, but still relevant). This applies also to our own hybrid KDE calibration-estimation method, except possibly close to boundaries in explanatory variable space due to the kernel truncation correction applied there (not described in our overview of the method). Second,

the numerical efficiency of our algorithms allow us to produce a large number (thousands) of calibration-allocation sequences for the same time step in a reasonable computation time. We checked on a number of test problems that post-allocation transition probabilities (recovered by our calibration-estimation procedure) converge in expectation value to the enforced one by averaging over these multiple allocations. These arguments provide an *a posteriori* validation of both procedures. Indeed, having errors or biases introduced by one procedure nearly exactly compensated by the other is beyond unlikely, considering the very different strategies used in their elaboration.

A similar concern may be raised about the fact that using our own allocation procedure to produce the $t = t_1$ LUC map for testing calibration-estimation procedures may favor our method over the others. This concern is misplaced for the same type of reason: the way this second map was produced is irrelevant precisely because there is no relation between our allocation procedure and any of the calibration-estimation procedures we have tested. Also, we have just shown (or at least convincingly argued, until the above-mentioned proofs are available in print), that our allocation procedure produces unbiased post-allocation probability distributions, so that the amount of bias (due in fact to statistical noise) produced on a single $t = t_1$ map is limited. In any case, it applies equally to all tested calibration methods, which are therefore treated on the same footing in this respect.

A.5. RESULTS AND DISCUSSION

In this section we apply the evaluation methods presented in section A.4. We thus define a case study that is intended to be representative of commonly encountered LUCC problems. We put to the test our own modules (section A.3) as well as the ones implemented in the existing model-building software introduced in section A.2.3. The parameters used for each model are specified in the Appendix.

A.5.1. CASE STUDY SHORT DESCRIPTION

We are interested in a study area of 94 square kilometers located in the Isère *département* in France. We focus on a smaller sector in the Southwest of the town of Grenoble; we have raster maps describing this smaller area at 15 meters of resolution (6.3 million pixels), and use 7 different LUC classes at the coarsest typology level (water bodies, mineral areas, forests, agricultural areas, urban ar-

eas, economic activity areas and other), and up to several tens at the finest level. These data have been used in a recent project, whose objective was to explore the future of ecosystem services at the 2040 horizon under various land planning scenarios Vannier et al. (2016, 2019a,b).

For our present purposes, we only focus on a single transition, namely from agricultural areas (u) to urban areas (v). This is one of the main transitions responsible for urban sprawl. We chose this transition for the sake of clarity and simplicity. The number of agricultural area pixels is ~ 3.3 million.

We have selected three explanatory variables to characterize this transition: elevation above sea level in meters (y_0), slope in degrees (y_1) and shortest distance from urban areas in meters (y_2). These are three of the main explanatory variables typically used in urban sprawl studies relying on statistical LUCC modeling frameworks.

In line with the evaluation strategy described in section A.4, we enforce a specific transition probability distribution, namely we adopt a multivariate Gaussian distribution:

$$P^*(v|u, \mathbf{y}) = \begin{cases} \mathcal{N}_{\mu, \Sigma}(y_0, y_1, y_2) & \text{if } \{y_0, y_1, y_2\} \in \mathcal{D} \\ 0 & \text{else} \end{cases} \quad (\text{A.4})$$

where $\mathcal{N}_{\mu, \Sigma}$ refers to a normal distribution of mean μ (vector of the means of the explanatory variables), and covariance matrix Σ (covariance matrix of the explanatory variables), (y_0, y_1, y_2) is the vector of explanatory variables, and $\mathcal{D}$ is a subset of the explanatory variable space where the probability density $P(\mathbf{y}|u)$ is larger than a (small) threshold (this avoids potential problems in the application of Bayes rule without introducing any significant bias in the transition probability distribution). The exact definition of μ , Σ and $\mathcal{D}$ are given in the Appendix.

A.5.2. MEAN ABSOLUTE ERROR COMPARISON

The pixel-averaged absolute errors are calculated with Eq. (A.3) of section A.4 and reported in Table A.1.

CLUMPY's $\varepsilon_{\text{calib}}$ is about four times lower than for the next best existing software with respect to this evaluation criterion, Dinamica EGO. This confirms the quality of the KDE estimator relative to other methods. As expected, $\varepsilon_{\text{tot}} > \varepsilon_{\text{calib}}$ whatever the model since the allocated map is a particular instantiation of the estimated probability distribution, which is itself an approximation of the exact one. We notice also the influence of the pruning factor F of Dinamica EGO: reducing this parameter results in a larger error (note that $F = 10$ is

TABLE A.1: Comparison of models through Eq. (A.3) for the calibration-estimation and calibration/allocation comparison processes. DE = Dinamica EGO, for two different pruning factors ($F = 10$ and 100). The last column results from an average over 100 allocations of the same time step.

model	$\varepsilon_{\text{calib}}$	ε_{tot}	$\varepsilon_{\text{tot}, 100}$
CLUMondo	$3.56e^{-3}$		
LCM LR	$3.35e^{-3}$	$6.59e^{-3}$	$6.59e^{-3}$
LCM SW	$1.75e^{-3}$	$5.85e^{-3}$	$5.85e^{-3}$
LCM MLP	$1.38e^{-3}$	$6.58e^{-3}$	$6.58e^{-3}$
DE F=10	$1.29e^{-3}$	$5.31e^{-3}$	$5.27e^{-3}$
DE F=100	$1.29e^{-3}$	$1.65e^{-3}$	$1.17e^{-3}$
CLUMPY	$3.37e^{-4}$	$9.84e^{-4}$	$3.39e^{-4}$

the default value of this parameter). This finding is consistent with the fact that Dinamica EGO’s pruning procedure performs a sharp truncation of the probability density (section A.3.2.1).

We can also repeat the allocation step in the combined calibration - estimation / allocation process. We have chosen to run it 100 times (last column of Table A.1), and average over these various runs in order to improve the precision of the estimation of the transition probability distribution. This is ineffective for LCM ($\varepsilon_{\text{tot}, 100} = \varepsilon_{\text{tot}}$), consistently with the fact that LCM allocation procedure is deterministic (see section A.3.2.1). For Dinamica EGO, the improvement is marginal, which reflects the error due to pruning. On the other hand, CLUMPY displays a significant improvement and comes very close to the value obtained for the calibration-estimation comparison: $\varepsilon_{\text{calib}} \approx \varepsilon_{\text{tot}, 100}$. The allocation error itself has become negligible compared to the calibration-estimation one, consistently with the fact that our allocation algorithm is bias-free.

A.5.3. ONE-DIMENSIONAL CUT COMPARISON

The graphs of the estimated distributions $\hat{P}(v|u, \mathbf{y})$ and $\tilde{P}(v|u, \mathbf{y})$ returned by the different models are visible in Fig. A.2. This shows a one-dimensional cut at fixed altitude and slope, while using the shortest distance to existing urban areas as abscissa. The exact transition probability, computed from $P^*(v|u, \mathbf{y})$, Eq. (A.4), is also represented. There are two subplots, one for calibration-estimation comparisons [$\hat{P}(v|u, \mathbf{y})$], one for global calibration / allocation comparisons [$\tilde{P}(v|u, \mathbf{y})$]. Although this is a limited and local comparison, this exemplifies the behavior of each of the model-building algorithms.

Fig. A.2a represents the transition probabilities estimated by the calibration-estimation comparison process. We observe a large disparity in the obtained es-

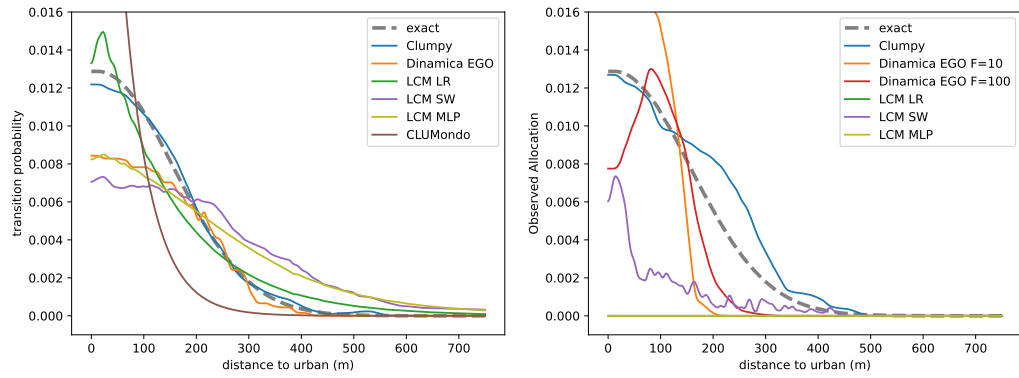
timations. It seems fair to say that the existing modeling environments fail to represent in an accurate way the exact probability distribution, although it was chosen to be relatively smooth. The dispersion in these results must clearly contribute to the problem discussed in introduction, a point that will be quantified more precisely in the future. Our own modeling environment (CLUMPY) performs very well, comparatively and in absolute.

Dinamica EGO displays a significant deviation from the exact curve which can be traced back to the assumption of independence of explanatory variables and to the pruning process (a point we will discuss in more detail elsewhere). LCM SW and LCM MLP deviate even more significantly, especially in regions where there have been very few, if any, observed transitions. Finally, CLUMondo and LCM LR follow the trend of the 'exact' curve. This is somewhat coincidental as these models are parametric, i.e., they perform a logistic regression for a specific type of curve, which is by chance similar to the one adopted here. Had we used a more complex probability distribution dependence on $\mathbf{y}$ instead, e.g., a bimodal distributions with two peaks, the result of LCM LR and CLUMondo would have been much less convincing (we checked this point).

Fig. A.2b represents the transition probabilities estimated from the whole calibration-estimation / allocation comparison process. Let us point out that this is a single run (no average over a series of allocation for the same time step is performed), which by definition presents a significant statistical noise (section A.5.2) as this noise adds up at every stage of the whole process. This being said, once again, CLUMPY performs significantly better than the other modeling environments. We can observe very clearly the differences produced by the pruning parameter and the assumption of statistically independent explanatory variables for Dinamica EGO (we will quantify elsewhere the respective importance of these two sources of bias, which are usually the most significant). The $F = 10$ (orange line) corresponds to the default value of this pruning parameter and produced a substantial bias in this example. We also notice that LCM LR and LCM MLP do not perform any allocation on this cut. Indeed, the LCM pruning method only selects the exact number of pixels to be transited (section A.3.2.1). The pixels with the highest probability $\hat{P}(\mathbf{y}|u, v)$ being obviously not on this slice, no transition is observed, which is a very clear illustration of the bias involved in this allocation method.

A.5.4. EXECUTION TIME COMPARISON

Finally, we focus on the calculation times of the different models. Fast computations are important to be able to perform a sufficient number of simula-



(A) Calibration-estimation comparison process (B) Global calibration/allocation comparison process (for a single allocation run)

FIGURE A.2: Transition probability one-dimensional cut from agricultural to urban areas with respect to distance to existing urban areas, with elevation set to 300 m and slope set to 2° .

TABLE A.2: Computation time of the different modeling environments for the case study of A.5.1, and for the calibration-estimation and global calibration/allocation comparison processes (total column) of section A.4. DE = Dinamica EGO with two different pruning factors ($F = 10$ and 100).

model	Calibration Estimation	Total
CLUMondo	45 sec	45 sec
LCM MLP	2 min 40 sec	3 min 18 sec
LCM SW	19 min 15 sec	19 min 53 sec
LCM LR	2 min 40 sec	3 min 18 sec
DE F=10	32 sec	35 sec
DE F=100		38 sec
CLUMPY	6 sec	11 sec

tions of the same problem (this is never done in LUCC modeling, but would in fact be required to extract meaningful statistical information, in agreement with the probabilistic and Markovian nature of projections). This would also allow the user to perform sensitivity analyses (something which again is never attempted).

The results are summarized in Table A.2. First we point out that these results do not do justice to the major computational advantage of CLUMPY on very large problems (tens or hundreds of millions of pixels and several LUCC transitions), where it outperforms all other modeling environments by a factor of at least 100 in computational time. However, for large problems, some of the methods evaluated do not even converge in 24h, while CLUMPY converges in a matter of minutes, which is why we chose a small enough problem, in order

for the comparison to be possible.

We still obtain a reasonable numerical efficiency for CLUMPY compared to the other models. Note that the KDE parameter q , which is set to 51 here linearly influences the calibration-estimation time. Having a lower value of q speeds up the process but increases the error in the estimated probability distribution. Also, the allocation algorithm presented in section A.3.2 implies to recompute $p(\mathbf{y}|u)$ frequently enough to obtain a bias-free allocation, and this also introduces a computation time penalty.

CLUMPY is always more efficient in computing dynamic distance maps than its closest competitor, Dinamica EGO. The larger the problem, the larger the gap in efficiency (CLUMPY is approximately quadratically more efficient than Dinamica EGO with increasing problem size). Conversely, the need to recompute probability distributions frequently enough during the allocation step (in order to ensure an unbiased allocation) is always a computation time penalty for CLUMPY. In fact, Dinamica EGO would greatly benefit from a change in its algorithm of dynamic distance updating (such as the python function `SCIPY.NDIMAGE` used by CLUMPY).

A.6. CONCLUSIONS

This paper introduces a new spatially explicit statistical LUCC modeling environment, CLUMPY. This environment is based on sound theoretical considerations, and is numerically efficient. In particular, we will show explicitly in a series of papers under preparation that our patch-oriented probabilistic formulation of LUCC state transitions is formally correct, and that our algorithmic implementations of these theoretical bases is bias-free. We will also present an investigation strategy that allowed us to identify the sources of biases and errors in existing software in a systematic way, and correct them in our new modeling environment. This endeavor is designed to help reduce the differences of behavior between existing LUCC modeling environments on a given problem and set of data pointed out in the introduction, and to provide at least one such environment where remaining errors (mostly due to statistical noise) are under strict control and can be precisely quantified.

In the process, we have introduced a new calibration strategy inspired from the large body of work performed on density evaluation in the machine-learning community. Our implementation of this strategy produces significant improvements in the precision of the calibration process, in comparison to existing calibration methods. We have also used a new, bias-free, patch-allocation algo-

rithm. This couple of calibration-allocation procedures is always unbiased and substantially more precise than existing ones. It is more efficient in terms of computational time on small (millions of pixels) problems, and significantly faster (up to ~ 100 times) than existing software on large or very large problems (tens to hundreds of millions of pixels).

We have finally proposed an evaluation method in section A.4 allowing us to perform effective comparisons of the performances of various modeling environments, including ours. This constitutes a first step towards a systematic validation procedure for LUCC models. This method takes advantage of the fact that it is both more relevant and more efficient to compare models in explanatory variable space rather than in physical space. Indeed, LUCC calibration data are often undersampled, by necessity, and the type of LUCC models analyzed here is statistical in nature. Both features imply that trying to reproduce transition locations exactly in physical space is often essentially impossible and misleading. Instead, one should focus on reproducing the correct probability structure in explanatory variable space, and, to a lesser extent, in patch parameter space (patch characteristics have not yet been seriously characterized in existing LUCC modeling environments).

All these points will be elaborated upon in detail in our forthcoming papers.

A.7. APPENDICES

A.7.1. CASE STUDY TRANSITION PROBABILITY FUNCTION

The parameters used to define the function $P^*(v|u, \mathbf{y})$ in Eq. (A.4) are the following:

$$\mu = (150, 0, 0) \quad (\text{A.5})$$

$$\Sigma = \begin{pmatrix} 25^2 & 843 & 325 \\ 843 & 10^2 & -16.3 \\ 325 & -16.3 & 10^2 \end{pmatrix} \quad (\text{A.6})$$

$$\mathcal{D} = \{\mathbf{y} \mid 0 \leq y_0 \leq 616, 0 \leq y_1 \leq 15, 0 \leq y_2 \leq 60\} \quad (\text{A.7})$$

A.7.2. MODELS PARAMETERS

In section A.5, we use various LUCC models with the following parameters.

A.7.2.1. DINAMICA EGO

We have used version 5.2.1 and all the calculations were performed on a single CPU. The binning parameters are the following. The parameter *increment* is fixed at 15 meters for the elevation, 5° for the slope and 10 meters for the distance to urban areas. The *minimum delta*, the *maximum delta* and the *tolerance angle* are respectively fixed at 50, 500, 000 and 5.0 for all explanatory variables.

A.7.2.2. IDRISI LCM

We have an Idrisi Selva license, which is relatively old (17.00). The estimation by logistic regression is done without sampling. The parameters of SimWeight are the default ones with notably the sample size fixed at 1000. All the default parameters of MLP are kept.

A.7.2.3. CLUMONDO

We have used version 1.4.0. The sampling parameter is fixed to 30% of all observations. The *number of cells distance between samples* is fixed to 2 with no data values excluding and balanced sample enabled.

A.7.2.4. CLUMPY

The KDE parameter q (section A.3.1.2) is fixed to 51. This is the only user-defined parameter in CLUMPY. This default value should be appropriate for most applications.

APPENDIX B | SUPPLEMENTARY MATERIAL OF CHAPTER 2

This supplementary material presents in some detail the architecture and algorithmic implementation of the fit and estimation procedure discussed in main text.

B.1. KERNEL DENSITY ESTIMATION IMPLEMENTATION

The paper presented in a general way a KDE approximation method (eKDE) that starts from the idea of finely binarizing the studied space. The fit and estimation steps have been introduced and we reproduce here the algorithm outline in order to allow the reader who wishes to do so to finely understand and reproduce the method. To help understanding, we represent in figure B.1 the call tree of the functions. The main key to obtaining a numerically efficient implementation lies in the way we store the sparse matrices of the observed bins during the fitting step (Alg. 4) and during the estimation step (Alg. 5). Indeed, we extend the principle of Compressed Sparse Row (also called Yale Format) to more than 2 dimensions (Alg. 8). Thus, it is then efficient to explore (Alg. 11) these sparse matrices using simple binary searches (Alg. 13). This method is implemented in an open-source python module called EKDE¹.

¹<https://github.com/fmazy/ekde>

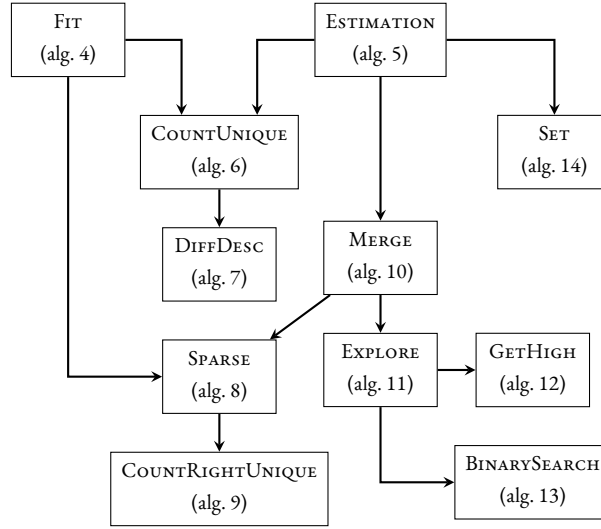


FIGURE B.1: Call tree of all the functions involved in the implementation of the eKDE method. An arrow means that the function at the origin of the arrow calls function at the arrow point.

Algorithm 4: FIT function involved in the KDE implementation. It is an algorithm outline. This is the main algorithm of the fitting step described in the main text.

Input: $\tilde{X}, h, s, q$

- 1 $\forall j < d, a_j = \min_{i < n}(x_{i,j})$
 - 2 $\forall i, j, \hat{x}_{i,j} = \text{FLOOR}\left(\frac{\tilde{x}_{i,j} - a_j}{hs/q}\right)$
 - 3 $\hat{X} := \text{SORT}(\hat{X})$
 - 4 $\hat{X}^*, \mu, \hat{X}_\downarrow = \text{COUNTUNIQUE}(\hat{X})$
 - 5 $\mathbb{S} := \text{SPARSE}(\hat{X}^*, \hat{X}_\downarrow)$
 - 6 **return** $\mathbb{S}, \mu, a$
-

Algorithm 5: ESTIMATE function involved in the KDE implementation. It is an algorithm outline.

Input: $\tilde{Y}, \hat{X}^*, \mu, X, h, s, q, \mathbf{a}$.

```

1  $\forall i, j, \hat{y}_{i,j} = \text{FLOOR} \left( \frac{\tilde{y}_{i,j} - a_j}{hs/q} \right)$ 
2  $\hat{Y} := \text{SORT}(\hat{Y})$ 
3  $\hat{Y}^*, \_, \hat{Y}_{\downarrow} := \text{COUNTUNIQUE}(\hat{Y})$  (the second return is not kept)
4  $\mathbf{g} := \text{MERGE}(\mathbb{S}, \mu, Y, \hat{Y}_{\downarrow}, q)$ 
5  $\mathbf{f} := \text{SET}(\hat{Y}_{\downarrow}, \mathbf{g})$ 
6  $\mathbf{f} := \frac{1}{n(2h)^d} \mathbf{f}$ 
7  $\mathbf{f}$  is ordered in order to inverse the sort operation made at line 2
8 return  $\mathbf{f}$ 

```

Algorithm 6: COUNTUNIQUE function involved in the KDE implementation. It is an algorithm outline.

Input: A of size (n, d) .

```

1  $A = \text{SORT}(A)$ 
2  $B = \text{DIFFDESC}(A)$ 
3 Let  $A^*$  be an empty array of  $d$  columns
4 Let  $\mu$  be an empty vector
5  $i := 0$ 
6 while  $i < n$  do
7   append  $a_i$  to  $A^*$ 
8   append  $b_{i,d-1}$  to  $\mu$ 
9    $i := i + b_{i,d-1}$ 
10 return  $A^*, \mu, B$ 

```

Algorithm 7: DIFFDESC function involved in the KDE implementation. It is an algorithm outline.

Input: A of size (n, d) (sorted).

```

1 Let  $B$  be an array of shape  $(n, d)$  full of 1
2 for  $i \in \{n-2, n-3, \dots, 1, 0\}$  do
3   for  $j \in \text{RANGE}(d)$  do
4     if  $a_{i,j} = a_{i+1,j}$  then
5       if  $j = 0$  then
6          $b_{i,j} := b_{i+1,j} + 1$ 
7       else if  $b_{i,j-1} > 1$  then
8          $b_{i,j} := b_{i+1,j} + 1$ 
9 return  $B$ 

```

Algorithm 8: SPARSE function involved in the KDE implementation. It is an algorithm outline.

Input:

$\mathbf{A}$, an array of size (n, d) (sorted)
 $\mathbf{B}$ its DIFFDESC array.

```

1  $C := \text{COUNTRIGHTUNIQUE}(\mathbf{B})$ 
2 For all  $j < d$ , let  $\mathbf{S}^j$  be an empty array of shape  $(n, 2)$ 
3  $\forall i < n$ ,  $s_{i,0}^{d-1} = a_{i,d-1}$  and  $s_{i,1}^{d-1} = i$ 
4  $\forall j < d - 1$ ,  $s_{0,0}^j = a_{0,j}$  and  $s_{0,1}^j = 0$ 
5 for  $j \in \text{RANGE}(d - 1)$  do
6    $i_a := 0$ 
7    $i_s := 0$ 
8   while  $i_a < n$  do
9      $s_{i_s,1}^j := s_{i_s-1,1}^j + c_{i_a,j}$ 
10     $i_a := i_a + b_{i_a,j}$ 
11     $s_{i_s,0}^j := a_{i_a,j}$ 
12  $\mathbb{S} := \{\mathbf{S}^0, \mathbf{S}^1, \dots, \mathbf{S}^{d-1}\}$ 
13 return  $\mathbb{S}$ 

```

Algorithm 9: COUNTRIGHTUNIQUE function involved in the KDE implementation. It is an algorithm outline.

Input: $\mathbf{A}$, a DIFFDESC array of size (n, d) (sorted).

```

1 Let  $\mathbf{B}$  be an array of shape  $(n, d - 1)$ 
2 for  $j \in \text{RANGE}(d - 1)$  do
3    $s := 0$ 
4   for  $i \in \{n - 1, n - 2, \dots, 1, 0\}$  do
5     if  $a_{i,j} = 1$  then
6        $s := 0$ 
7     if  $a_{i,j+1} = 1$  then
8        $s := s + 1$ 
9      $b_{i,j} := s$ 
10 return  $\mathbf{B}$ 

```

Algorithm 10: MERGE function involved in the KDE implementation. It is an algorithm outline.

Input:

$\mathbb{S} = \{\mathbf{S}^j, \forall j < d\}$, the sparse collection of unique observed elements,
 μ the weights vector of size n ,
 $\mathbf{Y}$, the array to evaluate of shape (m, d) (sorted)
 $\hat{\mathbf{Y}}_{\downarrow}$, DIFFDESC array of $\mathbf{Y}$,
 q , the binarization parameter.

```

1  $\mathbb{T} := \text{SPARSE}(\mathbf{Y}, \mathbf{Y}_{\downarrow})$ 
2 Let  $\mathbf{f}$  be a vector of size  $n$  full of 0
3  $a_s := 0$ 
4  $b_s := \# \mathbf{S}^j$ 
5 for  $i_t \in \text{RANGE}(\# \mathbf{T}^0)$  do
6    $\mathbf{f}, a_s := \text{EXPLORE}(\mathbb{S}, \mu, \mathbb{T}, \mathbf{f}, j = 0, a_s, b_s, i_t, q)$ 
7 return  $\mathbf{f}$ 

```

Algorithm 11: EXPLORE function involved in the KDE implementation (box kernel case). It is an algorithm outline.

Input:

$\mathbb{S} = \{\mathbf{S}^j, \forall j < d\}$, the sparse collection of unique observed elements,
 μ , the weights of unique observed elements
 $\mathbb{T} = \{\mathbf{T}^j, \forall j < d\}$, the sparse collection of unique elements to estimate,
 $\mathbf{f}$, the density vector being completed,
 j , the current column,
 a_s , the low index bound for $\mathbf{S}^j$,
 b_s , the high index bound for $\mathbf{S}^j$,
 i_t , the current index in $\mathbf{T}^j$,
 q , the binarization parameter.

```

1  $a_t := T_{i_t,1}^j$ 
2  $b_t := \text{GETHIGH}(\mathbb{T}, j, i_t)$ 
3  $a_s := \text{BINARYSEARCH}(\mathbf{S}^j, t_{i_t,0}^j - (q-1)/2, a_s, b_s, \text{"left"})$ 
4  $b_s := \text{BINARYSEARCH}(\mathbf{S}^j, t_{i_t,0}^j + (q-1)/2, a_s, b_s, \text{"right"})$ 
5 if  $j < d - 1$  then
6   for  $i_s \in \text{RANGE}(a_s, b_s)$  do
7      $a'_s := s_{i_s,1}^j$ 
8      $b'_s := \text{GETHIGH}(\mathbb{S}, j, i_s)$ 
9     for  $i'_t \in \text{RANGE}(a_t, b_t)$  do
10       $\mathbf{f}, a'_s := \text{EXPLORE}(\mathbb{S}, \mu, \mathbb{T}, \mathbf{f}, j + 1, a'_s, b'_s, i'_t, q)$ 
11 else
12   for  $i_s \in \text{RANGE}(a_s, b_s)$  do
13      $f_{i_t} := f_{i_t} + \mu_{i_t}$ 
14 return  $\mathbf{f}, a_s$ 

```

Algorithm 12: GETHIGH function involved in the KDE implementation.

Input:

$\mathbb{S} = \{\mathbf{S}^j, \forall j < d\}$, the sparse collection of unique observed elements,
 j , the current column index,
 i , the current row index.

```

1 if  $i < \#\mathbf{S}^j - 1$  then
2   return  $s_{i+1,1}^j$ 
3 else
4   if  $j < d - 1$  then
5     return  $\#\mathbf{S}^{j+1}$ 
6   else
7     return  $s_{i,1}^j + 1$ 

```

Algorithm 13: BINARYSEARCH function involved in the KDE implementation. It is an algorithm outline.

Input:

$\mathbf{R}$, an array of shape $(n, 2)$ (sorted)
 x the target,
 a the lower index bound ($a \geq 0$),
 b the upper index bound ($b \leq n$)
 $side \in \{\text{"left"}, \text{"right"}\}$ the binary search side.

```

1 if  $a \geq b$  then
2   if  $left\ side$  then
3     return  $a$ 
4   else
5     return  $b$ 
6 if  $x < r_{a,0}$  then
7   return  $a$ 
8 if  $x > r_{b-1,0}$  then
9   return  $b$ 
10 while  $a < b$  do
11    $m := \text{FLOOR}\left(\frac{a+b}{2}\right)$ 
12   if  $r_{m,0} > x$  then
13      $b := m$ 
14   else
15      $a := m + 1$ 
16 return  $m$ 

```

Algorithm 14: SET function involved in the KDE implementation.

Input:

$\hat{\mathbf{Y}}_{\downarrow}$, the DIFFDESC of shape (n, d)
 $\mathbf{g}$, the density estimation of unique rows.

```

1 Let  $\mathbf{f}$  be an empty vector of size  $n$ 
2  $i_g := 0$ 
3 while  $i < n$  do
4   for  $k \in \text{RANGE}(y_{\downarrow, i, d-1})$  do
5      $f_{i+k} := g_{i_g}$ 
6    $i_g := i_g + 1$ 
7    $i := i + y_{\downarrow, i, d-1}$ 

```

APPENDIX C | RÉSUMÉ DÉTAILLÉ EN FRANÇAIS

Cette annexe répond aux contraintes imposées aux thèses de l'*Université Grenoble Alpes* rédigées en langue anglaise en proposant une description de son contenu en français. Il ne s'agit pas de proposer une traduction intégrale de la thèse mais bien d'apporter assez de détails pour en comprendre les éléments et résultats principaux. Dans cette perspective, j'ai fait le choix de traduire une grande partie du chapitre d'introduction qui permettra au lecteur francophone de saisir les enjeux et les éléments constitutifs de cette thèse (section C.1). Puis, une traduction des abstracts de chacun des chapitres suivant permet de connaître leurs résultats principaux (sections C.2, C.3, C.4 et C.5). Enfin, une traduction de la conclusion clôt cette annexe et permet de comprendre les perspectives ouvertes par ce travail de thèse (section C.6).

C.1. INTRODUCTION

Une partie significative de l'introduction (chapitre 1) est traduite ici.

Tout d'abord, définissons plus précisément le sujet central de cette thèse. Un changement d'utilisation et d'occupation des sols (*Land Use and Cover Change*, soit LUCC en anglais) est la modification de l'environnement d'un lieu. Il peut être le résultat direct ou indirect de l'activité humaine à un endroit donné. Deux grandes causes historiques et mondiales sont la déforestation pour l'agriculture, et la perte de zones naturelles ou agricoles au profit du développement urbain. L'utilisation des sols se distingue de l'occupation des sols, qui est un concept plus générique qui ne se limite pas aux activités humaines. Ainsi, des causes naturelles peuvent provoquer un changement dans l'occupation des sols, alors que

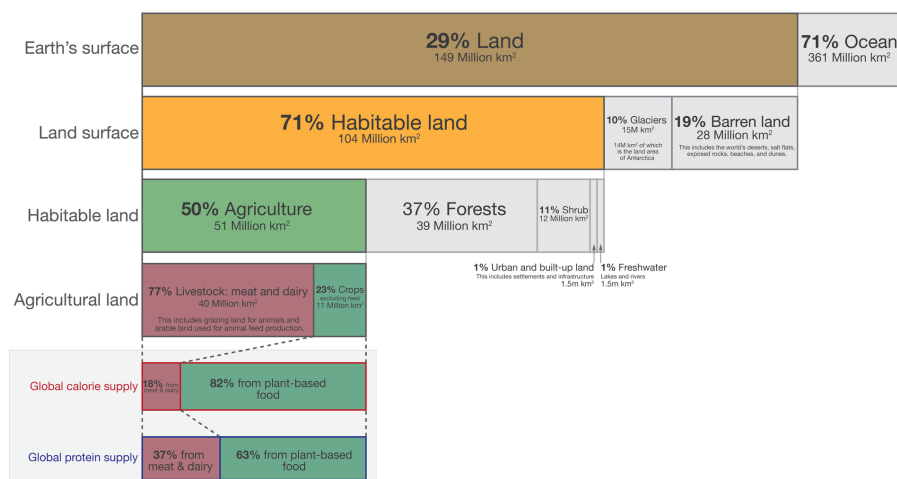


FIGURE C.1: Distribution mondiale de l'utilisation et de la couverture des sols, avec un accent sur l'agriculture. Source des données : Organisation des Nations unies pour l'alimentation et l'agriculture (FAO). Figure sous licence CC-BY par les auteurs Hannah Ritchie et Max Roser (2019).

seule l'intervention humaine peut être une source de changement dans l'utilisation des sols. Dans cette thèse, j'utilise principalement le terme général d'utilisation et de couverture des sols (*Land Use and Cover*, soit LUC en anglais).

Dans l'histoire de l'humanité, le monde était principalement constitué de forêts, de prairies et de montagnes. Cet état de fait a changé radicalement au cours du 20^e siècle, lorsque plus de 50% de toutes les terres ont commencé à être affectées et modifiées par l'homme. Aujourd'hui, si l'on exclut les glaciers et les terres stériles (déserts, salines, rochers nus, plages et dunes), la surface consacrée à l'agriculture représente 50% des terres (Fig. C.1). Il reste donc 37% pour la forêt, 11% pour la brousse, 1% pour l'eau douce et 1% pour les terres urbanisées. Cette dernière quantité peut paraître étonnamment faible et est généralement surestimée par les non-experts. Celle-ci représente 1,5 million de km². Ensuite, si l'on descend encore d'un cran dans le détail au sein des terres agricoles, on constate un grand déséquilibre : plus des trois quarts de la surface sont dédiés à la production de viande et de produits laitiers (y compris la production d'aliments dédiés aux animaux) et le quart restant est affecté aux produits alimentaires consommés par l'homme. Nous pourrions encore affiner : au sein de la surface agricole dédiée à l'alimentation humaine, quelle est la part des grandes cultures, des arbres fruitiers ou du maraîchage ?

L'introduction de la thèse a pour but de poser le contexte en abordant la notion de changement d'usage et de couverture des sols depuis le cadre général jusqu'aux méthodes de modélisation et leurs applications. Ainsi, je rappelle d'abord l'interaction entre changement global et LUCC (section 1.1). Ensuite, je présente la modélisation des LUCC à travers les applications et les principes

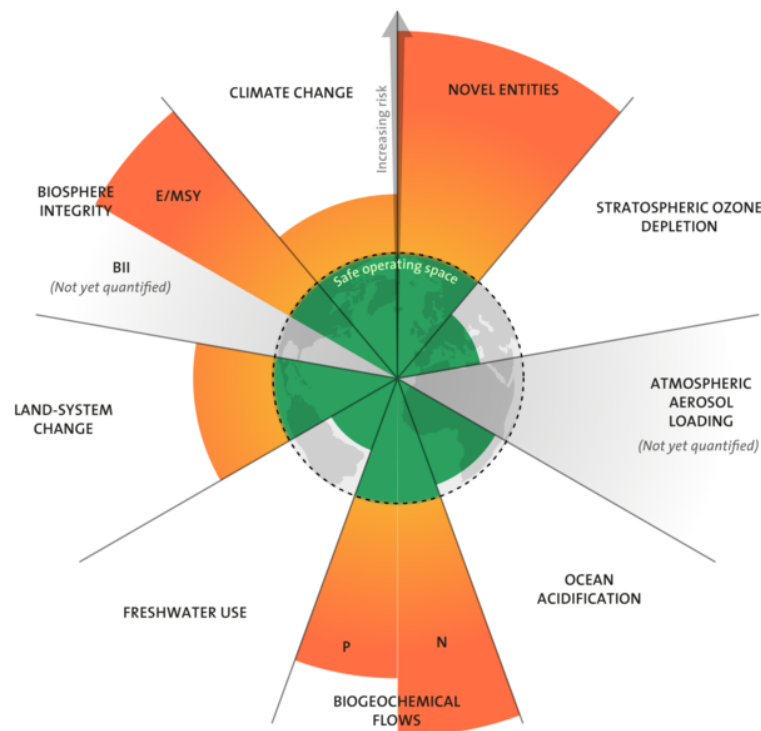


FIGURE C.2: Diagramme des limites planétaire. Le seuil normalisé à ne pas dépasser est en pointillé. Les coins en dessous de ce seuil sont représentés en vert tandis que ceux au-dessus sont en orange. Données mises à jour en 2022. Source de la figure : Stockholm Resilience Centre¹ – CC-BY-4.0.

fondamentaux de ces modèles (section 1.2). Je présente ensuite plus spécifiquement les modèles statistiques et spatialement explicites de LUCC qui font l'objet de cette thèse (section 1.3). Enfin, les objectifs de la thèse sont énoncés ; je détaille ma contribution personnelle dans les articles reproduits dans ce manuscrit et je décris la structure de cette thèse (section 1.4).

C.1.1. CHANGEMENT D'USAGE ET DE COUVERTURE DES SOLS ET CHANGEMENTS GLOBAUX

L'impact de l'homme sur son environnement est si important qu'il met désormais en péril les conditions de la vie humaine telle que nous la connaissons (IPCC, 2022). Rockström et al. (2009) ont introduit la notion de limites planétaires dans laquelle ils décrivent neuf seuils à ne pas dépasser pour pouvoir vivre durablement dans un écosystème sûr en évitant les modifications rapides et non réversibles de l'environnement à l'échelle planétaire (Fig. C.2). Ce concept de limites planétaires a été mis à jour depuis (Wang-Erlandsson et al., 2022). Parmi ces limites figure la modification de la LUC. Son niveau de charge est déterminé par le taux d'occupation des forêts intactes, qui devrait être supérieur à 75% à

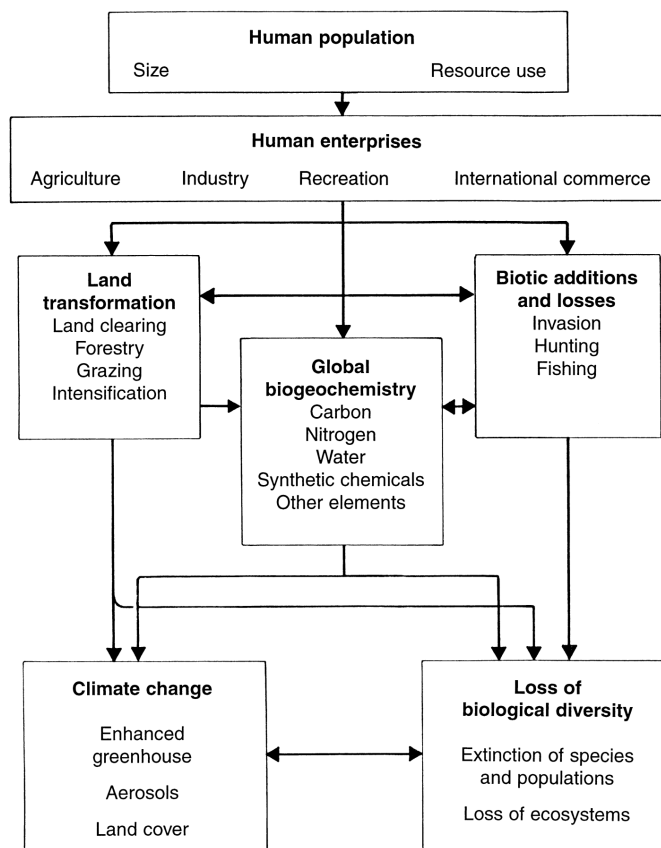


FIGURE C.3: Les effets directs et indirects de l'humanité sur le système terrestre. Modèle conceptuel et figure fournis par Vitousek et al. (1997).

l'échelle mondiale (Steffen et al., 2015). Cette limite est affinée en distinguant les forêts boréales (limite de 85%), les forêts tempérées (limite de 50%) et les forêts tropicales (limite de 85%). Steffen et al. (2015) estime à la date de publication que le taux global de forêts intactes est de 62%, en dessous de la limite mondiale.

En outre, il n'existe pas de lien de causalité simple entre les usages et les couverture des sols et les changement globaux (section 1.1.1) sachant que les changement globaux ont également un impact sur les usages et les couvertures des sols (section 1.1.2). Des boucles de rétroaction peuvent ainsi se produire.

C.1.1.1. UN MOTEUR DE CHANGEMENT GLOBAL

Un changement global est un phénomène d'origine anthropique — c'est-à-dire causé par les activités humaines — qui a un impact significatif sur le système terrestre. Les deux principales sources de changement global sont le réchauffement climatique et la perte de biodiversité. Bien qu'aucun lien direct ne puisse être établi entre les activités humaines et ces deux résultats, des facteurs de changement ont été identifiés, tels que l'altération des cycles biogéochimiques (cycles du carbone, de l'eau et de l'azote), le déséquilibre des biotiques ou la modifica-

tion des usages et des couvertures des sols. Vitousek et al. (1997) propose un modèle schématique des liens directs et indirects des activités humaines sur ces moteurs de changement (Fig C.3). Il est difficile de mesurer l'impact du LUCC sur le climat ou la perte de biodiversité. Néanmoins, malgré les incertitudes que l'on peut identifier, les LUCC ont un impact important et certain sur ces changements globaux (Vitousek et al., 1997; IPCC, 2019).

Nous pouvons rapidement distinguer trois types d'impact du LUCC, discutés plus en détail dans l'introduction de cette thèse : l'action sur le climat local, les émissions de gaz à effet de serre (GES) et les effets sur la biodiversité locale.

C.1.1.2. IMPACT DU CHANGEMENT GLOBAL SUR L'USAGE ET LA COUVERTURE DES SOLS

Les changements d'usage et de couverture des sols est le résultat d'une longue histoire d'adaptation et de développement de l'espèce humaine, notamment en ce qui concerne les conditions climatiques localement dominantes (Stephens et al., 2019). Cependant, le changement global actuel est disproportionnellement plus rapide que par le passé. En effet, alors que les changements climatiques précédents correspondent à des cycles relativement longs (des milliers à des centaines de milliers d'années), nous sommes aujourd'hui confrontés à un changement global sur une ou deux centaines d'années seulement. Cette rapidité accentue les effets sur les espèces végétales et animales, ainsi que sur les sociétés humaines.

Le changement global a un impact sur la production agricole à l'échelle régionale (Wheeler and von Braun, 2013) et induit des changements dans l'habitabilité des écosystèmes et des modifications de la LUC (Hulme et al., 1999; Pecl et al., 2017). Ainsi, une distinction est faite entre les impacts positifs et négatifs, même si, dans l'ensemble, les impacts négatifs sont dominants (Jia et al., 2019).

C.1.2. MODÉLISER LES CHANGEMENTS D'USAGE DES SOLS

La section précédente était dédiée aux interactions bidirectionnelles entre le changement global et les LUC. Cette observation est la toile de fond de nombreuses études et analyses des changements d'usage et de couverture des sols. En effet, de nombreux domaines de recherche peuvent être concernés par ce type de phénomène. De plus, dans de nombreux cas, il ne s'agit pas seulement d'analyser et de déterminer les moteurs des LUCC passée à l'aide de cartes d'état d'usage et de couverture des sols (Meyfroidt et al., 2018), mais aussi d'interroger les changements futurs à travers divers scénarios (Geist et al., 2006). Aussi, certaines applications simulent des cartes LUCC future, projection tangible des

scénarios étudiés par le modélisateur.

C.1.2.1. APPLICATION DES MODÈLES

Nous ne nous intéressons pas aux LUCC simplement pour le phénomène lui-même, mais surtout pour les raisons qui guident ces changements et leurs implications. Ainsi, les modèles LUCC sont à considérer comme un outil permettant de répondre à diverses questions de recherche qui se situent pour la grande majorité au carrefour des sciences de l'environnement et de la géographie. En particulier, une grande majorité des applications des modèles LUCC traitent d'une manière ou d'une autre de l'adaptation des sociétés et des écosystèmes aux changements globaux.

En sciences de l'environnement, il est très courant de s'appuyer sur des scénarios pour guider les analyses et les modélisations (Verburg et al., 2016). Ces scénarios correspondent à des situations que l'utilisateur souhaite évaluer à la lumière de sa question de recherche. Il est d'abord nécessaire de les traduire dans le cadre du modèle, puis de produire les simulations associées. La construction d'un scénario n'est pas une tâche triviale et sa conception et sa transposition dans un modèle LUCC vont évidemment influencer les résultats. Afin d'éviter ces écueils, Escobar et al. (2018) propose un guide de construction de scénarios qui tend à standardiser leur conception.

En géographie, l'utilisation de cartes est monnaie courante. C'est un domaine qui a particulièrement bénéficié de la démocratisation des outils informatiques à travers le développement des logiciels de systèmes d'information géographique (SIG).

Enfin, face à des enjeux globaux, de meilleures décisions d'aménagement du territoire peuvent être prises en matière de LUCC grâce à des politiques dédiées à ce sujet (Reid et al., 2006). Les modèles LUCC peuvent être des outils d'aide à la décision très intéressants en raison de leurs résultats de simulation faciles à appréhender, notamment par le biais de cartes des états futurs probables de la LUC pour un scénario donné (Koomen and Borsboom van Beurden, 2011). Les décideurs ainsi que les citoyens sont aujourd'hui très friands d'outils interactifs avec lesquels il est possible de faire varier les paramètres de modélisation rapidement et visuellement (Voinov et al., 2016). Il s'agit souvent de comparer différents scénarios d'aménagement contrastés pour illustrer les phénomènes et ainsi aider à la prise de décision, que ce soit à l'échelle locale ou globale (Ornetsmüller et al., 2016).

C.1.2.2. CLASSIFICATION DES MODÈLES LUCC

Depuis la fin du 20^{ème} siècle et l'accessibilité plus facile aux outils informatiques, de nombreux modèles (assistés par ordinateur) des LUCC ont été développés pour répondre à de nombreuses questions de recherche (Sec. 1.2.1). Parmi eux, certains se concentrent sur les volumes globaux de changement et permettent ainsi de modéliser les attributions d'usage et de couverture des sols sur une zone entière de manière globale. Cependant, ces modèles n'ont pas de considérations spatiales explicites, c'est-à-dire qu'ils n'interagissent pas avec les cartes LUC mais seulement avec des tableaux spatialement non localisés (ou grossiers). Dans cette thèse, je me concentre sur les questions qui impliquent explicitement une spatialisation du problème étudié et qui constituent un champ de recherche et d'innovation logicielle (Verburg et al., 2006). Ainsi, j'exclus les modèles qui n'ont pas de dimension spatiale et je me concentre uniquement sur les modèles spatialement explicites.

On observe aujourd'hui une grande diversité dans ces modèles au point qu'il n'est pas aisé d'en constituer une taxonomie, d'autant que certains d'entre eux reposent sur des approches radicalement opposées ou des méthodes hybrides. National Research Council (2014) a proposé cinq catégories de modèles LUCC spatialement explicites (plus une catégorie de modèles hybrides) en fonction du paradigme général qui les définit :

1. Modèles Pattern-Based (PBM). Il s'agit d'une approche statistique (qui peut être similaire à l'apprentissage automatique dans d'autres contextes) qui est calibrée sur les LUCC passés en fonction de variables explicatives.
2. Automates cellulaires (AC). Ici, le modèle reçoit des cartes d'adéquation pour chaque LUC, des paramètres de voisinage et un volume de transition global à atteindre. Les pixels suivent un processus d'automate cellulaire jusqu'à ce qu'ils atteignent les surfaces transitionnelles attendues.
3. Modèles économiques sectoriels. On considère que la LUC est soumise au marché économique de l'offre et de la demande. Ce type de modèle est particulièrement pris en compte dans les échanges interrégionaux.
4. Approche économique désagrégée spatialement. Ici, nous considérons la LUC comme un équilibre économique qui est perturbé par des phénomènes causaux.
5. Modèles d'agents. Chaque acteur économique lié à ou influençant les changements d'usage et de couverture des sols est représenté et leurs interactions entre eux et avec leur environnement sont modélisées. Ce type de

modèle s'est rapidement développé ces dernières années, bien qu'ils soient construits spécifiquement pour chaque question de recherche et ne soient pas très généralisables (O'Sullivan et al., 2016).

Selon cette classification, j'étudie dans cette thèse les modèles qui entrent dans la catégorie PBM. Cependant, comme nous le verrons plus tard, je m'intéresse également aux modèles qui sont calibrés statistiquement (PBM) mais qui simulent en allouant les transitions d'une manière qui pourrait relever de la catégorie des automates cellulaires (CA). Cette approche hybride s'avère assez courante puisqu'elle s'appuie à la fois sur les statistiques pour analyser les changements et sur les automates cellulaires pour construire des cartes allouées de manière réaliste et cohérente avec la calibration statistique (voir section 1.3.3 pour les considérations spatiales et section 1.3.5.1 pour les méthodes d'allocation).

C.1.2.3. VALIDATION ET COMPARAISON DES MODÈLES LUCC

Rétrospectivement, la multiplicité des modèles a été une source d'interrogation croissante dans la littérature, bien que le nombre d'articles consacrés à cette question soit relativement modeste. D'autant plus que les différents paradigmes qui sont à l'origine de la classification National Research Council (2014) ne sont pas nécessairement exclusifs dans leurs applications possibles, *i.e.* la même question de recherche peut souvent trouver une réponse avec deux cadres de modélisation issus de catégories différentes.

Une revue de littérature à ce sujet à la section 1.2.4 permet de souligner quelques éléments : i) la comparaison de modèles n'est pas une chose facile à faire devant la multitude de modèles ; ii) cette pratique n'est pas courante dans la communauté ; iii) souvent, la méthode de comparaison / validation n'est pas satisfaisante d'un point de vue statistique (des détails sur la précision spatiale et les modèles LUCC sont donnés dans la section 1.3.6) ; iv) les sources de différences entre les modèles spatialement explicites basés sur des motifs ne sont pas clairement identifiées. Une certaine filiation directe peut être observée entre ces différents articles et cette thèse, comme le montre la description du processus de pensée suivi dans cette étude (section 1.4.2) et des objectifs poursuivis (section 1.4.3).

C.1.3. ENVIRONNEMENT DE MODÉLISATION LUCC PATTERN-BASED

Le sujet de cette thèse est la modélisation LUCC pattern-based. Cette classe de modèles présente des caractéristiques particulièrement intéressantes. J'énumère ici quelques attributs de ces modèles : i) ils permettent de comprendre les moteurs du LUCC à l'aide d'outils statistiques plus ou moins sophistiqués qui

renvoient des cartes de probabilité de transition (voir 1.3.4) ; ii) ils traitent des cartes de LUC qui ont un attrait visuel indéniable, tant pour l'aide à la décision que pour les études écologiques (voir section 1.2.1) ; iii) elles sont basées sur des variables explicatives qui constituent une manière compréhensible d'analyser la LUCC (voir section 1.3.2) ; iv) elles ne nécessitent pas de matériel informatique spécifique ; v) elles permettent l'utilisation de scénarios dans la production de projections LUCC futures (voir section 1.3.4.3).

Il existe plusieurs environnements de modélisation de modélisation statistique et spatialement explicite des LUCC. Parmi eux, citons CLUMondo, Dinamica EGO, LCM et CLUMPY, ce dernier étant l'environnement de modélisation conçu conjointement à cette thèse (cf chapitre 5). Une attention particulière leur a été consacrée dans cette thèse puisque je compare leurs résultats sur des études de cas artificiels dans les chapitres 2 et 3.

Bien que les modèles de cette catégorie présentent des caractéristiques communes, il existe de nombreuses différences entre eux, que ce soit au niveau de la conception, des méthodes utilisées van Vliet et al. (2016) ou des résultats obtenus sur une même étude de cas (Mas et al., 2014). Il est possible d'esquisser une architecture commune à laquelle la plupart des modèles correspondent plutôt bien. Cette architecture est composée de deux modules principaux : un module de calibration-estimation et un module d'allocation (Fig. C.4). Le premier permet aux utilisateurs d'analyser les modèles LUCC passés pour déterminer (ou plutôt estimer) les probabilités de transition des pixels. Le second est un processus qui modélise les LUCC réels et permet ainsi aux utilisateurs de simuler (allouer) des cartes de LUC probables, en accord avec les probabilités retournées par le module de calibration. Il est possible de répéter cette opération d'allocation pour simuler plusieurs pas de temps consécutifs (itération).

C.1.3.1. CARTES D'USAGE ET DE COUVERTURE DES SOLS

Avant toute chose, les modèles statistiques et spatialement explicites basés sur des modèles LUCC nécessitent des cartes LUC passées et des cartes de variables explicatives (présentées dans la section 1.3.2). Les cartes d'usage et de couverture des sols sont traditionnellement présentées en format matriciel (c'est-à-dire en pixels) à des fins de modélisation des LUC.

En général, dans les environnements de modélisation existants, la procédure de calibration statistique est effectuée sur la base de deux cartes LUC de la même zone d'étude à des dates passées différentes. A titre d'illustration, un détail de deux cartes LUC est reproduit dans la figure C.5.

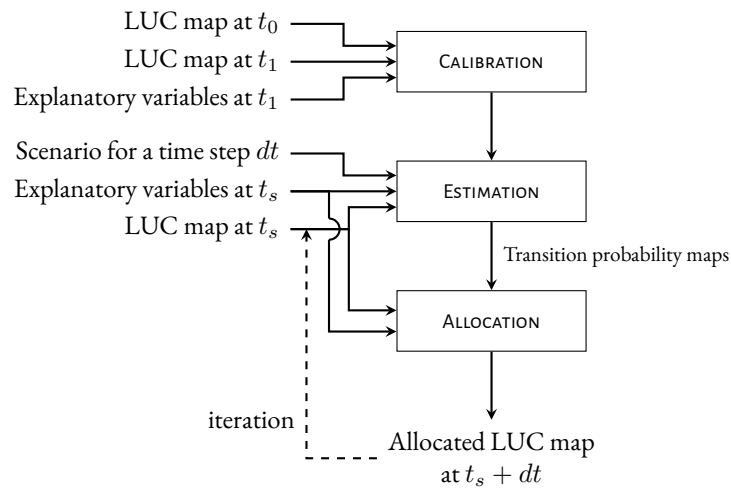


FIGURE C.4: Architecture générale d'un modèle LUCC pattern-based statistique et spatialement explicite.

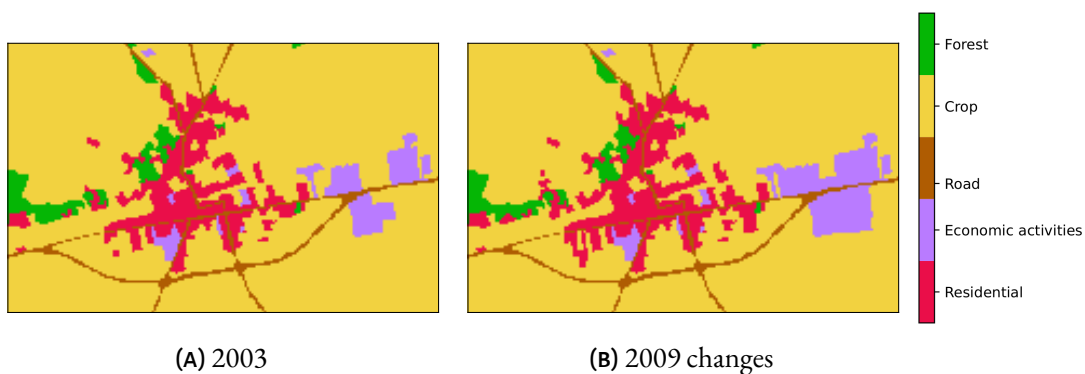


FIGURE C.5: Détail de deux cartes LUC à deux dates différentes (2003 et 2009). Les changements d'usage des sols des zones de cultures vers des zones urbaines et d'activités économiques sont mis en évidence dans la deuxième carte. Ces cartes sont utilisées dans l'étude de cas illustrative de la section 4.5.1. Elle est zoomée sur Marcillolles 38260, Auvergne Rhône Alpes, France.

C.1.3.2. VARIABLES EXPLICATIVES

Une variable explicative (*Explanatory Variable*, soit EV en anglais) est une variable associée à chaque pixel – elle peut donc être représentée sous la forme d’une carte – qui représente une quantité que l’on soupçonne d’être corrélée aux LUCC observés dans le passé (et dans le futur).

Un EV peut être continue ou discrète. Une EV continue est caractérisée par le fait que l’EV peut prendre n’importe quelle valeur réelle sur sa plage de définition. Cette continuité est souvent perceptible dans l’espace spatial où des pixels proches auront des valeurs d’EV proches. Des exemples typiques d’EV continus dans l’espace sont l’altitude, la pente ou la distance à un élément particulier du paysage (comme la route la plus proche). Il existe également des VE continues qui n’auront pas nécessairement de continuité spatiale, comme le prix des terrains et des biens immobiliers. Un EV discret fournit des informations catégorisées. Par exemple, il est courant d’utiliser des cartes de type de sol ou des cartes de régime foncier. Certaines variables discrètes (comme la carte des zones protégées) peuvent nécessiter un paramétrage spécifique du cadre de modélisation et exiger la division de la zone d’étude de cas en différentes sous-régions (voir section 1.3.4.2). Les variables discrètes sont mélangées aux variables continues (le cas échéant) et il appartient à la méthode de calibration-estimation de traiter les deux types d’EV.

C.1.3.3. CALIBRATION-ESTIMATION

La calibration-estimation est le premier des modules d’un modèle LUCC qui est analysé dans cette thèse. Dans une étude de cas réelle, les variables explicatives doivent déjà être sélectionnées avant que la calibration puisse être effectuée. La sélection des variables explicatives est abordée dans le chapitre 4.

Comme nous l’avons souligné précédemment, il est nécessaire de déterminer les probabilités de transition de chaque pixel. Pour mener à bien ce processus, il est nécessaire d’effectuer ce que l’on appelle une calibration (on parle aussi de ”calibrer le modèle”). La calibration est un processus qui consiste à collecter les valeurs des EV de calibration pertinentes et les pixels qui leur sont associés. Diverses transformations préliminaires de ces EV sont également effectuées. L’estimation est le processus fournissant des cartes de probabilité de transition. Il est possible de distinguer les modèles LUCC par la méthode de calibration-estimation qu’ils utilisent.

C’est ici qu’intervient le caractère ”statistique” des modèles de LUCC que j’étudie dans cette thèse. En effet, la méthode de calibration-estimation vise à inférer des probabilités de LUCC en fonction des changements observés sur les cartes de calibration et pour une sélection de EV. Cette approche peut être as-

similée à de l'apprentissage automatique, sans exclure des méthodes statistiques très simples. Différentes méthodes sont employées dans la littérature parmi lesquelles CA_Markov, la régression logistique, le réseau de neurones ou encore les poids d'évidence. Une nouvelle méthode est introduite dans cette thèse : Bayes-eKDE.

Ce module est donc divisé en deux sous-modules dans la Fig. C.4 car le processus d'estimation est effectué avec un scénario donné et qu'il peut être appelé sans avoir à recalibrer le modèle (notamment dans le cas d'une modélisation itérative).

C.1.3.4. SCÉNARIOS

La modélisation de scénarios est une caractéristique commune des applications des modèles LUCC (voir section 1.2.1). Le modélisateur construit un ou plusieurs scénarios contrastés pour tenter de répondre à sa question de recherche. Dans la structure du module de calibration-estimation adoptée dans cette thèse, les scénarios sont implémentés uniquement dans le sous-module d'estimation et modifient les probabilités de transition.

Souvent, le premier scénario est une extrapolation, et se base uniquement sur les changements observés sur les cartes de calibration, projetant simplement les mêmes modèles de changement dans le futur. Il est traditionnellement appelé *business as usual* et peut être considéré comme un scénario de référence. Il est généralement plus facile de construire des scénarios contrastés en modifiant à divers degrés le scénario de référence, mais ce n'est pas une nécessité.

C.1.3.5. ALLOCATION

Le module de calibration-estimation produit des cartes de probabilité de LUCC pour chaque pixel. Le module d'allocation permet aux utilisateurs de simuler des cartes d'usage et de couverture des sols réelles sur la base de ces cartes de probabilité. Au niveau du pixel, le terme "allocation d'un pixel" est parfois utilisé.

Ainsi, la carte allouée contient en quelque sorte moins d'informations que les cartes de probabilité, puisqu'elle constitue une simulation, un tirage, parmi tous les autres possibles qui conviennent tout autant. Néanmoins, certaines applications nécessitent une allocation particulière (en écologie du paysage par exemple, voir section 1.2.1). De plus, une carte allouée est observée et analysée visuellement par le modélisateur ou le destinataire de l'étude (par exemple, un décideur politique) de manière plus facile que les cartes de probabilité de transition.

Ce point définit implicitement une exigence de simulation : en plus d'être

basées sur des probabilités LUCC, les cartes d'allocation doivent être réalistes. Le caractère réaliste d'une allocation fait référence à la forme des pixels contigus qui sont passés et que nous appelons un patch, ou une tache. Un patch est considéré comme réaliste — au niveau actuel des exigences formelles de la modélisation LUCC — lorsque, à l'œil, sa forme semble cohérente avec les caractéristiques qualitatives ou semi-quantitatives de la transition d'état LUCC considérée. Par exemple, un changement d'utilisation agricole d'une parcelle se produira très souvent sur l'ensemble de la parcelle (ou sur une sous-parcelle résultant d'une certaine forme de division géométrique de celle-ci) alors que la construction de maisons est un phénomène plus limité qui se traduit par la transition d'un plus petit nombre de pixels agglomérés. Bien entendu, considérer une carte allouée comme réaliste est assez subjectif et il s'agit surtout d'éviter des attributions très surprenantes et manifestement en contradiction avec la réalité.

Comme pour le module de calibration-estimation, plusieurs méthodes ont été conçues pour mettre en œuvre l'allocation. Ces méthodes doivent répondre à deux exigences : i) être statistiquement correctes et ii) être "réalistes". La manière dont ces contraintes sont implémentées (en particulier la première) est détaillée dans le chapitre 3.

La première exigence peut sembler superflue. Cependant, tous les cadres de modélisation LUCC que j'ai étudiés violent ce point dans une certaine mesure (voir chapitre 3). Il y a plusieurs raisons derrière ce manquement, et elles varient d'un logiciel à l'autre. Par exemple, un algorithme peut échouer à implémenter une allocation formellement correcte lorsque plusieurs transitions ont une probabilité non négligeable de se produire. En effet, comment arbitrer entre deux transitions ?

De plus, en général, on alloue un patch (ensemble de pixels contigus) et non un seul pixel. Un algorithme d'allocation doit donc d'abord s'intéresser au lien entre les distributions de probabilité des pixels de transition d'état LUC et la production d'un patch de pixels transités. Une façon de le faire est de tirer un seul pixel "graine" d'une telle distribution, puis de produire un patch de pixels contigus autour du pixel graine et de recommencer le processus. La cohérence exige de recalculer les probabilités de transition des pixels après la formation de chaque patch. Nous décrivons une telle approche dans la section et montrons qu'elle est non biaisée dans 3.4. Le choix de la procédure de construction du patch ("patcher") est également caractéristique de la méthode d'allocation.

C.1.3.6. PRÉCISION SPATIALE DES MODÈLES LUCC

Par conception, on ne peut pas s'attendre à ce qu'une approche probabiliste reproduise exactement les LUCC observé. En fait, si les changements sont com-

plètement aléatoires (spatialement parlant), ils ne peuvent pas être reproduits avec une précision significative, sauf par un coup de chance dont la probabilité d'occurrence est infime, car cela reviendrait à attendre une prédiction correcte d'une série de lancers de dés aléatoires. Bien sûr, si le dé est chargé, le niveau de précision de la "prédiction" augmente, et si le lancer devient déterministe (toujours le même visage qui apparaît, par exemple), on peut s'attendre à le prédire exactement. La modélisation probabiliste du LUCC est basée sur l'impossibilité ou l'extrême difficulté de connaître les causes exactes du LUCC (de la même manière, connaître la dynamique exacte d'un dé est impossible ou extrêmement difficile). Par conséquent, la comparaison des modèles LUCC sur la précision de la prédiction spatiale aboutit en fait à une confusion entre deux questions différentes (encore une fois, en supposant que la qualité des données n'est pas un problème en soi, et que les logiciels remplissent correctement et précisément leur objectif, hypothèses qui sont discutables en soi) : le niveau d'aléa spatial du problème pour un ensemble "exact" d'EV, et l'identification correcte d'un ensemble pertinent de variables explicatives (voir section 1.3.2.2). La première est liée à l'inhomogénéité spatiale des variables explicatives, mais même si elles sont très inhomogènes, la précision spatiale de l'allocation sera faible si la quantité de changement est suffisamment faible par rapport à ce niveau d'inhomogénéité.

C'est pourquoi, en fin de compte, une certaine précision spatiale "raisonnable" sera obtenue si : i/ le cadre de modélisation LUCC choisi, mis en œuvre dans un logiciel donné, est correct ; ii/ le problème est suffisamment bien compris pour que les causes sous-jacentes du changement soient connues à un niveau suffisant, de sorte que les VE puissent être choisis judicieusement pour représenter efficacement ces causes sous-jacentes ; iii/ la qualité des données n'est pas un problème grave. Une conséquence pratique de ceci est que l'on doit être capable de juger de l'exactitude d'un cadre de modélisation indépendamment de la précision de la prédiction spatiale, de la pertinence des VE et de la qualité des données. C'est précisément l'un des principaux objectifs de notre ligne de recherche.

Pour résumer, en fin de compte, la précision spatiale est limitée par le niveau intrinsèque d'aléa du problème à traiter. L'amélioration sur ce plan ne peut pas venir de la modélisation de la LUCC elle-même, mais de la compréhension des causes sous-jacentes du changement de la LUC par les sciences sociales et environnementales. Cependant, comme une prédiction exacte des changements de la LUCC est presque impossible, une part d'aléatoire subsistera toujours. Je remarque une imputation incorrecte des problèmes de précision spatiale dans les études de cas LUCC réelles aux erreurs potentielles de sélection des VE, à la

qualité limitée des données et aux erreurs et biais de mise en œuvre du cadre et du logiciel de modélisation.

C.2. UNE MÉTHODE PRÉCISE DE CALIBRATION-ESTIMATION BASÉE SUR L'ESTIMATION DE DENSITÉ PAR NOYAUX

L'abstract du chapitre 2 est simplement traduit ici.

Plusieurs stratégies de modélisation ont été proposées pour étudier les changements d'utilisation et de couverture des sols (LUCC). Cependant, des divergences importantes ont été constatées entre différents modèles pour le même problème, remettant en question leur fiabilité et leur reproductibilité globales. Pour relever ce challenge, nous élaborons un cadre théorique générique, formellement correct, pour la modélisation de l'utilisation et de la couverture des sols basée sur des modèles, qui est mis en œuvre dans notre propre logiciel, CLUMPY (Comprehensive Land Use [and cover] Modeling in PYthon).

Le présent travail se concentre sur la calibration. Nous concevons une méthode de calibration-estimation de la densité du noyau (Bayes-eKDE) qui s'avère à la fois précise et algorithmiquement efficace sur des données artificielles synthétiques. Nous introduisons également une méthode d'évaluation générique qui nous permet de comparer l'efficacité de la calibration des modèles existants. Le gain en précision et en temps de calcul de notre méthode de calibration est ainsi précisément quantifié.

C.3. L'ALLOCATION REVISITÉE : FONDATIONS FORMELLES ET IDENTIFICATION DES BIAIS

L'abstract du chapitre 3 est simplement traduit ici.

La modélisation pattern-based de l'occupation et de l'utilisation des sols (ou modélisation LUCC) est mise en œuvre dans divers logiciels, tels que la famille CLUE, LCM ou Dinamica EGO. Ces outils sont maintenant relativement matures, mais leurs fondements conceptuels sont peu discutés dans la littérature. En particulier, ces environnements de modélisation présentent des comportements sensiblement différents pour le même problème et les mêmes données de

calibration.

Ces incohérences inter-modèles sont revisitées *ab initio*, en se concentrant sur l'allocation. Le concept de biais d'allocation est défini et nous permet d'identifier un certain nombre d'erreurs conceptuelles, de biais et d'imprécisions algorithmiques dans les environnements de modélisation LUCC existants. Nous décrivons des méthodes d'allocation exemptes d'erreurs et de biais, mises en œuvre dans notre propre logiciel CLUMPY, qui s'avère nettement plus performant que les logiciels existants en termes de correction formelle et de précision.

Il est crucial que la validation soit effectuée dans l'espace des variables explicatives, et non sur des cartes, contrairement à la pratique la plus courante dans le domaine.

C.4. SÉLECTION DES VARIABLES EXPLICATIVES PERTINENTES ET NON REDONDANTES

L'abstract du chapitre 4 est simplement traduit ici.

La sélection optimale des variables explicatives dans les modèles pattern-based de changement d'occupation et d'utilisation des sols (LUCC) est une question qui ne semble pas avoir été traitée de manière approfondie dans la littérature spécialisée. Dans ce travail, nous passons d'abord en revue les fondements du problème et exposons une stratégie possible pour mettre en œuvre une approche algorithmique conçue pour maximiser la pertinence des variables explicatives d'une part, et minimiser la redondance d'autre part. Les quelques paramètres de cette méthode semi-automatique doivent être choisis avec soin et nous illustrons cette approche dans une étude de cas simple. Cette méthode de sélection des variables explicatives est mise en œuvre dans notre propre logiciel, CLUMPY (Comprehensive Land Use [and cover] Model in PYthon).

C.5. CLUMPY, UN ENVIRONNEMENT DE MODÉLISATION DE CHANGEMENT D'USAGE DES SOLS

L'introduction du chapitre 5 est simplement traduit ici.

Toutes les méthodes présentées dans les chapitres 2, 3 et 4 ont été mises en

œuvre dans un package Python appelé CLUMPY pour Comprehensive Land Use [and cover] Model in PYthon. Il peut être configuré pour une étude de cas de deux manières différentes : la première permet à un utilisateur avancé de modéliser un problème en utilisant nos méthodes par le biais d'un script Python et la seconde est une interface utilisateur graphique (GUI) qui permet à l'utilisateur de configurer une étude de cas simple. J'ai utilisé ce package pour modéliser toutes les études de cas présentées dans cette thèse.

C.6. CONCLUSION

L'intégralité de la conclusion (chapitre 6) est traduite ici.

Cette thèse s'est concentrée sur la modélisation mathématique du LUCC à travers une approche pattern-based statistique et spatialement explicite. Cela n'implique en aucun cas que les analyses et théories conceptuelles de la géographie et de l'aménagement du territoire (et d'autres sciences sociales également) ne sont pas importantes pour le phénomène LUCC. En fait, elles sont de première importance en elles-mêmes. Dans le contexte plus limité de la modélisation basée sur un logiciel LUCC, elles sont essentielles pour développer une compréhension claire de ce qu'un logiciel LUCC peut et ne peut pas faire, et fournissent un guide majeur pour mettre en place correctement une étude de cas LUCC où un tel logiciel est utilisé. A cet égard, un logiciel LUCC peut apporter des informations utiles s'il est utilisé correctement, mais il ne peut pas se substituer à des analyses préalables (utilisation des terres, sciences sociales, etc.) des moteurs des LUCC et de leurs implications. Mais avec Pierre-Yves Longaretti, nous nous concentrons sur la partie que nous pouvons aider à améliorer : les mathématiques appliquées et la mise en œuvre algorithmique. Ce type d'amélioration peut être très important en termes de pertinence et de précision des résultats, et ne devrait pas être écarté a priori, même si le champ d'application du logiciel LUCC est intrinsèquement limité.

Arrivé au terme de ce manuscrit de thèse, il peut être salutaire de prendre du recul et de tirer quelques conclusions sur ce travail. Les conclusions respectives des chapitres ne sont pas reprises ici mais résumées en mettant l'accent sur les contributions de cette thèse (section 6.1). De plus, de nouveaux éléments de perspective seront introduits afin d'esquisser des développements futurs possibles dans la modélisation du LUCC mais aussi dans les applications (section 6.2).

C.6.1. CONTRIBUTIONS DE LA THÈSE

La liste assez détaillée des objectifs (section 1.4.3) peut constituer une énumération quantitative des contributions de cette thèse que je ne répéterai pas ici. La première question qui a conduit à l'élaboration de ce sujet de thèse était basée sur l'observation de différences significatives de résultats entre les modèles existants pour un même problème et des données identiques. Il s'agissait donc d'identifier les origines de ces différences, dont l'existence soulève de sérieuses questions sur la validité des résultats obtenus jusqu'à présent avec ce type de modèle. Est-ce dû au logiciel, aux données utilisées, aux choix des utilisateurs, aux mathématiques formelles ou à une combinaison de ces raisons ? Avec Pierre-Yves Longaretti, nous avons donc entrepris une étude de ces modèles qui se voulait la plus complète possible. A ma connaissance, une telle entreprise est sans précédent dans la communauté.

Nous avons d'abord clarifié l'architecture des modèles LUCC statistiques et spatialement explicites (section 1.3). Ainsi, nous avons pu diviser le problème en différents modules qui pouvaient être étudiés indépendamment afin de simplifier l'étude du modèle global. Nous avons ensuite suivi le même schéma pour chacun d'entre eux : critiquer l'appareil mathématique choisi par le modèle, vérifier son implémentation et son bon fonctionnement (parfois par rétro-ingénierie) puis proposer des méthodes alternatives plus précises (calibration) et non biaisées (allocation) que nous comparons à l'existant. Ceci a été fait pour les modules de calibration (chapitre 2), d'allocation (chapitre 3) et de sélection d'EV (chapitre 4) (toutefois sans l'étape de comparaison pour ce dernier). L'idée était de couvrir autant de terrain que possible afin de s'assurer que toutes les questions pertinentes pour la modélisation du LUCC soient abordées. Bien sûr, il faut savoir où arrêter le raffinement de la modélisation et c'est pourquoi nous nous sommes tenus aux questions de premier et de second ordre, mais pas au-delà, ce qui est plus que suffisant compte tenu du type actuel d'applications de ces modèles.

Aussi, il me semble essentiel de revenir sur la manière dont on peut évaluer ces modules. En effet, à ma connaissance, il n'existait pas dans la littérature sur les modèles LUCC de méthode formelle permettant d'évaluer qualitativement et quantitativement une méthode de calibration-estimation ou d'allocation du point de vue algorithmique et également du point de vue de la précision obtenue autre que la précision d'allocation spatiale par rapport à des changements observés. Afin de vérifier le bon comportement de chacun des modules, nous avons donc développé des protocoles d'évaluation. Celui dédié à l'évaluation du module de calibration-estimation est basé sur l'utilisation d'un cas d'étude entièrement contrôlé où toutes les probabilités sont parfaitement connues (sec-

tion 2.6). Cette approche est assez courante dans de nombreux domaines (en machine learning en particulier) et est à ma connaissance sans précédent pour les modèles LUCC. Cependant, c'est la seule façon appropriée d'évaluer objectivement une méthode d'estimation de probabilité. Concernant l'évaluation du module d'allocation, il s'agit de mesurer la différence entre la densité de probabilité mise en entrée du module et la densité de probabilité obtenue par une série de simulations. Nous avons ainsi défini le concept de biais d'allocation (section 3.3.2). Cette approche est également nouvelle dans la communauté.

Ainsi, nous avons caractérisé et montré la compétitivité de notre méthode de calibration-estimation (Bayes-eKDE, section 2.5) et de nos méthodes d'allocation sans biais (uSAM et uPAM, section 3.4). Ces méthodes ont montré des résultats supérieurs ou égaux à toutes les autres en termes de résultats obtenus et de temps de calcul numérique (sections 2.7.3 et 3.7). J'ai implémenté toutes ces méthodes dans un package Python nommé CLUMPY (chapitre 5) qui peut être considéré comme un co-produit de cette thèse. Celui-ci n'a pas pour but de concurrencer un logiciel aussi complet que Dinamica EGO mais d'inviter les développeurs de modèles existants à revoir certaines de leurs méthodes (voir section 1.4.1 sur les destinataires de cette thèse), tout en ayant à leur disposition un point de comparaison précis, efficace et impartial. J'espère que les utilisateurs seront également intéressés à le tester sur leurs propres études de cas.

Nous avons constaté que tous les développeurs font, à des degrés divers et probablement sans le savoir, des hypothèses mathématiques injustifiées et en fait incorrectes dans le cadre de modélisation mis en œuvre dans leur logiciel. Il s'agit d'une source majeure d'erreurs et de biais. Je peux enfin répondre aux questions soulevées précédemment. Les différences de résultats observées entre les différents modèles LUCC sont principalement dues (par ordre décroissant d'importance) à des

NnoteExcluant les considérations relatives à la qualité des données d'entrée qui sont considérées comme acquises dans cette thèse. De plus, nous avons complètement évacué ce problème en traitant des études de cas parfaitement maîtrisées. : i) des choix mathématiques intrinsèques aux modèles (pruning, méthode de calibration-estimation...) ; ii) des paramètres utilisateurs dont les choix ne sont pas triviaux et manquent d'indication (binning, pruning, MLP...) ; et iii) des implémentations logicielles discutables (pruning, patcher...).

C.6.2. PERSPECTIVES DE TRAVAIL FUTUR

Après avoir résumé les principales contributions de la thèse, je souhaite maintenant esquisser brièvement quelques directions possibles de travaux de recher-

che futurs. Le logiciel CLUMPY, ambassadeur des méthodes introduites dans cette thèse, fait également l'objet de perspectives concrètes d'extension et d'amélioration (section 6.2.2). Enfin, je me permets ici d'imaginer de nouvelles applications qui pourraient tirer parti de certains des résultats obtenus au cours de cette thèse (section 6.2.3).

C.6.2.1. ENVIRONNEMENT DE MODÉLISATION LUCC

Une formalisation des modèles LUCC pattern-based a été entreprise dans cette thèse. La stratégie de recherche adoptée a été motivée par deux considérations : proposer une théorie formelle valide et mathématiquement correcte tout en étant compréhensible par la communauté des développeurs de modèles existants. Cependant, certains points mathématiques fondamentaux ont été laissés de côté. Ceux-ci étant soit considérés comme acquis par la communauté, soit totalement impensés, nous ne les avons pas abordés dans cette thèse pour nous concentrer sur des éléments immédiatement compréhensibles et mobilisables par les développeurs de modèles. Pourtant, ces concepts constituent la base des modèles LUCC. En particulier, tous les modèles basés sur des motifs considèrent que l'étude de la distribution de probabilité des pixels est équivalente à l'étude de la distribution de probabilité des pixels d'ensemencement des patches (un concept rapidement introduit dans la section 1.3.3.1). Cependant, ce postulat contient une approximation qui devrait être examinée au niveau formel afin d'évaluer son importance et les conséquences possibles de sa violation, le cas échéant ; d'autres hypothèses connexes ont été mentionnées (par exemple, l'indépendance statistique entre le pixel-semence et tous les autres paramètres du patch) qui devraient également être analysées formellement. De plus, j'ai mentionné dans la section 1.3.2.2 qu'il n'y a en fait que deux variables explicatives réellement indépendantes sans développer de preuves formelles. Une étude mathématique des variables explicatives et de leur nombre "idéal" pourrait également faire l'objet d'un développement formel intéressant. Une autre hypothèse qui n'a jamais été formellement examinée est que les projections futures de la couverture et de l'usage des sols peuvent être simulées dans une approximation markovienne. Cependant, l'évaluation de ce dernier point impliquerait très probablement des méthodes sophistiquées qui pourraient, par exemple, être empruntées aux domaines de la physique statistique, de la cinétique physique, des équations différentielles stochastiques, etc. mais qui dépasseraient probablement les approches et le niveau mathématique de tout ce qui a été fait jusqu'à présent dans la communauté. L'avantage de justifier dans une certaine mesure l'approximation markovienne doit être mis en balance avec le fait qu'une telle analyse serait trop abstraite pour la communauté.

Je souligne également certaines perspectives dans les méthodes utilisées dans les différents modules. De nouvelles approches peuvent être envisagées dans le cadre proposé comme une alternative à Bayes-eKDE qui est plus efficace numériquement par exemple. On peut aussi imaginer proposer des procédures de construction de patchs basées sur d'autres paradigmes que l'utilisation d'automates cellulaires comme une bibliothèque de formes (mentionnée section 1.3.5.1). Celles-ci ne seraient pas destinées à remplacer complètement la méthode actuelle mais pourraient être particulièrement adéquates pour modéliser des phénomènes LUCC particuliers. Le cadre de modélisation en modules proposé dans cette thèse permet aux utilisateurs et aux développeurs de considérer les méthodes comme interchangeables et de choisir celle qui est la plus adaptée à la situation pour chaque transition d'un même cas d'étude. Cette interchangeabilité doit ensuite être permise par le logiciel pour être opérationnelle (voir section 6.2.2).

Il pourrait également être intéressant de proposer des mécanismes pour aider les utilisateurs à configurer les paramètres du modèle. En effet, les modèles LUCC peuvent être complexes à utiliser et leur validité est souvent fortement dépendante des choix des utilisateurs. Nous avons esquissé une telle approche dans l'illustration de notre méthode de sélection des variables explicatives en décrivant le choix des différents paramètres (section 4.5.1). Cependant, cette approche doit être prise avec précaution. En effet, les outils ne font pas à eux seuls une bonne science. Il est indispensable de comprendre les tenants et les aboutissants d'un problème, et il ne faut jamais utiliser un logiciel comme une boîte noire, c'est-à-dire sans comprendre suffisamment ce qu'il fait réellement. Cette remarque invite les utilisateurs à la prudence, mais aussi les concepteurs de cadres de modélisation à proposer des outils conçus de cette manière.

C.6.2.2. CLUMPY

Le logiciel CLUMPY livré avec cette thèse est une première version stable (1.0.0). Bien qu'il puisse être utilisé tel quel par les utilisateurs, il existe de nombreuses perspectives de développement futur. CLUMPY est un logiciel open-source et est donc ouvert aux contributions, notamment par l'ajout de nouvelles méthodes. Son architecture traite les méthodes comme des briques interchangeables et il est facile d'élaborer et d'implémenter une nouvelle méthode tant qu'elle respecte les entrées et sorties demandées.

De plus, l'interface graphique proposée dans ce premier livrable est rudimentaire et ne permet que de concevoir une étude de cas simple (voir section 5.2.2). Toute personne est la bienvenue pour l'améliorer en veillant à ne pas rendre l'expérience utilisateur trop ardue, ce qui est un véritable défi compte tenu de la complexité de certaines études de cas. Il serait également souhaitable d'éviter

une approche de boîte noire entièrement automatisée en proposant des choix réels de paramètres et en alertant graphiquement l'utilisateur lorsque les valeurs de ceux-ci sont inadaptées.

Enfin, la documentation de CLUMPY a un rôle important dans sa diffusion et son accessibilité aux utilisateurs potentiels. La rédaction de cette documentation doit apporter les clés de compréhension pour une bonne utilisation du package à travers des tutoriels, des cas d'illustration et l'énumération de toutes les fonctions et classes. Le retour des utilisateurs doit permettre d'améliorer la documentation en fonction de leurs expériences. Actuellement, la documentation est exhaustive concernant les fonctions et les objets (documentation API) mais n'inclut pas encore de tutoriels.

C.6.2.3. APPLICATIONS

CLUMPY étant un logiciel Python, il est possible pour un utilisateur expérimenté et averti de faire une utilisation très avancée du logiciel. En effet, Python est un langage informatique particulièrement apprécié dans la recherche scientifique. Il a l'avantage d'être de haut niveau, polyvalent et relativement facile à utiliser. De plus, il est possible d'appeler des fonctions écrites en C ou d'écrire des fichiers en Cython pour profiter de la vitesse d'exécution du C pour certaines opérations critiques. Ainsi, étant donné l'efficacité numérique de CLUMPY qui permet d'effectuer un grand nombre d'estimations et d'allocations en un temps raisonnable, il est tout à fait possible d'effectuer des analyses de sensibilité des paramètres avec ce logiciel pour améliorer sa compréhension des pilotes LUCC. Le fait que CLUMPY soit écrit en Python lui permet également d'être facilement interfacé et inclus dans un modèle plus large qui le considère comme une fonction avec ses propres entrées et sorties, ou de le coupler avec d'autres modèles (par exemple économétriques) qui spécifieraient un scénario à partir d'autres entrées scientifiques.

Ainsi, j'espère que toutes les méthodes et les processus introduits dans cette thèse ainsi que CLUMPY auront une influence positive sur la communauté des développeurs de modèles LUCC mais aussi sur la précision et la validité des résultats obtenus sur les futures études de cas. Les applications des modèles LUCC sont appelées à prendre une place importante dans les années à venir, notamment dans l'évaluation des politiques publiques d'aménagement du territoire. En effet, comme je l'ai expliqué dans l'introduction (section 1.1), les LUCC sont bidirectionnellement liés aux changements globaux qui menacent les conditions de vie sur Terre telles que nous les connaissons aujourd'hui. Dès lors, la possibilité d'évaluer en amont les politiques publiques d'aménagement du territoire est déterminante et les modèles LUCC peuvent apporter des élé-

ments de réponse aux questions pratiques d'aménagement du territoire tout en fournissant des cartes de simulation très compréhensibles par les acteurs (qu'il s'agisse de responsables politiques, de fonctionnaires ou encore d'assemblées de citoyens). Ainsi, il est par exemple possible d'interroger avec les modèles LUCC de nouvelles contraintes pour favoriser la relocalisation et l'autonomie alimentaire d'un territoire, ou pour planifier l'arrêt de la déforestation et de l'artificialisation des sols. Cette thèse a fourni les premières étapes nécessaires pour que les modèles LUCC pattern-based statistiques et spatialement explicites soient à la hauteur des questions auxquelles ils prétendent aider à répondre.

BIBLIOGRAPHY

- Agarwal, C., Green, G. L., Grove, M., Evans, T., and Schweik, C. A review and assessment of land-use change models: dynamics of space, time and human choice. *Gen. Tech. Rep. NE-297. Newton Square, PA: U.S. Department of Agriculture, Forest Service, Northeastern Research Station. 61 p.*, 2000. doi:10.2737/NE-GTR-297.
- Agresti, A. *Categorical data analysis, 2nd edition*. John Wiley, 2002. ISBN 978-0-471-45876-0.
- Alexander, P., Prestele, R., Verburg, P. H., Arneth, A., Baranzelli, C., Batista e Silva, F., Brown, C., Butler, A., Calvin, K., Dendoncker, N., Doelman, J. C., Dunford, R., Engström, K., Eitelberg, D., Fujimori, S., Harrison, P. A., Hasegawa, T., Havlik, P., Holzhauser, S., Humpenöder, F., Jacobs-Crisioni, C., Jain, A. K., Krisztin, T., Kyle, P., Laval, C., Lenton, T., Liu, J., Meiyappan, P., Popp, A., Powell, T., Sands, R. D., Schaldach, R., Stehfest, E., Steinbuks, J., Tabeau, A., van Meijl, H., Wise, M. A., and Rounsevell, M. D. A. Assessing uncertainties in land cover projections. *Global Change Biology*, 23 (2):767–781, 2017. doi:10.1111/gcb.13447.
- Anderson, J. R., Hardy, E. E., Roach, J. T., and Witmer, R. E. A land use and land cover classification system for use with remote sensor data. *Professional Paper*, 1976. ISSN 2330-7102. doi:10.3133/pp964.
- Arneth, A., Denton, F., Agus, F., Elbehri, A., Erb, K., Elasha, B. O., Rahimi, M., Rounsevell, M., Spence, A., and Valentini, R. Framing and context. in: Climate change and land: an ipcc special report on climate change, desertification, land degradation, sustainable land management, food security, and greenhouse gas fluxes in terrestrial ecosystems framing and context. in: Climate change and land: an ipcc special report on climate change, desertification, land degradation, sustainable land management, food security, and greenhouse gas fluxes in terrestrial ecosystems [p.r. shukla, j. skea, e. calvo

- buendia, v. masson-delmotte, h.-o. pörtner, d.c. roberts, p. zhai, r. slade, s. connors, r. van diemen, m. ferrat, e. haughey, s. luz, s. neogi, m. pathak, j. petzold, j. portugal pereira, p. vyas, e. huntley, k. kissick, m. belkacemi, j. malley, (eds.)). *In press.*, 2019.
- Backurs, A., Indyk, P., and Wagner, T. Space and time efficient kernel density estimation in high dimensions. In Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Garnett, R., editors, *Advances in Neural Information Processing Systems*, volume 32, pages 15773–15782. Curran Associates, Inc., 2019.
- Beer, T. The impact of extreme weather events on food security. In *Climate Change, Extreme Events and Disaster Risk Reduction*, pages 121–133. Springer International Publishing, 2017. doi:10.1007/978-3-319-56469-2_8.
- Bezner Kerr, R., Hasegawa, T., Lasco, R., Bhatt, I., Deryng, D., Farrell, A., Gurney-Smith, H., Ju, H., Lluch-Cota, S., Meza, F., Nelson, G., Neufeldt, H., and Thornton, P. Food, fibre, and other ecosystem products. in: Climate change 2022: Impacts, adaptation, and vulnerability. contribution of working group ii to the sixth assessment report of the intergovernmental panel on climate change [h.-o. pörtner, d.c. roberts, m. tignor, e.s. poloczanska, k. mintenbeck, a. alegría, m. craig, s. langsdorf, s. löschke, v. möller, a. okem, b. rama (eds.)]. *Cambridge University Press. In Press.*, 2022.
- Bielecka, E. Gis spatial analysis modeling for land use change. a bibliometric analysis of the intellectual base and trends. *Geosciences*, 10(11):421, 2020. doi:10.3390/geosciences10110421.
- Bonham-Carter, G. F. Tools for map analysis: Map pairs. In *Geographic Information Systems for Geoscientists*, pages 221–266. Elsevier, 1994. doi:10.1016/b978-0-08-041867-4.50013-8.
- Camacho Olmedo, M. T., Paegelow, M., Mas, J. F., and Escobar, F. Geomatic Approaches for Modeling Land Change Scenarios. An Introduction. In Camacho Olmedo, M. T., Paegelow, M., Mas, J.-F., and Escobar, F., editors, *Geomatic Approaches for Modeling Land Change Scenarios*, Lecture Notes in Geoinformation and Cartography, pages 1–8. Springer International Publishing, 2018. doi:10.1007/978-3-319-60801-3_1.
- Camacho Olmedo, M. T., Pontius, R. G., Paegelow, M., and Mas, J.-F. Comparison of simulation models in terms of quantity and allocation of

- land change. *Environmental Modelling & Software*, 69:214–221, 2015. doi:10.1016/j.envsoft.2015.03.003.
- Chacón, J. E. and Duong, T. *Multivariate Kernel Smoothing and its Applications*. Chapman and Hall/CRC, 2018. doi:10.1201/9780429485572.
- Chamayou, J. M. F. Averaging shifted histograms. *Computer Physics Communications*, 21(2):145–161, 1980. ISSN 0010-4655. doi:10.1016/0010-4655(80)90037-5.
- Chandrashekar, G. and Sahin, F. A survey on feature selection methods. *Computers & Electrical Engineering*, 40(1):16–28, 2014. doi:10.1016/j.compeleceng.2013.11.024.
- Charikar, M. and Siminelakis, P. Hashing-Based-Estimators for Kernel Density in High Dimensions. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1032–1043, 2017. doi:10.1109/FOCS.2017.99.
- Cho, Y. and Kim, S. Volume of Hypercubes Clipped by Hyperplanes and Combinatorial Identities. *The Electronic Journal of Linear Algebra*, 36:228–255, 2020. doi:10.13001/ela.2020.5085.
- Cochran, W. G. *Sampling Techniques, 3rd Edition*. Wiley, 1977. ISBN 978-0-471-16240-7.
- Cohen, J. *Statistical Power Analysis for the Behavioral Sciences*. Routledge, 1988. doi:10.4324/9780203771587.
- Davis, L. W. and Gertler, P. J. Contribution of air conditioning adoption to future energy use under global warming. *Proceedings of the National Academy of Sciences*, 112(19):5962–5967, 2015. doi:10.1073/pnas.1423558112.
- de Almeida, C. M., Monteiro, A. M. V., Camara, G., Soares-Filho, B. S., Cerqueira, G. C., Pennachin, C. L., and Batty, M. Empiricism and stochastics in cellular automaton modeling of urban land use dynamics. *UCL Working paper series n° 42*, 2002.
- Diggle, P. A Kernel Method for Smoothing Point Process Data. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 34(2):138–147, 1985. doi:10.2307/2347366.
- Ding, C. and Peng, H. Minimum redundancy feature selection from microarray gene expression data. *Journal of Bioinformatics and Computational Biology*, 3(2):185–205, 2005. doi:10.1142/S0219720005001004.

- Duffy, K. A., Schwalm, C. R., Arcus, V. L., Koch, G. W., Liang, L. L., and Schipper, L. A. How close are we to the temperature tipping point of the terrestrial biosphere? *Science Advances*, 7(3), 2021. doi:10.1126/sciadv.aay1052.
- Duong, T. and Hazelton, M. Plug-in bandwidth matrices for bivariate kernel density estimation. *Journal of Nonparametric Statistics*, 15(1):17–30, 2003. doi:10.1080/10485250306039.
- Duong, T. and Hazelton, M. L. Cross-validation Bandwidth Matrices for Multivariate Kernel Density Estimation. *Scandinavian Journal of Statistics*, 32(3):485–506, 2005. doi:10.1111/j.1467-9469.2005.00445.x.
- Eastman, J. R. and Toledano, J. A Short Presentation of CA_markov. In Camacho Olmedo, M. T., Paegelow, M., Mas, J.-F., and Escobar, F., editors, *Geomatic Approaches for Modeling Land Change Scenarios*, Lecture Notes in Geoinformation and Cartography, pages 481–484. Springer International Publishing, Cham, 2018a. ISBN 9783319608013. doi:10.1007/978-3-319-60801-3_33.
- Eastman, J. R. and Toledano, J. A Short Presentation of the Land Change Modeler (LCM). In Camacho Olmedo, M. T., Paegelow, M., Mas, J.-F., and Escobar, F., editors, *Geomatic Approaches for Modeling Land Change Scenarios*, Lecture Notes in Geoinformation and Cartography, pages 499–505. Springer International Publishing, Cham, 2018b. ISBN 9783319608013. doi:10.1007/978-3-319-60801-3_36.
- Eastman, J. R., Jin, W., Kyem, P., and Toledano, J. Raster Procedure for Multi-Criteria/Multi-Objective Decisions. *Photogrammetric Engineering & Remote Sensing*, 61:539–547, 1995.
- Ellis, E. C., Klein Goldewijk, K., Siebert, S., Lightman, D., and Ramankutty, N. Anthropogenic transformation of the biomes, 1700 to 2000. *Global Ecology and Biogeography*, 19(5):589–606, 2010. ISSN 1466-8238. doi:10.1111/j.1466-8238.2010.00540.x.
- Engelbrecht, F., Adegoke, J., Bopape, M.-J., Naidoo, M., Garland, R., Thatcher, M., McGregor, J., Katzfey, J., Werner, M., Ichoku, C., and Gatebe, C. Projections of rapidly rising surface temperatures over Africa under low mitigation. *Environmental Research Letters*, 10(8):085004, 2015. ISSN 1748-9326. doi:10.1088/1748-9326/10/8/085004.
- Enora, B., Locatelli, B., Vendel, F., Bergeret, A., Elleaume, N., Grosinger, J., and Lavorel, S. Historical reconfigurations of a social-ecological system adapting

- to economic, policy and climate changes in the French Alps. *Regional Environmental Change*, 21, 2021. doi:10.1007/s10113-021-01760-8.
- Escobar, F., van Delden, H., and Hewitt, R. LUCC Scenarios. In Camacho Olmedo, M. T., Paegelow, M., Mas, J.-F., and Escobar, F., editors, *Geomatic Approaches for Modeling Land Change Scenarios*, Lecture Notes in Geoinformation and Cartography, pages 81–97. Springer International Publishing, Cham, 2018. ISBN 9783319608013. doi:10.1007/978-3-319-60801-3_5.
- García-Álvarez, D., Delden, H., Camacho Olmedo, M. T., and Paegelow, M. Uncertainty Challenge in Geospatial Analysis: An Approximation from the Land Use Cover Change Modelling Perspective. *Key Challenges in Geography*, 2019a. doi:10.1007/978-3-030-04750-4_15.
- García-Álvarez, D., Lloyd, C. D., van Delden, H., and Camacho Olmedo, M. T. Thematic resolution influence in spatial analysis. An application to Land Use Cover Change (LUCC) modelling calibration. *Computers, Environment and Urban Systems*, 78:101375, 2019b. ISSN 0198-9715. doi:10.1016/j.compenvurbsys.2019.101375.
- García-Álvarez, D., Camacho Olmedo, M. T., Van Delden, H., Mas, J.-F., and Paegelow, M. Comparing the structural uncertainty and uncertainty management in four common land use cover change (lucc) model software packages. *Environmental Modelling & Software*, 153:105411, 2022. doi:https://doi.org/10.1016/j.envsoft.2022.105411.
- Geist, H., McConnell, W., Lambin, E. F., Moran, E., Alves, D., and Rudel, T. Causes and Trajectories of Land-Use/Cover Change. In Lambin, E. F. and Geist, H., editors, *Land-Use and Land-Cover Change. Global Change - The IGBP Series*, Global Change - The IGBP Series, pages 41–70. Springer, Berlin, Heidelberg, 2006. ISBN 9783540322023. doi:10.1007/3-540-32202-7_3.
- Gillies, S. et al. Rasterio: geospatial raster i/o for Python programmers. *Github*, 2013–. URL <https://github.com/rasterio/rasterio>.
- Girard, M.-C., Girard, C., Courault, D., Gilliot, J.-M., Loubersac, L., Meyer-Roux, J., Monget, J.-M., Seguin, B., and Rao, N. V. Corine land cover. In *Processing of Remote Sensing Data*, pages 331–344. Routledge, 2018. doi:10.1201/9780203741917-19.
- Hahs-Vaughan, D. L. and Lomax, R. G. *An introduction to statistical concepts, 4th edition*. Routledge, 2020. doi:10.4324/9781315624358.

- Hanzl, M. Urban Sprawl in Europe: Landscapes, Land-use Change and Policy. *Planning Practice & Research*, 25(2):273–274, 2010. ISSN 0269-7459. doi:10.1080/02697451003740312.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., and Oliphant, T. E. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi:10.1038/s41586-020-2649-2.
- Helber, P., Bischke, B., Dengel, A. R., and Borth, D. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12:2217–2226, 2019.
- Hernández-Morcillo, M., Plieninger, T., and Bieling, C. An empirical review of cultural ecosystem service indicators. *Ecological Indicators*, 29:434–444, 2013. ISSN 1470-160X. doi:10.1016/j.ecolind.2013.01.013.
- Hulme, M., Barrow, E. M., Arnell, N. W., Harrison, P. A., Johns, T. C., and Downing, T. E. Relative impacts of human-induced climate change and natural climate variability. *Nature*, 397(6721):688–691, 1999. ISSN 1476-4687. doi:10.1038/17789.
- Hunter, J. D. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi:10.1109/MCSE.2007.55.
- Hurlbert, M., Krishnaswamy, J., Davin, E., Johnson, F., Mena, C., Morton, J., Myeong, S., Viner, D., Warner, K., Wreford, A., Zakieldeen, S., and Zommers, Z. Risk management and decision making in relation to sustainable development. in: Climate change and land: an ipcc special report on climate change, desertification, land degradation, sustainable land management, food security, and greenhouse gas fluxes in terrestrial ecosystems [p.r. shukla, j. skea, e. calvo buendia, v. masson-delmotte, h.-o. pörtner, d.c. roberts, p. zhai, r. slade, s. connors, r. van diemen, m. ferrat, e. haughey, s. luz, s. neogi, m. pathak, j. petzold, j. portugal pereira, p. vyas, e. huntley, k. kissick, m. belkacemi, j. malley, (eds.)]. *In press*, 2019.
- IPBES. The regional assessment report on biodiversity and ecosystem services from europe and central asia biodiversity [rounsevell, m., fischer, m., torremarin rando, a. and mader, a. (eds.)]. *Secretariat of the Intergovernmental*

- tal Science-Policy Platform on Biodiversity and Ecosystem Services, Bonn, Germany*, page 892 pp, 2018.
- IPCC. Climate change and land: an ipcc special report on climate change, desertification, land degradation, sustainable land management, food security, and greenhouse gas fluxes in terrestrial ecosystems [p.r. shukla, j. skea, e. calvo buendia, v. masson-delmotte, h.-o. pörtner, d. c. roberts, p. zhai, r. slade, s. connors, r. van diemen, m. ferrat, e. haughey, s. luz, s. neogi, m. pathak, j. petzold, j. portugal pereira, p. vyas, e. huntley, k. kissick, m. belkacemi, j. malley, (eds.)]. *In press.*, 2019.
- IPCC. Climate change 2022: Impacts, adaptation, and vulnerability. contribution of working group ii to the sixth assessment report of the intergovernmental panel on climate change [h.-o. pörtner, d.c. roberts, m. tignor, e.s. poloczanska, k. mintenbeck, a. alegría, m. craig, s. langsdorf, s. löschke, v. möller, a. okem, b. rama (eds.)]. *Cambridge University Press. In Press.*, 2022.
- Jeong, J.-H., Kug, J.-S., Linderholm, H. W., Chen, D., Kim, B.-M., and Jun, S.-Y. Intensified arctic warming under greenhouse warming by vegetation–atmosphere–sea ice interaction. *Environmental Research Letters*, 9(9): 094007, 2014. doi:10.1088/1748-9326/9/9/094007.
- Jia, G., Shevliakova, E., Artaxo, P., Noblet-Ducoudré, N. D., Houghton, R., House, J., Kitajima, K., Lennard, C., Popp, A., Sirin, A., Sukumar, R., and Verchot, L. Land–climate interactions. in: Climate change and land: an ipcc special report on climate change, desertification, land degradation, sustainable land management, food security, and greenhouse gas fluxes in terrestrial ecosystems [p.r. shukla, j. skea, e. calvo buendia, v. masson-delmotte, h.-o. pörtner, d.c. roberts, p. zhai, r. slade, s. connors, r. van diemen, m. ferrat, e. haughey, s. luz, s. neogi, m. pathak, j. petzold, j. portugal pereira, p. vyas, e. huntley, k. kissick, m. belkacemi, j. malley, (eds.)]. *In press.*, 2019.
- Kelly Letcher, R. A., Jakeman, A. J., Barreteau, O., Borsuk, M. E., ElSawah, S., Hamilton, S. H., Henriksen, H. J., Kuikka, S., Maier, H. R., Rizzoli, A. E., van Delden, H., and Voinov, A. A. Selecting among five common modelling approaches for integrated environmental assessment and management. *Environmental Modelling & Software*, 47:159–181, 2013. ISSN 1364-8152. doi:10.1016/j.envsoft.2013.05.005.
- Kessy, A., Lewin, A., and Strimmer, K. Optimal Whitening and Decorrelation. *The American Statistician*, 72(4):309–314, 2018. doi:10.1080/00031305.2016.1277159.

- Knuth, D. E. *The art of computer programming. Volume 2, seminumerical algorithms, 3rd edition*. Addison-Wesley, 1998.
- Koomen, E. and Borsboom van Beurden, J. A. M., editors. *Land-use modelling in planning practice*. Number v. 101 in The GeoJournal library. Springer, New York, 2011. ISBN 978-94-007-1821-0.
- Lambin, E. F. and Geist, H., editors. *Land-Use and Land-Cover Change*. Springer Berlin Heidelberg, 2006. doi:10.1007/3-540-32202-7.
- Langrené, N. and Warin, X. Fast and Stable Multivariate Kernel Density Estimation by Fast Sum Updating. *Journal of Computational and Graphical Statistics*, 28(3):596–608, 2019. doi:10.1080/10618600.2018.1549052.
- Lawrence, D. and Vandecar, K. Effects of tropical deforestation on climate and agriculture. *Nature Climate Change*, 5(1):27–36, 2015. ISSN 1758-6798. doi:10.1038/nclimate2430.
- Leite-Filho, A. T., Soares-Filho, B. S., Davis, J. L., Abrahão, G. M., and Börner, J. Deforestation reduces rainfall and agricultural revenues in the Brazilian Amazon. *Nature Communications*, 12(1):2591, May 2021. ISSN 2041-1723. doi:10.1038/s41467-021-22840-7. URL <https://www.nature.com/articles/s41467-021-22840-7>.
- Lickley, M. and Solomon, S. Drivers, timing and some impacts of global aridity change. *Environmental Research Letters*, 13(10):104010, 2018. ISSN 1748-9326. doi:10.1088/1748-9326/aae013.
- Lobell, D. B., Schlenker, W., and Costa-Roberts, J. Climate trends and global crop production since 1980. *Science*, 333(6042):616–620, 2011. doi:10.1126/science.1204531.
- Lohr, S. L. *Sampling: Design and Analysis: Design and Analysis, 2nd edition*. Chapman and Hall/CRC, 2019. doi:10.1201/9780429298899.
- Longaretti, P.-Y. and Mazy, F.-R. Towards a Generic Theoretical Framework for Pattern-Based LUCC Modeling. A maximum relevance / minimum redundancy selection procedure of explanatory variables. *Submitted to Environmental Software and Modeling*, 2022.
- Mas, J.-F., Kolb, M., Paegelow, M., Camacho Olmedo, M. T., and Houet, T. Inductive pattern-based land use/cover change models: A comparison of four software packages. *Environmental Modelling & Software*, 51:94–111, 2014. doi:10.1016/j.envsoft.2013.09.010.

- Mas, J., García Álvarez, D., Paegelow, M., Domínguez-Vera, R., and Castillo-Santiago, M. Metrics Based on a Cross-Tabulation Matrix to Validate Land Use Cover Maps. In *Land Use Cover Datasets and Validation Tools. Validation Practices with QGIS*, pages 127–151. Springer, Cham, 2022. doi:10.1007/978-3-030-90998-7_8.
- Mazy, F.-R. and Longaretti, P.-Y. Towards a Generic Theoretical Framework for Pattern-Based LUCC Modeling. An accurate and powerful calibration-estimation method based on Kernel density estimation. *Environmental Software and Modeling*, 2022a.
- Mazy, F.-R. and Longaretti, P.-Y. Towards a Generic Theoretical Framework for Pattern-Based LUCC Modeling. Allocation Revisited: Formal foundations and bias identification. *Submitted to Environmental Software and Modeling*, 2022b.
- Mazy, F.-R. and Longaretti, P.-Y. A Formally Correct and Algorithmically Efficient LULC Change Model-building Environment. In *Proceedings of the 8th International Conference on Geographical Information Systems Theory, Applications and Management - GISTAM*, pages 25–36. SciTePress, 2022c.
- Meyfroidt, P., Roy Chowdhury, R., de Bremond, A., Ellis, E. C., Erb, K.-H., Filatova, T., Garrett, R. D., Grove, J. M., Heinimann, A., Kuemmerle, T., Kull, C. A., Lambin, E. F., Landon, Y., le Polain de Waroux, Y., Messerli, P., Müller, D., Nielsen, J. Ø., Peterson, G. D., Rodriguez García, V., Schlüter, M., Turner, B. L., and Verburg, P. H. Middle-range theories of land system change. *Global Environmental Change*, 53:52–67, 2018. ISSN 0959-3780. doi:10.1016/j.gloenvcha.2018.08.006.
- National Research Council. Advancing land change modeling: opportunities and research requirements. *The National Academies Press, Washington DC*, 2014.
- Newbold, T. Future effects of climate and land-use change on terrestrial vertebrate community diversity under different scenarios. *Proceedings of the Royal Society B: Biological Sciences*, 285(1881):20180792, 2018. doi:10.1098/rspb.2018.0792.
- Newbold, T., Hudson, L. N., Hill, S. L. L., Contu, S., Lysenko, I., Senior, R. A., Börger, L., Bennett, D. J., Choimes, A., Collen, B., Day, J., De Palma, A., Díaz, S., Echeverria-Londoño, S., Edgar, M. J., Feldman, A., Garon, M., Harrison, M. L. K., Alhusseini, T., Ingram, D. J., Itescu, Y., Kattge, J., Kemp, V., Kirkpatrick, L., Kleyer, M., Correia, D. L. P., Martin, C. D., Meiri, S.,

- Novosolov, M., Pan, Y., Phillips, H. R. P., Purves, D. W., Robinson, A., Simpson, J., Tuck, S. L., Weiher, E., White, H. J., Ewers, R. M., Mace, G. M., Scharlemann, J. P. W., and Purvis, A. Global effects of land use on local terrestrial biodiversity. *Nature*, 520(7545):45–50, 2015. ISSN 1476-4687. doi:10.1038/nature14324.
- Noszczyk, T. A review of approaches to land use changes modeling. *Human and Ecological Risk Assessment: An International Journal*, 25(6):1377–1405, 2019. doi:10.1080/10807039.2018.1468994.
- Nuissl, H. and Siedentop, S. Urbanisation and Land Use Change. In Weith, T., Barkmann, T., Gaasch, N., Rogga, S., Strauß, C., and Zscheischler, J., editors, *Sustainable Land Management in a European Context. Human-Environment Interactions, vol 8*, Human-Environment Interactions, pages 75–99. Springer International Publishing, Cham, 2021. ISBN 9783030508418. doi:10.1007/978-3-030-50841-8_5.
- Ornetsmüller, C., Verburg, P. H., and Heinimann, A. Scenarios of land system change in the Lao PDR: Transitions in response to alternative demands on goods and services provided by the land. *Applied Geography*, 75:1–11, 2016. ISSN 0143-6228. doi:10.1016/j.apgeog.2016.07.010.
- O’Brien, T. A., Kashinath, K., Cavanaugh, N. R., Collins, W. D., and O’Brien, J. P. A fast and objective multidimensional kernel density estimation method: fastKDE. *Computational Statistics & Data Analysis*, 101:148–160, 2016. doi:10.1016/j.csda.2016.02.014.
- O’Sullivan, D., Evans, T., Manson, S., Metcalf, S., Ligmann-Zielinska, A., and Bone, C. Strategic directions for agent-based modeling: avoiding the YAAWN syndrome. *J Land Use Sci*, 11:177–187, 2016. doi:10.1080/1747423X.2015.1030463.
- Paegelow, M., Camacho Olmedo, M. T., and Mas, J. F. Techniques for the Validation of LUCC Modeling Outputs. In Camacho Olmedo, M. T., Paegelow, M., Mas, J.-F., and Escobar, F., editors, *Geomatic Approaches for Modeling Land Change Scenarios*, Lecture Notes in Geoinformation and Cartography, pages 53–80. Springer International Publishing, Cham, 2018. ISBN 9783319608013. doi:10.1007/978-3-319-60801-3_4.
- Paegelow, M., Camacho Olmedo, M. T., Mas, J.-F., and Houet, T. Benchmarking of LUCC modelling tools by various validation techniques and error analysis. *Cybergeog : European Journal of Geography*, 2014. ISSN 1278-3366. doi:10.4000/cybergeog.26610.

- Papoulis, A. and Pillai, S. U. *Probability, random variables and stochastic processes, 4th edition*. McGraw Hill, 2002. ISBN 978-0071226615.
- Parmesan, C., Morecroft, M., Trisurat, Y., Adrian, R., Anshari, G., Arneth, A., Gao, Q., Gonzalez, P., Harris, R., Price, J., Stevens, N., and Talukdar, G. Terrestrial and freshwater ecosystems and their services. in: Climate change 2022: Impacts, adaptation, and vulnerability. contribution of working group ii to the sixth assessment report of the intergovernmental panel on climate change [h.-o. pörtner, d.c. roberts, m. tignor, e.s. poloczanska, k. mintenbeck, a. alegría, m. craig, s. langsdorf, s. löschke, v. möller, a. okem, b. rama (eds.)]. *Cambridge University Press. In Press.*, 2022.
- Pecl, G. T., Araújo, M. B., Bell, J. D., Blanchard, J., Bonebrake, T. C., Chen, I.-C., Clark, T. D., Colwell, R. K., Danielsen, F., Evengård, B., Falconi, L., Ferrier, S., Frusher, S., Garcia, R. A., Griffis, R. B., Hobday, A. J., Janion-Scheepers, C., Jarzyna, M. A., Jennings, S., Lenoir, J., Linnetved, H. I., Martin, V. Y., McCormack, P. C., McDonald, J., Mitchell, N. J., Mustonen, T., Pandolfi, J. M., Pettorelli, N., Popova, E., Robinson, S. A., Scheffers, B. R., Shaw, J. D., Sorte, C. J. B., Strugnell, J. M., Sunday, J. M., Tuanmu, M.-N., Vergés, A., Villanueva, C., Wernberg, T., Wapstra, E., and Williams, S. E. Biodiversity redistribution under climate change: Impacts on ecosystems and human well-being. *Science (New York, N.Y.)*, 355(6332):eaai9214, 2017. ISSN 1095-9203. doi:10.1126/science.aai9214.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. doi:10.5555/1953048.2078195.
- Peng, H., Long, F., and Ding, C. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8):1226–1238, 2005. doi:10.1109/TPAMI.2005.159.
- Perugini, L., Caporaso, L., Marconi, S., Cescatti, A., Quesada, B., Noblet-Ducoudré, N. d., House, J. I., and Arneth, A. Biophysical effects on temperature and precipitation due to land cover change. *Environmental Research Letters*, 12(5):053002, 2017. ISSN 1748-9326. doi:10.1088/1748-9326/aa6b3f.
- Phillips, J. M. ϵ -samples for kernels. In *Proceedings of the 2013 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages

- 1622–1632. Society for Industrial and Applied Mathematics, 2013. doi:10.1137/1.9781611973105.116.
- Phillips, J. M. and Tai, W. T. Near-Optimal Coresets of Kernel Density Estimates. In Speckmann, B. and D., T. C., editors, *34th International Symposium on Computational Geometry (SoCG 2018)*, volume 99 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 66:1–66:13. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2018a. doi:10.4230/LIPIcs.SoCG.2018.66.
- Phillips, J. M. and Tai, W. T. Improved coresets for kernel density estimates. In *Proceedings of the 2018 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2718–2727, 2018b. doi:10.1137/1.9781611975031.173.
- Pontius, R. G. Statistical methods to partition effects of quantity and location during comparison of categorical maps at multiple resolutions. *Photogrammetric Engineering & Remote Sensing*, 68(10):1041–1049, 2002.
- Pontius, R. G., Boersma, W., Castella, J.-C., Clarke, K., de Nijs, T., Dietzel, C., Duan, Z., Fotsing, E., Goldstein, N., Kok, K., Koomen, E., Lippitt, C. D., McConnell, W., Mohd Sood, A., Pijanowski, B., Pithadia, S., Sweeney, S., Trung, T. N., Veldkamp, A. T., and Verburg, P. H. Comparing the input, output, and validation maps for several models of land change. *The Annals of Regional Science*, 42(1):11–37, 2008. ISSN 1432-0592. doi:10.1007/s00168-007-0138-2.
- Poore, J. and Nemecek, T. Reducing food’s environmental impacts through producers and consumers. *Science*, 360(6392):987–992, 2018. doi:10.1126/science.aag0216.
- Porter, J., Xie, L., Challinor, A., Cochrane, K., Howden, S., Iqbal, M., Lobell, D., and Travasso, M. Food security and food productionsystems. climate change 2014: Impacts, adaptation and vulnerability.part a: Global and sectoral aspects. contribution of working group ii tothe fifth assessment report of the intergovernmental panel on climatechange [field, c.b., v.r. barros, d.j. dokken, k.j. mach, m.d. mastrandrea,t.e. bilir, m. chatterjee, k.l. ebi, y.o. estrada, r.c. genova, b. girma,e.s. kissel, a.n. levy, s. maccracken, p.r. mastrandrea, and l.l. white(eds.)]. *Cambridge University Press, Cambridge, United Kingdom and New York, NY, USA*, page 485–533, 2014.
- Prestele, R., Alexander, P., Rounsevell, M. D. A., Arneth, A., Calvin, K., Doelman, J., Eitelberg, D. A., Engström, K., Fujimori, S., Hasegawa, T., Hav-

- lik, P., Humpenöder, F., Jain, A. K., Krisztin, T., Kyle, P., Meiyappan, P., Popp, A., Sands, R. D., Schaldach, R., Schüngel, J., Stehfest, E., Tabeau, A., Van Meijl, H., van Vliet, J., and Verburg, P. H. Hotspots of uncertainty in land-use and land-cover change projections: a global-scale model comparison. *Global Change Biology*, 22(12):3967–3983, 2016. doi:10.1111/gcb.13337.
- Prevedello, J. A., Winck, G. R., Weber, M. M., Nichols, E., and Sineru, B. Impacts of forestation and deforestation on local temperature across the globe. *PLOS ONE*, 14(3):e0213368, 2019. ISSN 1932-6203. doi:10.1371/journal.pone.0213368.
- Reback, J., jbrockmendel, McKinney, W., den Bossche, J. V., Roeschke, M., Augspurger, T., Hawkins, S., Cloud, P., gfyong, Hoeffler, P., Sinhrks, Klein, A., Petersen, T., Tratner, J., She, C., Ayd, W., Shadrach, R., Naveh, S., Garcia, M., Darbyshire, J., Schendel, J., Wörtwein, T., Hayden, A., Saxton, D., Gorelli, M. E., Li, F., Zeitlin, M., Jancauskas, V., McMaster, A., and Li, T. pandas-dev/pandas: Pandas. *Zenodo*, 2022. doi:10.5281/zenodo.7037953.
- Reid, R. S., Tomich, T. P., Xu, J., Geist, H., Mather, A., DeFries, R. S., Liu, J., Alves, D., Agbola, B., Lambin, E. F., Chhabra, A., Veldkamp, T., Kok, K., van Noordwijk, M., Thomas, D., Palm, C., and Verburg, P. H. Linking Land-Change Science and Policy: Current Lessons and Future Integration. In Lambin, E. F. and Geist, H., editors, *Land-Use and Land-Cover Change. Global Change - The IGBP Series*, Global Change - The IGBP Series, pages 157–171. Springer, Berlin, Heidelberg, 2006. ISBN 9783540322023. doi:10.1007/3-540-32202-7_7.
- Rockström, J., Steffen, W., Noone, K., Persson, Å., Chapin, F. S., Lambin, E. F., Lenton, T. M., Scheffer, M., Folke, C., Schellnhuber, H. J., Nykvist, B., de Wit, C. A., Hughes, T., van der Leeuw, S., Rodhe, H., Sörlin, S., Snyder, P. K., Costanza, R., Svedin, U., Falkenmark, M., Karlberg, L., Corell, R. W., Fabry, V. J., Hansen, J., Walker, B., Liverman, D., Richardson, K., Crutzen, P., and Foley, J. A. A safe operating space for humanity. *Nature*, 461(7263):472–475, 2009. ISSN 1476-4687. doi:10.1038/461472a.
- Rodrigues, H. and Soares-Filho, B. A Short Presentation of Dinamica EGO. In Camacho Olmedo, M. T., Paegelow, M., Mas, J.-F., and Escobar, F., editors, *Geomatic Approaches for Modeling Land Change Scenarios*, Lecture Notes in Geoinformation and Cartography, pages 493–498. Springer International Publishing, Cham, 2018. ISBN 9783319608013. doi:10.1007/978-3-319-60801-3_35.

- Rudemo, M. Empirical Choice of Histograms and Kernel Density Estimators. *Scandinavian Journal of Statistics*, 9(2):65–78, 1982.
- Rwanga, S. S. and Ndambuki, J. M. Accuracy assessment of land use/land cover classification using remote sensing and gis. *International Journal of Geosciences*, 08:611–622, 2017.
- Sain, S. R., Baggerly, K. A., and Scott, D. W. Cross-validation of multivariate densities. *Journal of the American Statistical Association*, 89(427):807–817, 1994. doi:10.1080/01621459.1994.10476814.
- Saporta, G. *Probabilités, analyse des données et statistique*. Editions Technip, 2008. ISBN 978-2710809807.
- Saunders, D. A., Hobbs, R. J., and Margules, C. R. Biological Consequences of Ecosystem Fragmentation: A Review. *Conservation Biology*, 5(1):18–32, 1991. ISSN 0888-8892.
- Schindler, A. *Bandwidth Selection in Nonparametric Kernel Estimation*. PhD thesis, Faculty of Economic Sciences of the Georg-August-Universität Göttingen, 2011.
- Schlaepfer, D. R., Bradford, J. B., Lauenroth, W. K., Munson, S. M., Tietjen, B., Hall, S. A., Wilson, S. D., Duniway, M. C., Jia, G., Pyke, D. A., Lkhagva, A., and Jamiyansharav, K. Climate change reduces extent of temperate drylands and intensifies drought in deep soils. *Nature Communications*, 8(1), 2017. doi:10.1038/ncomms14196.
- Schuur, E. A. G., McGuire, A. D., Schädel, C., Grosse, G., Harden, J. W., Hayes, D. J., Hugelius, G., Koven, C. D., Kuhry, P., Lawrence, D. M., Natali, S. M., Olefeldt, D., Romanovsky, V. E., Schaefer, K., Turetsky, M. R., Treat, C. C., and Vonk, J. E. Climate change and the permafrost carbon feedback. *Nature*, 520(7546):171–179, 2015. doi:10.1038/nature14338.
- Scott, D. *Multivariate density estimation: Theory, practice, and visualization: Second edition*. John Wiley, 2015. doi:10.1002/9781118575574.
- Searchinger, T., Heimlich, R., Houghton, R. A., Dong, F., Elobeid, A., Fabiosa, J., Tokgoz, S., Hayes, D., and Yu, T.-H. Use of U.S. Croplands for Biofuels Increases Greenhouse Gases Through Emissions from Land-Use Change. *Science*, 319(5867):1238–1240, 2008. doi:10.1126/science.1151861.

- Sharpe, D. Your chi-square test is statistically significant: Now what? *Practical Assessment, Research & Evaluation*, 20(8), 2015. doi:10.7275/tbfa-x148.
- Siminelakis, P., Rong, K., Bailis, P., Charikar, M., and Levis, P. Rehashing Kernel Evaluation in High Dimensions. In *International Conference on Machine Learning*, pages 5789–5798. PMLR, 2019.
- Smith, P., Bustamante, M. M. C., Ahammad, H., Clark, H., Dong, H., El-siddig, E. A., Haberl, H., Harper, R., House, J. I., Jafari, M., Masera, O., Mbow, C., Ravindranath, N. H., Rice, C. W., Abad, C. R., Romanovskaya, A. Y., Sperling, F., and Tubiello, F. N. *Agriculture, Forestry and Other Land Use (AFOLU)*, page 811–922. Cambridge University Press, 2015. doi:10.1017/CBO9781107415416.017.
- Soares-Filho, B. S., Coutinho Cerqueira, G., and Lopes Pennachin, C. Dinamica — A stochastic cellular automata model designed to simulate the landscape dynamics in an Amazonian colonization frontier. *Ecological Modelling*, 154(3):217–235, 2002. doi:10.1016/S0304-3800(02)00059-5.
- Soares-Filho, B., Rodrigues, H., and Follador, M. A hybrid analytical-heuristic method for calibrating land-use change models. *Environmental Modelling & Software*, 43:80–87, May 2013. ISSN 1364-8152. doi:10.1016/j.envsoft.2013.01.010. URL <https://www.sciencedirect.com/science/article/pii/S1364815213000236>.
- Steffen, W., Richardson, K., Rockström, J., Cornell, S., Fetzer, I., Bennett, E., Biggs, R., Carpenter, S., Vries, W., de Wit, C., Folke, C., Gerten, D., Heinke, J., Persson, L., Ramanathan, V., Reyers, B., and Sörlin, S. 'Planetary Boundaries: Guiding Human Development on a Changing Planet'. *Science*, 2015. doi:10.1126/science.1259855.
- Stephens, L., Fuller, D., Boivin, N., Rick, T., Gauthier, N., Kay, A., Marwick, B., Armstrong, C. G., Barton, C. M., Denham, T., Douglass, K., Driver, J., Janz, L., Roberts, P., Rogers, J. D., Thakar, H., Altaweel, M., Johnson, A. L., Sampietro Vattuone, M. M., Aldenderfer, M., Archila, S., Artioli, G., Bale, M. T., Beach, T., Borrell, F., Braje, T., Buckland, P. I., Jiménez Cano, N. G., Capriles, J. M., Diez Castillo, A., Çilingiroğlu, Ç., Negus Cleary, M., Conolly, J., Coutros, P. R., Covey, R. A., Cremaschi, M., Crowther, A., Der, L., di Lernia, S., Doershuk, J. F., Doolittle, W. E., Edwards, K. J., Erlandson, J. M., Evans, D., Fairbairn, A., Faulkner, P., Feinman, G., Fernandes, R., Fitzpatrick, S. M., Fyfe, R., Garcea, E., Goldstein, S., Goodman, R. C., Dalpoim Guedes, J., Herrmann, J., Hiscock, P., Hommel, P., Horsburgh,

- K. A., Hritz, C., Ives, J. W., Junno, A., Kahn, J. G., Kaufman, B., Kearns, C., Kidder, T. R., Lanoë, F., Lawrence, D., Lee, G.-A., Levin, M. J., Lindskoug, H. B., López-Sáez, J. A., Macrae, S., Marchant, R., Marston, J. M., McClure, S., McCoy, M. D., Miller, A. V., Morrison, M., Motuzaite Matuzeviciute, G., Müller, J., Nayak, A., Noerwidi, S., Peres, T. M., Peterson, C. E., Proctor, L., Randall, A. R., Renette, S., Robbins Schug, G., Ryzewski, K., Saini, R., Scheinsohn, V., Schmidt, P., Sebillaud, P., Seitsonen, O., Simpson, I. A., Sołtysiak, A., Speakman, R. J., Spengler, R. N., Steffen, M. L., Storozum, M. J., Strickland, K. M., Thompson, J., Thurston, T. L., Ulm, S., Ustunkaya, M. C., Welker, M. H., West, C., Williams, P. R., Wright, D. K., Wright, N., Zahir, M., Zerboni, A., Beaudoin, E., Munevar Garcia, S., Powell, J., Thornton, A., Kaplan, J. O., Gaillard, M.-J., Klein Goldewijk, K., and Ellis, E. Archaeological assessment reveals Earth's early transformation through land use. *Science*, 365(6456):897–902, 2019. doi:10.1126/science.aax1192.
- Sun, H., Forsythe, W., and Waters, N. Modeling Urban Land Use Change and Urban Sprawl: Calgary, Alberta, Canada. *Networks and Spatial Economics*, 7(4):353–376, 2007. ISSN 1572-9427. doi:10.1007/s11067-007-9030-y.
- Taha, H. Urban climates and heat islands: albedo, evapotranspiration, and anthropogenic heat. *Energy and Buildings*, 25(2):99–103, 1997. ISSN 0378-7788. doi:10.1016/S0378-7788(96)00999-1.
- Tai, W. M. Optimal Coreset for Gaussian Kernel Density Estimation. In Goaoc, X. and Kerber, M., editors, *38th International Symposium on Computational Geometry (SoCG 2022)*, volume 224 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 63:1–63:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.SoCG.2022.63.
- Taud, H. and Mas, J. Multilayer Perceptron (MLP). In Camacho Olmedo, M. T., Paegelow, M., Mas, J.-F., and Escobar, F., editors, *Geomatic Approaches for Modeling Land Change Scenarios*, Lecture Notes in Geoinformation and Cartography, pages 451–455. Springer International Publishing, Cham, 2018. ISBN 9783319608013. doi:10.1007/978-3-319-60801-3_27.
- Terrell, G. The Maximal Smoothing Principle in Density Estimation. *Journal of the American Statistical Association*, 85(410):470–477, 1990. doi:10.1080/01621459.1990.10476223.
- Tiku, M. L. Power Function of the F-Test Under Non-Normal Situations. *Journal of the American Statistical Association*, 66(336):913–916, 1971. doi:10.1080/01621459.1971.10482371.

- van Asselen, S. and Verburg, P. H. Land cover change or land-use intensification: simulating land system change with a global-scale land change model. *Global Change Biology*, 19(12):3648–3667, 2013. doi:10.1111/gcb.12331.
- van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., Yu, T., and the scikit-image contributors. scikit-image: image processing in Python. *PeerJ*, 2:e453, 2014. ISSN 2167-8359. doi:10.7717/peerj.453.
- van Vliet, J. and Verburg, P. H. A Short Presentation of CLUMondo. In Camacho Olmedo, M. T., Paegelow, M., Mas, J.-F., and Escobar, F., editors, *Geomatic Approaches for Modeling Land Change Scenarios*, Lecture Notes in Geoinformation and Cartography, pages 485–492. Springer International Publishing, Cham, 2018. ISBN 9783319608013. doi:10.1007/978-3-319-60801-3_34.
- van Vliet, J., Malek, Z., and Verbug, P. *The CLUMondo land use change model, manual and exercises*, 2015.
- van Vliet, J., Bregt, A. K., Brown, D. G., van Delden, H., Heckbert, S., and Verburg, P. H. A review of current calibration and validation practices in land-change modeling. *Environmental Modelling & Software*, 82:174–182, 2016. doi:10.1016/j.envsoft.2016.04.017.
- Vannier, C., Lefebvre, J., Longaretti, P.-Y., and Lavorel, S. Patterns of Landscape Change in a Rapidly Urbanizing Mountain Region. *Cybergeo*, 2016. doi:10.4000/cybergeo.27800.
- Vannier, C., Bierry, A., Longaretti, P.-Y., Nettier, B., Cordonnier, T., Chauvin, C., Bertrand, N., Quétier, F., Lasseur, R., and Lavorel, S. Co-constructing future land-use scenarios for the Grenoble region, France. *Landscape and Urban Planning*, 190:103614, 2019a. doi:10.1016/j.landurbplan.2019.103614.
- Vannier, C., Lasseur, R., Crouzat, E., Byczek, C., Lafond, V., Cordonnier, T., Longaretti, P.-Y., and Lavorel, S. Mapping ecosystem services bundles in a heterogeneous mountain region. *Ecosystems and People*, 15:74–88, 2019b. doi:10.1080/26395916.2019.1570971.
- Verburg, P. H., de Koning, G. H. J., Kok, K., Veldkamp, A., and Bouma, J. A spatial explicit allocation procedure for modelling the pattern of land use change based upon actual land use. *Ecological Modelling*, 116(1):45–61, 1999. doi:10.1016/S0304-3800(98)00156-2.

- Verburg, P. H., Kok, K., Pontius, R. G., and Veldkamp, A. Modeling Land-Use and Land-Cover Change. In Lambin, E. F. and Geist, H., editors, *Land-Use and Land-Cover Change. Global Change - The IGBP Series*, Global Change - The IGBP Series, pages 117–135. Springer, 2006. doi:10.1007/3-540-32202-7_5.
- Verburg, P. H., Alexander, P., Evans, T., Magliocca, N. R., Malek, Z., Rounsevell, M. D. A., and van Vliet, J. Beyond land cover change: towards a new generation of land use models. *Current Opinion in Environmental Sustainability*, 38:77–85, 2019. doi:10.1016/j.cosust.2019.05.002.
- Verburg, P. H., Dearing, J. A., Dyke, J. G., Leeuw, S. v. d., Seitzinger, S., Steffen, W., and Syvitski, J. Methods and approaches to modelling the Anthropocene. *Global Environmental Change*, 39:328–340, 2016. ISSN 0959-3780. doi:10.1016/j.gloenvcha.2015.08.007.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020. doi:10.1038/s41592-019-0686-2.
- Vitousek, P. M., Mooney, H. A., Lubchenco, J., and Melillo, J. M. Human Domination of Earth’s Ecosystems. *Science*, 277(5325):494–499, 1997. ISSN 0036-8075, 1095-9203. doi:10.1126/science.277.5325.494.
- Vogel, E., Donat, M. G., Alexander, L. V., Meinshausen, M., Ray, D. K., Karoly, D., Meinshausen, N., and Frieler, K. The effects of climate extremes on global agricultural yields. *Environmental Research Letters*, 14(5):054010, 2019. doi:10.1088/1748-9326/ab154b.
- Voinov, A., Kolagani, N., McCall, M. K., Glynn, P. D., Kragt, M. E., Ostermann, F. O., Pierce, S. A., and Ramu, P. Modelling with stakeholders – Next generation. *Environmental Modelling & Software*, 77:196–220, 2016. ISSN 1364-8152. doi:10.1016/j.envsoft.2015.11.016.
- Wand, M. and Jones, C. Multivariate plug-in bandwidth selection. *Computational Statistics*, 9(2):97–116, 1994.

- Wand, M. P. Error analysis for general multivariate kernel estimators. *Journal of Nonparametric Statistics*, 2(1):1–15, 1992. doi:10.1080/10485259208832538.
- Wang-Erlandsson, L., Tobian, A., van der Ent, R. J., Fetzer, I., te Wierik, S., Porkka, M., Staal, A., Jaramillo, F., Dahlmann, H., Singh, C., Greve, P., Gerten, D., Keys, P. W., Gleeson, T., Cornell, S. E., Steffen, W., Bai, X., and Rockström, J. A planetary boundary for green water. *Nature Reviews Earth & Environment*, 3(6):380–392, 2022. ISSN 2662-138X. doi:10.1038/s43017-022-00287-8.
- Wells, J. R. and Ting, K. M. A new simple and efficient density estimator that enables fast systematic search. *Pattern Recognition Letters*, 122:92–98, 2019. doi:10.1016/j.patrec.2018.12.020.
- Wheeler, T. and von Braun, J. Climate change impacts on global food security. *Science*, 341(6145):508–513, 2013. doi:10.1126/science.1239402.
- Wilcox, R. *An introduction to robust estimation and hypothesis testing*, 4th edition. Elsevier, 2020. doi:10.1016/C2010-0-67044-1.
- Yang, Y. and Newsam, S. Bag-of-visual-words and spatial extensions for land-use classification. In *Proceedings of the 18th SIGSPATIAL International Conference on Advances in Geographic Information Systems*, GIS '10, pages 270–279, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781450304283. doi:10.1145/1869790.1869829.
- Zheng, Y., Jests, J., Phillips, J. M., and Li, F. Quality and efficiency for kernel density estimates in large data. In *SIGMOD '13: Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 433–444, 2013. doi:10.1145/2463676.2465319.